



Programa Profesional en Inteligencia Artificial y Data
Science

Módulo 3. Aprendizaje Automático Aplicado

Tema 1. Introducción al lenguaje automático

- 1.1. Introducción y objetivos
- 1.2. Introducción

Tema 2. Análisis exploratorio de datos y preprocesamiento

- 2.1. Introducción y objetivos
- 2.2. Análisis exploratorio de datos y preprocesamiento I
- 2.3. Análisis exploratorio de datos y preprocesamiento II

Tema 3. Algoritmos de aprendizaje automático supervisado

- 3.1. Introducción y objetivos
- 3.2. Algoritmos de aprendizaje automático supervisado I
- 3.3. Algoritmos de aprendizaje automático supervisado II

Tema 4. Algoritmos de aprendizaje automático no supervisado

- 4.1. Algoritmos de aprendizaje automático no supervisado I
- 4.2. Algoritmos de aprendizaje automático no supervisado II
- 4.3. Aprendizaje automático

Tema 5. Redes neuronales y deep learning

- 5.1. Introducción y objetivos
- 5.2. Redes neuronales y Deep learning (continuación)

Tema 6. Despliegue de modelos de aprendizaje automático

- 6.1. Introducción y objetivos
- 6.2. Despliegue de modelos de aprendizaje automático

Tema 1. Introducción al lenguaje automático

1.1. Introducción y objetivos

En esta unidad formativa se hace una introducción al aprendizaje automático. Inicialmente describimos y caracterizamos la inteligencia artificial fuerte y débil.

Luego se introducen los conceptos principales de aprendizaje automático y sus tipos para, a continuación, ir esbozando los conceptos de conjunto de datos, los tipos de variables y su clasificación para terminar haciendo un recorrido por los algoritmos y modelos de aprendizaje automático y su proceso.

Tema 1. Introducción al lenguaje automático

1.2. Introducción

Probablemente ya conozcas de lecturas anteriores, por videos u otra vía, los elementos esenciales de la inteligencia artificial. Probablemente sepas también, que se tiene como inicios de este campo de las ciencias de la computación, el año 1956 cuando se juntaron, en lo que se dio a llamar conferencia de Dartmouth en Estados Unidos, científicos de la talla de John McCarthy, Marvin Minsky, Claude Shannon, Allen Newell, y Herbert. A. Simon. En esta conferencia se acuñó el término «inteligencia artificial» para denominar el nuevo campo que acaba de nacer.

El objetivo de dicha conferencia era examinar la posibilidad de que cada aspecto del aprendizaje o cualquier otra característica de la inteligencia pudiera ser descrita con tanta precisión que se pudiera construir una máquina que la simulara. Se intentaría encontrar la forma para que las máquinas usen el lenguaje, creen abstracciones y conceptos, resuelvan los tipos de problemas que actualmente solo pueden solucionar los humanos y se vayan autoperfeccionando. Pensaron que diez personas tardarían dos meses en conseguirlo. Sin embargo, conseguir ese objetivo no ha sido tan sencillo como inicialmente pensaban estos pioneros de la IA. La imposibilidad de cumplir este objetivo llevó al filósofo John Searle a concebir la IA como dividida en dos bandos, dos partes, a los cuales los llamó IA fuerte y IA débil.

Para él, la IA fuerte es la que se dedica, aún, a perseguir el objetivo inicial de la IA. Aquel que pretende desarrollar máquinas capaces de comportarse como un humano más, este campo también se ha nombrado Inteligencia Artificial General.

Por otra parte, la inteligencia artificial débil, es aquella que se centra en resolver problemas más concretos o estrechos, tales como el reconocimiento del habla, la traducción, el reconocimiento de imágenes, etc.

Tema 1. Introducción al lenguaje automático

En la siguiente tabla mostramos una caracterización de cada una de estas Inteligencias Artificiales.

Inteligencia artificial débil	Inteligencia artificial fuerte
Existe en la vida Real	Solo existe en la ciencia ficción
AlphaGo, Watson, etc	3CP0, Wall-e, etc.
Orientada a Problemas muy concretos. Por ejemplo, aprender a clasificar imágenes.	Orientada a resolver problemas abiertos. Ejemplo, modelar la conciencia o el sentido común.
Reactivo	Proactivo
Inflexibles. Solo funcionan para el problema que se diseña	Flexibles. Se adaptan a cualquier situación. Aprenden de sus límites y posibilidades
Son programadas o entrenadas por humanos	Se autoprograman
Implementadas sobre miles de neuronas artificiales. (Deep Learning por ejemplo que pueden tener cientos de capas ocultas)	Implementadas sobre miles de millones de redes o neuronas artificiales u otras tecnologías. (Computación cuántica)
No razonan, solo computan	Imitan el comportamiento humano
Aprenden a partir de ejemplos similares	Aprenden como los humanos
Tareas repetitivas	Aprenden nuevas tareas
No pueden salirse de su marco de trabajo	Se adaptan a nuevos entornos de trabajo

Tabla 1. Caracterización de IA. Fuente: elaboración propia.

Uno de los nombres que representa de forma inequívoca la persecución de los objetivos iniciales de la IA y por tanto representa a los investigadores de la IA fuerte es el del investigador Ben Goertzel.

En un intento por desarrollar lo que se conoce como IA fuerte, Goertzel quería crear un cerebro de bebé digital y lanzarlo a Internet, donde creía que crecería para volverse completamente consciente de sí mismo y mucho más inteligente que los seres humanos. «Estamos en vísperas de una transición de igual magnitud a la aparición de la inteligencia o al surgimiento del lenguaje», dijo al Christian Science Monitor en 1998.

Tema 1. Introducción al lenguaje automático

A pesar de los esfuerzos, la IA Fuerte aún no se ha logrado y no se tiene claro cuando se logrará. Al punto que el padre de la Inteligencia Artificial John McCarthy, casi al final de su carrera investigadora declara «Para crear una verdadera IA se necesitaría el trabajo de 1,7 Einsteins, 2 Maxwells, 5 Faradays y la financiación de 0,3 Proyectos Manhattan», en una clara alusión a la imposibilidad de conseguirla.

Sin embargo, mientras la IAG, conseguía poco éxito, la IA débil alcanzaba hitos importantes. A continuación, se expone una lista, que, sin ser exhaustiva, muestra verdaderos logros de este campo.

Algunos hitos de la IA débil:

- ▶ 1979 el programa BKG 9,8 gana al campeón mundial de Backgammon.
- ▶ 1980 aparece la primera furgoneta guiada por visión artificial.
- ▶ 1988 se aplica la IA a la traducción de Inglés a francés.
- ▶ 1994 dos vehículos autónomos recorren 1000 km en las autopistas de París.
- ▶ 1996 nacen los agentes inteligentes que perciben el entorno.
- ▶ 1997 la IA Deep Blue gana al campeón mundial de Ajedrez y gran maestro Garry Kasparov.
- ▶ 2004 vehículos autónomos recorren 100 kilómetros en el desierto de Mojave en una competencia.
- ▶ 2008 Google lanza la primera aplicación de reconocimiento de Voz.
- ▶ 2011 Watson, un programa desarrollado por IBM, gana el concurso Jeopardy a contrincantes humanos.
- ▶ 2012 aparecen las primeras aplicaciones de Aprendizaje computacional que son capaces de reconocer y clasificar imágenes con gatos.

Tema 1. Introducción al lenguaje automático

- ▶ 2013 Boston Dynamics construye su robot Atlas, bípedo de rescate.
- ▶ 2014 el programa Eugene Goostman pasa el test de Turing. (El 33 % de los jueces creyeron que hablaban con un humano)
- ▶ 2016 AlphaGo gana al campeón mundial de Go. El MIT lanza el primer Taxi autónomo en Singapur.
- ▶ 2017 el programa Libratus vence a cuatro oponentes jugando Poker. La experiencia adquirida por Deepmind puede ahora ser transferida a otro juego. La Ginoide Sophia, se convierte en ciudadana saudí. (Ginoide, término utilizado para denominar a un androide de aspecto antropomorfo femenino).
- ▶ 2018 Alpha Zero aprende a jugar Ajedrez por sí misma. El robot bípedo de Boston Dynamic aprende a hacer Parkour.

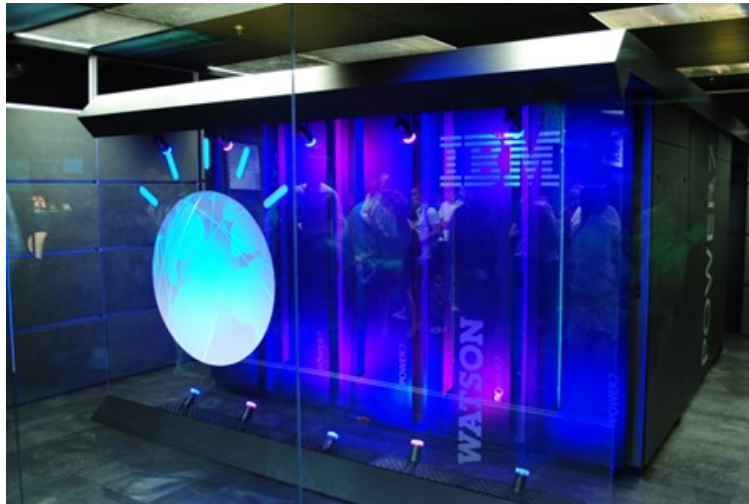


Figura 1. Soporte de Hardware del programa de IBM, Watson.

Tema 1. Introducción al lenguaje automático



Figura 2. Atlas, el robot de Boston Dynamic.

El problema del aprendizaje automático

Uno de los elementos esenciales de la inteligencia humana, es la capacidad de aprender. El interés por el aprendizaje y el razonamiento no es nuevo. Ya desde Sócrates y Aristóteles se estudia de forma analítica la inferencia deductiva e Inductiva y de manera general el aprendizaje en animales y humanos. El desarrollo de los ordenadores ha permitido contar con una nueva herramienta para su investigación y ha engendrado un nuevo campo de investigación, el aprendizaje automático.

El aprendizaje tiene que ser visto como un proceso y uno de los elementos centrales de este es el razonamiento. Este es, a su vez, un proceso del pensamiento por medio del cual, a partir de determinadas premisas, se llega a conclusiones y por tanto se genera conocimiento.

Si las premisas son casos concretos, particulares, que se comparan entre sí, y determinando lo que le es común y ese algo que es común no es casual sino esencial a ellos y por tanto podemos llegar a conclusiones tales como «Todos los

Tema 1. Introducción al lenguaje automático

elementos de tipo x tienen la cualidad y » estamos en presencia de razonamiento inductivo y de esta forma se descubren leyes y principios. La principal cualidad del razonamiento inductivo es que parte de premisas puntuales, particulares y llega a conclusiones generales. (Generalización)

Por el contrario, si partimos de elementos generales o premisas generales, del tipo «Todos los gatos tienen pelo» y tenemos un gato, teniendo en cuenta este tipo de razonamiento, por tanto, el gato tiene que tener pelo» entonces estamos hablando de razonamiento deductivo y como puede verse su rasgo principal es que parte de premisas generales para llegar a conclusiones particulares y por tanto a generar conocimiento.

Podemos, entonces, definir estos dos tipos de razonamiento, aunque grosso modo, de la siguiente manera:

- ▶ **Razonamiento inductivo:** parte de premisas particulares y obtiene conclusiones generales.
- ▶ **Razonamiento deductivo:** parte de premisas generales y obtiene conclusiones particulares.

Teniendo claro que el proceso de aprendizaje tiene entre sus componentes esenciales estos tipos de razonamiento, cabe preguntarse ¿Cómo entonces podría una máquina aprender?

Grosso modo, puede decirse que el campo del aprendizaje automático trata con los métodos que permiten dotar a los ordenadores de la capacidad de aprender. Establezcamos entonces los conceptos más importantes que nos permitan explicar cómo tiene lugar.

Supongamos que tenemos un programa de ordenador al que llamamos L y tenemos un profesor o entrenador al cual llamamos P y este, somete a L a un proceso de aprendizaje para ejecutar una tarea T , entregándole una experiencia D y además

Tema 1. Introducción al lenguaje automático

contamos con un método para evaluar lo que L ha aprendido y que llamaremos m que viene de métrica. Tanto L como P, se desenvuelven en un entorno llamado E.

Entonces, si L mejora su desempeño en la ejecución de la tarea T, dentro del entorno o ambiente que llamamos E, a medida que se le somete a la experiencia D, evaluado por P por medio de m, puede decirse que L, aprende.

Este proceso se muestra en la siguiente Figura.

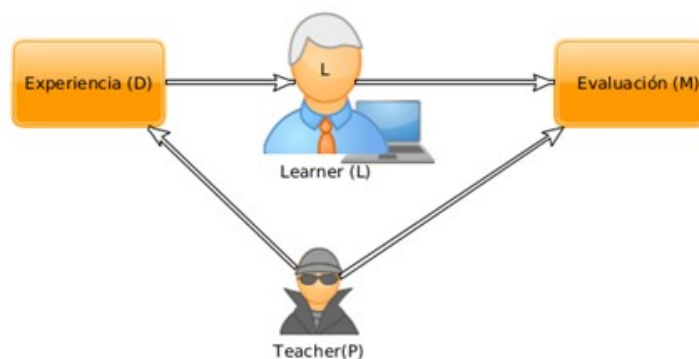


Figura 3. El proceso de aprendizaje grosso modo.

Pongamos como ejemplo, para entender mejor el proceso de aprendizaje automático, un programa de aprendizaje automático que debe aprender a jugar ajedrez.

En este caso concreto el aprendiz L debe aprender a jugar ajedrez, la cual es la tarea, a partir de una experiencia que tomaría la forma de partidas de ajedrez jugadas por grandes maestros. La métrica con la que evaluaríamos la mejora de L en la tarea T sería por medio de las jugadas ganadas, puede que jugando contra si mismos u otros L.

En los epígrafes siguientes iremos viendo la forma exacta que toman cada uno de estos elementos que hemos introducido de manera informal en este parte.

Tema 1. Introducción al lenguaje automático

Con estos elementos podemos, entonces, ver de manera también muy informal, los tipos de aprendizaje que pueden tener lugar.

Tipos de aprendizaje. Una aproximación informal

En un primer escenario, quizás el más elemental, pues es de los que utilizamos casi de manera intuitiva cuando educamos a nuestros hijos o por el que se entrenan las mascotas, es el siguiente:

Supongamos de nuevo que tenemos el aprendiz, llamado L y que interactúa de manera libre con el ambiente E, bajo la supervisión del profesor P y este lo remunera por las buenas acciones o lo castiga por las malas.

Este tipo de aprendizaje se le denomina aprendizaje con reforzamiento. Aunque este tipo de aprendizaje puede realizarse sin la supervisión de P y es E el que determina la remuneración o penalización a L. Este tipo de aprendizaje con reforzamiento se le llama por prueba y error.

Cuando el aprendizaje con reforzamiento se usa la remuneración de las buenas acciones entonces estamos haciendo un aprendizaje con reforzamiento positivo y viceversa cuando penalizamos las malas acciones hacemos uso del aprendizaje con reforzamiento negativo.

Esta es la manera en que amaestramos mascotas. Por ejemplo, si luego de indicar la ejecución de una acción, la mascota la ejecuta de manera correcta, pues se premia con un cubo de azúcar u otra acción. A esto se le llama reforzamiento positivo. Si por el contrario luego de ejecutar una acción fuera de las indicadas, como, por ejemplo, un perro que ladra a un transeúnte se tira de la correa y se regaña, se le da una reprimenda, se le penaliza esa conducta, entonces estamos en presencia de una forma de reforzamiento negativo.

Tema 1. Introducción al lenguaje automático

En este mismo escenario, donde tenemos, al mismo aprendiz que ya habíamos llamado L y un profesor llamado P y que actuaban en un ambiente E, también puede darse otro tipo de aprendizaje. En este caso, a L, el profesor P le entrega una lista de preguntas y sus respuestas, luego evalúa lo que L ha aprendido por medio de preguntas y puede con preguntas que no estaban en el cuestionario inicial. A este tipo de aprendizaje es al que se le llama aprendizaje supervisado.

- ▶ Aprendizaje supervisado

El detalle distintivo de este tipo de aprendizaje es precisamente que a L se le entregan las respuestas junto al cuestionario. Este L entonces aprende por medio de una experiencia ya obtenida y trata de responder preguntas que no están en el cuestionario.

También puede darse otra forma de aprendizaje, en la cual, a L, se le entregan solo las preguntas y luego P evalúa la manera en que L aprendió, es decir cómo se las arregló con las preguntas que se le hicieron. A este tipo de aprendizaje se le llama Aprendizaje no supervisado.

Y el rasgo distintivo en este tipo de aprendizaje es que, a L, no se les subministran las respuestas a las preguntas, él tiene que encontrarlas y eso es lo que se evalúa.

Para ir resumiendo, como hemos visto, en aprendizaje computacional podemos encontrarnos con tres tipos fundamentales de aprendizaje y digo fundamentales porque hay en la literatura algunos otros que no trataremos aquí.

- ▶ 1. Aprendizaje supervisado.
- ▶ 2. Aprendizaje no supervisado.
- ▶ 3. Aprendizaje con reforzamiento.

Tema 1. Introducción al lenguaje automático

Hemos establecido, de manera informal, cada uno de los tipos de aprendizaje y de la misma manera vamos a descubrir los elementos que conforman estos métodos y los acercaremos poco a poco al campo de ciencia de la computación.

Si analizamos detenidamente cada uno de estos métodos de aprendizaje, podemos darnos cuenta de que en todos ellos hay un elemento que se llama L , este representa el algoritmo o método de aprendizaje. Luego iremos viendo poco a poco que aspecto tiene y las formas que puede adoptar. También tenemos a un elemento P y este representa a los algoritmos de entrenamiento o los ingenieros de aprendizaje con sus herramientas que también poco a poco iremos poniéndole cara y cuerpo. Por otro lado, está, otro de los elementos importantes, la manera en que se le presenta a L la lista de cosas a aprender y que en la vida real toma forma de un conjunto de datos. Por último y no menos importante la manera en que T evalúa lo que L ha aprendido. Esta manera de evaluar lo que cada algoritmo aprende se denominan métricas y suelen ser variadas.

Por tanto, tenemos aquí, que en el aprendizaje computacional intervienen los siguientes elementos:

- ▶ Un conjunto de datos (la lista de preguntas con las respuestas o no).
- ▶ Un Algoritmo de aprendizaje (L en los ejemplos que pusimos).
- ▶ Una métrica de evaluación (en los ejemplos anteriores usamos m para denotarla)

1.2.3. El conjunto de datos, los tipos de variables y su clasificación

En esta parte estaremos aportando los detalles necesarios para comprender la forma que toma la experiencia, de la que hablamos en la sección anterior, que es la que se usa en el entrenamiento de los algoritmos de aprendizaje.

Esta experiencia de la que hemos hablado en las secciones anteriores toma la forma de Conjunto de Datos o Dataset. Específicamente a esta forma de experiencia se le

Tema 1. Introducción al lenguaje automático

llama Tabla de Decisión.

El conjunto de datos es el nombre que se le da, por tanto, a los datos que van a ser usados en el proceso de entrenamiento de los algoritmos de aprendizaje y están organizados de forma tabular, es decir, en filas y columnas al estilo de un libro de Excel.

Height	Weight	Age	Male
151.765	47.825606	63.0	1
139.700	36.485807	63.0	0
136.525	31.864838	65.0	0

Tabla 2. Tabla en Excel. Fuente: elaboración propia.

La Tabla anterior es un ejemplo de un *dataset* real, este contiene el alto, el peso, la edad y el sexo de 544 personas del pueblo Kung San, recopiladas por Nancy Howell en 1960. Este pueblo se encuentra ubicado al oeste del desierto de Kalahari entre Ovamboland y Botswana, al sur de África. Aquí solo se muestran tres casos; pero como ya se comentó este tiene 544 casos.

Un *dataset* está formado por filas y columnas. A las filas se les llama casos y aquí cada una de ellas representa a una persona a la que se les midió ciertas características (El peso, el alto, la edad y el sexo). Cada columna es una medición hecha a cada caso y se llaman variables, rasgos, predictores o características. Estos nombres se usan de forma equivalente en la literatura.

Estas variables o rasgos pueden clasificarse por el tipo de valor que asume o por la función que tienen a la hora de entrenar un modelo de aprendizaje, veamos en más detalles.

Si consideramos el uso de estas variables o rasgos, entonces pueden clasificarse en:

- Variables dependientes (variable predicha).

Tema 1. Introducción al lenguaje automático

- ▶ Variable Independientes (predictores).

Explicemos un poco esto para que se vea con claridad, si fuésemos a aprender a predecir el peso de una persona o caso del conjunto anterior y nos basamos en las otras variables que ya tenemos en él, como el alto, la edad y el sexo, entonces la variable dependiente es la queremos predecir, en este caso el peso y el resto de las variables son llamadas predictoras o variables independientes.

Si, por el contrario, queremos clasificar en hombre o no a los casos de este *dataset* entonces la variable *male*, sería la variable dependiente o predicha y el resto las variables independientes.

Ahora bien, estas variables también se pueden clasificar por su naturaleza, es decir por el tipo de valores que puede asumir y existen dos grandes grupos:

- ▶ Las cualitativas.
- ▶ Las cuantitativas.

A su vez, cada una de las anteriores se dividen dos:

Las variables cualitativas pueden dividirse en:

- ▶ Nominales.
- ▶ Ordinales.

Por otro lado, las cuantitativas se pueden dividir en:

- ▶ Discretas.
- ▶ Continuas.

Tema 1. Introducción al lenguaje automático



Figura 4. Clasificación de las variables según su naturaleza.

Ya sabemos que los cuatro tipos de variables son las siguientes:

- ▶ Nominales (categóricas).
- ▶ Ordinales.
- ▶ Discretas.
- ▶ Continuas.

Definamos y pongamos ejemplo de cada una de ellas.

Variables nominales (categóricas): estas se consideran como el nivel básico de medición. Esto entraña un proceso de codificación o un proceso por medio del cual se asignan números de forma arbitraria a las diferentes categorías que conforman la variable.

Las diferentes categorías simplemente constituyen una clasificación, ellas no pueden ordenarse de ninguna forma, son simplemente diferentes.

Tema 1. Introducción al lenguaje automático

Por ejemplo, cuando queremos preguntar a algún grupo de personas porque razón trabajan, las razones pueden ser:

- ▶ Porque pagan bien.
- ▶ Porque el trabajo da una sensación de bienestar.
- ▶ Porque no hay mucha supervisión y se puede tomar sus propias decisiones.
- ▶ Porque el trabajo es excelente.

Por tanto, asignando un valor numérico a cada tipo de respuesta a esta pregunta estamos asignando una etiqueta a cada una de las posibles respuestas.

Hay veces que solo tenemos dos categorías Hembra/Varón, femenino/masculino, a estas además se les puede conocer como variables dicotómicas o binarias.

Este es el caso de la variable *male*, del *dataset* que analizamos como ejemplo, el de los pesos y el alto de 544 personas. En este caso la variable toma solo dos valores 0 y 1, para representar si el caso pertenece a un hombre o mujer.

Una de las características de las variables nominales es que carecen de orden.

Por tanto, una variable puede ser tratada como nominal cuando sus valores representan categorías que no obedecen a una clasificación intrínseca. Por ejemplo, el departamento de la compañía en el que trabaja un empleado. Algunos ejemplos de variables nominales son: región, código postal o filiación religiosa.

Otros ejemplos de variable nominal:

Outcome, en el *dataset* *pima-indians-diabetes-database*. Esta variable toma dos valores: 1 si el caso pertenece a un paciente con diabetes confirmada y 0 de lo contrario.

Tema 1. Introducción al lenguaje automático

Variables ordinales: este tipo de variable contiene valores que pueden ser ordenados. Por ejemplo, si le preguntamos a un grupo de trabajadores que escojan de entre las siguientes respuestas su nivel de satisfacción, entonces la variable puede considerarse ordinal:

Indique cuál de las siguientes respuestas representa su nivel de satisfacción en su trabajo.

- ▶ 1. Insatisfecho
- ▶ 2. Ni satisfecho ni insatisfecho
- ▶ 3. Medianamente satisfecho
- ▶ 4. Satisfecho
- ▶ 5. Muy satisfecho

Es evidente que aquí el número ofrece una medida de la cantidad de satisfacción con 1 siendo insatisfecho y 5 muy satisfecho. Eso es una escala.

Este tipo de variables es muy sencilla de identificar y no creo que sea necesario que pongamos muchos más ejemplos.

Variables discretas: son aquellas que toman valores puntuales en la recta numérica. Es decir, toman valores discretos.

Puede llamarse discreta a la variable Profesores, que contiene los números de profesores de Matemática en cada una de las universidades de España.

Otro ejemplo sería, la cantidad de niños que asisten a una clase y es importante hacer notar que, a la clase no pueden participar 8 niños y medio, por tanto, esta variable solo toma valores enteros.

Tema 1. Introducción al lenguaje automático

Otros ejemplos serían la cantidad de escalones de una escalera, el número de palabras en un texto, etc.

Variables Continuas: Este tipo de variables pueden tomar cualquier valor numérico. Es decir, estos valores pueden ser enteros o no.

Por ejemplo, la altura de todos los estudiantes de la escuela x , o la longitud de las barras de acero salidas de una planta, o el ingreso de los trabajadores de una fábrica.

Otro ejemplo de este tipo de variable sería la distancia recorrida por un vehículo al frenarlo a una determinada velocidad. O el Valor del Tiempo de Vida de un consumidor de una gran cadena de supermercados.

En el caso del *dataset* que pusimos de ejemplo, las variables *height* y *weight*, son variables continuas.

Los algoritmos y los modelos de aprendizaje automático

Ya sabemos, de los epígrafes anteriores, que se entiende por Inteligencia Artificial débil a aquellos métodos que se concentran en solucionar problemas puntuales de la vida real, como poder clasificar las variantes de covid-19 a partir de su perfil genético o a clasificar imágenes de personas en sistemas de alarmas que usan IA.

También, sabemos que el aprendizaje automático se desarrolló como una herramienta para resolver los problemas anteriores y por tanto es uno de los métodos de la IA débil.

Sabemos también que se puede aprender por medio del razonamiento inductivo o deductivo y que el primero se basa en premisas particulares, específicas y puede llegar a conclusiones generales. Es muy importante que tomemos nota de este detalle pues todos los algoritmos de aprendizaje automático tienen este mecanismo de razonamiento y al cual se le llama principio inductivo.

Tema 1. Introducción al lenguaje automático

Es, este principio inductivo la forma práctica o computacional que toma el razonamiento o inferencia inductivos cuando es aplicado al aprendizaje automático.

En esta sección veremos, aunque también de manera muy informal, cuáles son los algoritmos más usados para cada uno de los métodos de aprendizaje que vimos en las secciones anteriores y estableceremos la diferencia entre algoritmo y modelo.

También sabemos, de manera más o menos clara la forma que toma la experiencia a partir de la cual los algoritmos pueden aprender; pero es necesario hacer algunas distinciones en lo que respecta a las tareas particulares que se pueden llevar a cabo dentro de cada uno de los tipos de aprendizaje.

¿A qué me refiero con esto?, pues, que en dependencia del tipo tarea que se pretende ejecutar así es el tipo de algoritmo que debemos escoger o tenemos disponible para usar.

Si queremos, por ejemplo, predecir al precio de una casa a partir del lugar donde está ubicada y los metros cuadrados, etc. Debemos tener en cuenta que la variable precio, debe ser una del tipo continua y eso quiere decir que puede tomar cualquier valor numérico en el conjunto de los reales positivos.

Por tanto, cuando tenemos esta situación, donde la tarea requiere predecir una variable continua, estamos en presencia de una tarea de Regresión y por tanto hay que escoger algoritmos de regresión.

Si, por el contrario, se necesita predecir el género de una persona a partir de rasgos como el peso y el alto, o se necesita predecir si un paciente padece cardiopatía hipertensiva a partir de determinados factores de riesgo, entonces la variable género, o la variable HTA son variables categóricas dicotómicas, entonces estamos ante una tarea de Clasificación.

Tema 1. Introducción al lenguaje automático

Entonces, podemos decir que en el aprendizaje supervisado se pueden acometer dos tipos de tareas básicamente:

Aprendizaje supervisado:

- ▶ Regresión.
- ▶ Clasificación.

Es necesario tener en cuenta, aunque por ahora a modo de información, que hay algoritmos de clasificación que en vez de retornar una variable dicotómica tal como Hombre/mujer, 0/1, retornan un valor que corresponde a la probabilidad de que un caso n pertenezca a una clase c . Es decir, el valor que obtendríamos sería 0,78, por ejemplo, o 0,51.

Si a diferencia de lo que pasa en aprendizaje supervisado, donde contamos con una tabla de datos que contiene una variable de decisión que ayuda a los algoritmos a aprender la dependencia de esta a partir de los predictores, tenemos una tabla con n casos con b variables y tenemos que aprender de ellos, entonces estamos ante una tarea de *clustering* o agrupamiento. Este tipo de tarea pertenece a los métodos de aprendizaje no supervisado.

En el agrupamiento (*clustering*) se utiliza una medida de distancia o similitud entre los elementos del conjunto para agruparlos. La idea es formar *clusters* con los elementos cuya similitud sea mayor que los de otros grupos.

Por tanto, el elemento a destacar de este tipo de aprendizaje son las medidas de similitud o de distancia que usamos. El método usado por excelencia en aprendizaje no supervisado es el de agrupamiento o *clustering* y este también puede dividirse de dos formas: jerárquico y particional.

Tema 1. Introducción al lenguaje automático

Aprendizaje no supervisado:

► *Clustering:*

- Jerárquico.
- Particional.

Ahora bien, el aprendizaje con reforzamiento es un paradigma completamente diferente a los casos de aprendizaje supervisado y no supervisado. A diferencia del caso supervisado que funciona bajo la idea de la inferencia inductiva, el aprendizaje con reforzamiento se basa en la idea del llamado «condicionamiento operante». Este principio, hablando grosso modo, sostiene la idea de que, si después de cada acción se ejecuta otra en forma de recompensa o castigo, se puede modificar la frecuencia de ejecución de las acciones anteriores por medio del refuerzo o debilitamiento de la motivación a ejecutarla y por tanto nos lleva a aprender.

A diferencia del aprendizaje supervisado, en este, solo existe una señal de recompensa. Es también, importante saber que en este tipo de aprendizaje la recompensa puede ser inmediata o demorada.

Para atrapar la idea de aprendizaje con reforzamiento, es necesario verlo como un proceso secuencial, donde en cada paso t , se observa el estado y del ambiente y se elige una acción a y se ejecuta y como resultado se obtiene una recompensa R . Este proceso se ejecuta hasta que se llega a un objetivo/meta.

Tema 1. Introducción al lenguaje automático

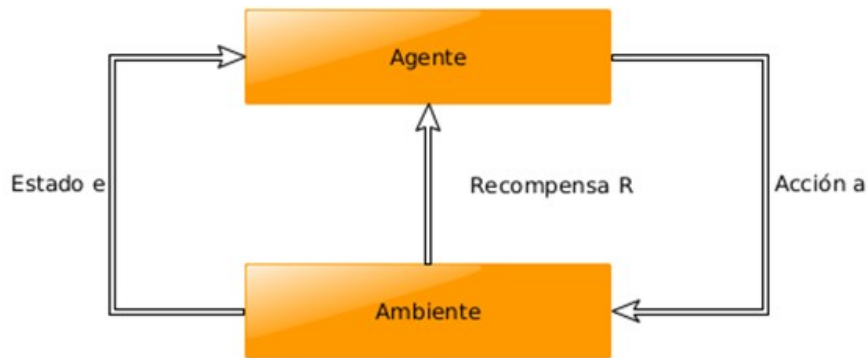


Figura 5. Aprendizaje con reforzamiento.

Aunque hay otros algoritmos de aprendizaje con reforzamiento, los dos más usados son Q-Learning y SARSA.

Entonces, a modo de conclusión de esta parte haremos un listado de los algoritmos disponibles para cada uno de los tipos de aprendizaje. Es necesario explicar que esta lista no es completa, ni pretende serlo, solo daremos los más usados o los que con más frecuencia se pueden encontrar en la literatura.

Algoritmos usados en aprendizaje supervisado:

► Algoritmos de regresión:

- Regresión lineal
- Regresión logística.

► Algoritmos de clasificación:

- KNN. (Algoritmo de los K vecinos más cercanos: k-Nearest Neighbor).
- CART (Arboles de clasificación y regresión).
- Random Forest. (Lo dejo en inglés a falta de un buen término para la su traducción).

Tema 1. Introducción al lenguaje automático

- Naives bayes.
- Máquinas con vectores de apoyo. (Support Vector Machines).
- Redes Neuronales. (Hay muchos modelos englobados en este término).

Los algoritmos de agrupamiento pueden dividirse a grandes rasgos en tres:

- ▶ Particionales.
- ▶ Jerárquicos.
- ▶ Basados en densidad.

Sin embargo, por su simpleza y uso solo tocaremos uno. Pero listaremos algunos.

Algoritmo de Clustering:

- ▶ K-Mean.
- ▶ C-Mean.
- ▶ Fuzzy C-Mean.
- ▶ DBSCAN.

Algoritmos de aprendizaje con reforzamiento:

- ▶ Q-Learning.
- ▶ SARSA.

Creo que sería muy útil poner algunos ejemplos de problemas en los que se puede aplicar las técnicas de aprendizaje automático y ver cómo se seleccionan basados en el tipo de problema.

Ejemplo 1

Tema 1. Introducción al lenguaje automático

Supongamos que somos parte de un equipo de análisis de datos de la empresa x y la dirección de Marketing nos encomienda la tarea de desarrollar un modelo para la predicción del CLV (Valor del tiempo de vida de un consumidor) a partir de los datos históricos almacenados.

Pues si ya tenemos claro que la tarea es la de predecir el Valor del tiempo de vida de un cliente, pero no sabemos cómo luce el modelo analítico que se usa para calcularlo, pero sabemos que contamos con varias mediciones que intervienen de manera directa en su valor, pues procedemos a juntar dichos rasgos y evaluar los datos con que contamos.

Inicialmente ya sabemos que CLV es una variable continua y la unidad de medida es el Euro. Esto nos dice claramente que como vamos a predecir una variable continua tenemos que usar algoritmos de regresión.

Entonces, lo que hay que sacar de este ejemplo es que si se predicen variables continuas los algoritmos que se pueden utilizar son los de regresión.

Ejemplo 2

Como en el ejemplo anterior, que ya formábamos parte del equipo de análisis de la empresa X, la dirección de marketing ahora nos propone otro problema, automatizar la tarea de segmentación de los clientes usando la técnica llamada RFM.

Bueno, al iniciar el análisis del problema, nos referimos como siempre a los datos con los que contamos para la solución y nos damos cuenta que los datos que tenemos para eso son la cantidad en dinero que ha gastado un cliente, la frecuencia con la que el cliente compra nuestro producto y la Recencia o el tiempo que media entre una compra y otra. En este tipo de

Tema 1. Introducción al lenguaje automático

problemas, no tenemos una variable o rasgo que nos permita aprender una dependencia funcional y por tanto tenemos que encontrar un método que aprenda de la propia estructura de los datos.

Cuando tenemos este tipo de problemas, estamos en presencia de un problema de Clustering o agrupamiento y podemos atacarlo usando k-mean u otro de los algoritmos.

Ejemplo 3

Supongamos, para este ejemplo, que somos parte de una empresa consultora Y y se nos acerca un cliente del sistema de salud interesado en un sistema que mejore las propiedades de los sistemas de ventilación mecánica para la atención a pacientes críticos de Covid-19. La mayoría de estos sistemas son controlados por controladores PID, pero sería posible desarrollar un sistema controlado por medio de Aprendizaje con reforzamiento y que aprenda por medio de miles de simulaciones. En este tipo de sistema de control el Learner aprendería por medio de la señal de recompensa cuando lo está haciendo bien o mal a partir de los signos asociados al sistema respiratorio.

Hasta aquí hemos visto los nombres de algunos de los algoritmos que se usan en aprendizaje automático. Hemos visto también algunos ejemplos de los tipos de problemas que pueden resolverse usando regresión/clasificación, clustering y aprendizaje con reforzamiento. Sin embargo, hay un elemento que no hemos mencionado y que es sumamente importante que se entienda desde el principio. En la literatura de aprendizaje automático se trata con muchísima frecuencia, de manera intercambiable los términos Modelo y algoritmo, lo que resulta confuso para los que se inician.

Tema 1. Introducción al lenguaje automático

Es importante entonces que dejemos claro que es un algoritmo de aprendizaje automático y que es un modelo.

Grosso modo, un algoritmo es una serie de pasos que se utilizan para la determinación o cálculo de los parámetros de un modelo a partir del conjunto de datos. Mientras el algoritmo es el mismo para todos los conjuntos de datos, el modelo puede ser distinto. Por tanto, de manera muy general podemos afirmar que el algoritmo es el método por el cual adquirimos el conocimiento para ejecutar una tarea, mientras que el modelo es el conocimiento en sí, el que nos permite luego responder preguntas.

Para ejemplificar la diferencia entre Algoritmo y modelo, tomemos el caso de una red neuronal. No importa que no entienda ahora todo lo relacionado con el algoritmo de Redes Neuronales.

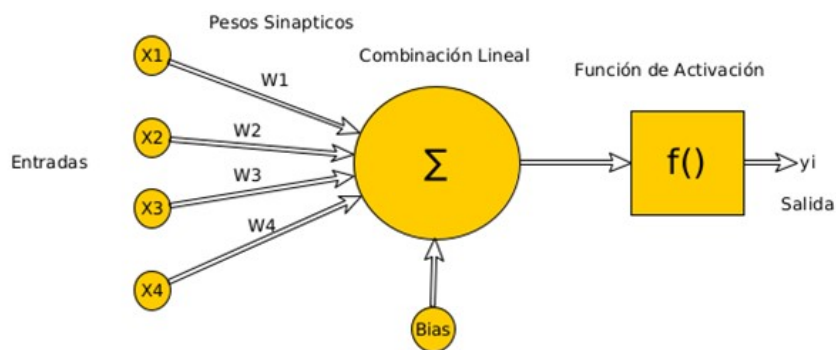


Figura 6. Esquema simplificado de una Red Neuronal.

Cuando usamos una red neuronal para la solución de algún problema de aprendizaje automático, al algoritmo se le pasa el conjunto de datos y este inicia un proceso que se denomina entrenamiento. Este «entrenamiento» no es otra cosa que una serie de pasos para calcular la matriz de pesos sinápticos w , a partir de cada una de los casos del conjunto de datos. Una vez calculada esta matriz de pesos podemos decir que tenemos un modelo entrenado. Es a esta matriz junto a otros metadatos a lo que

Tema 1. Introducción al lenguaje automático

se puede llamar modelo y ser guardado o desplegado a producción.

En la imagen, los círculos con nombre $x_1...x_4$ son los rasgos o variables del conjunto de datos y los $w_1...w_4$ son los llamados pesos sinápticos. A $f()$ se le llama función de activación. El resto de los detalles de cómo funciona este tipo de algoritmo se dará en la parte correspondiente a los detalles de cada uno de los modelos.

Podemos definir entonces:

Algoritmo de aprendizaje automático es el conjunto de pasos necesarios para el cálculo de los parámetros del modelo a partir del conjunto de datos de entrada.

Modelo de aprendizaje automático es el conjunto de parámetros arrojado por algún algoritmo de aprendizaje después del entrenamiento. Este nos permite hacer predicciones.

El proceso de aprendizaje automático

Utilizar un algoritmo de aprendizaje automático implica de alguna forma un conjunto de tareas que se ordenan de forma natural y que facilitan la obtención de modelos y su uso.

Hay autores que dividen este proceso hasta en 7 etapas, nosotros lo dejaremos en cuatro que incluyen todos los pasos necesarios para el uso del aprendizaje automático.

Por tanto, las etapas que componen el proceso de uso del aprendizaje automático son:

- ▶ Establecimiento del problema.
- ▶ Recolección y limpieza de los datos.
- ▶ Construcción y evaluación de los modelos.

Tema 1. Introducción al lenguaje automático

- Despliegue.

Etapas del proceso de aprendizaje automático:

Etapas de establecimiento del problema:

En esta etapa se comienza a tratar de entender el problema que necesitamos resolver. Esto implica, muchas veces, reuniones frecuentes con los que saben del problema. Si tomamos como ejemplo los casos que ya vimos en el epígrafe anterior, pues deberíamos tener sesiones interminables con la gente de Marketing para entender de que trata el problema de la predicción del valor del tiempo de vida del cliente y revisar los métodos por los que antes se hacía y leer documentación, buscar experiencias de otras empresas. En fin tratar de entender que se nos pide.

Etapas de recolección y limpieza de los datos:

Una vez que entendemos que es lo que nos piden, podemos pasar a evaluar los datos que tenemos y si no los tenemos pues tratar de encontrarlos. Luego que ya recopilamos los datos que pensamos necesitamos, iniciamos una etapa de análisis exploratorio, donde tratamos de entender que nos cuentan los datos. Revisar si faltan valores, si los datos tienen anomalías o están en el formato correcto para que los algoritmos puedan procesarlos.

Esta es una de las etapas a las que normalmente se le dedica más tiempo.

Es sumamente importante que se entienda desde el inicio que no todos los algoritmos pueden funcionar con todos los tipos de variables que puede contener los conjuntos de datos.

También en esta etapa hay que ejecutar varias acciones sobre los datos que de no hacerse provocarían que los modelos no pudieran operar de forma correcta. Entre ellas se encuentra el balanceo de los casos, la selección de las variables que no son necesarias, la normalización, el centrado, etc. Estas operaciones las trataremos en

Tema 1. Introducción al lenguaje automático

mayor profundidad en la Unidad correspondiente a Análisis Exploratorio de los datos.

Etapa de construcción y evaluación de los modelos:

En esta etapa se construyen los modelos, definiendo los parámetros con los que se entrenarán y luego poniendo en marcha el proceso de entrenamiento. Luego que el proceso concluye, se evalúan, si al compararlos con determinadas pautas no nos satisface, escogemos nuevos modelos y repetimos. O variamos los parámetros del modelo y tratamos de sintonizar nuestro modelo.

Esta evaluación de los modelos se hace por medio de métricas que de alguna manera están ya estandarizadas y nos permiten comparar modelos y decidir.

Cuando tenemos un modelo que creemos que se aproxima a lo que esperamos podemos pasar a la siguiente fase.

Etapa de despliegue:

Esta etapa depende de las herramientas que estemos utilizando para nuestro proceso de aprendizaje. El hecho es que, en esta etapa, el modelo que ya está entrenado se despliega a producción para ser usado por otras aplicaciones o por nuestros clientes.

El proceso de despliegue puede que sea diferente en cada organización. En una el personal de IT solo debe escribir un par de letras y pasarla al equipo de gobierno de datos para que se apruebe, mientras que en otras a lo mejor hay que escribir un informe y la decisión demore.

Ya en producción los modelos están sometidos a una evaluación constante que determina cuando hay que eliminarlo o reentrenarlo.

Conclusiones

Hasta aquí hemos hecho un recorrido por los conceptos que consideramos que

Tema 1. Introducción al lenguaje automático

deben formar parte de una introducción al aprendizaje automático, sin embargo, hay conceptos que aunque deben tocarse desde un primer momento son más complejos y requeriría de mucho más espacio para su explicación. Me refiero a conceptos como el compromiso Bias/Varianza, Overfitting/underfitting, etc.

Ejercicios:

- ▶ El curso favorito de los alumnos de una escuela.
- ▶ 2. Cantidad de libros en un anaquel.
- ▶ 3. Diámetro de una esfera.
- ▶ 4. Cantidad de clientes atendidos en un restaurante en un día.
- ▶ 5. Lugar que ocupa un nadador en una competencia.
- ▶ 6. Volumen de agua dentro de una lavadora de 200 litros de capacidad máxima.
- ▶ 7. Longitud de 150 tornillos producidos en una fábrica.
- ▶ 8. Número de pétalos que tiene una flor.
- ▶ 9. Color de cabello de los niños que audicionan para una película de Netflix.
- ▶ 10. Tiempo requerido para responder las llamadas en un call center.
- ▶ 11. Candidato al cuál apoyan los votantes en las elecciones presidenciales de Perú.
- ▶ 12. Número de televisores en una casa.
- ▶ 13. Número de páginas de una serie de libros de estadística.
- ▶ 14. Tiempo de vuelo de los aviones que van de Lima a Santiago.
- ▶ 15. Marcas de autos que se venden en tu país.
- ▶ 16. Grado de satisfacción laboral en una empresa.

Tema 1. Introducción al lenguaje automático

- ▶ 17. Número de presidentes que ha tenido tu país en los últimos 5 años.
- ▶ 18. Peso de una persona.
- ▶ 19. Resultado de tirar dos dados.
- ▶ 20. Se define una variable como la fracción de focos defectuosos en una inspección de 100 focos escogidos aleatoriamente en el almacén de una fábrica. ¿Qué tipo de variable es?

Respuestas:

- ▶ 1. Variable cualitativa nominal.
- ▶ 2. Variable cuantitativa discreta.
- ▶ 3. Variable cuantitativa continua.
- ▶ 4. Variable cuantitativa discreta.
- ▶ 5. Variable cualitativa ordinal.
- ▶ 6. Variable cuantitativa continua.
- ▶ 7. Variable cuantitativa continua.
- ▶ 8. Variable cuantitativa discreta.
- ▶ 9. Variable cualitativa nominal.
- ▶ 10. Variable cuantitativa continua.
- ▶ 11. Variable cualitativa nominal.
- ▶ 12. Variable cuantitativa discreta.
- ▶ 13. Variable cuantitativa discreta.

Tema 1. Introducción al lenguaje automático

- ▶ 14. Variable cuantitativa continua.
- ▶ 15. Variable cualitativa nominal.
- ▶ 16. Variable cualitativa ordinal.
- ▶ 17. Variable cuantitativa discreta.
- ▶ 18. Variable cuantitativa continua.
- ▶ 19. Variable cuantitativa discreta.
- ▶ 20. Variable cuantitativa discreta.

Tema 2. Análisis exploratorio de datos y preprocesamiento

2.1. Introducción y objetivos

En esta unidad formativa se trata toda la etapa de preprocesamiento y exploración de los datos con los que serán entrenados los modelos de aprendizaje.

La unidad estará dividida en dos lecciones: en la primera se trata todo lo relacionado con la preparación, limpieza y reducción de los datos, mientras que en la segunda se trata el manejo de datos con clases desbalanceadas, la división del conjunto de datos en datos en conjunto de entrenamiento y prueba y por ultimo el análisis exploratorio de los datos.

La idea detrás de esta unidad formativa es que se adquieran las habilidades necesarias en el manejo de los conjuntos de datos con los que se entrenan los modelos de aprendizaje automático, la corrección a los problemas que se pueden presentar y las técnicas básicas para ello.

El objetivo de esta lección es mostrar el conjunto de técnicas que se engloban dentro de lo que se conoce como etapa de preprocesamiento de los datos y el análisis exploratorio de datos.

El preprocesamiento está formado por las siguientes actividades: Integración, limpieza, normalización y transformación e identificación de ruido más aquellas que pertenecen a lo que se conoce como reducción de datos, como la selección de rasgos, la selección de casos y la discretización.

No siempre es necesario aplicar todas las técnicas a los datos, de lo que se trata es de ir adaptando los datos a los requerimientos de los algoritmos y modelos que necesitamos para resolver el problema que tenemos a mano.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Conocer las etapas de preprocesamiento de los datos.
- ▶ Conocer y aplicar las técnicas de manejo de casos y rasgos duplicados en esta etapa.
- ▶ Conocer y aplicar las técnicas de limpieza de datos, como el manejo de datos faltantes y anómalos.
- ▶ Conocer y aplicar las técnicas de normalización de datos.
- ▶ Conocer y aplicar las técnicas de selección de rasgos y casos.
- ▶ Conocer y aplicar las técnicas de discretización.
- ▶ Conocer y aplicar técnicas de manejo de datos desbalanceados.
- ▶ Conocer y aplicar las técnicas para la división del conjunto de datos en conjunto de entrenamiento y conjunto de pruebas.
- ▶ Conocer y aplicar las técnicas de análisis exploratorio de datos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

2.2. Análisis exploratorio de datos y preprocesamiento I

De la lección anterior sabemos que el proceso de aprendizaje automático está dividido en las siguientes etapas:

- ▶ Establecimiento del problema.
- ▶ Recolección y limpieza de los datos (preprocesamiento).
- ▶ Construcción y evaluación de los modelos.
- ▶ Despliegue.

La segunda etapa, la de recolección y limpieza de datos, es lo que conocemos como etapa de preprocesamiento. La función de esta etapa es la de preparar los datos recolectados para que puedan suministrarse a los algoritmos de aprendizaje.

¿Por qué hay que preprocesar los datos?, esta pregunta aflora de forma natural cuando comenzamos el estudio del aprendizaje automático y nuestro objetivo es responder esta pregunta y mostrar el conjunto de técnicas que se aplican para aliviar cada uno de los problemas que se presentan.

Durante la recolección de los datos es normal que estos, estén contenidos en fuentes distintas, por ejemplo, en libros de Excel, tablas de bases de datos, ficheros planos, etc. Sería necesario entonces integrar todos estos datos en un solo conjunto de datos, esto puede traer consigo duplicados o que los datos que estemos integrando le falten valores o que contengan ruido y valores extremos. Si usáramos estos datos en la etapa de construcción de los modelos, los algoritmos podrías arrojar errores y cuando no lo hagan, de ninguna forma los resultados pudieran ser confiables.

Por tanto, la importancia de esta etapa es vital para el buen desempeño de los

Tema 2. Análisis exploratorio de datos y preprocesamiento

modelos. Si los datos no son correctamente preparados, no se pueden asegurar buenos resultados en las prestaciones de los modelos.

Hay autores que sostienen que a esta etapa se le dedica más del 60% del tiempo de desarrollo de un modelo de aprendizaje automático.

Es importante entender desde el principio que esta etapa no es una etapa con reglas de actuación rígidas, es una etapa donde los problemas que tiene el conjunto de datos se van descubriendo poco a poco y se van solucionando de forma conjunta con el Análisis exploratorio de datos.

Por eso hemos combinamos en una sola lección lo correspondiente a estos dos procesos.

La etapa de procesamiento comprende dos conjuntos de técnicas dirigidas a objetivos diferentes, la primera, conocida como preparación de los datos y la segunda llamada como reducción de datos.

Entonces, la etapa de preprocesamiento de datos está formada por las siguientes tareas:

- ▶ Preparación de los datos.
- ▶ Reducción de datos.
- ▶ División del conjunto de datos. (Entrenamiento, validación y prueba)

La tarea de preparación de los datos, a su vez, está compuesta por las siguientes técnicas:

- ▶ Integración.
- ▶ Manejo de datos faltantes. (Missing Values)
- ▶ Detección de Datos Anómalos. (Outliers)

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Normalización

Por otra parte, la tarea de reducción de datos, está compuesta de las siguientes técnicas:

- ▶ Selección de rasgos (Variables).
- ▶ Selección de casos o instancias.
- ▶ Discretización.

En esta lección, por tanto, comenzaremos por el estudio de las técnicas de preparación de datos, luego las de reducción y por último, todo lo concerniente al análisis exploratorio.

¿Qué tipos de datos esperan cada uno de los métodos de aprendizaje automático? La respuesta a esta pregunta es importante por la simple razón que la mayor parte de esta etapa la dedicamos a adaptar los datos que recolectamos a los métodos o algoritmos que vamos a usar para resolver la tarea que nos planteamos.

Para tratar de responder esa pregunta hagamos un breve recorrido por los métodos de aprendizaje y su clasificación y luego veamos, aunque de forma muy somera, algunas de las exigencias que le impone cada algoritmo de aprendizaje a los datos.

Los algoritmos de aprendizaje automático pueden clasificarse en dos grandes grupos si nos basamos en su función:

- ▶ Predictivos. (aprendizaje supervisado)
- ▶ Descriptivos. (no supervisado)

A su vez, los predictivos se pueden dividir en:

- ▶ Estadísticos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Simbólicos.

Dentro de los métodos Predictivos y estadísticos tenemos métodos tales como:

- ▶ Regresión.
- ▶ Redes Neuronales.
- ▶ Aprendizaje Bayesiano.
- ▶ Aprendizaje basado en instancias. (K-NN)
- ▶ SVM. (Máquinas con vectores de apoyo)

Por otra parte, dentro de los métodos Predictivos y Simbólicos, están:

- ▶ Aprendizaje de Reglas.
- ▶ Árboles de decisión.

Métodos de regresión

Estos métodos son usados en tareas de estimación. Los métodos más conocidos son la Regresión Lineal, la Cuadrática y la logística. Aunque tienen muchos primos cercanos como Lasso y Mínimos cuadrados parciales (PLS), entre otros.

Este tipo de método requiere de atributos numéricos, es decir con rasgos o variables cuantitativas. No toleran muy bien los datos faltantes en los rasgos y son muy sensibles a datos anómalos. (Outliers)

Este tipo de método usa todos los rasgos presentes en el conjunto de datos sin hacer distinción si son útiles o no o si están correlacionados.

Redes neuronales

Al igual que los modelos de regresión, necesitan rasgos numéricos y no toleran

Tema 2. Análisis exploratorio de datos y preprocesamiento

valores faltantes (Missing Values). Sin embargo, si son correctamente configuradas son relativamente tolerantes a datos anómalos (outliers) y ruido.

Aprendizaje bayesiano

Este tipo de método usa la Teoría de las Probabilidades como marco para la toma de decisiones racionales bajo incertidumbre. Se basa en el teorema de Bayes. El método más aplicado es el conocido como Naives Bayes. La definición inicial de este algoritmo solo trabaja con rasgos categóricos, debido a que el cálculo de la probabilidad solo puede hacerse sobre dominios discretos.

Este tipo de algoritmo es muy sensible a valores faltantes (Missing Values).

Debido a que asumen que los rasgos o atributos son independientes entre si, son muy sensibles a la redundancia tanto en los rasgos como en los casos, así como a los datos anómalos (outliers) o ruidosos.

Aprendizaje basado en instancias

Dentro de los métodos que se pueden incluir en esta clase el más conocido es K-NN. La diferencia entre estos métodos es la medida de distancia o similaridad que usan.

Uno de los principales problemas que sufre este tipo de método es la cantidad de espacio de almacenamiento que usa y una baja tolerancia al ruido. Esto hace que sea necesario tratar los datos por medio de procedimientos de reducción.

Máquinas con vectores de apoyo (Support Vector Machines)

En la literatura en español puede que la traducción sea Máquinas con vectores de soporte, sin embargo, para nosotros estos términos (Apoyo y soporte) son sinónimos y solo hace referencia a que usa algunos de los casos del conjunto de datos como apoyo o soporte en la construcción del hiperplano separante entre las clases. No importa que ahora no entiendan completamente esta idea, pero si debe quedar claro el hecho del uso de estas diferencias en la traducción del SVM se refieren a la misma

Tema 2. Análisis exploratorio de datos y preprocesamiento

cosa.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Requiere datos con rasgos numéricos y sin valores faltantes y comúnmente son robustos antes datos anómalos y ruidos.

Aprendizaje Basado en Reglas

Los tipos de algoritmos que se clasifican bajo este nombre, por la naturaleza de su funcionamiento, normalmente necesitan rasgos o atributos nominales o discretizados. Los valores faltantes o anómalos, así como ruidosos pueden impactar de forma negativa la calidad del modelo final.

Ejemplo de algoritmos que se incluyen en esta categoría están: AQ, CN2, RIPPER, PART y FURIA.

Arboles de decisión

Estos algoritmos trabajan intentando separar los datos usando una de las variables independiente, dentro de subgrupos homogéneos. Estos están emparentados con los métodos de aprendizaje basado en reglas y sufren de las mismas dificultades que ellos. Usan rasgos nominales o discretizados. Los valores faltantes o anómalos así como ruidosos pueden impactar de forma negativa la calidad del modelo final.

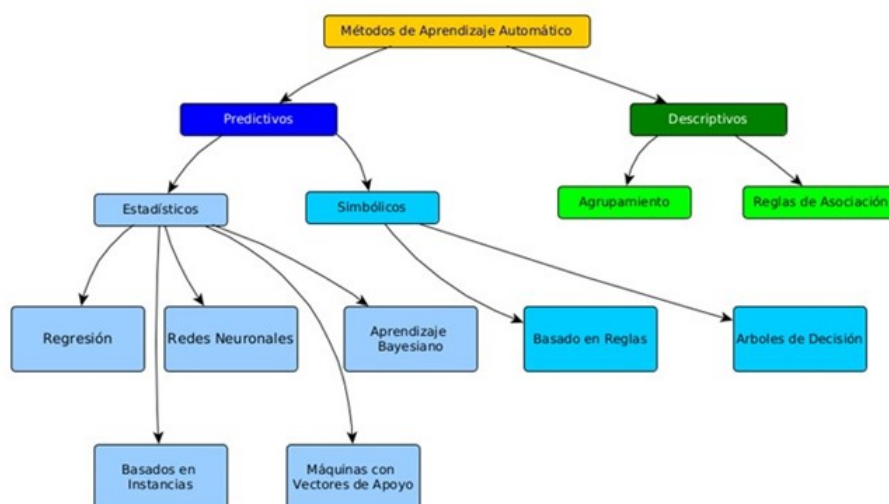


Figura 1. Clasificación de los métodos de aprendizaje automático.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Los métodos correspondientes a la clasificación Descriptiva, no los veremos en detalles, pues están de manera íntima asociados a los predictivos en lo que respecta al uso de los datos.

Preparación de los datos

En esta sección analizaremos las técnicas usadas en la preparación de los datos antes de poder entrenar los modelos de Aprendizaje. Como ya mencionamos al principio de esta lección, la recolección de los datos es una etapa susceptible a errores que si no se tratan adecuadamente pueden poner en duda los resultados de los modelos de aprendizaje.

En la recolección de datos pueden darse problemas como la duplicación de casos y de rasgos. Pueden faltar valores a los rasgos de algunos de los casos o pueden ser anómalos, es decir que estén alejados de forma considerable de los valores considerados normales. Como los datos, puede que se integren desde medios diferentes, estos, pueden estar en unidades de medida diferente o los rasgos, aunque se refieran a los mismo, se llamen diferente. Etc.

Es en esta etapa donde se comienzan a lidiar con este tipo de problemas y las técnicas para su solución las iremos describiendo de manera más o menos en detalles.

Integración de datos

En esta etapa se trata de integrar en un solo conjunto, los datos que pueden provenir de diferentes fuentes. Estas fuentes pueden ser variadas, tales como tablas en bases de datos relacionales, documentos almacenados como libros de Excel, ficheros planos, etc.

Uno de los problemas que se da en este proceso de integración es la duplicación de casos o tuplas. (Una tupla es una fila en una tabla de una BD relacional) Por otro lado puede que también, por algún motivo, tengamos rasgos o atributos redundantes.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Estos problemas, pues, harán más demorado el proceso de Aprendizaje en el mejor de los casos y como norma evitará resultados confiables.

Técnicas para la detección de rasgos duplicados o redundantes.

El problema de la redundancia de los atributos puede hacer que el tamaño del conjunto de datos crezca, provocando que el tiempo de entrenamiento de un modelo también crezca y puede conducir a un sobre ajuste. (En la parte correspondiente a la división de un dataset antes de usarlo en el entrenamiento veremos que es el sobre ajuste).

Se dice que un rasgo o atributo es redundante cuando su información está contenida de alguna forma o puede ser derivada de otro atributo. La detección de este tipo de redundancia puede encontrarse por medio de análisis de correlación.

Aquí, lo más importante es entender que si dos rasgos están altamente correlacionados puede, de manera segura, eliminarse uno de los dos.

Hay varias maneras de acometer el análisis de correlación y en la práctica se lleva a cabo de formas diferentes también. Sin embargo, las técnicas básicas son las siguientes:

- ▶ Cuando los atributos o rasgos son nominales y los valores finitos se usa χ^2 (Llamada Ji Cuadrado).
- ▶ Cuando los atributos son numéricos se usa los coeficientes de correlación y la covarianza.

No nos vamos a detener en el análisis de las formulaciones matemáticas de cada una de estas pruebas, pues existe muchísima documentación al respecto y librerías como pandas en python o R pueden ejecutarlas. Aquí nos centraremos en cómo aplicarlas y la interpretación de los resultados.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Veamos cómo se aplica la prueba χ^2 :

Supongamos que tenemos un conjunto de datos sobre el que estamos trabajando y recién hemos integrado. Bueno, necesitamos comprobar si hay atributos redundantes. Además, sabemos que los atributos son nominales, tales como, género por ejemplo, que contiene valores del tipo masculino/femenino.

Tomemos dos rasgos y apliquemos la prueba ji cuadrado:

Como esta es una prueba de hipótesis, entonces la llamada hipótesis nula, es que las dos variables son independientes. Y podemos aceptarla o rechazarla en función del valor del valor p arrojado. (p-value)

Si $p > \alpha$: Los rasgos son independientes. No están relacionados. Acepta la hipótesis nula.

Si $p \leq \alpha$: Los rasgos están correlacionados. Se rechaza la hipótesis nula.

A α se le conoce como nivel de significación estadística y un valor aceptado de forma general es de 0,05 o 5%.

Cuando los rasgos son numéricos, entonces usamos los coeficientes de correlación tales como los de Pearson o Spearman.

En el caso de usar el coeficiente de correlación de Pearson para determinar si dos rasgos son redundantes, pues tenemos que buscar en los valores que arroja este al usarlos entre dos rasgos. Aquí tampoco nos detendremos en la formulación matemática de este coeficiente y si en la interpretación de los resultados.

Si aplicar el coeficiente a un par de rasgos o atributos con el fin de determinar si son redundantes o no, la interpretación de los resultados sería:

Si $r=1$: Los dos rasgos están perfectamente correlacionados. Es decir, son redundantes. Esto implica que uno de los dos puede eliminarse.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Si $0 < r < 1$: Existe una correlación positiva. Aquí puede tomarse como altamente correlacionados los mayores que 0,85 o 0,90. Es un criterio que depende de cada caso.

Si $-1 < r < 0$: Entonces existe una correlación negativa.

Si $r = -1$: Existe una correlación negativa perfecta.

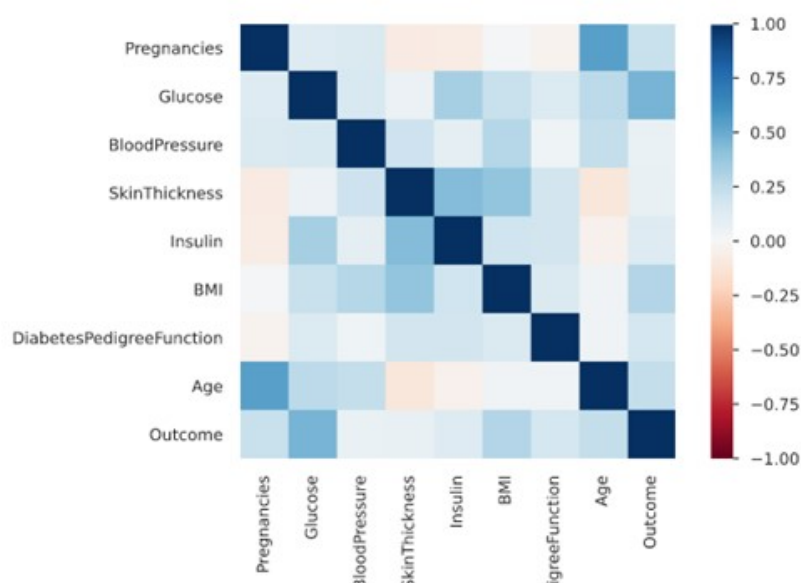


Figura 2. Ejemplo de una matriz de correlación usando coeficiente de correlación de Pearson.

En la parte correspondiente al análisis exploratorio de datos de esta lección daremos muchos más detalles de estos coeficientes.

Técnicas para la detección de casos duplicados o redundantes

Tener tuplas duplicadas o casos duplicados puede ser problemático, no solo por el hecho de desperdiciar espacio y tiempo en el entrenamiento de los modelos, sino porque puede ser una fuente importante de inconsistencia. Además es un problema formidable y difícil de atacar por la complejidad y sutileza de las formas en que puede

Tema 2. Análisis exploratorio de datos y preprocesamiento

presentarse.

Estos problemas pueden presentarse, por ejemplo, a lo hora de integrar los datos, podrían darse el caso que con identificadores diferentes represente tuplas iguales. Este tipo de problema puede pasar inadvertido.

Otro de los problemas puede ser la aparición de tuplas o casos que aunque sean iguales los atributos estén, por ejemplo, representado en una con el Sistema Internacional de unidades y en la otra con Sistema métrico decimal.

Las técnicas para la detección de casos duplicados no están tan formalizadas como otras técnicas y los enfoques más usados son los siguientes:

- ▶ Enfoque probabilista: en este enfoque se formula el problema de detección de duplicados como uno de inferencia bayesiana.
- ▶ Enfoque aprendizaje supervisado: en este enfoque se han usado algoritmos tales como CART y SVM.
- ▶ Enfoque basado en distancia: en este enfoque se usan métricas de distancia o similaridad para la detección de duplicados.

La mayoría de las veces se usan técnicas a la medida cuando de filas duplicadas se trata. El proceso de integración puede ejecutarse de muchas maneras, pueden usarse herramientas propias de las bases de datos, herramientas de integración como el Servicio de Integración de SQL Server o Pentaho; pero también puede hacerse desde Python o R.

En cualquiera de los casos es muy conveniente que tenga un especial cuidado con el tema de los duplicados.

Tema 2. Análisis exploratorio de datos y preprocesamiento



Figura 3. Integración de datos, según la ubicación.

En la Figura 3, puede verse que, durante la etapa de integración, los datos pueden estar físicamente en cualquier lugar. Pueden estar almacenados en Datalakes en la nube, en servidores de terceros a los cuales accedemos por medio de API REST u otros métodos, pueden estar en los Datawarehouses de las empresas o en redes locales y hasta en lo que se conoce como Silos de Información. Es decir, en los ordenadores de analistas en formas de ficheros, aunque esto es cada vez menos frecuente.

Los datos que necesitamos para el entrenamiento de nuestro modelo, pueden estar disponible, también, en diferentes tecnologías de almacenamiento, pueden estar en Bases de Datos relacionales como SQL Server, en BD de MySQL o en servidores Oracle. Pueden estar en Servidores noSQL como MongoDB u otro. Puede también, que estén en Bases de Datos de Grafos como Neo4J. Ver la Figura 4.

Tema 2. Análisis exploratorio de datos y preprocesamiento

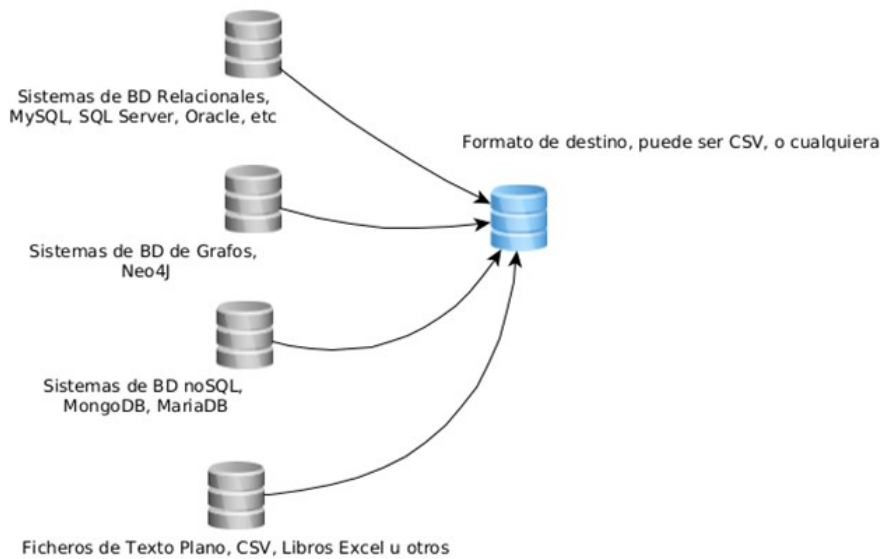


Figura 4. Integración de Datos desde diferentes tecnologías de almacenamiento.

El destino de la integración puede ser una tabla en una base de datos SQL o por lo común un fichero CSV, o de valores separados por coma. Sin embargo, luego de la integración no ha terminado todo, pasamos a una siguiente etapa, la de limpieza de los datos.

Limpieza de datos (Data Cleaning)

Después que se han integrado los datos en un dataset, no quiere decir que los datos estén correctos. Los datos pueden estar representados de manera no estándar, faltar datos o tener ruidos.

Todos estos problemas en los datos impactan de manera negativa en los resultados de los modelos de aprendizaje y por tanto hay que manejarlos de manera adecuada.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Manejo de valores faltantes (Missing Values)

Uno de los problemas que impacta de manera directa la calidad de los modelos es el llamado “Valores Faltantes” o Missing Values y puede darse de muchas formas. Para ejemplificar una de tantas formas en que puede mostrarse, supongamos que tenemos un dataset donde una de las variables es la tensión sanguínea, como en el caso del dataset Pima-indians-diabetes-database. En este dataset hay un rasgo llamado BloodPressure que tiene 35 ceros de los 768 casos con que cuenta. Está claro, de manera intuitiva, que la presión sanguínea no puede ser cero y por tanto se hace evidente que hay un problema a la hora de capturar los datos. Ahora bien, ¿cómo sustituimos estos ceros? ¿Qué valores les asignamos?

Para resolver el problema de los Missing Values o MV, se han desarrollado varias técnicas, llamadas técnicas de imputación.

La solución más rápida sería la de eliminar aquellos casos que contienen atributos con MV. Sin embargo, corremos el riesgo de provocar sesgo en los modelos al perder información valiosa contenida en estos casos. Esto es aún peor si el conjunto de datos tiene relativamente pocos casos.

		Atributos						
		r1	r2	r3	.	.	.	rn
Casos	x1			?				
	x2						?	
	x3							
	.			?				
	.							
	.					?		
	.							
	.							
	xn						?	

Figura 5. Representación general de un dataset con MV.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Dentro de las técnicas más sencillas para el manejo de MV, están las de sustituir sus valores con otros del mismo rasgo. A continuación, las mencionaremos.

Sustitución por valores del mismo rasgo:

- ▶ Sustituir MV, con la media o la mediana del rasgo si es cuantitativo.
- ▶ Sustituir MV, con la moda del rasgo si es categórico.

Si el conjunto tiene varias clases, los MV puede sustituirse con la media o la moda de la clase.

Otros métodos más elaborados de imputación de MV son los siguientes:

- ▶ Métodos basados en Máxima Verosimilitud. (Maximum Likelihood Imputation Methods).
- ▶ Métodos basados en Maximización de la Esperanza. (Expectation-Maximization).
- ▶ Imputación Múltiple.
- ▶ Métodos de imputación basados en Análisis de componentes principales Bayesiano. (Bayesian Principal Component Analysis (BPCA)).
- ▶ Imputación con KNN. (K vecinos más cercanos (KNNI)).
- ▶ Imputación con KNN Ponderado. (Weighted K-NN (WKNNI)).
- ▶ Imputación por medio de clustering usando k-mean. (K-means Clustering Imputation (KMI)).
- ▶ Imputación usando Fuzzy K-mean. (Imputation with Fuzzy K-means Clustering (FKMI)).
- ▶ Imputación usando SVM. (Support Vector Machines Imputation (SVMI)).

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Imputación usando Descomposición de Valores Singulares. (Singular Value Decomposition Imputation (SVDI)).
- ▶ Imputación por medio de Mínimos Cuadrados. (Local Least Squares Imputation (LLSI)).

Aunque, como puede verse, de la lista anterior hay varios métodos de imputación, sin embargo, ninguno es universal o mejor que otro.

Desde una perspectiva práctica podemos explorar los métodos de imputación disponibles en el módulo scikit-learn de python.

En python los valores faltantes o MV suelen representarse por medio de NaN, que significa “No es un Número”.

Este módulo contiene un conjunto de clases que cubren dos grupos de métodos de imputación de MV. Un grupo llamado métodos univariados y el otros, métodos multivariados, además de varios métodos basados aprendizaje automático.

Los métodos univariados son aquellos que usan el valor del rasgo con MV para sustituir los valores faltantes. En contraparte, los multivariados usan el dataset completo.

Estos métodos están disponibles en sklearn.impute y los métodos univariados se pueden usar por medio de la clase SimpleImputer, mientras que los multivariados, por ejemplo, podrían usar IterativeImputer.

El listado que mostramos a continuación es un ejemplo de uso de un método invariado.

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
In [1]: import numpy as np
from sklearn.impute import SimpleImputer
#*****
imp = SimpleImputer(missing_values=np.nan, strategy='mean')
imp.fit([[1, 2], [np.nan, 3], [7, 6]])
X = [[np.nan, 2], [6, np.nan], [7, 6]]
print(imp.transform(X))

[[4.          2.         ]
 [6.          3.6666667]
 [7.          6.         ]]
```

Figura 6. Método invariado. Fuente: elaboración propia.

Otra de las formas en que se puede acometer esta sustitución de MV, es usando la librería Pandas.

La función `fillna()`, ofrece varios métodos para hacerlo. Si se le pasa como argumento un cero, por ejemplo, este sustituirá todos los nan con 0. Si se le pasa como argumento `method="bfill"` sustituye el nan con el valor del mismo atributo del próximo caso. Si se le pasa el argumento `method="ffill"` entonces lo hará en el valor del caso anterior. Ver Figura 7.

Original		data.fillna(0)		data.fillna(method = 'bfill')		data.fillna(method = 'ffill')	
One	Two	One	Two	One	Two	One	Two
0	2	0	2	0	2	0	2
1	3	1	3	1	3	1	3
NaN	0	0	0	2	0	1	0
2	1	2	1	2	1	2	1

Figura 7. Funcionamiento del método `fillna()` de Pandas para el tratamiento de MV.

Una vez que tengamos resuelto el problema de los MV, es necesario chequear otras posibles anomalías en nuestro Dataset.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Manejo de datos anómalos (Outliers)

Al igual que el problema de los valores faltantes o Missing Values, el problema de datos anómalos, extremos o outliers es muy frecuente. Se les llama outliers a aquellas observaciones que se desvían en alguna dirección respecto al comportamiento general del resto de los valores.

Este tipo de valores atípicos pueden tener un efecto adverso en algunos de los métodos de aprendizaje y por tanto es importante tratarlos de manera adecuada.

Una de las definiciones más usadas en la literatura es la dada por Hawkins(1980):

“Un outlier es una observación que se desvía tanto del resto de las observaciones que hace sospechar que fue generada por otro mecanismo”

La intuición detrás de esto es que todos los datos son generados por determinado mecanismo, por ejemplo, algún proceso estadístico. Los datos considerados normales tienen propiedades comunes y si aparece alguno que se desvía demasiado de estas, puede hacer que se sospeche que su mecanismo de generación es otro.

Para entender esto de los mecanismos de generación pongamos como ejemplo el dado por Barnett.

En este ejemplo, el hijo del señor Mr. Hadlum nació 349 días después de que él salir del servicio militar, si como promedio el periodo de gestación humano es de 280 días (40 semanas), entonces este periodo de 349 días puede considerarse estadísticamente un outlier.

Los outliers pueden catalogarse en univariados o multivariados en dependencia de si tomamos en cuenta solo los valores de un rasgo o variable o si se tienen en cuenta todos los rasgos de una instancia.

En el caso de la detección de outliers univariados, la técnica más sencilla es la de

Tema 2. Análisis exploratorio de datos y preprocesamiento

tratar como anómalos todos aquellos valores que están por encima o por debajo de 1,5 veces el rango intercuartílico.

El rango intercuartílico es la diferencia entre el tercer y primer cuartil.

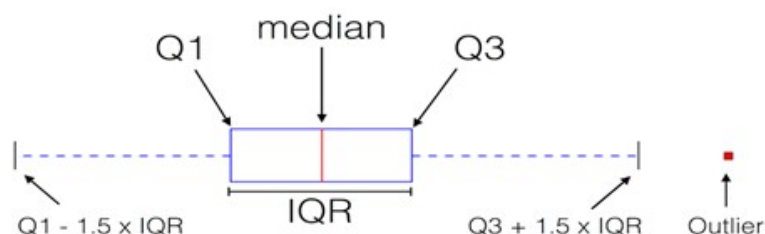
$$IQR = Q3 - Q1$$

Por tanto, podemos tratar como anómalos leves a aquellos valores que estén por debajo de 1,5 veces el rango intercuartílico o por encima de ellos.

$$x < Q1 - 1,5 * IQR \text{ o } x > Q3 + 1,5 * IQR$$

También se le llama valores anómalos severos a aquellos que están hasta 3 veces por debajo o por encima de IQR.

La manera más sencilla de aplicar esta técnica es usando los llamados gráficos de caja y bigote. (Boxplot) Estos gráficos muestran de manera sencilla los valores anómalos, ubicándolos a la derecha o encima del fin del bigote o por debajo o a la izquierda del inicio de él.



Q1: Quartile 1, or median of the *left* data subset after dividing the original data set into 2 subsets via the median (25% of the data points fall below this threshold)

Q3: Quartile 3, median of the *right* data subset (75% of the data points fall below this threshold)

IQR: Interquartile-range, $Q3 - Q1$

Outliers: Data points are considered to be outliers if
value $< Q1 - 1.5 \times IQR$ or
value $> Q3 + 1.5 \times IQR$



Sebastian Raschka, 2016

This work is licensed under a Creative Commons Attribution 4.0 International License

Figura 8. Gráfico de caja y bigote. (Boxplot).

Tema 2. Análisis exploratorio de datos y preprocesamiento

Una vez que detectamos los outliers, estamos en condiciones de determinar cómo debemos tratarlos. La vía más fácil y la del primer impulso sería eliminarlos. Si contamos con muchos datos y los anómalos o extremos son pocos a lo mejor sería buena idea hacerlo; pero sería prudente preguntarse si estos valores se deben a errores en el proceso de integración o en la recolección de los datos o son una muestra de valores genuinos. En cualquiera de las situaciones es necesario preguntarse el origen y tomar la vía de tratamiento en función de eso.

Lo cierto es que, cuando tratamos con valores extremos univariados, podemos aplicar algunas variantes, como sustituirlos por la media de los valores considerados no extremos o con la mediana del conjunto. Otras serían, sustituirlos por los valores que lindan con los bordes de los bigotes en el gráfico de caja y bigote. En cualquiera de los casos el método de tratamiento de estos valores depende del problema que tengamos a mano.

Es importante no perder de vista que las técnicas de detección de datos extremos o anómalos pueden usarse en dos contextos diferentes, uno en el proceso de limpieza de datos y el otro en la detección de eventos anómalos. Y lo llamamos así para diferenciarlos de alguna manera. En el primero, el objetivo es evitar que estos datos influyan de manera negativa en el resultado del entrenamiento de los métodos de aprendizaje automático. Mientras en el segundo, su objetivo es detectar eventos que se desvían de lo normal, como, por ejemplo, fraudes en tarjetas de crédito.

Para la detección de outliers multivariados se han desarrollado varios métodos; pero de manera general todos caen en alguna de las siguientes categorías:

- ▶ Métodos basados en pruebas estadísticas.
- ▶ Métodos basados en profundidad.
- ▶ Métodos basados en desviación.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Métodos basados en distancia.
- ▶ Métodos basados en densidad.
- ▶ Métodos de alta dimensionalidad.

No describiremos ninguno de estos métodos aquí; pero es importante que se tengan en cuenta y se conozcan. Basta luego, si necesitamos usarlas, con buscar en la documentación de las herramientas si están implementadas o buscar documentación adicional para hacerlo. La librería PyOD implementa más de 30 algoritmos de detección.

Veamos un ejemplo en python de cómo se usa pyod.

```
import pandas as pd
from pyod.models.abod import ABOD
from pyod.utils.data import evaluate_print
from pyod.utils.example import visualize
from sklearn.model_selection import train_test_split
from pyod.utils.data import generate_data
#*****
contamination = 0.1 # percentage of outliers
n_train = 500      # number of training points
n_test = 500       # number of testing points
n_features = 2     # number of features

X_train, X_test, y_train, y_test = generate_data(n_train=n_train, n_test=n_test, n_features=n_features,
                                                contamination=contamination, behaviour="new")
#*****
OD_Name = "ABOD"
od=ABOD(method="fast")
od.fit(X_train)
#*****
y_train_pred = od.labels_ # binary labels (0: inliers, 1: outliers)
y_train_scores = od.decision_scores_ # raw outlier scores
y_test_pred = od.predict(X_test) # outlier labels (0 or 1)
y_test_scores = od.decision_function(X_test) # outlier scores
visualize(OD_Name, X_train, y_train, X_test, y_test, y_train_pred, y_test_pred, show_figure=True, save_figure=False)
```

Figura 9. Código Python para un detector de Outlier usando ABOD. (Angle Based Outlier Detection).

Fuente: elaboración propia.

Tema 2. Análisis exploratorio de datos y preprocesamiento

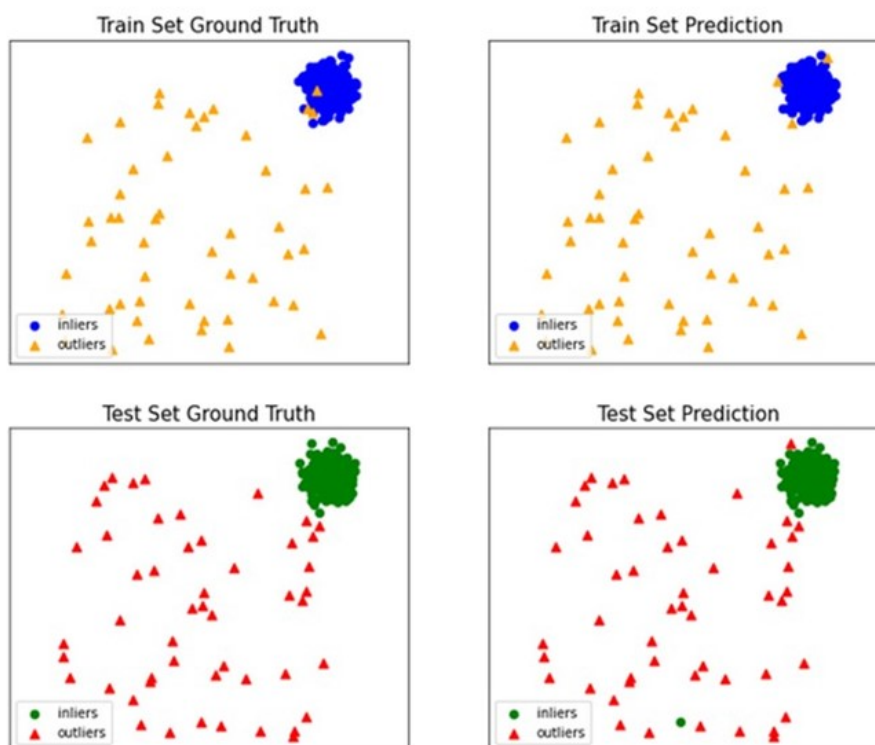


Figura 10. Resultados de la corrida del detector de outliers.

Normalización

Hay varios algoritmos de aprendizaje que son especialmente susceptibles a las escalas de los valores de los rasgos, por ejemplo, las redes neuronales y los algoritmos de regresión o los métodos basados en medidas de distancia. La normalización, representa todos los atributos de un dataset en una escala común, evitando que determinados valores influyan más que otros en los resultados de los modelos.

El objetivo de la normalización es cambiar los valores de los atributos numéricos en el dataset a una escala común, sin distorsionar las diferencias en los rangos de valores. Para el aprendizaje automático, todos los conjuntos de datos no requieren normalización.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Existen varias formas de normalizar, listemos dos de ellas:

- ▶ Normalización min-max.
- ▶ Normalización z-Score.

Normalización min-max:

El objetivo de la normalización min-max es escalar todos los valores numéricos x de un rasgo o atributo dado a un rango especificado denotado por $[\text{new-minA}, \text{new-maxA}]$.

Esto se hace aplicando la siguiente expresión a x para obtener x' :

$$x' = (x - \min x) / (x_{\max} - x_{\min})$$

Donde $\min A$ es el valor mínimo del atributo A , $\max A$ es el valor máximo del atributo A .

En la literatura “normalización” se refiere a un tipo particular de normalización min-max en la cual el intervalo final es $[0,1]$, aunque también es típico usar un intervalo de $[-1, 1]$.

Normalización z-Score:

Otro de los métodos de normalización usados comúnmente es el z-score. Este puede obtenerse aplicando a cada valor del atributo la siguiente expresión:

$$x' = (x - \bar{X}) / \sigma X$$

donde $\bar{X}$ es la media del atributo y σX es la desviación estándar.

Una vez más, con el fin de ver en la práctica como se usan estas técnicas apelamos al módulo scikit-learn de Python.

Este contiene una clase llamada MinMaxScaler que permite normalizar por el método

Tema 2. Análisis exploratorio de datos y preprocesamiento

min-max los valores numéricos de un dataset.

Vamos a mostrar un ejemplo simple, pues en el módulo de programación de inteligencia artificial profundizarán en estos temas o pueden hacerlo buscando información de cómo usar estas técnicas en python.

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
data_scaled = scaler.fit_transform(data)
```

Reducción de datos

Hasta aquí hemos visto las tareas necesarias para integrar los datos desde un conjunto de fuentes distintas y como tratar con muchos de los problemas que se presentan en ellos antes que se puedan usar para el entrenamiento de los algoritmos de aprendizaje automático. Sin embargo hay otro grupo de problemas que, aunque no impactan de manera directa los valores individuales del conjunto de datos, si lo hacen a nivel de las prestaciones y complejidad de los modelos.

En los últimos 20 años, la dimensionalidad de los conjuntos de datos usados en aprendizaje automático ha aumentado drásticamente. (Dimensionalidad es la cantidad de filas por columnas del conjunto) En los años 80 la dimensionalidad rondaba el número 100 y en los 2000 ya superaba los 1500.

Cada día es mucho más común encontrar conjuntos de datos con millones de casos y miles de rasgos. Mientras más grandes son estos conjuntos más lento y complejo se hace el entrenamiento de los modelos. Esto puede conducir a sobreajuste (overfitting) y los modelos tienden a ser menos interpretables.

Para ayudar a resolver estos problemas se usan técnicas de reducción de datos que se agrupan de la forma siguiente:

- ▶ Técnicas de reducción de la dimensionalidad.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Técnicas de reducción del número de instancias o casos (numerosity reduction).
- ▶ Técnicas de reducción de la cardinalidad.

Las técnicas de reducción de dimensionalidad se pueden agrupar en:

- ▶ Selección de rasgos.
- ▶ Construcción/extracción de rasgos.

Las técnicas de reducción del número de instancias o casos se pueden agrupar en:

- ▶ Técnicas paramétricas. (no las veremos)
- ▶ Técnicas no paramétricas.

Las técnicas de reducción del número de instancias o casos no paramétricas se pueden agrupar en:

- ▶ Muestreo de datos.
- ▶ Selección de instancias. (casos)

Mientras que las técnicas de reducción de la cardinalidad se pueden clasificar en:

- ▶ Compartimentación. (binning)
- ▶ Discretización.

Es importante que dejemos claro el concepto de “dimensión” de un conjunto de datos. Cuando se habla de la dimensión de un conjunto, se hace referencia a la cantidad de rasgos de un conjunto de datos, es decir a la cantidad de columnas de esta. Es necesario recordar que un conjunto de datos está compuesto por filas y columnas y que a las columnas se les llaman atributos, rasgos, variables y ahora dimensiones, por otro lado a las filas se les llama casos, instancias, etc.

Tema 2. Análisis exploratorio de datos y preprocesamiento

¿Por qué se les llama dimensiones a los atributos o rasgos de un conjunto de datos? La respuesta es sencilla. Todos los algoritmos de aprendizaje automático usan conceptos matemáticos para modelar las cosas de la vida real. En esta situación específica, cada una de las instancias o casos de un conjunto son vistas como vectores embebidos en un espacio n -dimensional. Cada uno de los rasgos se hace coincidir con una de estas dimensiones.

Curso de la dimensionalidad

Uno de los principales problemas con los que nos encontramos en aprendizaje automático, cuando el número de dimensiones de un conjunto de datos es grande es el llamado “curso de la dimensionalidad”. Hay una relación lineal entre el número de ejemplos o casos de entrenamiento y la dimensionalidad para obtener modelos de alta calidad. Debido a esto, este problema se le llama “maldición de la dimensionalidad”.

Técnicas de reducción de la dimensionalidad

Uno de los problemas representativos de la reducción de dimensionalidad es el conocido como “Selección de Rasgos”. Por eso, iniciaremos esta sección analizando las técnicas más comunes usadas para atacar este problema y luego veremos las correspondientes a la construcción/extracción de rasgos.

Selección de rasgos

Se le denomina selección de rasgos al proceso por medio del cual se elige un subconjunto óptimo de rasgos, de entre todos los que componen el conjunto, basado en algún criterio.

La idea detrás de la solución de este problema es encontrar el subconjunto óptimo de rasgos, es decir los rasgos más importantes y descargar aquellos que son irrelevantes o redundantes.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Las razones para ejecutar un proceso de selección de rasgos son:

- ▶ Eliminar datos irrelevantes.
- ▶ Mejorar la calidad de la predicción en los modelos obtenidos.
- ▶ Reducir los costos computacionales del proceso del aprendizaje.
- ▶ Reducir la complejidad de la descripción de los modelos aprendidos.

Primero que todo, el problema de selección de rasgos, puede considerarse como un problema de búsqueda, donde cada estado del espacio de búsqueda es un subconjunto de rasgos. Para entender este problema, supongamos que tenemos 3 rasgos o dimensiones. Si sabemos que cada rasgo, puede formar parte o no de cada una de las combinaciones, entonces podemos formar 2^m subconjuntos, si m es el número de rasgos originales. Entonces probaríamos cada uno de los subconjuntos o combinaciones y determinamos el que genere el mejor modelo.

Rara vez los conjuntos de datos de la vida real tienen tan pocos rasgos y lo normal en problemas de ciencia o la salud es que tengan muchos. En tareas como la clasificación de textos donde se usan técnicas de procesamiento de lenguaje natural las dimensiones se cuentan en miles. Supongamos que tenemos solamente 32 rasgos, pues la cantidad de combinaciones que habría que explorar para encontrar el subconjunto óptimo serían 4294967296. Esto demuestra la importancia de contar con algoritmos de selección sofisticados. (Ver la Figura 10).

Tema 2. Análisis exploratorio de datos y preprocesamiento

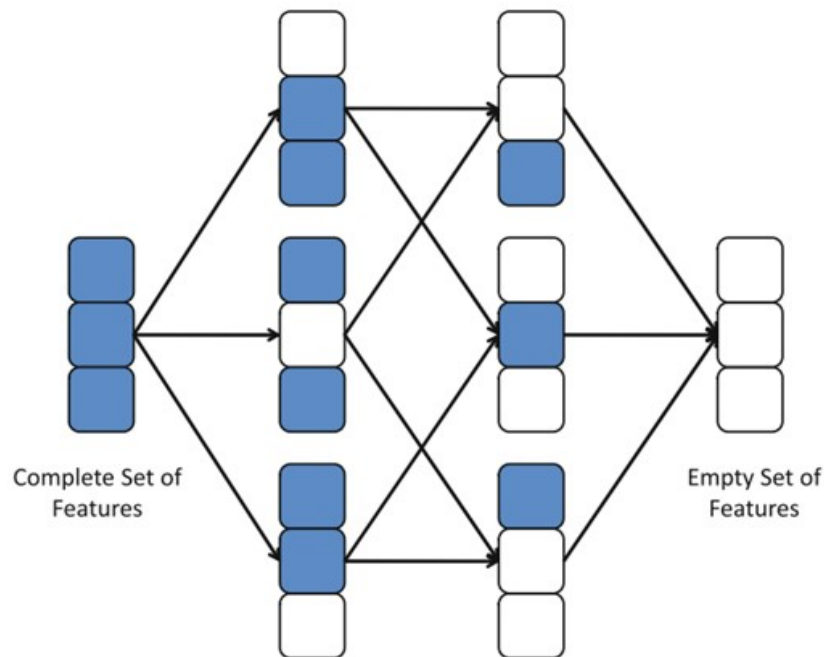


Figura 10. Espacio de búsqueda para selección de rasgos ($m=3$).

Los algoritmos de selección de rasgos, con el fin de encontrar el subconjunto óptimo, pueden comenzar a explorar todas las posibles combinaciones de rasgos de cuatro formas a la que en la literatura se les conoce como direcciones de búsqueda.

La primera, sería comenzar con un subconjunto vacío e ir añadiendo rasgos de manera secuencial e ir evaluándolo. A esta se le llama Generación Secuencial hacia adelante, el conjunto va creciendo hasta que alcanza el conjunto de datos originales. El criterio de parada puede ser un determinado umbral para rasgos relevantes o simplemente todos los posibles subconjuntos en una búsqueda por fuerza bruta.

La segunda, se le llama Generación Secuencial hacia atrás, y comienza con el subconjunto completo de rasgos e iterativamente va eliminando el peor cada vez. Al final quedará un solo rasgo, considerado el más importante o informativo. Como en el caso anterior, puede definirse un criterio de parada.

La tercera forma, es llamada Generación Bidireccional, esta comienza en ambas

Tema 2. Análisis exploratorio de datos y preprocesamiento

direcciones simultáneamente, ejecutando las dos anteriores de forma concurrente. Este tipo de búsqueda terminará en cualquiera de dos casos: 1) Cuando una de las búsquedas encuentra el mejor subconjunto m antes de alcanzar el medio exacto. 2) cuando las búsquedas alcanzan el medio del espacio de búsqueda.

La cuarta y última forma de búsqueda es la llamada Generación aleatoria, esta comienza la búsqueda en una combinación aleatoria.

Seleccionar una dirección de búsqueda es solo una de las decisiones; pero no la única. Si necesitamos escoger un subconjunto óptimo de rasgos, además de escoger la dirección en la que recorreremos las combinaciones o espacio de búsqueda, también debemos elegir cómo las recorreremos y a esto se le llama estrategia de búsqueda. A continuación esbozamos algunas:

- ▶ **Búsqueda Exhaustiva:** En esta estrategia se recorren todas y cada una de las combinaciones posibles para encontrar el subconjunto óptimo. Este tipo de estrategia no es práctica en dataset con un número de rasgos muy alto.
- ▶ **Búsqueda Heurística:** Este tipo de búsqueda hace uso de heurísticas con el fin de evitar recorrer de forma exhaustiva todas las combinaciones de rasgos. Este problema se ha atacado con muchas metaheurísticas tales como Optimización por colonias de hormigas (ACO). Este se considera un problema de Optimización combinatorial.
- ▶ **Búsqueda no determinista:** Este tipo de búsqueda aflora de la combinación complementaria de las dos anteriores.

Hasta aquí hemos visto la manera en que se recorren las posibles combinaciones de los m rasgos que componen nuestra conjunto de datos; pero aún no sabemos cómo escogemos el mejor subconjunto, a esto se le llama criterio de selección y veremos algunos a continuación.

Los criterios de selección se pueden clasificar de la siguiente manera:

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Criterios basados en medidas de información.
- ▶ Criterios basados en medidas de distancia.
- ▶ Criterios basados en medidas de dependencia.
- ▶ Criterios basados en medidas de consistencia.
- ▶ Criterios basados en medidas de calidad de la clasificación (Accuracy).

Criterios basados en medidas de información

Este tipo de criterio se basa en alguna medida de ganancia de información o medida de reducción de la incertidumbre, por ejemplo la entropía de Shannon.

$$-\sum_i P(c_i) \log_2 P(c_i).$$

En este tipo de criterio, un atributo A es escogido basado en una función de ganancia de información IG, si $IG(A) > IG(B)$.

La forma exacta que toma esta medida depende del algoritmo.

Criterios basados en medidas de distancia

También conocidas como distancias de separabilidad, discriminación o divergencia. Este tipo de criterio es similar al anterior, aunque en este las funciones son medidas de distancia.

Las medidas más usadas en este tipo de criterio están resumidas en la siguiente Tabla.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Nombre	Forma matemática
Distancia euclidiana	$D_e = \left[\sum_{i=1}^m (x_i - y_i)^2 \right]^{\frac{1}{2}}$
Distancia City-Block	$D_{cb} = \sum_{i=1}^m x_i - y_i $
Distancia de Cebyshev	$D_{ch} = \max_i x_i - y_i $
Distancia de Minkowski de orden m	$D_m = \left[\sum_{i=1}^m (x_i - y_i)^m \right]^{\frac{1}{m}}$
Distancia cuadrática Q	$D_q = \sum_{i=1}^m \sum_{j=1}^m (x_i - y_i) Q_{ij} (x_j - y_j)$
Distancia Canberra	$D_{ca} = \sum_{i=1}^m \frac{ x_i - y_i }{x_i + y_i}$
Separación angular	$D_{as} = \frac{\sum_{i=1}^m x_i \cdot y_i}{[\sum_{i=1}^m x_i^2 \sum_{i=1}^m y_i^2]^{\frac{1}{2}}}$

Tabla 2. Criterios basados en medidas de distancia. Fuente: elaboración propia.

Criterios basados en medidas de dependencia

A este tipo de medida se les conoce también como medidas de correlación o asociación. Su objetivo es determinar cuan correlacionadas o asociadas están dos variables, de forma tal que conociendo el valor de una podamos derivar el valor de la otra.

El procedimiento común en selección de rasgos es establecer una correlación entre cada uno de los rasgos y la variable de decisión tal que si $R(a)$ es la correlación entre el atributo a y la clase y $R(b)$ es la correlación del atributo b y la clase, entonces escogemos al atributo a , si: $R(a) > R(b)$.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Dicho en otras palabras, se escogen los rasgos con mayor correlación. Si cualquiera de los atributos del dataset es estadísticamente independiente de la clase, su eliminación no tendrá efecto alguno en la separabilidad entre las clases en un problema de clasificación.

Sin embargo, este tipo de correlación no puede manejar correlaciones no lineales entre atributos, por tanto hay que apelar a otras medidas de correlación, por ejemplo la medida de dependencia de Bhattacharyya.

Una de las medidas más usadas es el coeficiente de correlación de Pearson:

$$\rho(x,y) = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\left[\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2 \right]^{\frac{1}{2}}}$$

Criterios basados en medidas de consistencia

A diferencia de las medidas anteriores, esta trata de encontrar el conjunto mínimo de rasgos que separaría las clases de la misma forma que lo hace el conjunto de rasgos completo. La idea de estas medidas es la de encontrar un subconjunto de rasgos que mantenga la misma consistencia en los datos que la consistencia arrojada teniendo en cuenta el conjunto de rasgos originales.

Aquí la idea de inconsistencia es la de dos casos con rasgos con los mismos valores de los rasgos; pero con clases diferentes asignadas.

Criterios basados en medidas de calidad de la clasificación

Este tipo de medida descansa en las formas de evaluación de la calidad de clasificación cuando se usa un clasificador para determinar la optimalidad de un subconjunto de rasgos. La idea detrás de este tipo de medida es someter a entrenamiento un clasificador con los subconjuntos de rasgos que pretendemos evaluar y escoger aquel que mejor calidad de clasificación nos arroje.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Hasta ahora no hemos hablado de las métricas necesarias para la evaluación de los modelos de aprendizaje supervisado y por tanto solo sumariaremos estas medidas en una tabla y luego en la sección correspondiente a los algoritmos de clasificación daremos más detalles sobre ellas.

	Mathematical form
Accuracy	$\frac{tp+fp}{tp+tn+fp+fn}$
Error rate	$1 - \text{Accuracy}$
Chi-squared	$\frac{n(fp \times fn - tp \times tn)^2}{(tp+fp)(tp+fn)(fp+tn)(tn+fn)}$
Information gain	$e(tp+fn, fp+tn) - \frac{(tp+fp)e(tp,fp)+(tn+fn)e(fn,tn)}{tp+fp+tn+fn}$ where $e(x, y) = -\frac{x}{x+y} \log_2 \frac{x}{x+y} - \frac{y}{x+y} \log_2 \frac{y}{x+y}$
Odds ratio	$\frac{tpr}{1-tpr} \bigg/ \frac{fpr}{1-fpr} = \frac{tp \times tn}{fp \times fn}$
Probability ratio	$\frac{tpr}{fpr}$

Tabla 3. Ejemplo. Fuente: elaboración propia.

Donde, tp es true positivo, fp es falso positivo, tn es true negativo y fn significa falso negativo. En la sección de evaluación de los algoritmos de clasificación se verán los detalles.

Los métodos de selección de rasgos se pueden agrupar, básicamente, en las siguientes categorías:

- ▶ Filtros.
- ▶ Envoltorios o Wrappers.
- ▶ Embebidos.
- ▶ Híbridos.

Métodos de selección de rasgos del tipo filtros

Los métodos que se clasifican como “filtros” operan antes de someter los datos al entrenamiento de los algoritmos de aprendizaje automático. Este nombre lo reciben a partir de que filtran los rasgos no deseados antes del proceso de aprendizaje.

Tema 2. Análisis exploratorio de datos y preprocesamiento



Figura 11. Operación de los métodos conocidos como filtros.

Es importante tener en cuenta que no todos los métodos de filtrado pueden usarse con todos los métodos de aprendizaje, por eso en la lista siguiente se clasifican según el método en los que se puede utilizar.

Hay muchos métodos de filtrado, en la tabla siguiente se contemplan algunos.

Nombre	Clase de Filtro	Tarea Aplicable
Information gain	Univariada, información	clasificación
Gain ratio	Univariada, información	clasificación
Correlation-based feature selection (CFS)	Multivariada, dependencia	Clasificación, regresión
Fast correlation-based filter (FCBF)	Multivariada, información	clasificación
Multi-Cluster Feature Selection (MCFS)	Multivariada, Similitud	Clustering

Tabla 4. Métodos de filtrado. Fuente: elaboración propia.

Estos métodos pueden usar medidas univariadas para elegir cada uno de los rasgos o multivariadas.

Veamos un ejemplo práctico de selección de rasgos usando el módulo Scikit-Learn de Python.

Este módulo contiene un grupo de clases que permite de forma muy sencilla e intuitiva implementar selección de rasgos en sus etapas de preprocesamiento de

Tema 2. Análisis exploratorio de datos y preprocesamiento

datos.

Este módulo tiene un método muy simple de usar, que permite eliminar rasgos que tienen varianza cero o muy cerca de ella o por debajo de un determinado umbral.

Veamos:

```
import pandas as pd
from sklearn import datasets
from sklearn.feature_selection import VarianceThreshold
from sklearn.preprocessing import StandardScaler
#*****
iris = datasets.load_iris()
#*****
v_thr = VarianceThreshold(threshold=0.6)
data= v_thr.fit_transform(iris.data)
print(v_thr.get_support())
data.shape

[ True False  True False]
(150, 2)
```

Figura 12. Ejemplo. Fuente: elaboración propia.

Como se puede ver en este ejemplo, la clase que permite este tipo de filtrado es `VarianceThreshold`, y toma como parámetro un valor que sirve como umbral para el filtrado. El valor por defecto es cero.

También incluye una clase llamada `SelectKBest`, que permite usar métodos de filtrado basados en medidas univariadas, como chi cuadrado, veamos un ejemplo y luego damos algunos detalles.

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
import pandas as pd
from sklearn import datasets
import numpy as np
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
#*****
iris = datasets.load_iris()
#*****
iris_new = SelectKBest(chi2, k=2).fit_transform(iris.data, iris.target)
iris_new.shape

(150, 2)
```

Figura 13. Ejemplo. Fuente: elaboración propia.

En este ejemplo se escogen los dos mejores rasgos según la medida chi cuadrado.

Las medidas que se pueden usar con esta clase son:

- ▶ `f_classif`: basada en ANOVA F-value y usada en tareas de clasificación.
- ▶ `mutual_info_classif`: Usa Información mutua, variable objetivo discreta.
- ▶ `Chi2`: Chi-cuadrado, estadístico de rasgos no negativos, clasificación .
- ▶ `f_regression`: F-value entre label/feature para tareas de regresión.
- ▶ `mutual_info_regression`: Información Mutua para variable objetivo continua.
- ▶ `SelectPercentile`: Selecciona los rasgos basado en los percentiles de los scores más altos.
- ▶ `SelectFpr`: Selecciona los rasgos basado en la tasa de falsos positivos.
- ▶ `SelectFdr`: Selecciona los rasgos basado en false discovery rate.
- ▶ `SelectFwe`: Selecciona los rasgos basado en family-wise error rate.
- ▶ `GenericUnivariateSelect`: Selecciona los rasgos de modo configurable.
- ▶ Para el resto de los detalles del uso de los métodos de selección de rasgos en el

Tema 2. Análisis exploratorio de datos y preprocesamiento

módulo Scikit-Learn, ver su documentación.

Métodos de selección de rasgos del tipo Wrappers

Este tipo de método integra dentro del proceso de selección de rasgos un algoritmo de aprendizaje automático como una caja negra que le permite evaluar la calidad de los subconjuntos de rasgos. Ahora bien, el problema con este tipo de método es que puede llevar a sobreajuste (overfitting) o sesgos y es más lento que los métodos de filtrado.

Con vistas a evitar el sobreajuste, dentro de estos métodos hay que implementar también algún método de validación estadística (como validación cruzada que aún no hemos tocado) lo que los hace más complejos que los anteriores.

La idea en el uso de este tipo de método es escoger el subconjunto de rasgos que mejor calidad de clasificación arroje. Estos métodos requieren de dos etapas de trabajo, una donde se selecciona el subconjunto de rasgos optimo o al menos uno suboptimo y la otra en la cual se entrenan los métodos de aprendizaje con el conjunto de rasgos obtenidos.



Figura 14. Operación de los métodos conocidos como Wrappers.

Métodos de selección de rasgos del tipo embebidos

Este enfoque es parecido a los métodos wrapper en que los rasgos son seleccionados específicamente para un algoritmo de aprendizaje. Sin embargo en este tipo de métodos los rasgos son seleccionados el mismo proceso de aprendizaje.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Estos métodos integran el proceso de selección de rasgos en el proceso de entrenamiento y eso los hace más eficientes en algunos aspectos al no tener que dividir los datos en entrenamiento y validación. Además, pueden obtener soluciones más rápido al no tener que reentrenar para cada subconjunto de rasgos, como los métodos anteriores.

Este tipo de método no es nuevo y algoritmos como C4.5 tienen métodos de selección de rasgos incorporados.

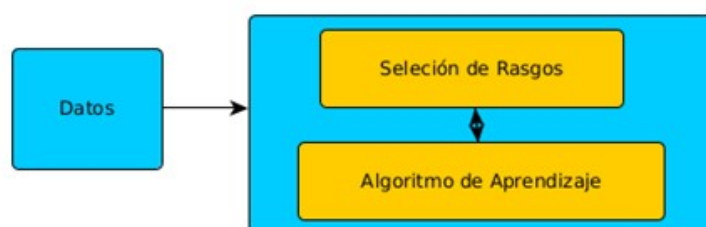


Figura 14. Operación de los métodos conocidos como embebidos.

Hasta aquí hemos hecho un breve recorrido por la clasificación más antigua y conocida del problema de Selección de rasgos; pero lo cierto es que la literatura es abrumadoramente grande y la cantidad de métodos y técnicas se extienden constantemente. Debido a esto es imposible que en este recorrido pudiéramos adentrarnos todo cuanto quisiéramos y dejamos de su parte la tarea de explorar los métodos más actuales.

Análisis de componentes principales (PCA)

Quizás el método de reducción de la dimensionalidad más conocido es el de análisis de componentes principales o PCA como se le conoce por sus siglas en inglés. Esto se debe en gran medida a que es uno de los métodos más antiguos, introducido en los trabajos de Pearson en 1901 y Hotelling en 1933.

Si vemos nuestro conjunto de datos como un grupo de vectores representados en un

Tema 2. Análisis exploratorio de datos y preprocesamiento

espacio n -dimensional, entonces la idea de PCA es encontrar un espacio p -dimensional donde $p < n$ y que se puedan representar la mayoría de los vectores del conjunto de datos perdiendo la menor cantidad de ellos y preservando su estructura.

Desde el punto de vista práctico el método PCA, toma un conjunto de datos de x observaciones y n variables y nos retorna un conjunto de x observaciones y p variables donde $p < n$. A las nuevas variables se les conoce como componentes principales y son una combinación lineal de las variables originales.

Es importante tener en cuenta que para la utilización de PCA los valores del dataset deben ser numéricos y antes de aplicarlo hay que normalizarlos.

Para poder captar la perspectiva práctica del uso de este método, veamos cómo se puede hacer con el módulo de aprendizaje automático Scikit-Learn.

Para el ejemplo que presentamos usamos el conjunto llamado Iris dataset. Este conjunto de datos contiene 150 ejemplares de flores de 3 categorías distintas, descritas por 4 variables numéricas. Las clases o categorías son: setosa, versicolor y virginica.

```
In [28]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D
from sklearn import datasets

# En esta sección se carga el dataset del conjunto de datasets internos
# de Scikit learn
iris_data = datasets.load_iris(as_frame=True)
iris_target = iris_data.target

# En esta se crea un objeto a partir de la clase PCA y como argumento
# toma la cantidad de componentes a los que queremos reducir nuestros
# dataset. Si no sabemos o no queremos establecer un número de componentes
# le pasamos como parámetro 0.95 que significa que queremos mantener el
# el 95% de la varianza de los datos
PCA_DR = PCA(n_components=3)
# Aquí se calculan los componentes, es decir esta es la aplicación de PCA
PCA_DR.fit(iris_data.data)
# Aquí se transforma el dataset a partir de los calculos anteriores
iris_tranf = PCA_DR.transform(iris_data.data)

fig = plt.figure(1, figsize=(8, 6))
ax = Axes3D(fig, elev=150, azim=110)
ax.scatter(iris_tranf[:, 0], iris_tranf[:, 1], iris_tranf[:, 2], c=iris_target, cmap=plt.cm.Set1, edgecolor='k', s=40)
ax.set_title("Tres componentes principales arrojados por PCA para el dataset iris")
ax.set_xlabel("1er eigenvector")
ax.waxis.set_ticklabels([])
ax.set_ylabel("2do eigenvector")
ax.waxis.set_ticklabels([])
ax.set_zlabel("3er eigenvector")
ax.waxis.set_ticklabels([])
plt.show()
```

Figura 14 Ejemplo de aplicación de un PCA al conjunto de datos Iris.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Al aplicar el PCA a este conjunto, como en el ejemplo dado en la fig 13. reducimos el dataset a 3 variables o componentes principales al pasar el argumento `n_components=3` al constructor de la clase PCA. Si por el contrario pasáramos el argumento `n_components=0.95` la implementación de este PCA nos retornaría los componentes principales que agrupan el 95% de la varianza de nuestros datos. Y en este caso, es decir con el dataset iris serían dos componentes. Esto significa que hemos reducido los datos originales con 4 rasgos a 3 en el caso que mostramos en el ejemplo con el fin de graficar los resultados. Ver Figura 15.

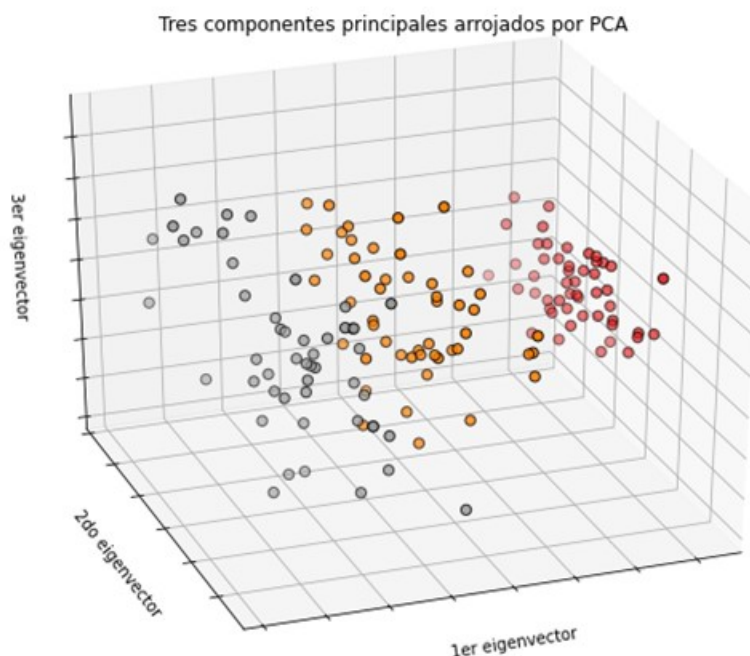


Figura 14. Gráfica de aplicación de un PCA al dataset Iris, para tres componentes.

Este módulo contiene varias implementaciones de PCA, para revisar más detalles de cómo aplicar este método, revisar: Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow. De Aurélien Géron.

Análisis de factor

Tema 2. Análisis exploratorio de datos y preprocesamiento

Al igual que el análisis de componentes principales o PCA, esta técnica de reducción de la dimensionalidad conduce a la deducción de un nuevo conjunto de variables que prácticamente describe la conducta de los datos originales. Aunque difiere en que este no trata de buscar transformaciones de los atributos dados, en vez, trata de descubrir factores ocultos en las variables originales.

La idea básica de esta técnica es intentar encontrar un grupo de factores ocultos, llamados factores latentes, de forma tal que los atributos originales puedan ser recuperados por medio de un conjunto de transformaciones lineales sobre estos factores.

La idea aquí es caracterizar la dependencia entre variables por medio de un pequeño número de factores.

¿Qué es un factor? Es una variable latente que describe la asociación entre el número de variables observadas. Cada factor explica una cierta cantidad de varianza en las variables observadas. Aquellos factores con la menor cantidad de varianza son eliminados.

¿Qué son los Loadings? Es una matriz que muestra las relaciones entre cada una de las variables y su factor subyacente. ¿Cómo se elige el número de factores? Generalmente un eigenvalue mayor que 1 es un buen criterio para seleccionar un factor. Otro método es a partir de la gráfica llamada scree, determinado donde la curva hace un codo.

No veremos aquí la forma matemática de llegar a los resultados; pero si veremos como en el caso de PCA, un ejemplo desarrollado usando scikit-learn.

Para iniciar con el ejemplo primero que todo debemos dejar claro las asunciones básicas que se requieren para la aplicación de esta técnica:

- Esta técnica asume que los datos son numéricos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Asume también que no tienen outliers.
- ▶ Que el tamaño de la muestra es mayor que la cantidad de factores.
- ▶ Y que no debe haber homocedasticidad entre las variables.
- ▶ Otro de los elementos a tener en cuenta antes de ejecutar el análisis de factores son las pruebas de adecuación de los datasets. Esto implica preguntarse si podemos encontrar factores en el dataset que tenemos a mano.

Básicamente hay dos test que se usan para esto:

- ▶ El test de Bartlett.
- ▶ El test de Kaiser-Meyer-Olkin.
- ▶ El test de Bartlett, es una prueba estadística que permite verificar si un conjunto de muestras tienen la misma varianza. Es una prueba paramétrica y la hipótesis nula H_0 es que todas las muestras tienen la misma varianza.

Por tanto, si el valor de p-value retornado por la prueba es mayor que un valor de significación determinado, por ejemplo 0.05 o 5% entonces podemos decir que la hipótesis nula es válida. Sin embargo, a los efectos del uso de esta prueba en el análisis de factor, lo que nos dice es que si todas las varianzas son iguales entonces no podemos aplicar el procedimiento de análisis de factor a estos datos.

Por tanto, si el resultado de la aplicación del test de esfericidad de Bartlett arroja un p-value > 0.05 , sus datos no son adecuados para aplicarle el análisis factorial.

El test de Kaiser-Meyer-Olkin, por su parte, considera que sus datos son adecuados para aplicar el análisis si el resultado es superior a 0.6. Los valores retornados por esta prueba están entre 0 y 1.

Como ejemplo, mostraremos uno desarrollado con el paquete Scikit-learn y el otro con un paquete llamado factor_analyzer.

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
import pandas as pd
import numpy as np
from sklearn import decomposition
from sklearn import datasets
from sklearn.decomposition import FactorAnalysis
from sklearn.preprocessing import StandardScaler
from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity
from factor_analyzer.factor_analyzer import calculate_kmo
#*****
# Cargando el dataset
iris = datasets.load_iris(as_frame=True)
#*****
#Estandarización de las variables
scaler = StandardScaler()
iris_scalado = scaler.fit_transform(iris.data)
#*****
chi_square_value,p_value=calculate_bartlett_sphericity(iris_scalado)
kmo_all,kmo_model=calculate_kmo(iris_scalado)
if p_value<0.05 and kmo_model>0.5:
    fa=FactorAnalysis(n_components=2)
    fa.fit(iris_scalado)
    iris_Transformed = fa.transform(iris_scalado)
    fig = plt.figure(figsize=(20,10))
    fig.suptitle("Análisis de Factor con el módulo Scikit-Learn", fontsize=14)
    ax = fig.add_subplot(2,3,1)
    ax.scatter(iris_Transformed[:,0], iris_Transformed[:,1], c=iris.target)
else:
    print("Sus datos no son adecuados para este tipo de técnica")
    print("Pues p_value es igual a "+str(p_value))
    print("y el valor de kmo es "+str(kmo_model))
```

Figura 15. Análisis de factores con Scikit-Learn. Fuente: elaboración propia.

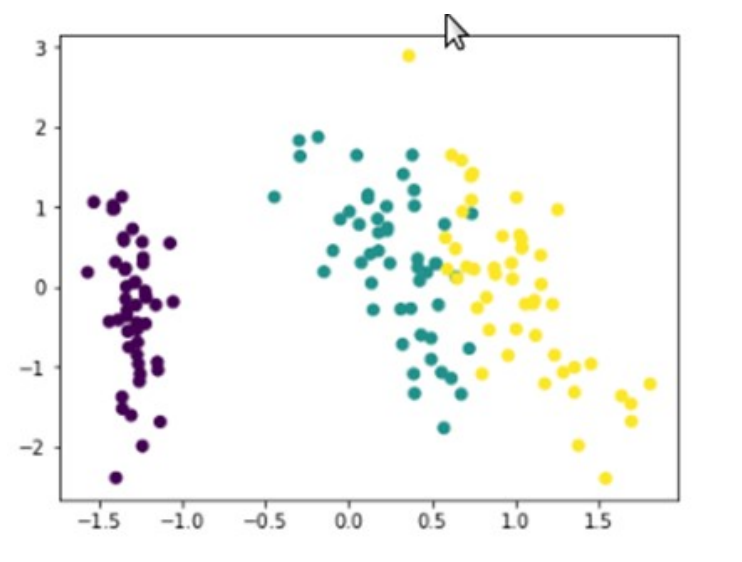


Figura 16. Resultado del Análisis de factores con Scikit-Learn. Fuente: elaboración propia.

En el ejemplo mostrado en la Fig. 15, hemos escogido representar un conjunto de datos con 4 rasgos por medio de uno que sola mente tiene 2 y contiene la mayoría

Tema 2. Análisis exploratorio de datos y preprocesamiento

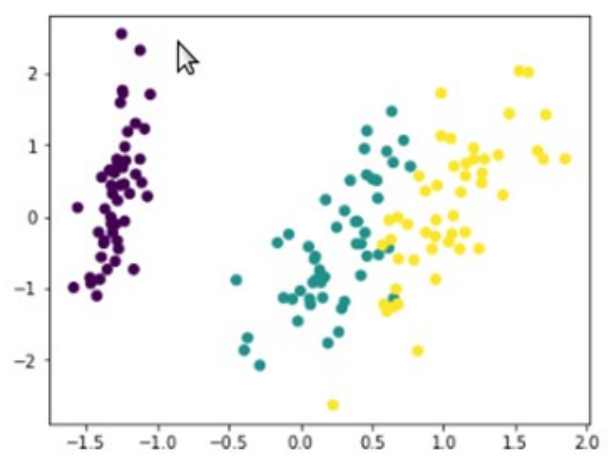
de la información del conjunto original.

No hablamos aquí de los tipos de rotación y su significado, pues a los efectos de la reducción de dimensionalidad es irrelevante.

Un método manual para escoger la cantidad de factores, es inspeccionar los eigenvalues y escoger los factores para los cuales estos son mayores que 1.

```
import pandas as pd
import numpy as np
from sklearn import datasets
from factor_analyzer import FactorAnalyzer
from sklearn.preprocessing import StandardScaler
from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity
from factor_analyzer.factor_analyzer import calculate_kmo
#*****
# Cargando el dataset
iris = datasets.load_iris(as_frame=True)
#*****
#Estandarización de las variables
scaler = StandardScaler()
iris_scalado = scaler.fit_transform(iris.data)
#*****
chi_square_value,p_value=calculate_bartlett_sphericity(iris_scalado)
kmo_all,kmo_model=calculate_kmo(iris_scalado)
if p_value<0.05 and kmo_model>0.5:
    fa = FactorAnalyzer(rotation=None)
    fa.fit(iris_scalado)
    iris_Transformed = fa.transform(iris_scalado)
    fig = plt.figure(figsize=(20,10))
    fig.suptitle("Análisis de Factor con el módulo factor_analyzer", fontsize=14)
    ax = fig.add_subplot(2,3,1)
    ax.scatter(iris_Transformed[:,0], iris_Transformed[:,1], c=iris.target)
else:
    print("Sus datos no son adecuados para este tipo de técnica")
    print("Pues p_value es igual a "+str(p_value))
    print("y el valor de kmo es "+str(kmo_model))
```

Figura 17. Análisis de Factores con factor_analyzer. Fuente: elaboración propia.



Tema 2. Análisis exploratorio de datos y preprocesamiento

Figura 18. Resultado del Análisis de Factores con factor_analyzer. Fuente: elaboración propia.

Locally Linear Embedding. (LLE)

A falta de un buen término para traducir el nombre de este método de reducción de la dimensionalidad, hemos preferido no hacerlo y referirnos a él como LLE.

Este método, lo traemos como representante de un tipo de método de reducción de la dimensionalidad en los que, a diferencia de PCA, pueden lidiar con relaciones no lineales entre las variables de un conjunto de datos. Estos métodos se engloban bajo el nombre de Manifold Learning.

Un Manifold, y lo dejamos sin traducción, es un objeto o estructura matemática, tomado de la topología. No pretendemos dar definiciones matemáticamente correctas aquí, sino desarrollar la intuición detrás de estos métodos y especialmente LLE.

Intuitivamente podemos imaginar un manifold como un objeto o figura representada en $\mathbb{R}^n$ en el cual es embebido en un espacio $\mathbb{R}^k$ donde $k > n$.

La base del uso de estos métodos en reducción de la dimensionalidad es la llamada “Hipótesis de Manifold” la cual establece que los datos altamente dimensionales de la vida real radican en Manifolds de más baja dimensión embebidos en dichos espacios.

Esto nos lleva a plantear este problema, de manera muy informal, como la búsqueda de un Manifold de más baja dimensión que contiene la información de nuestros datos, en un espacio de dimensiones mayores en donde está representado.

Dejamos el estudio de la mecánica de trabajo del algoritmo para que sea estudiada a partir de otras fuentes y veremos de forma práctica como se usa a partir de un ejemplo escrito en Python.

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import decomposition
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import LocallyLinearEmbedding
#*****
# Cargando el dataset
n_points = 1000
X, color = datasets.make_swiss_roll(n_points, random_state=0)
#*****
fig = plt.figure(figsize=(20,10))
fig.suptitle("LLE del swiss roll con 1000 puntos, 15 vecinos", fontsize=14)
ax = fig.add_subplot(231, projection='3d')
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=color, cmap=plt.cm.Spectral)
ax.view_init(4, -72)
#*****
lle = LocallyLinearEmbedding(n_neighbors=15, n_components=2, method='modified')
X_transformed = lle.fit_transform(X)
ax = fig.add_subplot(2, 3, 2)
ax.scatter(X_transformed[:, 0], X_transformed[:, 1], c=color, cmap=plt.cm.Spectral)
```

Figura 19. Ejemplo de python que aplica LLE al dataset llamado swiss roll. (Locally Linear Embedding).

Fuente: elaboración propia.

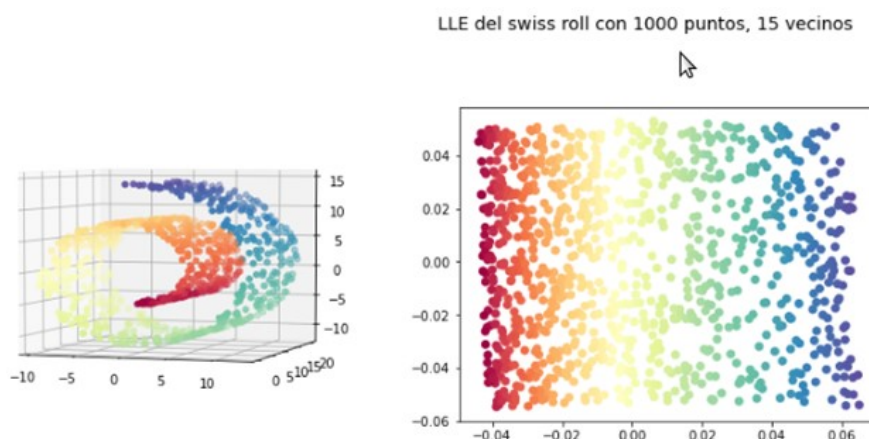


Figura 20. Resultado de la aplicación del método LLE al dataset llamado swiss roll. (Locally Linear Embedding). Fuente: elaboración propia.

¿Qué es lo que podemos ver del resultado de la aplicación de LLE al dataset swiss roll? Claramente, el dataset es un manifold no lineal de dos dimensiones embebido en un espacio de tres. Al aplicar el método, lo que estamos tratando de hacer es reducir la dimensionalidad del dataset sin perder la estructura de los datos. La gráfica de la izquierda muestra el dataset original dado en un espacio tridimensional y la gráfica de la derecha muestra el mismo dataset después de aplicar el método LLE en

Tema 2. Análisis exploratorio de datos y preprocesamiento

dos dimensiones. Es importante que nos demos cuenta que la estructura de los datos es la misma aún representada en 2D.

Dentro de esta clase de métodos de reducción de la dimensionalidad y que se llama Manifold Learning también hay otros métodos, tales como: Isomap, MDS y t-SNE que están implementados en el módulo Scikit-Learn.

Hasta este punto hemos tratado lo concerniente a las técnicas de reducción de la dimensionalidad de un conjunto de datos y comenzaremos a revisar algunas técnicas de reducción de la numerosidad como Selección de casos o Instancias entre otras.

Selección de casos

En un conjunto de datos, puede haber casos que, o son redundantes o ruidosos o simplemente no aportan información para la tarea a mano. Debido a esto, como en el caso de selección de rasgos, la selección de casos permite disminuir la numerosidad del conjunto mejorando así el rendimiento de los algoritmos de aprendizaje automático.

Por tanto, la tarea de Selección de Instancias o casos se puede definir a groso modo como: Dado un conjunto de entrenamiento T , el objetivo de la selección de instancias, es obtener un subconjunto $S \subset T$ tal que S no contenga instancias redundantes o superfluas y que $Acc(S) \approx Acc(T)$ Donde $Acc(X)$ denota la Accuracy obtenida, usando a X como conjunto de entrenamiento.

De manera general todos los métodos de selección de casos (Selección de Prototipos) tienen características comunes por las que se les puede ir clasificando. De manera general se toman como criterios para hacerlo, los siguientes: Dirección de búsqueda, la estrategia de selección y la evaluación de la búsqueda.

Si tomamos en cuenta la forma en que comienza la evaluación de las instancias, entonces podemos clasificarlos en Incrementales o Decrementales, en Batch (por lotes), mezcladas o Prefijadas.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Clasificación de los métodos de Selección de casos, según la dirección de búsqueda:

- ▶ Incrementales.
- ▶ Decrementales.
- ▶ Batch.
- ▶ Mezcladas.
- ▶ Prefijadas.

Métodos Incrementales: Inician con el subconjunto S vacío ($S=\emptyset$) y van incorporando instancias a medida que operan según un criterio predefinido.

Métodos decrementales: comienzan con el subconjunto S ($S=T$) con todas las instancias del conjunto T y las van eliminando a medida que operan según el criterio preseleccionado por el método en particular.

Batch: La idea con este tipo de métodos es que marcan a cada instancia como elegible o no según algún criterio antes de eliminarlas y luego se eliminan de una vez.

Mezcladas: En este tipo de métodos el procesamiento se inicia con un subconjunto preseleccionado o aleatoriamente o por medio incremental o decremental e interativamente se le van añadiendo o eliminando instancias que satisfagan un determinado criterio.

Prefijadas: Este tipo de método constituye una subfamilia de los de direcciones mezcladas, pero con la diferencia que los números de operaciones de eliminación o adición son las mismas. Es decir, el número de instancias o prototipos al final es decidido al inicio de la fase y nunca es cambiado.

Muchos de los métodos de selección de casos se basan en la clasificación de las

Tema 2. Análisis exploratorio de datos y preprocesamiento

instancias a partir de su ubicación dentro de las clases a las que pertenecen. Hay casos o instancias que se ubican en los bordes de las clases y otras más al centro de ellas. A las primeras se les llama instancias de frontera y las segundas instancias interiores.

Por tanto, se dice que una instancia $i_j \in T$ es de frontera de la clase C_i si $i_j \in C_i$ y es vecino más cercano de una instancia $i_k \notin C_i$, de otra forma se le conoce como instancia interior de la clase C_i .

Basados en este tipo de clasificación de las instancias los métodos de selección de casos se pueden dividir en (estrategia de selección):

- ▶ Condensación.
- ▶ Edición.
- ▶ Híbridos.

Condensación: La idea básica de este tipo de métodos es retener las instancias de frontera, pues éstas afectan mucho más la calidad de la clasificación que las interiores. Sin embargo, se ha demostrado que, aunque mantienen la calidad de la clasificación en el conjunto de entrenamiento, la generalización se ve afectada. Estos métodos tienen una gran capacidad de reducción, pues la cantidad de instancias de frontera es mucho menor que las interiores.

Edición: En este tipo de métodos, al contrario de los de condensación, lo que se busca es mantener las interiores y descargar las de frontera. Eliminan también las que son ruidosas o no están de acuerdo a sus vecinos.

Híbridos: Estos métodos buscan encontrar el subconjunto S menor que maximice la generalización, es decir la calidad de la clasificación sobre un conjunto de prueba. Esto les permite eliminar un tipo u otra de instancia basado en dos estrategias determinadas.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Los métodos de selección de casos, también se pueden clasificar, basándonos en la manera en cómo se decide si se elimina o se mantiene la instancia, en:

- ▶ Métodos Filtros.
- ▶ Métodos envoltorios. (Wrappers)

Filtros: En este tipo de método la decisión de de mantener o eliminar una instancia se hace por medio de una Heurística o determinada regla.

Envoltorios (Wrapper): En este tipo de métodos, la decisión se toma basado en un clasificador, usualmente KNN.

Otro de los elementos a tener en cuenta para la selección de los métodos de selección de instancias es la complejidad computacional, que, aunque no la tocaremos, sí que es uno de los elementos importantes.

Aunque se han propuesto cientos de algoritmos en la literatura, todos los días se proponen nuevos y es un área de investigación de constante actividad. Con esa gran cantidad de métodos es imposible que podamos abarcar muchos de ellos. En la siguiente figura se muestran muchos de ellos clasificados por los criterios anteriores.

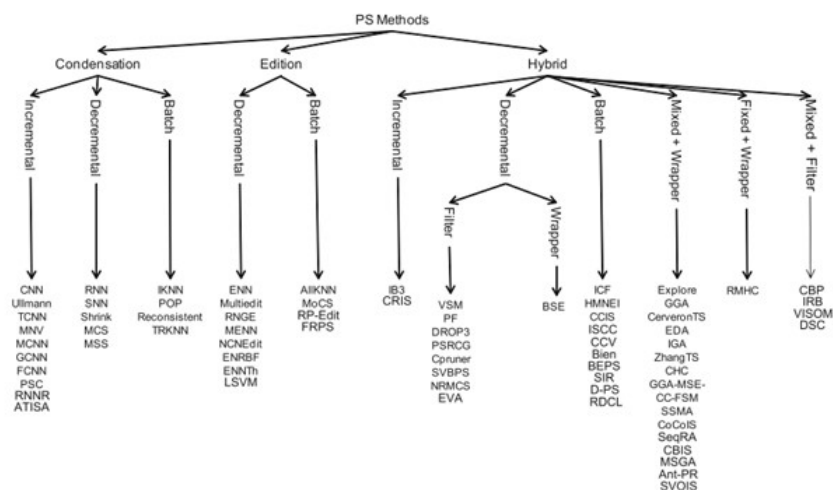


Figura 21. Clasificación de los métodos de selección de casos o instancias. (Data Preprocessing in Data

Tema 2. Análisis exploratorio de datos y preprocesamiento

Mining). Fuente: elaboración propia.

Con la imposibilidad de describir la mayoría de estos métodos, elegimos algunos que son representativos, tales como:

- ▶ CNN. (Condensed Nearest Neighbours)
- ▶ ENN. (Edited Nearest Neighbour)
- ▶ DROP. (Decremental Reduction Optimization)
- ▶ ICF. (Iterative Case Filtering)

Condensed Nearest Neighbours (CNN): Este método se considera la primera propuesta formal del uso de KNN para el problema de selección de casos. Este método se basa en el concepto de consistencia de un conjunto.

Definición de consistencia:

Se dice que un conjunto S es consistente con otro X , si:

Siendo X un conjunto de entrenamiento y S un subconjunto de X , entonces un clasificador KNN, usando S como conjunto de entrenamiento, puede clasificar correctamente todas las instancias en X .

Siguiendo esta definición entonces S , debe ser consistente con X y por tanto, idealmente, más pequeño.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Algorithm 1: Condensed Nearest Neighbour (CNN)

Input: A training set $X = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$

Output: The set of selected instances $S \subseteq X$

```
1  $S = \{\mathbf{x}_1\}$ 
2 foreach  $\mathbf{x} \in X$  do
3   if  $\mathbf{x}$  is misclassified using  $S$  then
4     Add  $\mathbf{x}$  to  $S$ 
5   Restart
6 return  $S$ 
```

Figura 20. Método Condensed Nearest Neighbours. (CNN). Fuente: elaboración propia.

Edited Nearest Neighbour. (ENN): Este es un método decremental, es decir, comienza con el conjunto S igual al conjunto de entrenamiento X y cada instancia es eliminada si no es bien clasificada por la regla KNN. El número de vecinos más cercanos es un parámetro del algoritmo y el artículo original se estable en 3.

ENN, elimina los casos ruidosos y las instancias de frontera. Las instancias interiores no son afectadas por el proceso de edición del método.

Algorithm 2: Edited Nearest Neighbours (ENN)

Input: A training set $X = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, the number of nearest neighbours k

Output: The set of selected instances $S \subseteq X$

```
1  $S = X$ 
2 foreach  $\mathbf{x} \in S$  do
3   if  $\mathbf{x}$  is misclassified using its  $k$  nearest neighbours then
4     Remove  $\mathbf{x}$  to  $S$ 
5 return  $S$ 
```

Figura 23. Edited Nearest Neighbour. (ENN). Fuente: elaboración propia.

Decremental Reduction Optimization Procedure. (DROP): Esta familia de algoritmos se propusieron en el 2000 y están considerados entre los mejores métodos para selección de instancias para la clasificación. El criterio de eliminación

Tema 2. Análisis exploratorio de datos y preprocesamiento

de las instancias se basa en dos conceptos “asociados” y “vecinos cercanos”. La relación de “asociado” es lo opuesto a vecino cercano. Por ejemplo, si una instancia p , tiene a q como vecino cercano, entonces, q tiene como asociado a p . Ver la fig. 20.

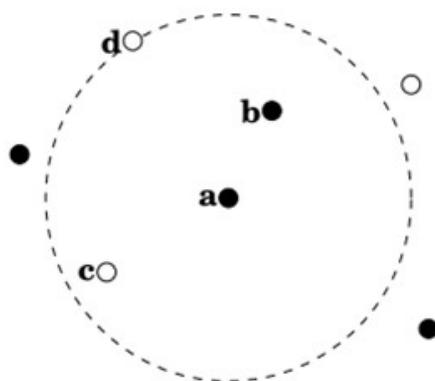


Figura 23. En knn con $k=3$. La instancia a tiene como vecinos más cercanos a b, c, d y ellos tienen como asociado a. Fuente: elaboración propia.

Algorithm 3: Decremental Reduction Optimization Procedure 3 (DROP3)

Input: A training set $X = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, the number of nearest neighbours k

Output: The set of selected instances $S \subseteq X$

```

1 Noise filtering: remove any instance in  $X$  misclassified by its  $k$  neighbours
2  $S = X$ 
3 Sort instances in  $S$  by the distance to their nearest enemy
4 foreach instance  $\mathbf{x} \in S$  do
5   Find  $\mathbf{x}.N_{1\dots k+1}$ , the  $k+1$  nearest neighbours of  $\mathbf{x}$  in  $S$ 
6   Add  $\mathbf{x}$  to each of its neighbour's list of associates
7 foreach instance  $\mathbf{x} \in S$  do
8   Let with = # of associates of  $\mathbf{x}$  correctly classified by  $\mathbf{x}$  as a neighbour
9   Let without = # of associates of  $\mathbf{x}$  correctly classified without  $\mathbf{x}$ 
10  if without  $\geq$  with then
11    Remove  $\mathbf{x}$  from  $S$ 
12    foreach associate  $\mathbf{a}$  of  $\mathbf{x}$  do
13      Remove  $\mathbf{x}$  from  $\mathbf{a}$ 's list of nearest neighbour
14      Find a new nearest neighbour for  $\mathbf{a}$ 
15      Add  $\mathbf{a}$  to its new neighbour's list of associates
16 return  $S$ 
```

Figura 24. Decremental Reduction Optimization Procedure. (DROP). Fuente: elaboración propia.

Iterative Case Filtering. (ICF): Este método, usa dos conceptos muy relacionados con conceptos de vecindad y asociado de DROP y son: coverage y reachable. La

Tema 2. Análisis exploratorio de datos y preprocesamiento

cobertura o coverage, de una instancia es su vecindad; pero en vez de considerar un número fijo de vecinos, k , todas las instancias más cercanas que su enemigo más cercano está dentro de su conjunto de cobertura. El conjunto alcanzable o accesible (reachable) de una instancia es el conjunto de todas las instancias para las cuales, esa instancia particular, está dentro de su conjunto de cobertura.

La regla de eliminación es como sigue: Si la cardinalidad del conjunto accesible o alcanzable es mayor que su conjunto de cobertura la instancia es eliminada del dataset.

Algorithm 4: Iterative Case Filtering (ICF)

Input: A training set $X = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, the number of nearest neighbours k

Output: The set of selected instances $S \subseteq X$

```
1  $S = X$ 
2 Noise filtering: remove any instance in  $S$  misclassified by its  $k$  neighbours
3 repeat
4   foreach instance  $\mathbf{x} \in S$  do
5     Compute coverage( $\mathbf{x}$ )
6     Compute reachable( $\mathbf{x}$ )
7   progress = False
8   foreach instance  $\mathbf{x} \in S$  do
9     if  $|\text{reachable}(\mathbf{x})| > |\text{coverage}(\mathbf{x})|$  then
10      Flag  $\mathbf{x}$  for removal
11      progress = True
12   foreach instance  $\mathbf{x} \in S$  do
13     if  $\mathbf{x}$  flagged for removal then
14       $S = S - \{\mathbf{x}\}$ 
15 until not progress
16 return  $S$ 
```

Figura 25. Iterative Case Filtering. (ICF). Fuente: elaboración propia.

Discretización y compartimiento (binning)

La discretización es una de las técnicas de reducción de datos básicas. El proceso de discretización transforma datos cuantitativos en cualitativos, es decir, convierte atributos numéricos en atributos discretos o nominales con un número finito de intervalos, obteniendo una partición del dominio continuo. Luego se establece una asociación entre en cada intervalo y un valor discreto.

Tema 2. Análisis exploratorio de datos y preprocesamiento

En la práctica, la discretización puede verse como un método de reducción de datos, debido a que mapea los datos correspondientes a un gran espectro de valores numéricos a un subconjunto reducido de valores discretos.

La discretización podría definirse de la siguiente forma: Supongamos que tenemos un dataset con N instancias y A atributos numéricos, entonces el algoritmo de discretización convertiría los atributos numéricos en un conjunto de m intervalos discretos $D = (d_0, d_1), (d_1, d_2), \dots, (d_{m-1}, d_m)$ Donde d_0 es el valor mínimo del atributo y d_m el máximo. Mientras que $d_i < d_{i+1}$ para todo $i=0,1,\dots,m-1$. A D se le llama esquema de discretización del atributo A y $P = (d_1, d_2, \dots, d_{m-1})$ el conjunto de puntos de corte.

Las motivaciones para discretizar pueden ser variadas, entre ellas que muchos algoritmos de aprendizaje automatizado trabajan solamente con rasgos o atributos nominales u otros solo con discretos. También, se sabe que hay algoritmos de aprendizaje cuyo rendimiento se ve afectado por el uso de variables que tienen distribuciones de probabilidad no estándar. Uno de los enfoques de solución sería convertir dichas variables a ordinales.

A este proceso de discretización se le conoce también como binning (Compartimentación, por darle algún nombre en español) y de lo que trata es de trasladar los valores de una variable cuantitativa a dos o más compartimentos o categorías. Los valores de la variable se agrupan en compartimentos o intervalos discretos, llamados bin y a cada uno se le asigna un único valor entero tal que se pueda mantener la relación de orden entre ellos.

Las estrategias que se pueden usar para acometerlo se pueden clasificar en:

- ▶ Uniforme: Cada bin tiene el mismo ancho.
- ▶ Quantile: Cada bin tiene el mismo número de valores.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- Clustered: Se identifican los grupos o clusters y los ejemplos se asignan a ellos.

Como en casi todos los ejemplos que presentamos a lo largo de esta lección, el módulo Scikit-Learn de Python viene a salvarnos.

Este módulo tiene una clase llamada KbinsDiscretizer, que permite implementar el proceso de discretización. Veremos un ejemplo; pero no profundizaremos en su uso.

```
from numpy.random import randn
from sklearn.preprocessing import KBinsDiscretizer
from matplotlib import pyplot
# genera una muestra de datos con distribución gaussian
data = randn(1000)
# Crea un histograma de los datos generados usa 25 bin
pyplot.hist(data, bins=25)
pyplot.show()
data = data.reshape((len(data),1))
# Crea el discretizador
kbins = KBinsDiscretizer(n_bins=10, encode='ordinal', strategy='uniform')
# Discretiza los datos usando 10 bin y una estrategia uniforme, retorna un ordinal
data_trans = kbins.fit_transform(data)
# Crea un histograma de los datos discretizados usa 10 bin
pyplot.hist(data_trans, bins=10)
pyplot.show()
```

Figura 26. Ejemplo de uso de un discretizador del módulo Scikit-learn. (KbinsDiscretizer). Fuente: elaboración propia.

Los histogramas generados por este ejemplo se muestran en las siguientes imágenes.

Tema 2. Análisis exploratorio de datos y preprocesamiento

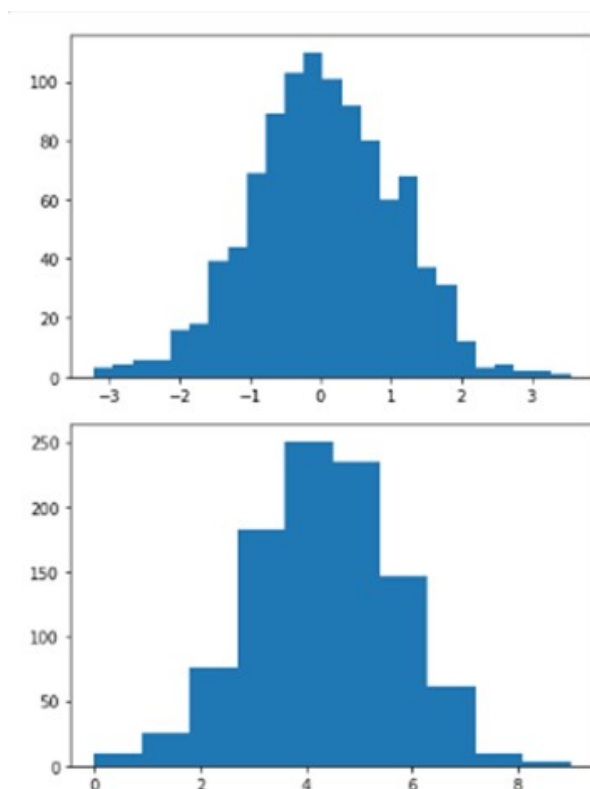


Figura 27. Resultados generados por el ejemplo de uso de un discretizador del módulo Scikit-learn. (KbinsDiscretizer). Fuente: elaboración propia.

Para precisar las opciones que brinda este discretizador, es necesario consultar la documentación del módulo Scikit-learn.

Hasta aquí hemos visto lo esencial de esta técnica y con eso concluimos esta lección.

Conclusiones

Hasta aquí hemos tocado aquellos conceptos y técnicas que componen una parte de la etapa de preprocesamiento de los datos y que antecede a la de entrenamiento de los modelos aprendizaje que escojamos para entrenar.

Para la próxima lección hemos dejado aquellos conceptos como balanceo de datos, separación en conjunto de entrenamiento y de prueba y lo relacionado con el análisis

Tema 2. Análisis exploratorio de datos y preprocesamiento

exploratorio de datos que de una forma u otra va descubriendo los problemas que se dan en los conjuntos de datos y para resolverlos entonces usamos las técnicas o métodos que hemos visto en mayor o menor profundidad en esta lección.

Toda la documentación usada está disponible en Internet y los ejemplos de código desarrollados en Google Colab y Kaggle. El campo de aprendizaje automático es basto y complejo, no importa que de momento no entiendan todos los conceptos, en la medida que busque información sobre ellos y lea va ir viendo como encajan unos con otros.

Todo conocimiento que necesita se construye como una casa, piedra sobre piedra, concepto sobre concepto.

Tema 2. Análisis exploratorio de datos y preprocesamiento

2.3. Análisis exploratorio de datos y preprocesamiento II

Objetivos

El objetivo de esta lección es continuar con todo lo referente a la etapa de preprocesamiento de los datos y los conceptos conexos y que forman la base necesaria para la comprensión del proceso de aprendizaje automático. En esta parte incluimos conceptos como «separabilidad de las clases de problemas de clasificación» o la división del conjunto de datos en conjunto de entrenamiento o conjunto de prueba, que son cruciales a la hora de entender el resto de los conceptos. Incluimos también, en esta lección, los conceptos de Análisis Exploratorio de Datos y aquellos que proceden de la estadística descriptiva como medidas de tendencia central y varianza y covarianza. También incluimos las pruebas de normalidad para las variables o rasgos.

Al finalizar esta lección, estará equipado con los conceptos que son necesarios para entender y usar de manera práctica el aprendizaje automático.

Introducción

En la lección anterior vimos los problemas que se presentan en la recolección e integración de los datos y el tratamiento de los problemas que nos podemos encontrar. Vimos, por ejemplo, la detección de valores faltantes y los métodos de lidiar con ellos, así como la detección de datos extremos o anómalos que se le llaman Outliers. Tratamos, también en la lección anterior, técnicas para la selección de subconjuntos óptimos o no, con el fin de aliviar la carga que significa el anejo de cientos de rasgos, llamadas técnicas de selección de rasgos. Revisamos, las técnicas de selección de casos y que disminuyen o condensan un conjunto de datos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Sin embargo, hay un problema que aún no se ha tocado y que puede afectar la calidad de la clasificación en modelos y es el correspondiente al desbalance entre la cantidad de casos pertenecientes a cada clase en un conjunto de datos.

Técnicas de tratamiento de conjuntos desbalanceados

Es bien conocido que los algoritmos de aprendizaje automáticos favorecen de alguna manera a las clases más pobladas, afectando la calidad de la clasificación y por tanto la habilidad de generalización de estos. Este problema de desbalance entre el número de casos en un dataset es muy común en la vida real y aparece, por ejemplo, en los conjuntos de datos en el área de salud, donde se pueden recolectar muchos casos de personas sanas y menos de enfermos. Pueden aparecer, también, en detección de fraudes, pues normalmente se recolectan muchas muestras de transacciones no fraudulentas y mucho menos de fraudulentas.

Con el fin de solucionar el efecto de este tipo de problemas en conjuntos de datos no balanceados se han desarrollado, a grandes rasgos dos grupos de técnicas:

- ▶ Técnicas a nivel de datos.
- ▶ Técnicas a nivel de algoritmos.

Las técnicas a nivel de datos pueden dividirse en:

- ▶ Muestreo de datos (*data sampling*).
- ▶ Selección de rasgos (no veremos ninguna aquí).

Las técnicas de muestreo de datos (*data sampling*), se dividen en dos grupos:

- ▶ Sobremuestreo (*Oversampling* e incluye SMOTE).
- ▶ Bajomuestreo (*undersampling*).

Tema 2. Análisis exploratorio de datos y preprocesamiento

Por otra parte, las técnicas a nivel de algoritmos se pueden dividir en:

- ▶ Costo-sensibles (*cost-sensitive*).
- ▶ Híbridas/ensamble.

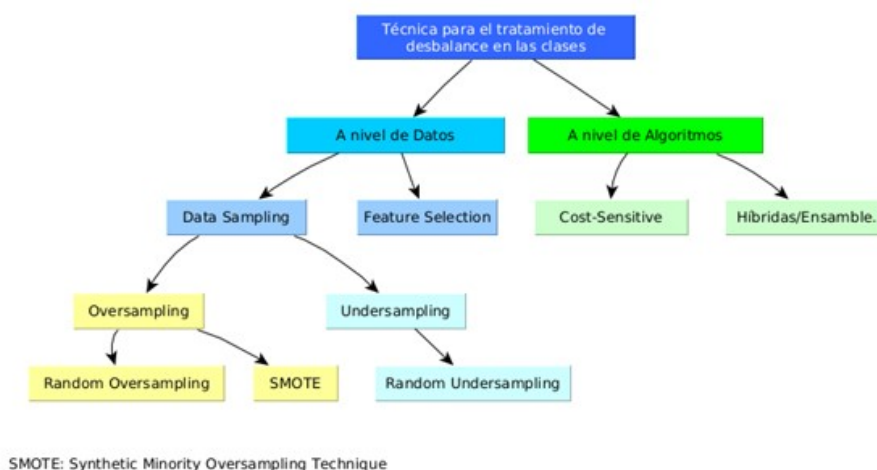


Figura 1. Clasificación de las técnicas de tratamiento de desbalance en las clases. Fuente: elaboración propia.

Las técnicas más usadas y con mejores rendimientos son las Data Sampling y por tanto son las que tocaremos aquí.

Básicamente, las técnicas que se engloban bajo este nombre de Data sampling, se basan en añadir o eliminar instancias de las clases minoritarias o mayoritarias ya sea de forma aleatoria o por algún método inteligente. Se le llama clase mayoritaria a aquella que tiene un número mayor de casos y minoritaria a las que están menos pobladas.

Random Undersampling: la idea detrás de este tipo de método es seleccionar ejemplos, de manera aleatoria, de la clase mayoritaria y eliminarlos del conjunto de entrenamiento. Este proceso puede repetirse de manera continuada hasta que las clases alcancen el balance. Es decir, las clases contengan el mismo número de

Tema 2. Análisis exploratorio de datos y preprocesamiento

casos.

El problema con este tipo de método es que, al eliminar casos, podemos perder información valiosa o aún peor, crítica para la construcción de las fronteras de decisión entre clases.

Random Oversampling: la idea detrás de este método, es añadir o duplicar de manera aleatoria los casos de la clase minoritaria en el conjunto de entrenamiento.

Cuando se usa este método se incurre en el peligro de provocar un sobreajuste (overfitting) en nuestro modelo ya que este tipo de técnica hace copias exactas de los ejemplos de la clase minoritaria. Así que hay que tomar precaución a la hora de utilizarlo.

Synthetic Minority Oversampling Technique(SMOTE): este método es el estándar por defecto de este tipo de técnica. A diferencia del método original de Oversampling, que duplica los casos de la clase minoritaria, este sintetiza ejemplos y los añade al conjunto de entrenamiento por medio de interpolación. Eliminando así el peligro de que aparezca overfitting.

Con vistas a ejemplificar el uso de cada una de estas técnicas, apelaremos a un módulo de Python que implementa la mayoría de estas.

El dataset que usaremos para explicar cada una de estas técnicas es el Pima indian diabetic dataset. Este dataset tiene 768 casos y 9 rasgos. De los 768 casos, 268 son diabéticos y los no diabéticos 500. Lo que hace a este un dataset con desbalance moderado en las clases.

La librería que usaremos para los ejemplos se llama Imbalanced-Learn y se puede instalar de la siguiente manera:

```
sudo pip install imbalanced-learn
```

Esta librería contiene un grupo de clases que permiten implementar estas

Tema 2. Análisis exploratorio de datos y preprocesamiento

estrategias, veamos.

define la estrategia de oversampling

```
oversample = RandomOverSampler(sampling_strategy='minority')
```

Este tipo de estrategia hace que si la clase mayoritaria tiene 1000 casos y la minoritaria 100, añade casos duplicados a esta hasta que alcance, también los 1000 casos. En el argumento `sampling_strategy`, en vez de una cadena se puede pasar un número de punto flotante que especifica la cantidad de casos de la clase minoritaria respecto a la mayoritaria. Por ejemplo, si se le pasa 0.5, indica que la clase minoritaria tendrá la mitad de los casos que la mayoritaria.

```
import pandas as pd
import numpy as np
from imblearn.over_sampling import RandomOverSampler
#*****
diabeteDF = pd.read_csv("diabetes.csv")
print("Diabéticos:"+str(len(diabeteDF[diabeteDF.Outcome==1])))
print("No Diabéticos:"+str(len(diabeteDF[diabeteDF.Outcome==0])))
#*****
Y = diabeteDF["Outcome"]
X= diabeteDF.drop("Outcome",axis=1)
#*****
inbalanceProcesor = RandomOverSampler(sampling_strategy='minority')
X_Processed, Y_Processed=inbalanceProcesor.fit_resample(X,Y)
X_DF = pd.DataFrame(X_Processed, columns=['Pregnancies','Glucose','BloodPressure',
                                           'SkinThickness','Insulin','BMI',
                                           'DiabetesPedigreeFunction','Age'])
Y_DF = pd.DataFrame(Y_Processed, columns=['Outcome'])
diabete_Res = pd.concat([X_DF,Y_DF],axis=1,)
print("Diabéticos con resampling:"+str(diabete_Res[diabete_Res.Outcome==1].shape[0]))
print("No Diabéticos con resampling:"+str(diabete_Res[diabete_Res.Outcome==0].shape[0]))

Diabéticos:268
No Diabéticos:500
Diabéticos con resampling:500
No Diabéticos con resampling:500
```

Figura 2. Ejemplo de uso de la técnica de Random Oversampling, con el módulo imbalanced-learn. La estrategia usada es minority. Fuente: elaboración propia.

En la figura correspondiente al ejemplo de uso de la clase `RandomOverSampler`, hemos establecido la estrategia de muestreo en `minority`; pero esta acepta las siguientes estrategias:

- 'minority': Hace un re-muestreo solo de la clase minoritaria

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ 'not minority': Hace un re-muestreo de todas las clases excepto la minoritaria.
- ▶ 'not majority': Hace un re-muestreo de todas las clases excepto la mayoritaria.
- ▶ 'all': Hace un re-muestreo de todas las clases;
- ▶ 'auto': Es equivalente a 'not majority'.
- ▶ No vamos a mostrar otro ejemplo para demostrar el uso de RandomUndersampler o SMOTE; porque el código es prácticamente el mismo, solo cambian las clases a utilizar y los resultados el mismo. Queda a los lectores explorar y experimentar con los parámetros de cada una de estas técnicas.
- ▶ Es importante que se revise la documentación del módulo imbalanced-learn, pues contiene algunas variantes a SMOTE, tales como SMOTENC, SMOTEN, ADASYN, BorderlineSMOTE, KmeansSMOTE y SVMSMOTE.

División del conjunto de datos. (Entrenamiento y prueba)

Hasta este momento hemos estado hablando de conjunto de entrenamiento o conjunto de prueba; sin embargo, no hemos dedicado ningún espacio a explicar los motivos por los que es necesaria esta separación.

En esta sección iremos explicando los motivos y haremos un ejemplo para que se entienda de manera práctica cómo se hace.

La intuición detrás de esta separación es probar cuan bien nuestros modelos generalizan. Veamos, si tomamos en cuenta que la mayoría de los modelos se construyen a partir de los conjuntos de datos que les damos entonces es buena idea dejar una parte de datos que no se usen para construir el modelo y usarlos para probar cuan bien generalizan.

De esta forma, podemos dividir el conjunto de datos que tenemos usando reglas como 80-20 para la división; pero esto no es fijo. También podría usarse 75-25, u otra. Esto quiere decir que usaríamos el 80% de los datos para construir el modelo y

Tema 2. Análisis exploratorio de datos y preprocesamiento

20 para probar su habilidad de generalización.

La idea detrás del entrenamiento es disminuir la cantidad de errores en el conjunto de entrenamiento, con la esperanza que esto nos lleve al mínimo de errores en el conjunto de prueba. Pero en la realidad es mucho más complejo y los detalles los veremos en la parte correspondiente a la evaluación de los modelos de aprendizaje automático.

En la literatura es común encontrarse con divisiones del tipo: Entrenamiento, Validación y Prueba. Lo cierto es que dividimos en entrenamiento y prueba, porque los algoritmos de entrenamiento usan el conjunto, y valga la redundancia, de entrenamiento para dividirlo por medio de una técnica que se llama validación cruzada en dos más: Entrenamiento y validación.

En esencia, solo dividimos en dos conjuntos: Entrenamiento y Prueba. Entonces el algoritmo de entrenamiento, al que le pasamos el primero, a su vez los divide según la validación cruzada en: Entrenamiento y validación; pero creo eso lo había dicho y solo lo repito con el propósito de reforzar la idea.

La manera más sencilla de hacer esta división sería hacerlo de forma lineal, es decir tomar el conjunto de datos y dividirlo a partir de una fila determinada, sin embargo, esta no es buena idea.

Lo ideal sería, si tenemos dos clases, por ejemplo y la emparejamos por medio de remuestreo, tomar de forma aleatoria la misma proporción de ambas en forma de 80-20. Esto nos garantiza que no introduzcamos ningún sesgo en el conjunto escogido y las clases tendrás la misma representación en ambos conjuntos. A este tipo de procedimiento se le conoce como muestreo aleatorio estratificado.

Esto es exactamente lo que hace la clase `train_test_split` del módulo Scikit-Learn de python y que usaremos para ejemplificar el procedimiento.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Es importante dejar claro que el balanceo de las clases por re-muestreo debe ejecutarse sobre el conjunto de entrenamiento.

Si queremos que `train_test_split`, deje de hacer muestreo aleatorio estratificado le pasamos como parámetro `shuffle = False`.

El tamaño del conjunto de entrenamiento o prueba se le pasa por medio del parámetro `train_size` o por medio de `test_size`, se puede definir uno solo y automáticamente se asume que el otro es el complemento.

Es decir, si se le pasa como argumento `test_size=.25` eso hace que el conjunto de entrenamiento sea del 75% de los casos del dataset.

```
import pandas as pd
import numpy as np
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
#*****
diabeteDF = pd.read_csv("diabetes.csv")
print("Diabéticos:"+str(len(diabeteDF[diabeteDF.Outcome==1])))
print("No Diabéticos:"+str(len(diabeteDF[diabeteDF.Outcome==0])))
#*****
Y = diabeteDF["Outcome"]
X = diabeteDF.drop("Outcome",axis=1)
X_train,X_test,Y_train,Y_test = train_test_split(X,Y, test_size=.25, random_state=0)
#*****
# Debe tenerse como norma que el balanceo de las clases debe ejecutarse sobre el conjunto de entrenamiento
inbalanceProcessor = SMOTE()
X_Processed, Y_Processed=inbalanceProcessor.fit_resample(X_train,Y_train)
X_DF = pd.DataFrame(X_Processed, columns=['Pregnancies','Glucose','BloodPressure',
                                           'SkinThickness','Insulin','BMI',
                                           'DiabetesPedigreeFunction','Age'])
Y_DF = pd.DataFrame(Y_Processed, columns=['Outcome'])
diabete_Res = pd.concat([X_DF,Y_DF],axis=1)
print("Diabéticos con resampling usando SMOTE:"+str(diabete_Res[diabete_Res.Outcome==1].shape[0]))
print("No Diabéticos con resampling usando SMOTE:"+str(diabete_Res[diabete_Res.Outcome==0].shape[0]))

Diabéticos:268
No Diabéticos:500
Diabéticos con resampling usando SMOTE:370
No Diabéticos con resampling usando SMOTE:370
```

Figura 3. Ejemplo de división del conjunto de datos en: Conjunto de entrenamiento(`X_train,Y_train`) y Conjunto de Pruebas (`X_test, Y_test`). Fuente: elaboración propia.

En la sección correspondiente a las métricas de evaluación de los modelos de aprendizaje, también detallaremos todo lo que respecta a la validación cruzada y como ejecutarla con Scikit-Learn.

Análisis exploratorio de datos

Desde la sección anterior y la parte inicial de esta, hemos estado analizando técnicas

Tema 2. Análisis exploratorio de datos y preprocesamiento

y métodos que corresponden a la etapa de preprocesamiento de los datos en el proceso de aprendizaje automático. Sin embargo, es durante el proceso de análisis exploratorio donde se detectan la mayoría de los problemas en nuestro conjunto de datos y que se resuelven o atacan con las técnicas que ya hemos visto. Esto quiere decir que Análisis Exploratorio de Datos está íntimamente ligada a la etapa de preprocesamiento de los datos, toda vez que nos permite ir visualizando, detectando y corrigiendo los problemas que pueden presentarse en dicha etapa.

¿Qué es el Análisis Exploratorio de Datos? Es un conjunto de técnicas que nos permiten entender los datos que tenemos a mano, detectar anomalía en ellos y corregirlas. De manera general, esta etapa pretende adaptar los conjuntos de datos a las necesidades de los modelos de aprendizaje que usaremos para resolver las tareas a mano.

Cada conjunto de datos es único y por tanto es muy difícil automatizar todas las tareas referente a su chequeo, análisis y corrección, por eso es importante un conjunto de pasos ordenados y técnicas definidas para su tratamiento, eso es el Análisis Exploratorio de datos.

Etapas de análisis exploratorio de datos

Las etapas que, de manera general, se llevan a cabo en AED, son las siguientes:

- ▶ Preparación de los Datos.
- ▶ Realizar una inspección general del conjunto de datos y de sus rasgos.
- ▶ Realizar un análisis de las correlaciones entre las variables o rasgos que conforman el conjunto.
- ▶ Detección de los valores faltantes y su corrección-
- ▶ Detección y corrección de datos anómalos, los llamados Outliers.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- Realizar una evaluación, si fuese necesario, de los supuestos básicos de las técnicas estadísticas que deben cumplir las variables tales como Normalidad, linealidad y homocedasticidad.

Lo importante a destacar aquí, es que esta no es una etapa que debe seguirse de forma lineal, puede verse como una etapa iterativa.

Aunque este AED, puede hacerse con herramientas gráficas tales como SPSS, STATA u otros, a los efectos de este trabajo usaremos Python y sus módulos para acometerlo.

Cómo ya hemos visto muchas de las técnicas que se deben aplicar en esta etapa, pues nos concentraremos en la visualización y tipos de gráficas que son útiles para el análisis, los conceptos relacionados con las distribuciones de frecuencias básicas, las pruebas de normalidad y homocedasticidad.

Teniendo en cuenta que las técnicas básicas para la integración de datos ya las vimos en secciones anteriores no nos detendremos a explicarlas y supondremos que tenemos un archivo ya integrado en formato CSV, es decir, de valores separados por coma.

Ahora bien, es de vital importancia estar familiarizado con los conceptos básicos de la estadística descriptiva y los gráficos más comunes para mostrar información y sus usos. Por tanto, antes de ver los detalles de cada una de las etapas del proceso, veremos los conceptos básicos de AED.

Conceptos de la estadística descriptiva básicos

Los conceptos que trataremos en este apartado son los usados para describir a una variable en estadística.

Medidas de tendencia central

Dentro de las medidas que permiten resumir un rasgo o variable, se encuentran:

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Media.
- ▶ Mediana.
- ▶ Moda.

Comencemos por la más simple, la media:

La media es un número que caracteriza el centro de una colección de valores, por ejemplo, un rasgo. Es a lo que comúnmente se conoce como promedio.

Si tomamos como ejemplo los siguientes valores: 3,5,1,2. Entonces la media sería la suma de ellos dividida por la cantidad.

$$(3+5+1+2) / 4 = 11/4 = 2.75$$

Su fórmula matemática es:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

Lo importante a notar de la media es que es muy susceptible a valores anómalos u Outliers, y puede ser engañosa cuando no se han detectado y tratados de forma correcta.

El cálculo de la mediana en Python para un rasgo como la glucosa, del dataset de diabetes sería:

Por ejemplo: `print("Media:"+str(np.mean(diabete_Res["Glucose"])))`

Aquí se usa la librería Numpy.

Mediana

Esta es otra de las medidas que caracterizan el centro de los valores de una variable. Esta se define como el valor del centro o medio de una variable ordenada. Si la

Tema 2. Análisis exploratorio de datos y preprocesamiento

variable es impar, la mediana coincide con el valor central. Si la variable es par entonces la mediana sería el promedio de los dos valores del centro.

Si, tomamos como ejemplo el mismo conjunto de valores anteriores: 3,5,1,2.

La mediana sería: $1,2,3,5 = 2+3/2 = 2.5$.

Si el conjunto estuviera compuesto por un número impar de valores:

mediana = $1,2,3,5,6 = 3$

La mediana es una medida de centralidad mucho más robusta a outliers que la media.

Introduzcamos a manera de ejemplo un outlier en el conjunto anterior y veamos cómo se comporta la mediana y la media.

mediana = $1,2,3,5,12 = 3$

media = $(3+5+1+2+12) / 5 = 23/5 = 4.6$

El cálculo de la mediana en Python para un rasgo como la glucosa, del dataset de diabetes sería:

```
print("Media:"+str(np.median(diabete_Res["Glucose"])))
```

Aquí se usa la librería Numpy.

Moda

La Moda es el valor que se repite con mayor frecuencia en un conjunto de datos. Cuando un conjunto de datos se representa por medio de las frecuencias de los valores que contiene, a esa representación se le llama distribución de frecuencias. Cuando en la distribución de frecuencias, aparece un solo valor con la máxima frecuencia, se le llama unimodal. Pues contiene una moda. Pueden aparecer varios valores con la misma frecuencia, en este caso se les llama bimodal. Cuando existe

Tema 2. Análisis exploratorio de datos y preprocesamiento

una distribución donde todos los valores tienen exactamente la misma frecuencia, se le llama distribución uniforme.

El cálculo de la moda en Python se hace por medio de las librerías `scipy` y `statistics`. `Pandas`, en su estructura de datos `DataFrame`, tiene métodos que permiten el cálculo de estas.

Cuando se usa `scipy`:

```
print("Moda según scipy:"+str(st.mode(diabete_Res["Glucose"])))
```

retorna:

```
Moda según scipy:ModeResult(mode=array([100]), count=array([17]))
```

Que significa que el valor con mayor frecuencia es 100 y que se repite 17 veces.

Cuando se usa el módulo `statistics`:

```
print("Moda según statistics:"+str(stat.mode(diabete_Res["Glucose"])))
```

El resultado es:

```
Moda según statistics:100
```

Cuando usamos `pandas`:

```
import pandas as pd
```

```
diabetes_df = pd.read_csv("diabetes.csv")
```

```
media = diabetes_df["Glucose"].mean()
```

```
mediana = diabetes_df["Glucose"].median()
```

```
moda = diabetes_df["Glucose"].mode()
```


Tema 2. Análisis exploratorio de datos y preprocesamiento

Medidas de dispersión

Estas medidas definen cuan agrupadas están las mediciones alrededor de una media o cuán lejos de ellas o dispersas. Por esto, también suelen llamárselas medidas de dispersión.

Entre estas se encuentran:

- ▶ Desviación.
- ▶ Varianza.
- ▶ Desviación estándar.
- ▶ Desviación Media Absoluta.
- ▶ Desviación Mediana absoluta desde la mediana.
- ▶ Rango.
- ▶ Percentil.
- ▶ Rango intercuartílico.

La intuición detrás de estas medidas de variabilidad es tratar de caracterizar la dispersión de los valores de un conjunto de datos de alguna de las medidas de tendencia central como la media o la mediana.

Para explicar este concepto, tomemos el conjunto de datos que usamos anteriormente: ,5,1,2,3.

Si tratáramos de encontrar una medida que caracterice este conjunto podríamos tomar las desviaciones alrededor de su media, por ejemplo:

5-2.75, 1-2.75, 2-2.75, 3-2.75.

Si sumamos todos los valores resultantes, oooooooh, nos da cero, bueno pues no

Tema 2. Análisis exploratorio de datos y preprocesamiento

es buena idea usar este procedimiento. Una mejor idea sería tomar el promedio de los valores absolutos de las desviaciones respecto a la media.

$$(2.25 + 1.75 + 0.75 + 0.25) / 4 = 1.25$$

A este tipo de procedimiento de caracterizar la desviación de los valores de un conjunto de datos se le llama **desviación media absoluta**.

Su fórmula matemática toma la siguiente forma:

$$\text{Desviación media absoluta} = \frac{\sum_{i=1}^n |(x_i - \bar{x})|}{n}$$

Esta medida es sumamente importante, pues si tomamos en vez de la media un valor conocido, esta medida podemos interpretarla como una medida de error y se llamaría Error Medio absoluto y nos ayudaría a caracterizar el error incurrido en la predicción de mediciones, por ejemplo. Esto lo veremos con mucho más detalle cuando estudiemos las métricas de evaluación de los modelos de aprendizaje automático.

Sin embargo, las más conocidas de las medidas de variabilidad son la varianza y la desviación estándar. A diferencia de la desviación absoluta, la varianza se basa en la desviación cuadrada, y su fórmula matemática es la siguiente:

$$S^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}$$

Mientras que la **desviación estándar**, es la raíz cuadrada de la varianza, y su fórmula es la siguiente:

$$S = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

Lo interesante de esta medida, la desviación estándar es que, es más fácil de interpretar, pues está expresada en la misma escala que los datos originales.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Exploración de las distribuciones de frecuencia de un conjunto de datos

Histograma:

Uno de las maneras habituales para visualizar la distribución de frecuencia de un conjunto es por medio de un histograma.

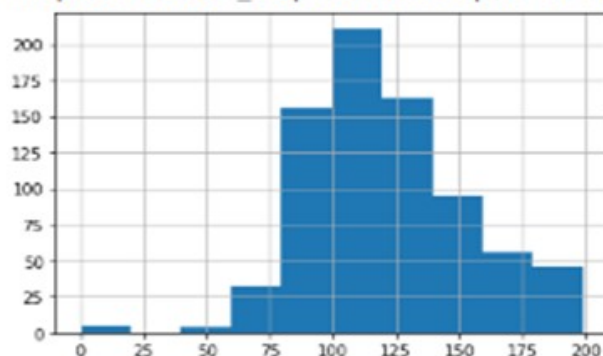
Los histogramas se caracterizan por la cantidad de bin, divisiones o compartimentos tiene.

El eje x se divide en compartimentos y el eje y del gráfico muestra la cantidad de observaciones. Veamos cómo se construye un histograma en Python.

Una de las formas más sencillas de mostrar un histograma en python es usando los métodos disponibles en la estructura DataFrame de Pandas, una de las librerías de Python para el análisis de datos.

```
import pandas as pd
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
#.....
diabeteDF = pd.read_csv("diabetes.csv")
print("Media:"+str(diabeteDF["Glucose"].mean()))
print("Mediana:"+str(diabeteDF["Glucose"].median()))
print("Moda:"+str(diabeteDF["Glucose"].mode()))
diabeteDF["Glucose"].hist()
```

```
Media:120.89453125
Mediana:117.0
Moda:0    99
1    100
dtype: int64
<matplotlib.axes._subplots.AxesSubplot at 0x7fdc3ce9b990>
```



Tema 2. Análisis exploratorio de datos y preprocesamiento

Figura 4. Histograma de un rasgo en un conjunto de datos. Fuente: elaboración propia.

¿Qué información podemos extraer de este tipo de gráfico? Pues a ojos ya podemos hacernos una idea de cómo están distribuidos los datos en este rasgo. De entrada podemos percatarnos que hay grupo de valores que están próximos al cero, si sabemos que los valores normales de glucosa en sangre son alrededor de 123, entonces nos damos cuenta que hay mediciones que contienen valores incorrectos. Es importante darse cuenta de los valores arrojados por la moda, que hay dos modas, una es 99 y la otra es 100.

En el eje x se ubican las concentraciones de glucosa en sangre de pacientes diabéticos o no. En el eje y se ubica el número de observaciones.

Veamos otra manera de construir un histograma en Python.

También puede usarse la librería `matplotlib.pyplot` para crear un histograma, especificando parámetros adicionales, como el número de bins o grupos en los que agrupar los datos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

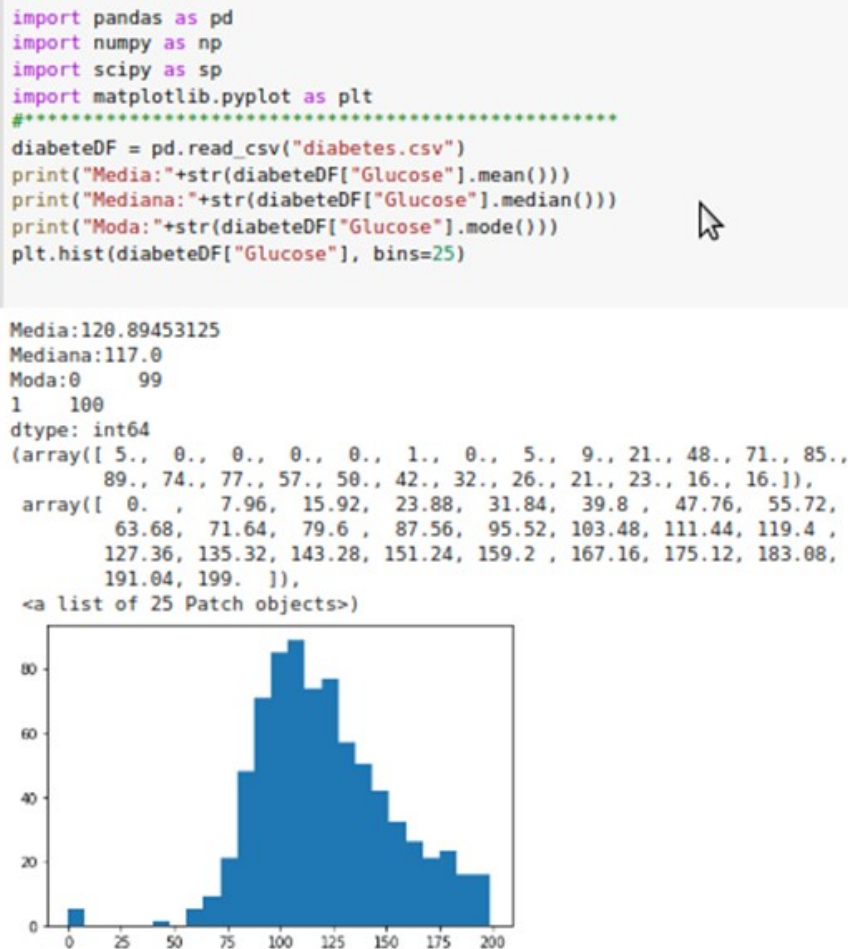


Figura 5. Ejemplo de histograma generado con pyplot, usando 25 bins.

Una conjunto de datos, entonces, puede quedar descrito por valores como la media, la mediana, la varianza y la desviación estándar. Sin embargo, los valores de este conjunto de datos pueden distribuirse de muchas maneras. Es a la forma en que se distribuyen a lo que se le llama, y valga la redundancia, distribuciones de frecuencias.

Los estadísticos, con el paso del tiempo, se han dado cuenta que los datos tomados, siempre tienen formas similares de distribuir sus valores y al ir sistematizando e investigando sus propiedades y características llegaron a generalizarlas para que así aparecieran las llamadas distribuciones teóricas de frecuencias.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Esa es la razón por la que la mayoría de las veces comparamos las distribuciones de nuestros datos con las distribuciones teóricas, pues de estas últimas ya conocemos sus propiedades y si nuestros datos actuales se ajustan a la forma de alguna de las teóricas podemos decir con alguna certeza que sus propiedades serán similares.

Ahora bien, ¿cuáles son las distribuciones de frecuencias más conocidas? Hay que saber, que si las variables que queremos comparar son continuas, las distribuciones a las que podemos comparar son a las llamadas distribuciones continuas:

- ▶ Distribución Normal.
- ▶ Distribución Ganma.
- ▶ Distribución Chi Cuadrado.
- ▶ Distribución t-Student.

Pero, también podemos listar algunas de las que se denominan distribuciones discretas:

- ▶ Distribución uniforme.
- ▶ Distribución Bernoulli.
- ▶ Distribución Geométrica.
- ▶ Distribución Hipergeométrica.
- ▶ Distribución Binomial.

En la Figura 6, se muestran la mayoría de las distribuciones más importantes y sus relaciones.

Tema 2. Análisis exploratorio de datos y preprocesamiento

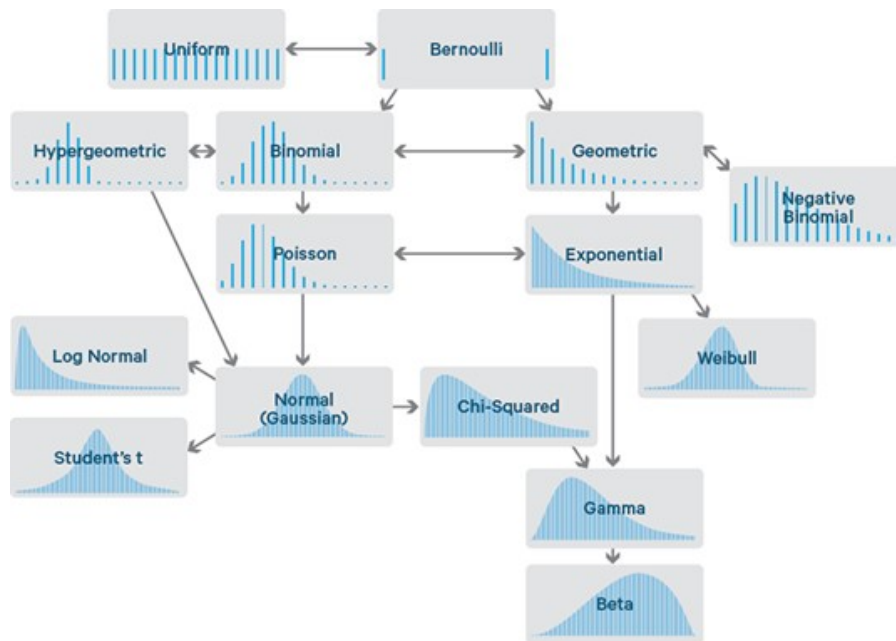


Figura 6. Distribuciones de frecuencias.

Distribución Normal: Se le conoce como distribución Gaussiana, distribución de Gauss, distribución campana, etc.

Esta distribución, al parecer, debe su popularidad a que muchos fenómenos naturales tienen distribuidos sus valores de esta forma.

Fenómenos como, las características morfológicas de los individuos, como la estatura y otros. Algunos efectos fisiológicos, como el efecto de los fármacos. El nivel de ruido en telecomunicaciones, etc.

Todos ellos tienen en común que sus valores están normalmente distribuidos. Las características principales de esta distribución son los siguientes:

- ▶ Es simétrica respecto a la media.
- ▶ En esta distribución, la media, la mediana y la moda son iguales.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ En el intervalo $[\mu - \sigma, \mu + \sigma]$ se encuentra, aproximadamente, el 68.26% de los valores de la distribución.
- ▶ En el intervalo $[\mu - 2\sigma, \mu + 2\sigma]$ se encuentra, aproximadamente, el 95.44% de los valores de la distribución.
- ▶ En el intervalo $[\mu - 3\sigma, \mu + 3\sigma]$ se encuentra, aproximadamente, el 99.74% de los valores de la distribución.
- ▶ Su expresión matemática es: $p(x; \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$

Donde μ es la media aritmética σ^2 es la desviación estándar.

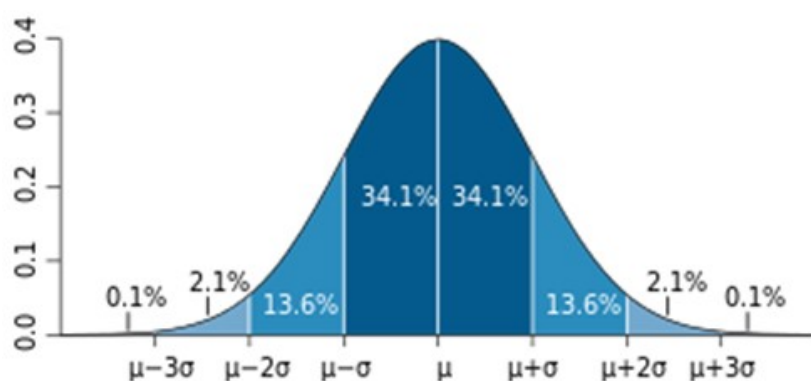


Figura 7. Intervalos en la gráfica de la distribución Gaussiana.

¿Por qué es tan importante la distribución normal? Esta pregunta aflora de manera natural en cuanto comenzamos a adentrarnos en el mundo del aprendizaje automático. Podríamos enumerar algunas, sin querer agotar el tema:

- ▶ Hay muchísimos fenómenos naturales que se pueden modelar por medio de esta distribución.
- ▶ Es más, según el teorema del límite central, cualquier fenómeno que puedan ser causados por un gran número de pequeños efectos, puede modelarse por esta distribución.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Tiene propiedades matemáticas muy bien conocidas y explotadas en la inferencia estadística y otras ramas.
- ▶ Muchos métodos de aprendizaje automático, incluyendo Análisis de Discriminante lineal y Regresión asumen que los datos de entrenamiento están normalmente distribuidos.
- ▶ Algunos modelos de redes neuronales, necesitan datos normalizados, para su entrenamiento.

Medidas de forma de las distribuciones

Conociendo que los conjuntos de datos, o rasgos, se pueden representar o caracterizar por la distribución de la frecuencia de sus valores, estamos en condiciones de aprender sobre algunas medidas que expresan la forma de esta distribución.

De estas medidas veremos la asimetría, que explica si la forma de la distribución está estirada a la derecha o a la izquierda, es decir es asimétrica.

Este tipo de comportamiento de la distribución de los valores de un rasgo es automáticamente reconocido por medio del histograma pero también medidas que lo sumarizan de manera numérica.

Asimetría (*Skewness*)

Es la medida que indica la simetría de la distribución de una variable respecto a la media aritmética, sin necesidad de hacer la representación gráfica. Los coeficientes de asimetría indican si contiene el mismo número de elementos a izquierda y derecha de la media.

Tema 2. Análisis exploratorio de datos y preprocesamiento

La asimetría arroja valores para identificar la forma de la distribución, que puede ser:

- ▶ **Asimetría negativa:** la cola de la distribución se alarga para valores inferiores a la media.
- ▶ **Simétrica:** contiene el mismo número de elementos a izquierda y derecha de la media. En este caso, coinciden la media, la mediana y la moda. La distribución se adapta a la forma de la campana de Gauss, o distribución normal.
- ▶ **Asimetría positiva:** la cola de la distribución se alarga (a la derecha) para valores superiores a la media.
- ▶ En la Figura 8 se muestran las formas respectivas.

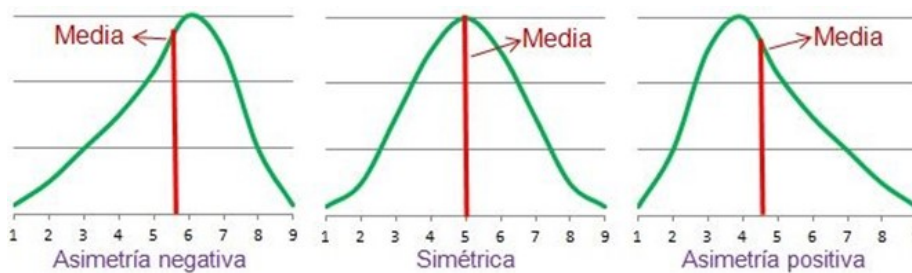


Figura 8. Tipos de asimetría.

Para calcular la asimetría de una variable, hay principalmente tres métodos:

- ▶ Coeficiente de asimetría de Fisher.
- ▶ Coeficiente de asimetría de Pearson.
- ▶ Coeficiente de asimetría de Bowley.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Coeficiente de asimetría de Fisher:

El coeficiente de asimetría de Fisher CAF evalúa la proximidad de los datos a su media $\bar{x}$. Cuanto mayor sea la suma $\sum (x_i - \bar{x})^3$, mayor será la asimetría. Sea el conjunto $X=(x_1, x_2, \dots, x_N)$, entonces la fórmula de la asimetría de Fisher es:

$$CA_F = \frac{\sum_{i=1}^N (x_i - \bar{x})^3}{N \cdot S_x^3}$$

siendo $\bar{x}$ la media y S_x la desviación típica

Y lo más importante es cómo analizamos los resultados que nos arroja esta correlación:

- ▶ Si $CAF < 0$: la distribución tiene una asimetría negativa y se alarga a valores menores que la media.
- ▶ Si $CAF = 0$: la distribución es simétrica.
- ▶ Si $CAF > 0$: la distribución tiene una asimetría positiva y se alarga a valores mayores que la media.

Esto significa que en la medida que el coeficiente de asimetría esté más cerca de cero, la distribución de los datos analizados está más cerca también de ser normalmente distribuidos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

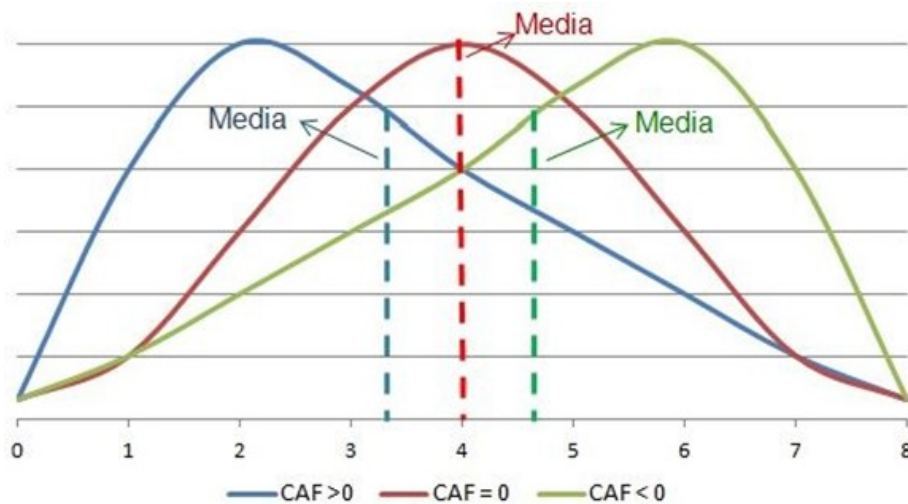


Figura 9. Resultados del coeficiente de asimetría de Fisher.

Coeficiente de asimetría de Pearson:

El coeficiente de asimetría de Pearson CAP mide la diferencia entre la media y la moda con respecto a la dispersión del conjunto $X=(x_1, x_2, \dots, x_N)$.

Este procedimiento, se emplea solamente para distribuciones unimodales y poco asimétricas. Su formulación matemática es:

$$CAP = \frac{\bar{x} - Mo(X)}{S_x}$$

siendo $\bar{x}$ la media, $Mo(X)$ la moda y S_x la desviación típica

Este tipo de coeficiente explota la característica de las distribuciones normales, en las cuales la media y la moda son iguales. Entonces tenemos que:

- ▶ Si $CAP < 0$: la distribución tiene una asimetría negativa, puesto que la media es menor que la moda.
- ▶ Si $CAP = 0$: la distribución es simétrica. Media y Moda coinciden.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- ▶ Si $CAP > 0$: la distribución tiene una asimetría positiva, ya que la media es mayor que la moda.

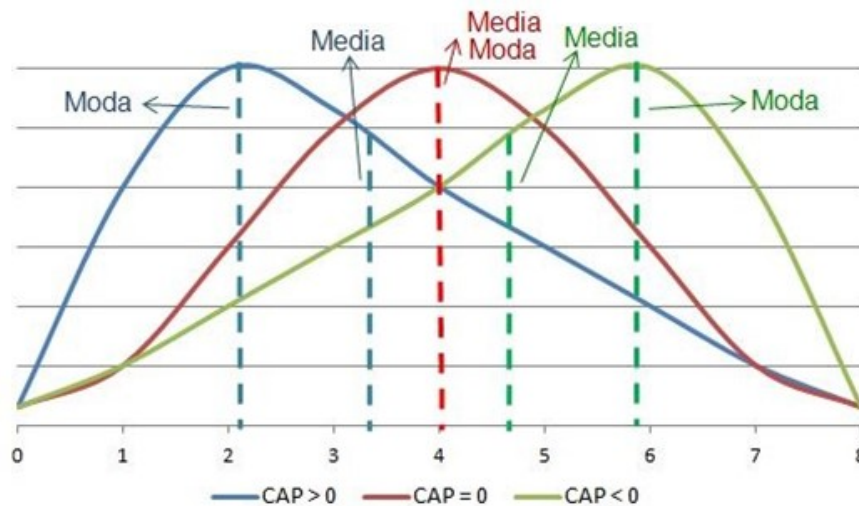


Figura 10. Interpretación de los resultados del Coeficiente de asimetría de Pearson.

El coeficiente de asimetría de Bowley CAB toma como referencia los cuartiles para determinar si la distribución es simétrica o no. Para aplicar este coeficiente, se supone que el comportamiento de la distribución en los extremos es similar. Sea el conjunto $X=(x_1, x_2, \dots, x_N)$, la asimetría de Bowley es:

$$CAB = \frac{Q_3 + Q_1 - 2Me(X)}{Q_3 - Q_1}$$

siendo Q_3 el tercer cuartil, Q_1 el primer cuartil y $Me(X)$ la mediana

Por tanto el resultado de este coeficiente como en los dos anteriores es el que sigue:

- ▶ Si $CAB < 0$: la distribución tiene una asimetría negativa, puesto que la distancia de la mediana al primer cuartil es mayor que al tercero.
- ▶ Si $CAB = 0$: la distribución es simétrica, ya que el primer y tercer cuartil están a la misma distancia de la mediana.

Tema 2. Análisis exploratorio de datos y preprocesamiento

- Si $CAB > 0$: la distribución tiene una asimetría positiva, ya que la distancia de la mediana al tercer cuartil es mayor que al primero.

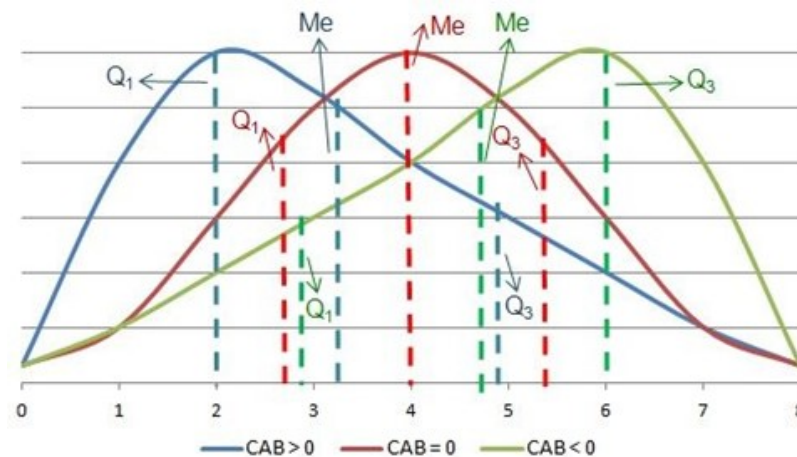


Figura 11. Interpretación de los resultados del Coeficiente de asimetría de Bowley.

Es importante revisar la literatura para ampliar, si se quiere, sobre las distintas formas de obtener esta medida.

Veamos ahora como calculamos la asimetría en Python. Hay varias formas de hacerlo y esta medida está disponible en varios módulos. Aquí, veremos cómo se calcula usando un método de la clase DataFrame de Pandas y la función disponible desde el módulo stats en scipy.

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
import pandas as pd
import numpy as np
import scipy.stats as st
#*****
diabeteDF = pd.read_csv("diabetes.csv")
print("Asimetría método de la Clase DataFrame: "+str(diabeteDF["Glucose"].skew()))
print("Asimetría función en Stats de scipy: "+str(st.skew(diabeteDF["Glucose"])))
diabeteDF["Glucose"].hist()
```

Asimetría método de la Clase DataFrame: 0.17375350179188992
Asimetría función en Stats de scipy: 0.17341395519987735
<matplotlib.axes._subplots.AxesSubplot at 0x7ff43416bb90>

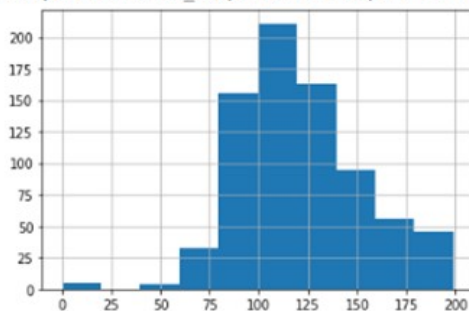


Figura 12. Cálculo del coeficiente de asimetría con scipy.stats y Pandas.

En `scipy.stats.skew`, la asimetría se calcula usando el coeficiente de Fisher-Pearson. Ver referencia de la API de `scipy` para más información.

La curtosis, al igual que la asimetría mide la forma de una distribución y permite, de forma analítica, sumarizar algún aspecto de esta. En este caso, el resultado de esta medida está asociado a las colas de la distribución y no a los picos.

- ▶ La Kurtosis de una distribución normal univariada es 3. Las distribuciones con kurtosis menores que 3 se les llama distribuciones platykurticas, mientras que a las que tienen un kurtosis mayor que 3 se les llaman leptokurticas.
- ▶ La Kurtosis se calcula de la misma forma que en el ejemplo anterior en Python.
- ▶ La kurtosis es la última de las medidas necesarias para caracteriza un rasgo o conjunto de datos univariado.

Por tanto, a modo de resumen, tenemos que cuando queremos caracterizar un

Tema 2. Análisis exploratorio de datos y preprocesamiento

rasgo, tenemos que calcular, la media, la mediana, la moda, la asimetría y la kurtosis y tratar de visualizar la distribución de sus valores por medio de un histograma.

Con este conocimiento a mano podemos pasar a evaluar algunas de las formas en podemos saber, de manera analítica la distribución de los rasgos de nuestro conjunto de datos es normal o no.

Pruebas de normalidad

A este tipo de prueba, se les llaman test de normalidad. Entre los más usados se encuentran:

- ▶ Test de Shapiro-Wilk.
- ▶ El Test de D'Agostino o K-cuadrado.

El test de Shapiro-Wilk tiene como hipótesis nula que los datos provienen de una población Normalmente distribuida. La decisión de apoyar o rechazar la hipótesis nula depende del valor de p-value arrojado de esta.

Si $p\text{-value} > 0.05$ Se apoya H_0 . (Los datos están normalmente distribuidos).

Si $p\text{-value} < 0.05$ Se rechaza H_0 . (Los datos no están normalmente distribuidos).

A este valor 0.05 se le llama nivel de significación y el valor normalmente escogido es el de 5%.

En este pedazo de código se muestra cómo se aplica en Python: un valor mayor que 0.05 dice apoya la hipótesis de que los datos están normalmente distribuidos

Tema 2. Análisis exploratorio de datos y preprocesamiento

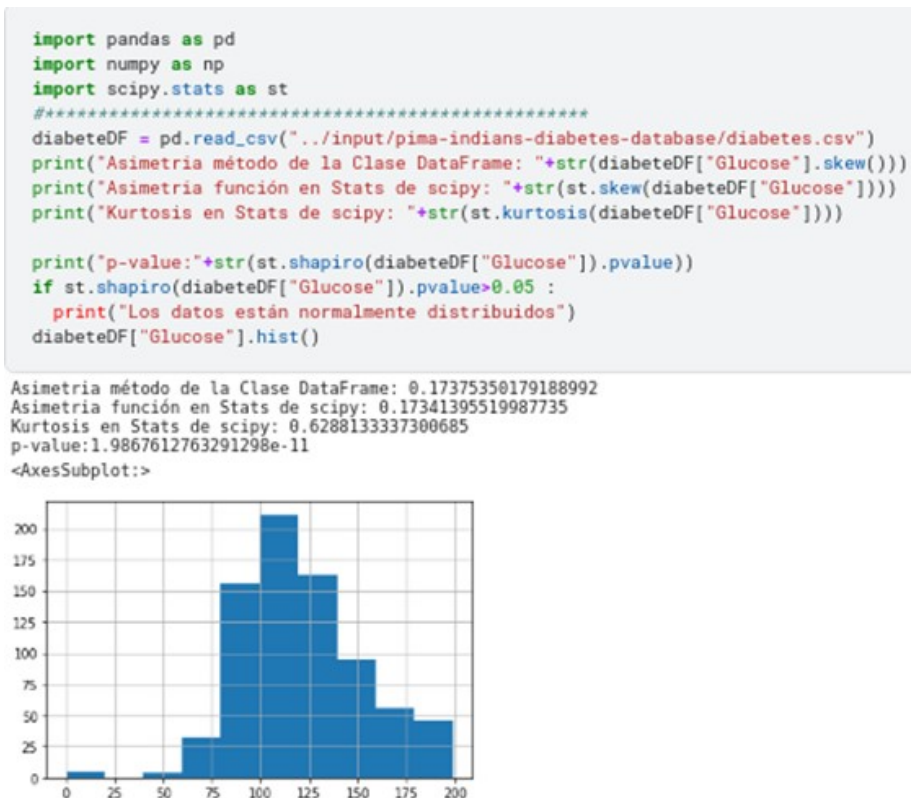


Figura 13. Ejemplo de uso del test Shapiro-Wilk para determinar si los datos están normalmente distribuidos.

Con el test de Dágostino K-Cuadrado o K2 sucede exactamente lo mismo que con el anterior. Este presupone una hipótesis nula que soporta el hecho de que los datos son tomados de una población normalmente distribuida. Para probar o refutar la hipótesis se calcula el valor pvalue y si este es mayor que el nivel de significación se apoya la H0 y de lo contrario se rechaza.

Un valor mayor que 0.05 dice que se apoya la hipótesis de que los datos están normalmente distribuidos

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
import pandas as pd
import numpy as np
import scipy.stats as st
*****
diabeteDF = pd.read_csv("../input/pima-indians-diabetes-database/diabetes.csv")
print("Asimetría método de la Clase DataFrame: "+str(diabeteDF["Glucose"].skew()))
print("Asimetría función en Stats de scipy: "+str(st.skew(diabeteDF["Glucose"])))
print("Kurtosis en Stats de scipy: "+str(st.kurtosis(diabeteDF["Glucose"])))

print("p-value: "+str(st.normaltest(diabeteDF["Glucose"]).pvalue))
if st.normaltest(diabeteDF["Glucose"]).pvalue>0.05 :
    print("Los datos están normalmente distribuidos")
diabeteDF["Glucose"].hist()
```

Asimetría método de la Clase DataFrame: 0.17375350179188992
Asimetría función en Stats de scipy: 0.17341395519987735
Kurtosis en Stats de scipy: 0.6288133337300685
p-value: 0.0020446506991363502
<AxesSubplot:>

Figura 14. Ejemplo de uso del test Dágostino K-Cuadrado para determinar si los datos están normalmente distribuidos.

Existen varias pruebas de normalidad, entre ellas se encuentran:

- ▶ Prueba de normalidad Kolmogorov-Smirnov.
- ▶ Prueba de Anderson-Darling.
- ▶ Prueba de normalidad de Ryan-Joiner.

Que aunque sigan vías diferentes los resultados son más o menos similares.

En análisis exploratorio de datos, no solo se analizan los rasgos individuales, es importante, para ganar alguna comprensión de los datos, también analizar la correlación entre los rasgos, en busca de altos valores de correlación o ausencia de ellas. Como ya explicamos en la sección anterior para eso se usan algunas medidas de correlación.

Análisis exploratorio de datos, etapa #1: recolección e integración de los datos

En la sección anterior en la parte correspondiente a los problemas que pueden presentarse en esta etapa, vimos cómo nos enfrentamos a problemas como rasgos redundantes o casos redundantes. También se describieron las fuentes de las cuales

Tema 2. Análisis exploratorio de datos y preprocesamiento

podríamos tomar los datos. Sin embargo no se han detallados los formatos más usuales en los que soportan los conjuntos de datos en las tareas de aprendizaje automático.

Por tanto en sección revisaremos formatos de archivos como los csv, o valores separados por coma, los archivos con formato json, entre otros.

Archivos con formato csv están especificado en el RFC 4180. Este, es un formato que permite representar arreglos rectangulares de valores textuales o numéricos. Estos arreglos rectangulares están conformados como filas y columnas. Estas columnas están separadas por el carácter coma (%x2C), mientras que las filas o records, se separan por medio de carácter de fin de línea. El RFC4180 especifica que sea CRLF. Cada línea debe contener el mismo número de campos.

Estos archivos pueden contener o no una primera fila que corresponde al nombre de los campos y debe contener el mismo número que los campos del resto del archivo. Aunque pueden haber variaciones, de manera general este es el formato que por defecto hemos estado usando en estas lecciones. En Python hay varias formas de leer este tipo de archivos. Una de las vías es hacerlo desde la librería estándar.

```
import os
os.chdir("C:\\Datos")
os.getcwd()

import csv
f= open("diabetes.csv")
reader = csv.reader(f)
for row in reader:
    print (row)
```

```
['6', '148', '72', '35', '0', '33.6', '0.627', '50', '1']
```

```
['1', '85', '66', '29', '0', '26.6', '0.351', '31', '0']
```

```
['8', '183', '64', '0', '0', '23.3', '0.672', '32', '1']
```

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
['1', '89', '66', '23', '94', '28.1', '0.167', '21', '0']
```

Este es el aspecto que tiene el contenido de un fichero con formato csv. También puede cargarse directamente a una estructura de datos llamada DataFrame de Pandas usando uno de los métodos de esta librería.

```
import pandas
```

```
filename = 'diabetes.csv'  
data = pandas.read_csv(filename, header=0)
```

Aunque también, librerías como Numpy tienen sus propios mecanismos para leer este tipo de formato.

Otro de los formatos que son muy comunes a la hora de cargar datos para el aprendizaje automático es el llamado JSON.

JSON significa, por sus siglas en inglés, JavaScript Object Notation. Es un formato de fichero estándar abierto y de intercambios de datos que usa texto completamente entendible por los humanos.

Aunque en sus inicios se derivó de JavaScript hoy se ha convertido en un formato de uso general. Desde el 2006 se ha publicado una especificación, la RFC 4627 y luego estandarizado en 2013 por la ECMA-404 y el RFC8259 en 2017.

El formato de un fichero con formato json:

Tema 2. Análisis exploratorio de datos y preprocesamiento

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

Por cuestiones de espacio y tiempo no nos detendremos en la descripción de formato de este tipo de archivo. Por favor, puede remitirse a los correspondientes documentos RFC.

Para importar datos almacenados en formato json en Python lo hacemos

de la siguiente forma:

```
import pandas as pd
```

```
data = pd.read_json("data.json")
```

Este es uno de los formatos más comunes, pues Excel es también una excelente herramienta de análisis de datos. Eso hace que en las empresas sea muy común

Tema 2. Análisis exploratorio de datos y preprocesamiento

encontrar grandes volúmenes de datos en forma de documentos .xlsx.

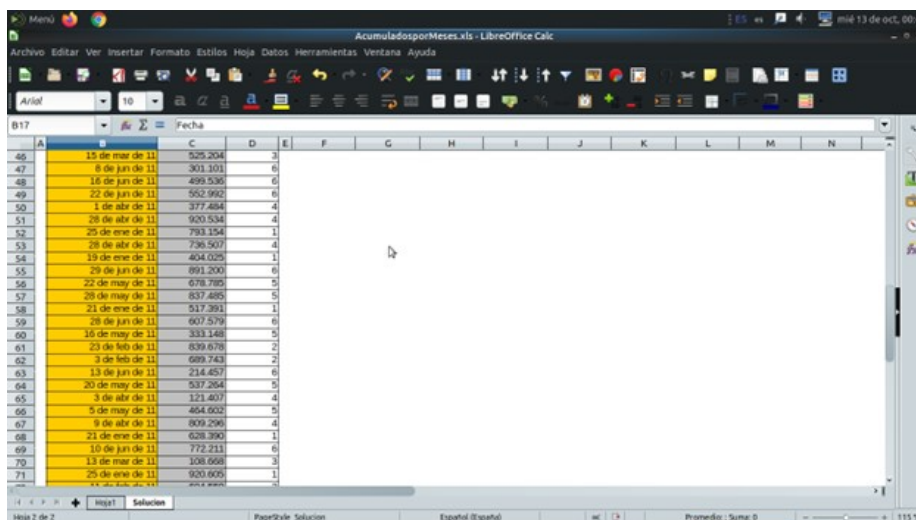


Figura 15. Ejemplo de un fichero .xls abierto en LibreOffice.

Este es un formato basado en XML y abierto de Microsoft. Este tipo de archivos, ya sea .xls o .xlsx, pueden leerse en Python de la siguiente forma:

```
import pandas as pd
df = pd.read_excel("../input/acumuladoxmeses-xls/AcumuladosporMeses.xls", sheet_name="Solucion")
df.describe
```

```
<bound method NDFrame.describe of Unnamed: 0 Unnamed: 1 Unnamed: 2 Unnamed: 3 \
0 NaN NaN NaN NaN
1 NaN NaN NaN NaN
2 NaN NaN NaN NaN
3 NaN NaN NaN NaN
4 NaN NaN NaN NaN
..
77 NaN 2011-05-08 23:45:55.273000 427486.933567 5
78 NaN 2011-02-28 20:05:05.015000 842768.382835 2
79 NaN 2011-03-05 11:39:25.071000 228402.734439 3
80 NaN 2011-02-25 06:26:57.411000 467385.039845 2
81 NaN 2011-05-03 10:17:08.541000 300136.624519 5

Unnamed: 4 Unnamed: 5 Unnamed: 6 Unnamed: 7 Unnamed: 8 Unnamed: 9 \
0 NaN NaN NaN NaN NaN NaN
1 NaN NaN NaN NaN NaN NaN
2 NaN NaN NaN NaN NaN NaN
3 NaN NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN NaN
..
77 NaN NaN NaN NaN NaN NaN
78 NaN NaN NaN NaN NaN NaN
79 NaN NaN NaN NaN NaN NaN
80 NaN NaN NaN NaN NaN NaN
81 NaN NaN NaN NaN NaN NaN
```

En el ejemplo anterior hemos cargado un documento con formato .xls a una estructura DataFrame de Pandas, en donde podemos tratarlo de forma adecuada para obtener los datos que necesitamos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

De esta misma forma, podemos cargar datos desde un archivo HDF5, ya que estos ficheros, como los de XML son autodescriptivos pueden contener grandes cantidades de datos con relaciones complejas. Como casi siempre, pandas viene a resolvernó los problemas:

```
import pandas as pd

df = pd.read_hdf('train.h5')
```

De igual manera podemos cargar archivos comprimidos en .zip o imágenes y hasta ficheros en formato pdf.

Bueno, hasta aquí hemos visto lo referente a los tipos de archivos y formatos de datos más usados en aprendizaje automático y que de seguro usaran en su día a día, sin embargo no es solo de ficheros o archivos desde donde provienen los datos que necesitamos en nuestro trabajo, también en muchas ocasiones los datos en tablas de bases de datos relacionales o documentos no sql, almacenados en servidores o en la nube.

Ahora, como lo hicimos con la manera que se cargan los datos desde archivos, haremos un recorrido de cómo podemos cargar los datos desde los sistemas de gestión de bases de datos más conocidos.

MySQL Server es uno de los sistemas de gestión de bases de datos relacionales más usados y populares. Una de las tantas formas es usando mysql.connector

```
import mysql.connector
conexion=mysql.connector.connect(host="localhost",
                                user="root",
                                passwd="",
                                database="Test_sch")

c_cursor=conexion.cursor()
c_cursor.execute("insert into testTable (id,nombre) values(200,'Evqueni');")
c_cursor.execute("select * from testTable")
for fila in c_cursor:
    print(fila)
conexion.close()

(200, 'Evqueni')
```


Tema 2. Análisis exploratorio de datos y preprocesamiento

Figura 16. Conectando con MySQL desde Python, usando mysql.connector.

Otro de los Sistemas de Gestión de Bases de Datos relacionales muy usado es el SQL Server de Microsoft. En esta parte mostramos como se puede ejecutar una consulta en un servidor de este tipo. En este caso haciendo uso de ODBC, por medio de pyodbc.

```
import pandas as pd
import pyodbc
server = 'tcp:localhost'
database = 'testDATA'
username = 'sa'
password = 'sasa'
cnxn = pyodbc.connect('DRIVER={ODBC Driver 17 for SQL Server};\
SERVER='+server+';DATABASE='+database+';UID='+username+';PWD='+ password)
cursor = cnxn.cursor()
#cursor.execute("INSERT INTO testTable (id,nombre) VALUES(101,'E');")
cursor.execute("SELECT id, nombre from testTable;")
row = cursor.fetchone()
while row:
    print(row)
    row = cursor.fetchone()

(100, 'E')
(101, 'E')
```

Figura 17. Conectando con MS SQL Server desde Python, usando pyodbc.

Otra de las fuentes comunes de datos y que cada vez gana más admiradores es MongoDB, de la familia de los sistemas noSQL. En la siguiente figura se muestra un ejemplo de cómo se efectúa dicha conexión.

```
import pymongo
from pymongo import MongoClient
# conexión
con = MongoClient('localhost',27017)
db = con['testDATA']
for i in range(20):
    db['testCollection'].insert_one({'identificador':i, 'nombre':'step_'+str(i)})
records = db['testCollection'].find()
print(list(records))
```

Figura 18. Conectando con MongoDB desde Python, usando pymongo.

Hasta aquí hemos visto, como conectarse y recuperar datos de tres tipos diferentes de Sistemas de Gestión de Bases de Datos. Una vez que los datos son importados,

Tema 2. Análisis exploratorio de datos y preprocesamiento

es posible procesarlo con pandas.

En esta etapa pueden eliminarse rasgos o añadirse o se pueden eliminar casos que se encuentren duplicados o añadir si se considera adecuado.

Siempre es importante destacar que este proceso de Análisis Exploratorio de Datos y sus etapas no son rígidas y tiene que verse más como un proceso iterativo. En este proceso se avanza y se retrocede de una fase a otro en la medida que la adaptación de los datos a las tareas de aprendizaje automático lo requieran.

Análisis exploratorio de datos, etapa #2: realización de una inspección general del conjunto de datos.

Aunque esta tarea de inspeccionar el conjunto de datos puede hacerse de manera manual, inspeccionando las variables una a una, nosotros mostraremos dos métodos que generan un perfil del conjunto de datos que aportan los gráficos y la información que de otra manera obtendríamos de forma más elemental.

El primer método que usaremos es el de generar un informe o reporte completo sobre el dataset que estemos trabajando por medio del uso del módulo sweetviz. Veamos el ejemplo y luego comentemos los detalles de la información que genera.

```
import pandas as pd
import sweetviz as sv
#*****
diabetesDF = pd.read_csv("diabetes.csv")
#*****
Report = sv.analyze(diabetesDF)
Report.show_html('diabetes_report.html')
```

Figura 19. Código para generar un informe en Html de un dataset usando SweetViz.

Como puede verse es sumamente sencillo y brinda muchísima información de todas las variables de dataset sin necesidad de ir mirándolos uno a uno. En la siguiente figura veremos parte del informe.

Tema 2. Análisis exploratorio de datos y preprocesamiento



Figura 20. Reporte en Html generado por el módulo SweetViz, del dataset diabetes.

En la parte central del reporte, se muestran los detalles generales del dataset, como la cantidad de casos, que para este en particular, son 768. Muestra la cantidad de rasgos y si tiene duplicados o no. En la parte izquierda, muestra cada uno de los rasgos del dataset junto a si histograma. Cuando se pone el puntero del mouse sobre uno de los rasgos se abre de forma inmediata a la derecha los detalles de este.

Cuando se pincha sobre el botón que se encuentra en la parte superior central que dice Associations, se muestra la matriz de correlación de estas variables, calculados usando el coeficiente de correlación de Pearson.

En la siguiente imagen se muestra la matriz.

Tema 2. Análisis exploratorio de datos y preprocesamiento

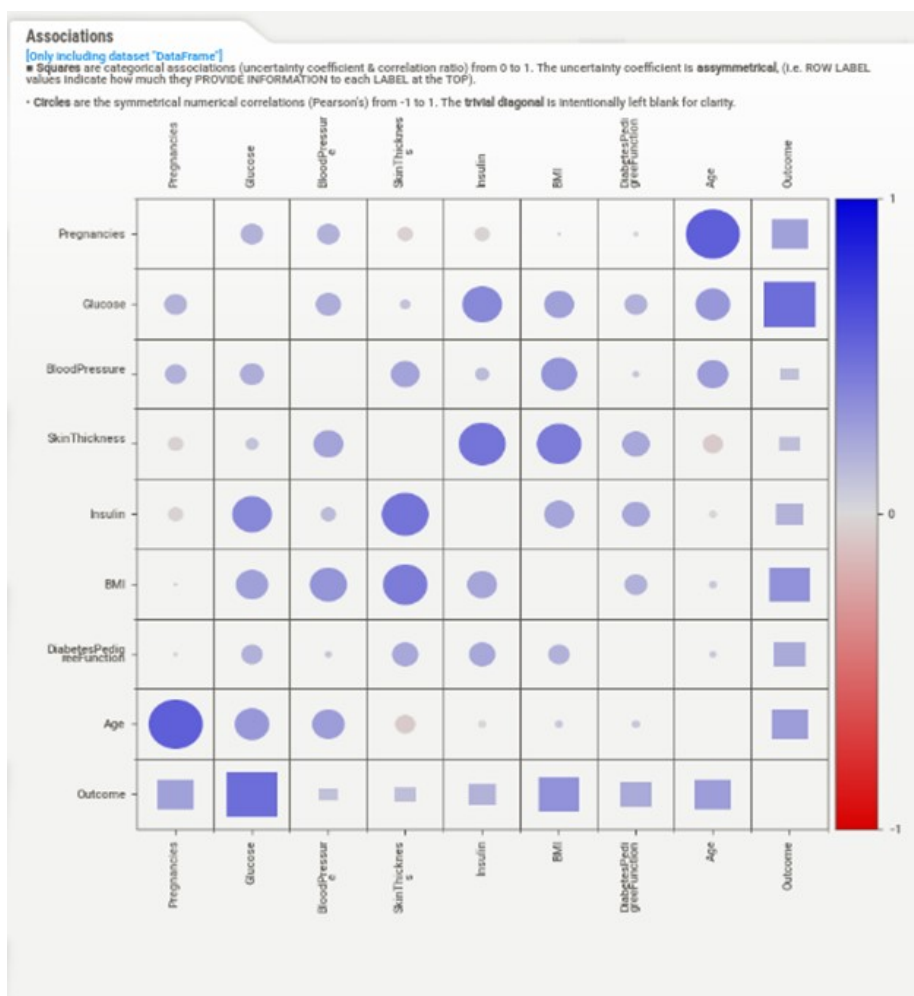


Figura 21. Matriz de correlaciones generada por el módulo SweetViz, usando el coeficiente de correlación de Pearson.

El segundo método que se puede usar, puede que sean muchos, es usando el módulo `pandas_profiling`. Veamos cómo se ejecuta el reporte en código y luego analicemos el resultado.

Este reporte es mucho más completo que el anterior y está compuesto por seis u ocho secciones que resumen diferentes aspectos del dataset bajo análisis.

Tema 2. Análisis exploratorio de datos y preprocesamiento



Figura 22. Código para generar un reporte del perfil del conjunto de datos que se analiza, usando el módulo `pandas_profiling`.

En las siguientes imágenes veremos cada una de las secciones en que está dividido este reporte y que información nos aporta. Las secciones en las que se divide este reporte son las siguientes:

- ▶ Overview.
- ▶ Variables.
- ▶ Interacciones.
- ▶ Correlaciones.
- ▶ Missing Values.
- ▶ Samples (Divida a su vez en First Rows y Last Rows).

Veamos la imagen y luego comentemos.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Overview Warnings 15 Reproduction		
Dataset statistics		
Number of variables	9	
Number of observations	768	
Missing cells	0	
Missing cells (%)	0.0%	
Duplicate rows	0	
Duplicate rows (%)	0.0%	
Total size in memory	54.1 KiB	
Average record size in memory	72.2 B	
Variable types		
Numeric	8	
Categorical	1	

Figura 23. Sección de Overview de un reporte generado por pandas_profiling.

Esta sección de reporte nos brinda una panorámica general del dataset que investigamos. Tiene tres subsecciones, aunque la más informativa es la que está destacada en azul.

Lo interesante de esta parte es que nos dice, la cantidad de casos, la cantidad de rasgos y el tipo de variable. Por ejemplo, este dataset tiene 9 variables, 8 numéricas y 1 categórica. Además nos dice que no tiene casos duplicados, etc.

Sección de variables:

En esta sección se incluye un perfil completo de cada una de las variables o rasgos del dataset.



Tema 2. Análisis exploratorio de datos y preprocesamiento

Figura 24. Sección de Variables de un reporte generado por pandas_profiling.

Incluye a la derecha de cada una de las secciones de las variables, un botón que de oprimirse se abre, debajo de esta, los detalles de cada una de las variables.

Esta sección nos permite observar, por ejemplo, que la variable *Pregnancies*, tiene 111 ceros. Si tenemos en cuenta la naturaleza de esta variable, que cuanta la cantidad de veces que una paciente ha estado embarazada, no es un problema y eso solo significa que no lo ha estado.

Sin embargo, en la siguiente variable, podemos ver que tiene cinco ceros. Esta variable mide la cantidad de glucosa en sangre y es imposible que tome valores cero y de hecho ya sabemos que aquí hay problema a solucionar. Estas son los tipos de problemas que debemos solucionar en esta etapa, aplicando las técnicas que ya hemos visto.

En la sección de interacciones, se muestran los scatterplot entre cada una de las variables del dataset.

En la siguiente imagen se muestra el grafico scatter plot entre las variables *BloodPressure* y *Insulin* del conjunto.

Tema 2. Análisis exploratorio de datos y preprocesamiento



Figura 25. Sección de Interactions de un reporte generado por pandas_profiling.

Información adicional de cómo usar Scatter Plot en Python puede encontrarse en Python DataScience Handbook.

La sección correlaciones muestra las correlaciones entre las variables del dataset usando varios coeficientes: Coeficiente de correlación de Pearson, el de Spearman, Kendall y Phik.

En estas matrices de correlaciones, se usan colores para denotar la fortaleza de estas. Un azul fuerte es indicativo de una correlación fuerte positiva, mientras que los colores rojos, denotan una correlación fuerte negativa. Cambiando de pestaña accedemos a la matriz de correlación de cada uno de los coeficientes. Este tipo de matriz es muy interesante y fácil de explorar cuando la cantidad de rasgos son relativamente pocos; pero cuando son muchos se hace difícil.

Tema 2. Análisis exploratorio de datos y preprocesamiento

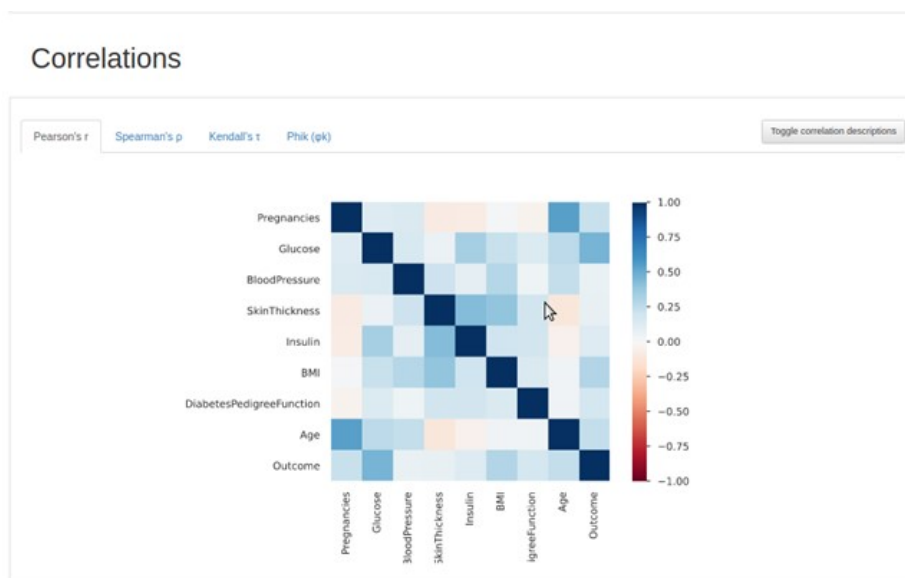


Figura 26. Sección de Correlaciones de un reporte generado por pandas_profiling.

Aunque estos tipos de reportes son visualmente atractivos y aportan mucha información, la manera más fácil y rápida de obtener un resumen informativo de un dataset es por medio de `DataFrame.describe()`.

Estamos asumiendo que el dataset está almacenado en una estructura `DataFrame` de Pandas, que una de las formas más comunes cuando analizamos datos con Python. Veamos una imagen que muestra el resultado de llamar el método `describe()` de un dataframe que almacena el dataset Pima Indian Diabetes.

```
import pandas as pd
pima_diabetesDF = pd.read_csv("diabetes.csv")
pima_diabetesDF.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Figura 27. Resultado de aplicar `describe` a un `DataFrame` de Pandas.

Tema 2. Análisis exploratorio de datos y preprocesamiento

Este método retorna la cantidad de casos en cada rasgo, la media, la desviación estándar, los valores mínimo y máximo del rasgo y el 1er, 2do y 3er cuartil.

Con esta información a mano, podemos identificar posibles anomalías en los datos que integramos y tratarlo de forma adecuada aplicando las técnicas que hemos visto a lo largo de esta lección. Si analizamos, por ejemplo, la variable Outcome a partir del reporte obtenido por medio de profiling, nos damos cuenta a simple vista que las clases están desbalanceadas. Veamos la imagen de la variable en el reporte.



Figura 28. Variable de decisión Outcome del Pima Indian Diabetes dataset.

Queda claro que el 65.1% de los casos, pertenecen a pacientes confirmados no diabéticos y el 34.9% a los que sí se sabe que lo son. La idea que pretendemos transmitir es que, es en la etapa de Análisis Exploratorio, donde se comienza a manosear, por decirlo de alguna forma, los datos y por medio de un proceso iterativo, resolver los problemas que se presentan en el dataset. Es importante subrayar que la idea, básica, de todo este proceso es adaptar los datos a los requerimientos de los métodos de aprendizaje que utilizaremos.

Es en la próxima lección donde comenzaremos a ver los detalles de cada uno de los métodos.

Conclusiones

Tema 2. Análisis exploratorio de datos y preprocesamiento

En esta lección hemos tratado de explicar, aunque en modo alguno muy resumido, las técnicas más importantes y conceptos que forman la base del análisis exploratorio de datos y sus etapas. También hemos visto la manera de realizar algunas tareas que son vitales para la adaptación de los datasets a los métodos de aprendizaje.

Es sumamente importante que se tenga en cuenta que el Análisis Exploratorio de Datos es un tema basto y lleno de conceptos que es imposible que cubramos en un curso de estas características y por lo vital de su entendimiento es necesario que consulten toda la literatura extra que les sea posible.

Tema 3. Algoritmos de aprendizaje automático supervisado

3.1. Introducción y objetivos

Esta unidad formativa está compuesta por dos lecciones que comprenden lo referente a los algoritmos y modelos de aprendizaje supervisado.

En la primera lección se inicia con una introducción a la regresión para luego pasar a la discusión de los conceptos de bias/varianza. Después de discutir este medular asunto se para a otro concepto no menos importante, el de validación cruzada para entonces analizar las métricas de evaluación de los modelos, tanto de regresión como de clasificación y por ultimo de analizan los principales algoritmos de regresión.

En la segunda lección se tratan los principales algoritmos de clasificación. Se tratan los algoritmos KNN, SVM, naives bayes, Arboles de decisión, random forest y por ultimo ensamblados de clasificadores.

En las lecciones anteriores revisamos todo lo correspondiente a la etapa de preprocesamiento de los conjuntos de datos con el fin de adaptarlos a los métodos de aprendizaje automático que necesitamos para resolver una tarea determinada. En esta lección y la siguiente estaremos revisando lo correspondiente a los algoritmos de aprendizaje automático y las métricas para su evaluación. Creemos necesario definir dichas métricas antes de comenzar el estudio mismo de los algoritmos pues entender como los evaluamos hará mucho más fácil entender cómo funcionan.

Por tanto los objetivos de esta lección serán: Definir aprendizaje supervisado por regresión y clasificación, definir los conceptos de Bias y Varianza y describir el compromiso bias/varianza, definir qué se entiende por validación cruzada, las métricas usadas para evaluar los modelos de regresión y clasificación y por ultimo describir los algoritmos más comunes.

Tema 3. Algoritmos de aprendizaje automático supervisado

Cuando termine estas dos lecciones usted conocerá los algoritmos de aprendizaje supervisado más usados, como se evalúan y cómo se aplican a problemas en la práctica.

Los objetivos que se perseguirán en este tema son:

- ▶ Conocer en que consiste la regresión y su diferencia respecto a la clasificación.
- ▶ Conocer los conceptos de bias y varianza de un modelo de aprendizaje automático.
- ▶ Conocer y aplicar el concepto de validación cruzada.
- ▶ Conocer y aplicar las métricas de evaluación de los modelos de regresión, así como los de clasificación.
- ▶ Conocer y aplicar algunos de los modelos de regresión más importantes.
- ▶ Conocer y aplicar algunos de los modelos de aprendizaje supervisado de clasificación más importantes, como KNN, SVM, entre otros.

Tema 3. Algoritmos de aprendizaje automático supervisado

3.2. Algoritmos de aprendizaje automático supervisado I

Desde la primera lección hemos estado hablando de manera muy informal de que el aprendizaje automático se divide en tres grandes grupos: Aprendizaje supervisado, el no supervisado y el aprendizaje con reforzamiento. En esta lección nos concentraremos en lo correspondiente al aprendizaje supervisado y sus dos tareas, regresión y clasificación. Para iniciar, estableceremos el problema del aprendizaje supervisado, revisaremos los conceptos de bias y varianza, luego revisaremos los métodos de evaluar los modelos de regresión y clasificación y por último veremos algunos de los algoritmos.

Desde las primeras lecciones cuando comenzamos a estudiar el problema del aprendizaje automático dejamos claro que los algoritmos de aprendizaje automático aprendían a partir de una experiencia que se daba en forma de un conjunto de datos. Este conjunto de datos le llamábamos dataset y estaba compuesto por casos y que estos casos estaban compuestos por rasgos o variables y que estos podrían ser clasificados según su función en predictores y variable predicha. Sabemos además que estos rasgos podían ser cuantitativos o cualitativos.

De manera general, el problema del aprendizaje, se resume en aprender un mapeo o función entre las variables predictoras, que en lo adelante llamaremos x y una variable predicha la cual llamaremos y .

Con estos elementos podemos establecer nuestra primera definición.

Definición #1: Aprendizaje supervisado

Se le llama aprendizaje automático supervisado al problema de aprender un mapeo entre el conjunto de variables predictoras x y la variable predicha y , del conjunto de datos D , tal que:

Tema 3. Algoritmos de aprendizaje automático supervisado

$$h_D: X \rightarrow Y$$

Definición #2: Regresión

Se le llama Regresión al problema de aprender un mapeo $h_D: X \rightarrow Y$, tal que la variable Y es una variable cuantitativa. (Numérica)

Definición #3: Clasificación

Se le llama Clasificación al problema de aprender un mapeo $h_D: X \rightarrow Y$, tal que la variable Y es una variable cualitativa.

Estas definiciones no tratan de ser matemáticamente precisas, solo pretenden de alguna manera desarrollar la intuición manteniendo al mínimo la complejidad matemática.

Podemos esclarecer un tanto estos conceptos si examinamos algunos de los conjuntos de datos que se usan a la hora de evaluar los modelos generados por los algoritmos de aprendizaje.

Revisemos el dataset Pima Indian Diabetes, que hemos estado usando en las lecciones anteriores. Este dataset tiene, 9 variables y 768 casos. Las variables predictoras, todas son variables numéricas, mientras que la variable predicha, la cual se llama Outcome, solo toma valores 0 y 1. Si nos atenemos a la definición #3 queda claro que este dataset pertenece a una tarea de clasificación. Específicamente con este dataset lo que se pretende es encontrar un mapeo h_D que clasifique los pacientes o casos en diabéticos o no a partir de los 8 rasgos predictores: Insulina en sangre, niveles de glucosa medidas, presión sanguínea, edad, etc.

Por el contrario, si revisamos el dataset de ventas de video juegos, vgsales.csv, este contiene 11 rasgos, 5 de ellos son cadenas de caracteres, 6 son numéricos. Si quisiéramos predecir las ventas globales de juegos para una de las plataformas determinadas, entonces estaríamos desarrollando una tarea de regresión, pues la

Tema 3. Algoritmos de aprendizaje automático supervisado

variable `Global_sales`, es una variable numérica.

Introducción a la regresión

La forma que toma el mapeo hD en el caso de una regresión lineal simple es el siguiente:

$$y = \beta_1 x_i + \beta_0 + \varepsilon$$

Donde y es la variable dependiente o predicha, x los rasgos del dataset o variables independientes o predictores, β_0 se le conoce como la intersección y β_1 el coeficiente de la regresión. Mientras que $\varepsilon \sim N(\mu, \sigma)$ es el error normalmente distribuido.

De esta forma, para predecir el valor de y a partir del valor de x tenemos que estimar los valores de β_0 y β_1 . La forma más simple de hacerlo la propuso Gauss en 1809 y se llama Mínimos Cuadrados.

Veamos en que consiste este método y como se estiman los parámetros a partir de los datos. Con este método β_1 se calcula de la siguiente forma:

$$\beta_1 = \frac{Cov(x, y)}{S_x^2} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Mientras que β_0 se estima de la siguiente forma: $\beta_0 = \bar{y} - \hat{\beta}_1 \bar{x}$

Hagamos un ejemplo muy sencillo para demostrar cómo funciona. Lo primero es revisar los datos y sobre los cuales entrenaremos nuestro algoritmo y luego haremos predicciones y veremos como evaluamos el error. Claro, esto lo haremos con una sola variable para simplificar todos los cálculos.

Es importante notar que no hemos hecho ninguna asunción desde el punto de vista estadístico y que nos hemos centrado en exponer los elementos básicos.

Supongamos que contamos con los datos de la producción de trigo y harina de

Tema 3. Algoritmos de aprendizaje automático supervisado

España:

Producción de trigo (Tonelada)	30	28	32	25	25	25	22	24	35	40
Precio de la Harina (Euros)	25	30	27	40	42	40	50	45	30	25

Con estos datos vamos a estimar los valores de la ecuación de regresión.

$$\widehat{\beta}_1 = \frac{\sum_{i=1}^{10} x_i y_i - n \bar{x} \bar{y}}{\sum_{i=1}^{10} x_i^2 - n \bar{x}^2} = \frac{9734 - 10 * 28,6 * 35,4}{8468 - 10 * (28,6)^2} = -1,3537$$

Ya tenemos el estimado del primer parámetro, hagamos la estimación del segundo.

$$\beta_0 = \bar{y} - \widehat{\beta}_1 \bar{x} = 35,4 + 13537 * 28,6 = 74,116$$

De esta forma, nuestra ecuación de regresión toma la forma siguiente:

$$\widehat{y} = 74,115 - 1,3537 * x$$

Con esta ecuación obtenida de los datos, podemos comenzar a predecir los valores de la harina a partir de valores la cantidad de trigo producido. Es a este proceso de calcular los valores de los estimadores a lo que se le llama proceso de entrenamiento y la ecuación que resulta es a lo que se llama modelo.

Hagamos un par de ejemplos y tratemos de predecir algunos valores con los que se entrenó, sustituyendo x en ella.

Tomemos a x=30 entonces y luego uno a uno el resto de los valores:

$$\widehat{y} = 74,115 - 1,3537 * 30 = 33,5$$

Ahora bien, como evaluamos si la función que generamos está correcta o se desvía demasiado de lo que queremos.

Tomemos todos los valores de la tabla y añadámosle los valores predichos y el error.

Tema 3. Algoritmos de aprendizaje automático supervisado

Producción de trigo (Tonelada)	30	28	32	25	25	25	22	24	35	40
Precio de la Harina(Euros)	25	30	27	40	42	40	50	45	30	25
Predicción	33.5	36.21	30.79	40.27	40.27	40.27	44.33	41.62	26.73	19.96
Residuo	-8,50	-6.21	-3.79	-0.27	1.72	-0.27	5.66	3.37	3.26	5.03

Después de calcular los residuales, el valor que nos queda:

$$S^2 = \frac{\sum_{i=1}^{10} (y - \hat{y})^2}{n - 2} = \frac{207,6}{10 - 2} = 25,95$$

Este 25,95 de alguna forma caracteriza la calidad de nuestra función.

Acabamos de hacer un breve recorrido por el entrenamiento de un modelo de aprendizaje o la construcción de un modelo de predicción. Con este ejemplo entendido es necesario ir introduciendo otros conceptos que son necesarios para ir descubriendo aspectos del aprendizaje automático que serán de gran ayuda en el futuro.

Sin embargo es instructivo que hagamos este mismo ejemplo usando las librerías de aprendizaje automático de Python con el fin de comprobar si nuestro procedimiento ha sido el correcto. Veamos este ejemplo:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import pearsonr
from sklearn.linear_model import LinearRegression
#*****
ProduccionTrigo = [30,28,32,25,25,25,22,24,35,40]
PrecioHarina = [25,30,27,40,42,40,50,45,30,25]
d={'ProduccionTrigo':ProduccionTrigo,'PrecioHarina':PrecioHarina}
#*****
regresionDF = pd.DataFrame(d)
X=regresionDF['ProduccionTrigo'].values.reshape(-1,1)
Y=regresionDF['PrecioHarina'].values
regress=LinearRegression()
regress.fit(X,Y)
print("Intercept:", regress.intercept_)
print("Coeficiente:", regress.coef_)
print("R-Cuadrado:", regress.score(X,Y))
plt.plot(X, (74.115-1.3537*X), color='red')
plt.scatter(X,Y)
plt.xlabel("Producción de Trigo(Toneladas)")
plt.ylabel("Precio de la harina (euro)")
plt.title("Producción de trigo vs precio de la harina")
plt.show()

Intercept: 74.11511789181691
Coeficiente: [-1.35367545]
R-Cuadrado: 0.7176465181664973
```

Figura 1. Ejemplo de regresión lineal, la cual ya habíamos calculado a mano.

Tema 3. Algoritmos de aprendizaje automático supervisado

Pues miremos las últimas líneas de la imagen anterior y ups, los mismos valores de intersección y la pendiente que obtuvimos en nuestros cálculos manuales.

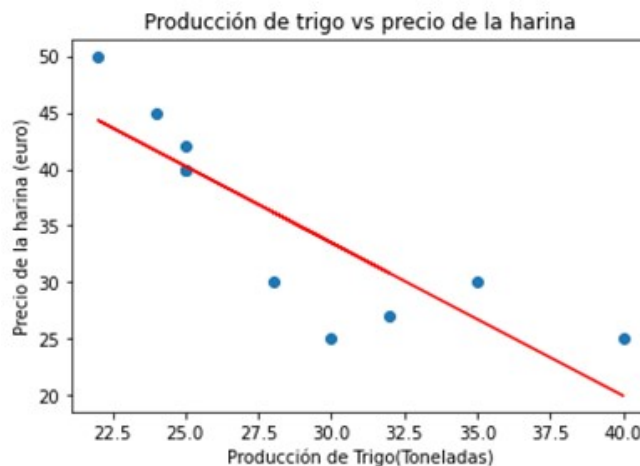


Figura 2. Gráfico resultante de la regresión que calculamos a mano. La línea roja es la $y=74,115-1,3537x$, llamada línea de regresión.

Bien, conociendo este tipo de modelo simple y sabiendo de primera mano que es lo que se hace durante el proceso de entrenamiento o de ajuste del modelo, podemos introducir conceptos que pensamos que son imprescindibles para continuar con el aprendizaje del uso de los métodos de aprendizaje automático.

Introducción de los conceptos de Bias, Varianza y compromiso Bias/Varianza de los modelos de aprendizaje automático

Para entender que es el bias o sesgo de un modelo de aprendizaje automático es necesario que introduzcamos varias ideas que pueden parecer complejas; pero trataremos de introducirlas haciéndolas de alguna manera entendible.

Establezcamos el problema del aprendizaje desde una perspectiva más matemática. Supongamos que necesitamos predecir el tiempo de gestación de las pacientes de un hospital. Ponemos este ejemplo, por el simple hecho de que el tiempo de gestación es un proceso natural del cual no tenemos control y es determinado por factores que no conocemos. Por tanto los tiempo de gestación dependen de factores

Tema 3. Algoritmos de aprendizaje automático supervisado

desconocidos y queremos ver qué relación tiene con medidas tales como: la edad de la gestante, si es primeriza o no, etc. A todas gestantes del mundo se le llama en estadística La población y denotaremos por P . Pero como no podemos ir una a una y seguirla para saber el tiempo de gestación, porque es imposible para nosotros, entonces la damos como desconocida.

Para poder realizar el estudio, tomamos una cantidad S de gestantes. Estas gestantes se toman de manera aleatoria. Al grupo de estas gestantes se les denomina Muestra. Entonces el problema sería: predecir el tiempo de gestación que llamaremos y , a partir de un conjunto de rasgos, que llamaremos x tomados de una muestra que llamamos S .

$h_s: X \rightarrow y$ Entonces, se trata de encontrar un mapeo o función que relacione la variable y o tiempo de gestación, con los rasgos X , que son la edad, si fuma, hemoglobina, entre otros. Todos los datos que se recopilan con estas gestantes se tabulan y se conforma un dataset. Que lo usaremos para entrenar nuestros modelos.

Matemáticamente, este mapeo toma la forma siguiente:

$$h_s: X \rightarrow y$$

Es a esta h_s , es a la que le llamamos función hipótesis y se escoge de entre varias disponibles para un método determinado, a esta clase de hipótesis la llamamos H . Revisemos las asunciones iniciales que hemos hecho hasta aquí:

- ▶ Hemos supuesto que las chicas que escogimos todas son normales.
- ▶ Que las hemos escogido de manera aleatoria.
- ▶ Que todas provengan de la misma población.
- ▶ Asunciones sobre los rasgos:
- ▶ Que los rasgos son representativos de lo que queremos medir.

Tema 3. Algoritmos de aprendizaje automático supervisado

- Que los casos de la muestra representan bien a la población.

Con todos estos elementos formulemos el problema:

La tarea es aprender a predecir la salida $y \in Y$ a partir de un conjunto de pares (x,y) que pertenecen a una distribución conjunta D . Para nosotros desconocida. Este problema consiste en aprender una función o mapeo. A partir de un dataset de entrenamiento, finito S , compuestos por m muestras de D , estas m muestras deben estar Independiente e idénticamente distribuidas.

A $h_s \in H$, la función aprendida, también se le conoce como hipótesis, la cual es elegida del conjunto de hipótesis o funciones que soporta el modelo. Para evaluar la calidad de lo aprendido por medio de la hipótesis $h_s \in H$ se usa una función llamada normalmente Riesgo o pérdida.

La función de esta función de pérdida o riesgo es cuantificar de alguna forma el error entre lo aprendido por la función o mapeo y el proceso real que generó el dato. En nuestro caso, la función de pérdida nos diría el error esperado entre los valores del tiempo de gestación de las gestantes y el valor real si conociéramos el proceso natural que decide este tiempo. Pero este proceso natural no lo conocemos. Por tanto, la única cosa que podemos hacer es usar el conjunto de entrenamiento con el fin de evaluar dicho error.

El objetivo, entonces, de los métodos de aprendizaje supervisado es minimizar el riesgo: $\min R(h_s) = E(x,y) \sim l(h(x), y)$.

Esto conduce a lo que se conoce como minimización del riesgo empírico donde se pretende aprender la hipótesis h minimizando $R(h)$. Ahora bien, no basta que la función aprendida h , prediga correctamente los casos que se usaron para derivarla, es necesario que esta hipótesis sea capaz de predecir correctamente casos que no estaban dentro de conjunto de entrenamiento. Es decir, si tomamos cualquier gestante que no esté dentro del grupo que elegimos con el fin de calcular la h , ¿esta

Tema 3. Algoritmos de aprendizaje automático supervisado

función sería capaz de predecir el tiempo de gestación de forma correcta?

Es a esta cualidad de h a la que se le denomina capacidad de generalización. Pero ¿qué puede ir mal cuando minimizamos el riesgo basado en el conjunto de entrenamiento en vez de usar el conjunto de todas las gestantes del mundo?

Aquí pueden pasar dos cosas, una que la h aprenda y memorice tan detalladamente los casos del conjunto de entrenamiento que luego no pueda generalizar de forma correcta y por otro lado puede pasar que h no aprenda muy bien y por tanto tampoco pueda.

Veamos esto en más detalles, veamos las imágenes siguientes:

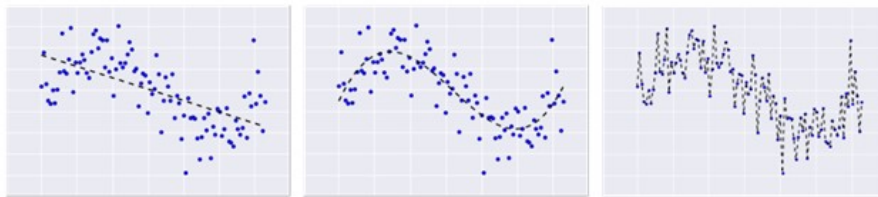


Figura 3. En la izquierda la función h no se ajusta a los datos, en el centro se nota algún ajuste, en la derecha la función se sobreajusta a los datos. Imágenes tomadas de On the Bias-Variance Tradeoff:

Textbooks Need an Update. Brady Neal.

En el caso de la imagen de la izquierda la complejidad de la función h , no permite que se ajuste a la complejidad de los datos y esto provoca que cometa muchos errores tanto en entrenamiento como en generalización. A esto se le conoce como: Bajo Ajuste, desajuste o UnderFitting.

En el caso de la imagen del centro, la función h , al parecer se ajusta a los datos, arrojando una baja cantidad de errores en el conjunto de entrenamiento y relativamente bajos errores en un conjunto de prueba de la generalización. A esto se le conoce como ajuste o se dice que la hipótesis se ajusta a los datos.

En la imagen de la derecha la hipótesis h , se ajusta demasiado a los datos, incluso a

Tema 3. Algoritmos de aprendizaje automático supervisado

los ruidos, provocando que prácticamente arroje cero errores en los conjuntos de entrenamiento, pero un elevado número de errores en los conjuntos de prueba de generalización. A esta situación se le llama Overfitting o sobreajuste de los datos.

Entonces podemos definir ahora que se entiende por bias y varianza:

Se define como Bias o sesgo, a la diferencia entre el valor esperado de una predicción hecha por nuestra hipótesis h de cualquier valor x , no visto anteriormente por ella y la verdadera función generadora de los casos.

Matemáticamente la representaremos así:

$$Bias = E(h_s(X)) - f(X)$$

Donde, h_s es la hipótesis que aprendimos usando el conjunto de entrenamiento s , E es el valor esperado y $f(x)$ es la función que obtendríamos si entrenáramos nuestros modelos con la población completa, cosa que es imposible.

En esencia el Bias de un modelo nos habla de cuan desviado anda el modelo de la realidad. El valor esperado de una variable aleatoria, por otro lado, es el promedio los valores que admite y su forma matemática es la que sigue:

$$E(x) = \sum_{i=1}^n x_i P(x_i) = \frac{1}{n} \sum_{i=1}^n x_i$$

Si la variable es continua se sustituye la sumatoria por una integral. $P(x_i)$ es la probabilidad de x_i .

La Varianza de una hipótesis h , es la medida que caracteriza la variación alrededor de la tendencia central de la hipótesis.

La forma matemática que toma es la siguiente:

$$Var(h_s) = E_s [(h_s(x) - E_s(h_s(x)))^2]$$

Tema 3. Algoritmos de aprendizaje automático supervisado

Hay un gráfico que puede que nos aclare de manera visual y nos permita desarrollar un tanto la intuición detrás de estos conceptos:

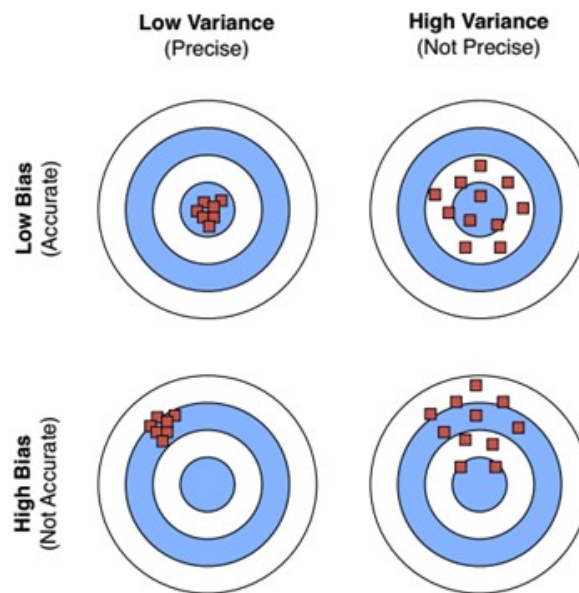


Figura 4. Gráfico para ayudar entender los conceptos de Bias y Varianza. Imágen tomada de STAT 479: Machine Learning Lecture Notes. Sebastian Raschka.

Los conceptos de Bias y varianza de los modelos de aprendizaje automático están también ligados a los conceptos de sobreajuste o overfitting y al de bajoajuste, desajuste o underfitting.

También están estrechamente ligado al concepto de complejidad de la hipótesis que escogemos para aprender y el de error. Históricamente se ha dicho que en la medida que escojamos hipótesis más complejas para un determinado problema corremos el riesgo de que el modelo incurra en overfitting y viceversa, si la complejidad de este no es suficiente pues corremos el riesgo de que incurra en underfitting. Esto nos lleva directamente a lo que se conoce como el compromiso Bias/Varianza.

Para ilustrar este tipo de compromiso, por favor vean la imagen en la Figura 5.

Tema 3. Algoritmos de aprendizaje automático supervisado

No vamos a profundizar aún más en estos elementos y creemos que se han aclarado los conceptos Bias, Varianza, over y under fitting, así como el de compromiso Bias-Varianza.

Lo práctico a sacar de aquí, grosso modo, es, ¿cómo sé que mi modelo adolece de overfitting?, bueno probablemente este arrojando un muy bajo valor de error en el conjunto de entrenamiento y un muy alto con el de prueba. ¿cómo sé que está haciendo underfitting? Lo normal en underfitting es que los valores de error sean grandes en ambos conjuntos.

Cuando el modelo genera valores de error relativamente bajos en ambos hay que pensar que vamos por el camino correcto.

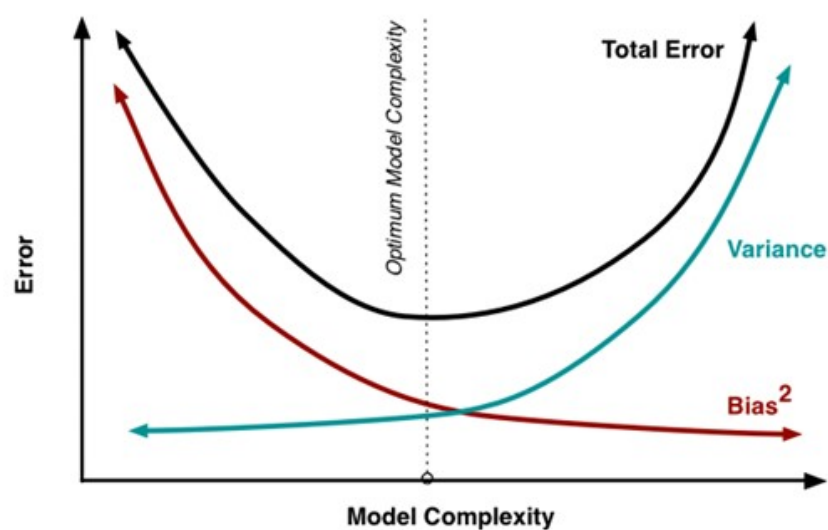


Figura 5. Compromiso Bias/Varianza. Imágenes tomadas de On the Bias-Variance Tradeoff :Textbooks
Need an Update. Brady Neal.

Con el fin de poder probar la capacidad de generalización de nuestros modelos, lo común, es que el conjunto de datos se divida en dos, una para el entrenamiento mientras que el otro se use para pruebas; pero es suficiente?

Antes de ver las métricas que usamos para evaluar cada uno de nuestros modelos

Tema 3. Algoritmos de aprendizaje automático supervisado

es necesario que aclaremos lo referente a la Validación Cruzada. ¿Qué es? Y ¿para qué se usa?

Tema 3. Algoritmos de aprendizaje automático supervisado

Validación cruzada

El objetivo que se persigue al entrenar los algoritmos de aprendizaje automático es obtener modelos que sean capaces de predecir de forma correcta casos que no estén incluidos en los que se usaron para su entrenamiento.

El primer instinto, a partir de lo que hemos ido aprendiendo, es dividir el dataset en dos, una parte, normalmente el 75% o 80% de los datos para entrenar y el resto para probar la capacidad de generalización del modelo. Aunque esto es correcto, tiene un par de inconvenientes: Al dejar una parte de los datos fuera perdemos información que puede ser vital para la construcción del modelo y como entrenamos y validamos sobre conjuntos fijos tendemos a sobre valorar la calidad de nuestros modelos, pues los valores de errores dependen de los conjuntos que elijamos. Si, separamos de nuevo los conjuntos y entrenamos de nuevo, probablemente obtendremos otros valores que puede que sean mejores o peores.

Para aliviar ese problema se desarrolló la técnica de validación cruzada. En este tipo de esquema el conjunto completo se usa para todo, es decir para entrenar, validar y probar. Aunque existen varios esquemas, solo explicaremos en llamado K-Fold.

Validación Cruzada usando K-Fold

En este esquema, el parámetro k define la subdivisión del conjunto de datos en entrenamiento y prueba y la cantidad de veces que se entrenará y probará el modelo. Esto quiere decir que este esquema divide el conjunto de entrenamiento en k partes y se entrenará y probará k veces nuestro modelo usando $k-1$ subdivisión del conjunto para entrenar y una para probar y al final los valores de errores obtenidos se promedian.

Tema 3. Algoritmos de aprendizaje automático supervisado



Figura 6. Validación cruzada con k=5.

En la imagen se denota con el color azul, aquellos subconjuntos que se usarán como conjuntos de prueba. Todos los errores de cada ciclo de entrenamiento/prueba se promedian y se dan como un solo valor que caracteriza la calidad del modelo.

Con estos conceptos a mano, podemos entender las métricas que se usan para medir la calidad de los modelos generados por los algoritmos de aprendizaje automático.

A continuación veremos las métricas correspondientes a los algoritmos de regresión y luego a los de clasificación.

Métricas para la evaluación de los algoritmos de regresión

Existen diferentes métricas de error que se pueden aplicar en un problema de regresión para obtener la función $f(x)$ ideal. Las más comunes o habituales y sus definiciones matemáticas son:

- **Error medio cuadrático o *mean square error* (MSE):** se define como la media de la diferencia entre el valor real y el valor predicho o estimado al cuadrado.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Tema 3. Algoritmos de aprendizaje automático supervisado

- **Error medio absoluto, *mean absolute error* (MAE):** se define como la diferencia en valor absoluto entre el valor real y el valor predicho.

$$MSE = \frac{1}{n} \sum_{i=1}^n |y_i - \widehat{y}_i|$$

- **Raíz del error cuadrático medio o *root mean square Error* (RMSE):** se define como la raíz cuadrada de la media de la diferencia entre el valor real y el valor predicho o estimado al cuadrado.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \widehat{y}_i)^2}$$

Esta métrica comparada con el error absoluto medio (MAE) amplifica y penaliza los errores grandes. Por otro lado, en MAE cada error contribuye al total del error en función de su valor absoluto.

- **Logaritmo de la raíz del error cuadrático medio, *root mean logarithmic square error* (RMLSE):**

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(y_i + 1) - \log(\widehat{y}_i + 1))^2}$$

Una de las mejores formas de evaluar un modelo de regresión es utilizando gráficos. Uno de los gráficos que más información proporcionan visualmente es un diagrama de dispersión de dos dimensiones donde el eje x son los valores reales y el eje y los valores predichos o estimados (o viceversa).

En el siguiente gráfico se muestra el gráfico de dispersión del ejemplo de la producción de trigo y precio de la harina.

Tema 3. Algoritmos de aprendizaje automático supervisado

Cada uno de los puntos corresponde a una de las estimaciones o predicciones realizadas. En el caso de que las predicciones o estimaciones fueran perfectas, caerían directamente sobre la diagonal del gráfico. Por otro lado, los puntos que caen por encima de la diagonal son sobre estimaciones de las predicciones, mientras que los puntos que caen por debajo de la diagonal son predicciones que se quedan cortas.

Una métrica común que se utiliza para medir la dispersión en este tipo de diagramas es el coeficiente de correlación, el cual cuantifica la relación lineal entre dos variables. Este coeficiente toma valores entre -1 y 1, donde 1 implica correlación lineal positiva de forma completa, -1 correlación lineal negativa de forma completa; y cuanto más cercano de 0 sea el valor menor correlación entre las dos variables.

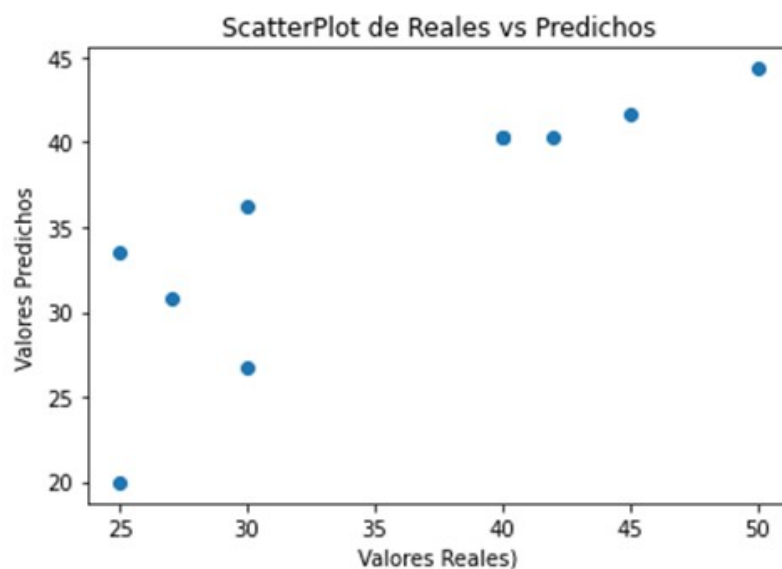


Figura 7. Gráfica de dispersión (ScatterPlot) de los valores predichos vs reales el precio de la harina en el ejemplo inicial de esta lección.

Una correlación lineal positiva implica que cuando el valor en el eje x crece, el valor en el eje y lo hace en la misma proporción. Por el contrario, una correlación lineal negativa implica que cuando el valor en el eje x crece el valor en y decrece de forma proporcional. Esta relación se define matemáticamente con la siguiente función.

Tema 3. Algoritmos de aprendizaje automático supervisado

Esta bondad de ajuste también se puede medir utilizando el coeficiente de determinación o R-Cuadrado. Este coeficiente indica la fracción de la variación explicada por la recta de regresión respecto a la variación total. Coincide con el cuadrado del coeficiente de correlación y puede tomar valores entre 0 y 1, siendo 1 el mejor ajuste y 0 el peor. En el ejemplo que hemos estado usando, este toma valor de 0.7176465181664973.

Métricas de evaluación de los algoritmos de clasificación

Dentro del aprendizaje supervisado, a continuación de los algoritmos de regresión, el siguiente gran grupo de algoritmos son los de clasificación.

A diferencia de los algoritmos de regresión cuyo objetivo es predecir un valor numérico, los algoritmos de clasificación tienen como objetivo obtener la clase más probable para cada una de las instancias.

Este tipo de técnicas pueden utilizarse para predecir o estimar la probabilidad de pertenencia a una clase de entre dos posibles, lo que se conoce como clasificación binaria. Por otro lado, también se pueden utilizar para predecir o estimar la probabilidad de pertenencia de una clase de entre varias posibles (más de dos), lo cual se conoce como clasificación multiclase.

A continuación, veremos las métricas de evaluación que se utilizan centrándonos en la matriz de confusión, precisión/recall, accuracy/sensitivity y f-measure.

Posteriormente, veremos las curvas ROC y la métrica de área bajo la curva.

Las métricas clásicas de evaluación de los clasificadores se extraen de la llamada Matriz de confusión. Veamos en detalle en que consiste esta matriz y para eso nos apoyaremos en la siguiente imagen:

Tema 3. Algoritmos de aprendizaje automático supervisado

		Predichos	
		Positivos	Negativos
Reales	Positivos	Verdaderos Positivos True Positive (TP)	Falsos Negativos False Negative (FN)
	Negativos	Falsos Positivos False Positive (FP)	Verdaderos Negativos True Negative (TN)

Figura 8. Matriz de Confusión.

Que significa cada uno de estos cuadrantes:

- ▶ True Positive (TP): Son los casos que en realidad son positivos y que el clasificador los clasificó como tal.
- ▶ True Negative (TN): Son los casos que en realidad son negativos y que el clasificador los clasificó de forma correcta.
- ▶ False Negative (FN): Son los casos que son positivos y el clasificador los clasifica como negativos.
- ▶ False Positive (FP): Son casos que en realidad son negativos y el clasificador los clasifica como positivos.

Veamos ahora las métricas que se derivan de esta matriz.

La primera de las métricas en la llamada Precisión o Accuracy está interpretado como el porcentaje total de casos clasificados correctamente. Esta medida toma valores en 0 y 1 y entre más alta es mejor.

Tema 3. Algoritmos de aprendizaje automático supervisado

Su interpretación matemática es la siguiente:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Es decir, esta medida son todos los clasificados correctamente contra el total. Aunque esta es una medida muy intuitiva y de las más fáciles de usar hay que tener cuidado con su uso.

Por ejemplo, si estamos tratando de predecir una enfermedad y las muestras de cada una de las clases está desbalanceada, esta medida nos puede arrojar una accuracy muy alta aunque sea completamente inútil. El problema es que esta métrica siempre favorece a la clase más poblada. En estos casos es aconsejable usar otras métricas.

Recall, también llamada sensibilidad:

$$Recall = \frac{TP}{TP + FN}$$

Precisión:

$$Precisión = \frac{TP}{TP + FP}$$

Especificidad:

$$Especificidad = \frac{TN}{TN + FP}$$

Medida F1:

$$F_1 Score = 2 \frac{precision * recall}{precision + recall}$$

Tasa de falsos positivos (*false positive rate*):

$$FalsePositiveRate (FPR) = \frac{FP}{TN + FP}$$

Tema 3. Algoritmos de aprendizaje automático supervisado

Tasa de verdaderos positivos (*true positive rate*):

$$\text{TruePositiveRate (TPR)} = \frac{TP}{TP + FN}$$

ROC: ROC es la curva Característica Operativa del Receptor (en inglés, Receiver Operating Curve), también conocida como ROC, es una representación gráfica de la sensibilidad frente a la especificidad según se varía el umbral. Es una curva que en el eje x se plotean los valores de False Positive Rate, mientras que en el eje y se plotean los valores correspondientes a True Positive Rate.

En la siguiente imagen se muestra un ejemplo de una curva ROC:

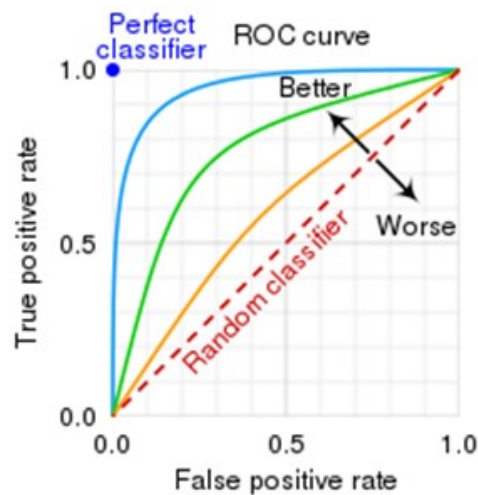


Figura 9. Forma de interpretar una gráfica ROC. Imagen tomada de Wikipedia.

Algoritmos de regresión

Como ya hemos comentado a lo largo de las lecciones anteriores la regresión es la tarea que pretende explicar el efecto que tienen las variables, llamadas independientes sobre la variable dependiente. Y también, se ha explicado que se da, cuando necesitamos predecir una variable que es continua en su naturaleza.

Si la dependencia de la variable a predecir u Outcome, es de una sola variable explicativa o independiente, entonces podemos hablar de una regresión lineal simple.

Tema 3. Algoritmos de aprendizaje automático supervisado

Si por el contrario, la variable dependiente, está relacionada con varias variables independientes o explicativas, nos podemos referir a este tipo de regresión como Regresión Lineal Múltiple.

Si la relación que se espera entre las variables dependientes e independientes es lineal, entonces la regresión se llama Lineal; pero si el tipo de relación que se espera no es de naturaleza lineal, entonces la regresión puede ser polinomial o multinomial.

Por tanto, hagamos una especie de clasificación de los métodos de regresión con el fin de entender su taxonomía. Si tenemos en cuenta la cantidad de variables relacionadas con la dependiente, entonces la regresión se puede clasificar como:

- ▶ Regresión lineal simple.
- ▶ Regresión lineal múltiple.

Si la relación que se espera entre variables dependientes e independientes es lineal o no, se pueden clasificar en:

- ▶ Regresión Lineal.
- ▶ Regresión Polinomial.

Hay muchas otras formas de clasificar los métodos de regresión si tomamos en cuenta uno u otro aspecto de sus características, en realidad hay muchos métodos de regresión y no es nuestro interés hacer una taxonomía cerrada de todos ellos. Solo destacar las principales ramas y las que acabamos de dar son las más notables.

A continuación haremos un listado de algunos de los métodos de regresión y una breve explicación de sus fundamentos, tampoco es nuestro interés hacer un listado exhaustivo de todos los métodos.

Tema 3. Algoritmos de aprendizaje automático supervisado

Algunos métodos de Regresión:

- ▶ Regresión Lineal.
- ▶ Ridge (Ridge Regression).
- ▶ Lasso.
- ▶ Elastic-Net.
- ▶ LAR (Least Angle Regression).
- ▶ OMP (Orthogonal Matching Pursuit).
- ▶ Bayesian Regression.
- ▶ Automatic Relevance Determination – ARD.
- ▶ Logistic regression.
- ▶ Generalized Linear Regression.
- ▶ Stochastic Gradient Descent – SGD.
- ▶ Passive Aggressive Algorithms.
- ▶ RANSAC: RANdom SAmple Consensus.
- ▶ Quantile Regression.
- ▶ Polynomial regression.

Los métodos que acabamos de listar, son modelos que implementados en el módulo `linear_model` de `scikit-learn`, Lo hemos hecho con toda intención, pues además de ser un paquete muy popular, les permite aplicarlo de manera inmediata a sus propios proyectos o simplemente coquetear con ellos para ganar en comprensión.

Tema 3. Algoritmos de aprendizaje automático supervisado

De manera general, los métodos de regresión esperan que la variable objetivo o dependiente sea explicada o dependa de una combinación

lineal de las variables independientes o rasgos. La notación matemática sería la siguiente:

$$\hat{y}(\beta, x) = \beta_0 + \beta_1 x_1 + \dots \beta_n x_n$$

A los $\beta_0, \beta_1, \dots, \beta_n$ se les llama coeficientes del modelo y son desconocidos. El método clásico de estimar estos coeficientes se llama Mínimos Cuadrados. La mayoría de las variantes de regresión se basan en variantes o métodos diferente de estimar estos coeficientes.

La idea detrás de los mínimos cuadrados es encontrar una combinación de estos coeficientes que minimice la Suma de los residuos cuadrados.

$$RSS = \sum_{i=1}^m (\hat{y} - y)^2$$

Regresión lineal

Este es el tipo de regresión con la comenzamos esta lección. Específicamente con la Regresión lineal simple, en la que la variable dependiente y está relacionada con una sola variable explicativa y se supone que la naturaleza de esta relación sea lineal.

La función de costo de este tipo de regresión es llamada Mínimos Cuadrados Ordinarios y la estimación de β_0, β_1 es como sigue:

$$\beta_1 = \frac{Cov(x, y)}{S_x^2} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Mientras que la intersección se calcula de la siguiente forma:

$$\beta_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Tema 3. Algoritmos de aprendizaje automático supervisado

En la Figura 1, se muestra un ejemplo de implementación de de una regresión lineal simple en Python.

La diferencia entre este tipo de regresión y la regresión lineal múltiple es la cantidad de predictores que se relacionan con la variable dependiente. Por tanto no la explicaremos.

Asunciones del modelo:

- ▶ Ausencia de multicolinealidad o autocorrelación entre las variables explicativas o rasgos.
- ▶ Los casos deben ser independientes e idénticamente distribuidos.
- ▶ No pueden haber Outliers.
- ▶ Entre otros.

Ridge regression

Este tipo de regresión resuelve alguno de los problemas de la regresión lineal y regresión lineal múltiple. Específicamente los de los mínimos cuadrados ordinarios. Especialmente en escenarios donde la variables están altamente correlacionadas.

En este tipo de regresión se modifica el método de los mínimos cuadrados ordinarios añadiendo un término de regularización llamado norma L2. Esto hace que se penalicen los coeficientes en dependencia de su tamaño.

La función de costo o pérdida de este tipo de regresión es:

$$Loss_{Ridge} = \sum_{i=1}^N (\hat{y} - y)^2 + \alpha \sum_{j=1}^m (\beta_j)^2$$

El parámetro a tener en cuenta en este tipo de regresión es el alfa.

Tema 3. Algoritmos de aprendizaje automático supervisado

Mientras mayor es el parámetro de complejidad alfa, el método se hace más robusto a la colinealidad. Si este alfa se hace igual a cero, el método se hace equivalente a una regresión lineal típica.

La forma de escoger el alfa que recomendamos es por medio de Validación Cruzada.

```
>>> import numpy as np
>>> from sklearn import linear_model
>>> reg = linear_model.RidgeCV(alphas=np.logspace(-6, 6, 13))
>>> reg.fit([[0, 0], [0, 0], [1, 1]], [0, .1, 1])
>>> RidgeCV(alphas=array([1.e-06, 1.e-05, 1.e-04, 1.e-03, 1.e-02, 1.e-01, 1.e+00, 1.e+01,
1.e+02, 1.e+03, 1.e+04, 1.e+05, 1.e+06]))
>>> reg.alpha_
0.01
```

Ejemplo de Ridge Regression en Python.

Lasso regression

Lasso significa **Least absolute shrinkage and selection operator** y fue introducido para mejorar la calidad de la predicción e interpretabilidad de los modelos de regresión. Al igual que Ridge introduce un término de regularización en su función de pérdida; pero en este caso en forma de norma L1.

$$Loss_{Lasso} = \sum_{i=1}^N (\hat{y} - y)^2 + \alpha \sum_{j=1}^m |\beta_j|$$

Al igual que Ridge, el cuándo alfa es cero, el modelo se comporta como lo hace los mínimos cuadrados ordinarios. En cuanto crece alfa, se van haciendo cero algunos coeficientes y teóricamente con alfa infinito todos los coeficientes se harían cero.

```
>>> from sklearn import linear_model
>>> reg = linear_model.Lasso(alpha=0.1)
>>> reg.fit([[0, 0], [1, 1]], [0, 1])
>>> Lasso(alpha=0.1)
>>> reg.predict([[1, 1]])
array([0.8])
```

Ejemplo de Lasso Regression.

Tema 3. Algoritmos de aprendizaje automático supervisado

Elastic-net Regression

En este tipo de regresión se incluye dos términos de regularización, la norma L1 y la norma L2.

La función de pérdida queda de la siguiente forma:

$$Loss_{Elasticnet} = \frac{1}{2n} \sum_{i=1}^N (\hat{y} - y)^2 + \alpha \rho \sum_{j=1}^m |\beta_j| + \frac{\alpha(1-\rho)}{2} \sum_{j=1}^m (\beta_j)^2$$

El grado en que influye cada una de las penalizaciones está controlado por el hiperparámetro `l1_ratio`. Su valor está comprendido en el intervalo `[0,1]`. Cuando `l1_ratio=0`, se aplica ridge y cuando `l1_ratio=1` se aplica lasso. La combinación de ambas penalizaciones suele dar lugar a buenos resultados. Una estrategia frecuentemente utilizada es asignarle casi todo el peso a la penalización L1 (`l1_ratio` muy próximo a 1) para conseguir seleccionar predictores y un poco a la L2 para dar cierta estabilidad en el caso de que algunos predictores estén correlacionados.

Tema 3. Algoritmos de aprendizaje automático supervisado

Conclusiones

Hasta este punto hemos visto los conceptos más importantes a la hora de evaluar, tanto los modelos de aprendizaje automático supervisados de regresión como los de clasificación y hemos visto un ejemplo de regresión lineal que a pesar de ser muy simple transmite la idea de cómo se aplican a problemas reales. Vimos, aunque de forma superficial, la idea detrás de los métodos para evaluar los modelos, sin embargo nos queda por ver los algoritmos de clasificación que veremos en la próxima lección.

Hemos tratado de escoger el material de las lecciones siempre pensando en la disponibilidad de documentación y la posibilidad de su rápida aplicación. No hemos querido irnos a detalles demasiado teóricos y listar la mayor cantidad de nombre de algoritmos y métodos que luego si es necesario se pueden buscar de forma fácil en Internet o literatura técnica.

Tema 3. Algoritmos de aprendizaje automático supervisado

3.3. Algoritmos de aprendizaje automático supervisado II

Objetivos

El objetivo de esta lección es continuar donde nos quedamos desde la anterior. En la anterior tocamos lo referente a conceptos como Bias y Varianza de un modelo, así como revisamos el concepto de validación cruzada. También vimos las métricas de evaluación de los algoritmos de regresión y clasificación y terminamos revisando algunos de los algoritmos de regresión. En esta lección revisaremos los algoritmos más conocidos de clasificación.

Cuando termine esta lección estará usted equipado con los conocimientos necesarios para poner en práctica soluciones que incluyan clasificación y evaluar sus resultados aplicando las métricas estudiadas en la lección pasada.

Introducción

En las lecciones anteriores hemos estudiado que dentro de las tareas de aprendizaje supervisado se encuentra la clasificación y que a diferencia de la regresión que trataba de predecir variables cuantitativas, la clasificación trata de métodos para predecir variables cualitativas.

En esencia de lo que tratan los métodos de aprendizaje automático para clasificación es de aprender una regla que permita decidir cuándo un caso pertenece a una clase o no a partir de los rasgos. Por ejemplo, decidir si un caso pertenece a un paciente diabético o no en función de rasgos o variables como la glusemia, la edad, o la presión. Para ello, la regla se deriva, por algún método a partir de un conjunto de casos de ejemplo, en este caso concreto puede ser a partir del Pima Indian Diabetes dataset u otro conjunto de casos.

Las tareas de clasificación pueden dividirse según el número de clases en las que pueden dividirse los casos en:

- ▶ **Clasificación binaria (Dos clases):** La variable Outcome solo toma dos valores.
- ▶ **Clasificación Multiclase:** La variable Outcome toma varios valores.

De manera general el problema de la clasificación podemos definirlo de la siguiente forma:

Dado un conjunto de casos y un conjunto de etiquetas, encontrar una función que aprenda el mapeo entre casos y etiquetas, y que luego sea capaz de asignar la etiqueta correcta a casos no vistos en el conjunto inicial.

Tema 3. Algoritmos de aprendizaje automático supervisado

Aunque esta es una definición informal del problema de clasificación, nos sirve perfectamente para comenzar la introducción de los principales métodos o algoritmos de aprendizaje supervisado y poner en práctica este tipo de tarea.

La calidad de la clasificación de un modelo de aprendizaje automático depende en gran medida de la separabilidad entre las clases. Una clase es un grupo de casos que comparten características comunes y que por tanto en el dataset con el que pretendemos entrenar, deben tener una etiqueta común. Sin embargo la diferencia entre las propiedades, en algunos casos, puede que no sean lo suficientemente diferenciables.

Eso nos lleva a examinar el concepto de separabilidad entre clases. Cuando la distancia entre las clases, hablando en términos de un espacio de rasgos, es suficiente, la clasificación no resulta problemática; pero cuando esta distancia no es suficiente y los casos que pertenecen a una clase u otra están muy cerca entonces ya no es tan sencilla la clasificación.

Si suponemos un espacio de rasgos, de dos dimensiones y se puede encontrar una línea recta, de forma tal que los casos de las dos clases se ubiquen a cada lado de esta recta se dice que las clases son linealmente separables.

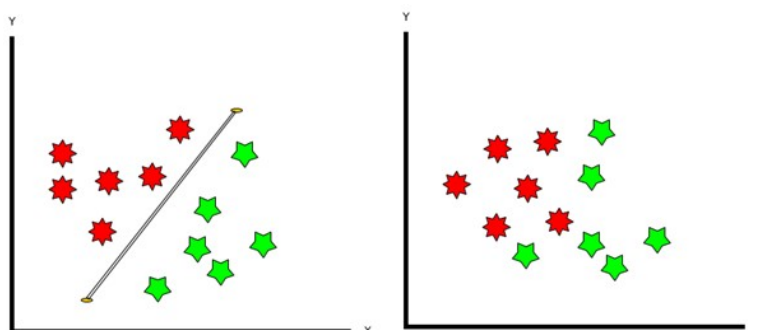


Figura 10. Figura de la izquierda: dos clases linealmente separables. Figura de la derecha: Clases que no son linealmente separables.

A los clasificadores que solo pueden lidiar con clases linealmente separables se les denominan clasificadores lineales, mientras a los que tratan con clases que no lo son, se les llaman clasificadores no lineales.

Tema 3. Algoritmos de aprendizaje automático supervisado

Estrategias de clasificación

Como mencionamos al inicio, el problema de la clasificación puede dividirse en binaria o multiclase. Esto hace que se hayan diseñado clasificadores que son capaces de enfrentar ambos tipos, sin embargo otros solo pueden hacer clasificación binaria.

Para poder usar estos clasificadores binarios por excelencia en clasificación multiclase se han desarrollado un par de estrategias: Uno contra Uno y Uno contra Todos.

Uno contra Todos. One-versus-all (OvA o OvR):

Supongamos que necesitamos clasificar los casos del Iris dataset en las tres categorías que ya conocemos: setosa, versicolor y virginica. En esta estrategia de uno-contra-todos crearíamos tres clasificadores: uno que distinguiese entre setosa y las otras dos. Otro que distinguiese entre versicolor y las otras, y un tercero que distinguiese entre virginica y el resto. A la hora de clasificar un caso, se ejecutarían los tres clasificadores y se escogería la especie que recibiese el mayor voto. En general, si tenemos N clases distintas, harán falta N clasificadores.

Uno contra Uno. One-vs-One (OvO):

En esta segunda estrategia entrenaríamos un clasificador para cada combinación posible de especie de Iris: uno para setosa-versicolor, otro para setosa-virginica y otro para versicolor-virginica. A la hora de clasificar una caso, se ejecutarían todos los clasificadores para ver qué clase "gana" la mayor parte de las confrontaciones (si el caso pertenece a la clase setosa, uno esperaría que fuese así detectada por el clasificador setosa-versicolor y por el setosa-virginica).

En esta estrategia, si tenemos N clases, harán falta $N \times (N-1) / 2$ clasificadores.

El problema de la clasificación ha sido bien estudiado en el marco de la estadística y del aprendizaje automático, se han desarrollado varios enfoques y cientos de algoritmos. Un par de libros serían insuficientes. En esta lección cubriremos algunos de los enfoques principales y trataremos de mantener la matemática al mínimo haciendo énfasis en la aplicabilidad.

En esta lección usaremos el dataset Pima Indian Diabetes, que de alguna manera lo hemos usado en las primeras lecciones. Este dataset pertenece a un problema de clasificación binaria. Sin embargo para demostrar el problema de clasificación multiclase, usaremos el dataset iris, que contiene 150 casos, pertenecientes a tres clases diferentes.

Tema 3. Algoritmos de aprendizaje automático supervisado

El dataset Pima Indian diabetes, consta con 9 variables, 8 variables son independientes y una la variable de clase o outcome que toma solo dos valores : 1 para los casos con diabetes confirmada y 0 para aquellos que no.

Algoritmos de clasificación

K-Nearest Neighbors (KNN):

Quizás el algoritmo de clasificación más simple de aplicar y de entender es el llamado KNN. Estas siglas significan K Nearest Neighbors o los K vecinos más cercanos y comenzaremos esta lección explicando cómo funciona y la manera de aplicarlo a un dataset.

La idea detrás de este algoritmo es muy sencilla pero potente y consiste en encontrar los k casos que más se parecen al nuevo caso que se quiere clasificar y asignarle la clase que más se repite o más frecuente entre todos los vecinos. Esta clase se elige por voto simple. Este número k puede ser definido por el usuario.

Como todos los algoritmos de aprendizaje automático tiene sus pros y sus contras, vemos:

Pros

- ▶ Es muy fácil de usar.
- ▶ No hace ninguna asunción sobre los datos. (No paramétrico)
- ▶ Muy rápido.

Contras

- ▶ Es necesario encontrar el k óptimo.
- ▶ Como no genera un modelo interno de los datos, los memoriza y luego en cada paso de clasificación los usa todos para tomar una decisión.

Veamos cómo funciona:

En la fase de entrenamiento, toma todos los datos de dataset y los almacena en una estructura interna, donde le permita, luego, hacer las comparaciones. En las implementaciones de scikit-learn usan principalmente dos, KD-Tree y Ball-Tree. Este proceso es relativamente rápido.

Luego, en la fase de test o clasificación, cuando se le presenta un caso a clasificar, busca, usando determinada estrategia, ya sea fuerza bruta u otra, los k casos más cercanos al que se desea clasificar.

Tema 3. Algoritmos de aprendizaje automático supervisado

Para determinar esta cercanía se usa una medida de distancia.

Una de las distancias que se usa muy frecuentemente con este algoritmo es la llamada distancia Euclideana.

$$d(p,q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Sin embargo la distancia por omisión de la implementación en scikit-learn es la Minkowski.

$$d(p,q) = \left(\sum_{i=1}^n |p_i - q_i|^p \right)^{\frac{1}{p}}$$

Aunque se pueden usar otras métricas de distancias.

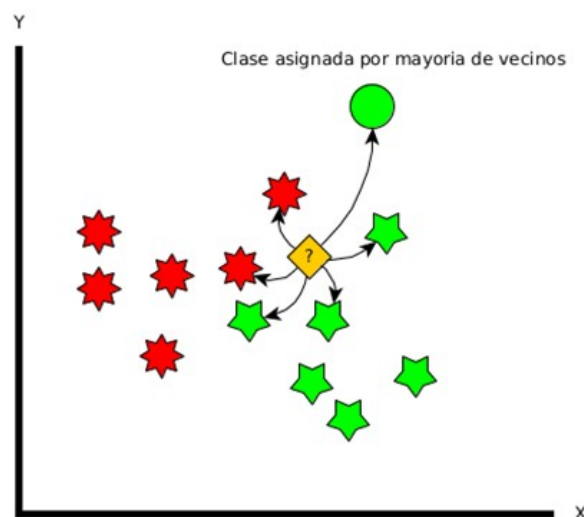


Figura 12. Clase asignada a un caso, por mayoría de votos. K=5.

El valor óptimo de K se puede ajustar por medio de validación cruzada.

En la imagen siguiente se muestra un ejemplo de KNN en Python usado para hacer predicciones sobre el iris Dataset.

Tema 3. Algoritmos de aprendizaje automático supervisado

```
#Clasificación del Iris Dataset con KNN
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
#=====
iris = datasets.load_iris(as_frame=True)

#=====
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size = 0.25, random_state = 0)
#=====
scaler = MinMaxScaler()
X_train = scaler.fit_transform(x_train)
X_test = scaler.transform(x_test)
#=====
n_neighbors = 7
knn = KNeighborsClassifier(n_neighbors)
knn.fit(X_train, y_train)
print('Accuracy de KNN con k= 7 training set: {:.2f}'
      .format(knn.score(X_train, y_train)))
print('Accuracy KNN con k=7 test set: {:.2f}'
      .format(knn.score(X_test, y_test)))
y_pred=knn.predict(X_test)
print(classification_report(y_test, y_pred))
```

Figura 13. Ejemplo de Implementación de KNN con Python.

En este ejemplo de cómo aplicar KNN a la clasificación de Iris, el código se divide en 5 partes. La primera donde se importa las librerías que contienen los métodos que necesitamos para la tarea a mano. Las partes se separan por líneas de asteriscos.

La segunda parte de este código es donde se importan el conjunto de datos. En la tercera etapa se separa el conjunto de datos en dos, uno con el fin de entrenar los métodos de clasificación y el otro con vistas a evaluar su calidad.

En la cuarta etapa, normalizamos los datos usando el MinMaxScaler.

Y en la quinta etapa entrenamos y evaluamos el modelo. En este caso de KNN hemos entrenado y probado el modelo con K=7.

Como en KNN la clase de un nuevo caso se asigna usando la mayoría de las clases de los K vecinos más cercanos, aconsejamos, siempre usar un K impar, para evitar que haya algún empate. También sugerimos que el valor optimo se encuentre usando Validación Cruzada.

El resultado de este ejemplo es el que sigue:

Accuracy de KNN con k= 7 training set: 0.97

Accuracy KNN con k=7 test set: 0.97

precision recall f1-score support

0 1.00 1.00 1.00 13

Tema 3. Algoritmos de aprendizaje automático supervisado

```
1 1.00 0.94 0.97 16
```

```
2 0.90 1.00 0.95 9
```

```
accuracy 0.97 38
```

```
macro avg 0.97 0.98 0.97 38
```

```
weighted avg 0.98 0.97 0.97 38
```

El valor por default de k, en la clase KNeighborsClassifier es 5. Los parámetros que esta clase admite son los siguientes:

- ▶ **n_neighbors:** Es un valor de tipo entero. Es el K al que nos referimos, el número de vecinos cercanos y el valor por omisión es 5.
- ▶ **weights:** Este parámetro permite añadir pesos a los vecinos más cercanos por varias estrategias diferentes, el valor por omisión es 'uniform':
 - **Uniforme (uniform):** Todos los vecinos tienen la misma importancia o peso.
 - **Basado en distancia (distance):** A los vecinos de un nuevo caso a clasificar se les asigna un peso que disminuye con la distancia. Eso significa que de los k vecinos más cercanos, los más cercanos reciben una importancia mayor que los más lejos.
 - **Función definida por el usuario(callable):** Por medio de este parámetro se le puede pasar una función la cual acepte un arreglo de distancias y retorne un arreglo de la misma forma con los pesos.
 - **Algorithm:** Este parámetro decide que algoritmo se usa para calcular los k vecinos más cercanos. El valor por omisión es Auto.
 - **ball_tree:** Usará el algoritmo Ball Tree para el cálculo de los vecinos más cercanos.
 - **kd_tree:** Usará el algoritmo de KD Tree para el cálculo de los k vecinos más cercanos.
 - **brute:** Usará Fuerza bruta como mecanismo de búsqueda de los k vecinos.
 - **auto:** Tratará de encontrar el mejor método basado en los parámetros pasados a fit().
- ▶ **Metric:** Este parámetro define la métrica que usará el algoritmo para determinar los vecinos más cercanos. Las medidas que se pueden usar son:

Tema 3. Algoritmos de aprendizaje automático supervisado

identifier	class name	args	distance function
"euclidean"	EuclideanDistance	•	$\sqrt{\sum (x - y)^2}$
"manhattan"	ManhattanDistance	•	$\sum (x - y)$
"chebyshev"	ChebyshevDistance	•	$\max(x - y)$
"minkowski"	MinkowskiDistance	p	$\sum (x - y)^p)^{1/p}$
"wminkowski"	WMinkowskiDistance	p, w	$\sum (w * (x - y)^p)^{1/p}$
"seuclidean"	SEuclideanDistance	V	$\sqrt{\sum (x - y)^2 / V}$
"mahalanobis"	MahalanobisDistance	V or VI	$\sqrt{(x - y)' V^{-1} (x - y)}$

El resto de los argumentos son dependientes de las métricas y de los argumentos. Para el resto de los detalles, por favor, refiérase a la documentación de Scikit-learn.

Máquinas con vectores de apoyo (SVM)

Este es uno de los algoritmos más robustos de aprendizaje automático. Se puede usar tanto para regresión como para clasificación, aunque principalmente se usa en clasificación.

Este algoritmo fue desarrollado por Vladimir Vapnik y algunos de sus colegas basados en la teoría del aprendizaje estadístico o teoría de Vapnik-Chervonenkis. (VC Theory).

La gran contribución de Vapnik al campo del aprendizaje automático ha consistido, precisamente, en proponer uno de los primeros modelos de estimación/predicción que no está basado en ningún modelo probabilístico, lo cual es un gran cambio de mentalidad. Puesto que todos los modelos que se usaban hasta ese momento estaban basados en la teoría estadística y de probabilidad.

Sin embargo, las máquinas de vector de soporte (SVM) se pueden considerar el primer modelo que utiliza un enfoque completamente distinto y que está basado en un modelado geométrico y que se resuelve mediante un problema de optimización con restricciones.

Una de las principales ideas detrás de este método es la de tratar encontrar un hiperplano que pueda separar las clases y que además lo haga con el mayor margen posible entre los vectores o casos que se encuentran en los bordes de las clases. A estos casos se les llaman Vectores de apoyo o de soporte. En la imagen siguiente se encuentran destacados con el centro en color amarillo y en sobre la línea discontinua.

Tema 3. Algoritmos de aprendizaje automático supervisado

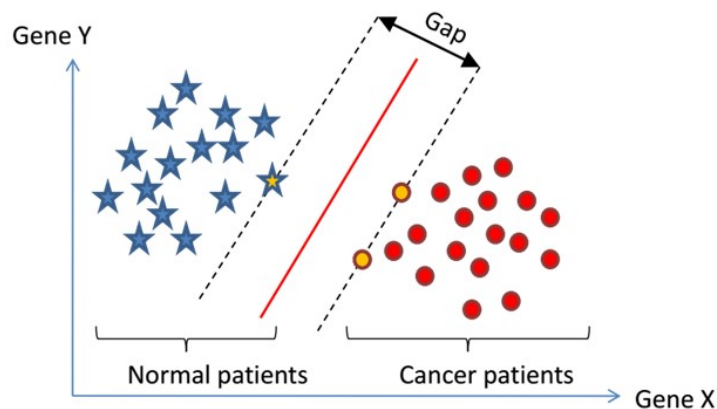


Figura 14. Hiperplano separante de margen máximo.

Se les denomina de esa forma porque son los que conforman o ayudan a delimitar los bordes del margen máximo entre las clases.

En caso de que sea imposible encontrar un hiperplano que linealmente separe las clases, entonces los datos se mapean a un espacio con una dimensionalidad mucho más alta llamado «Espacio de Rasgos» donde trata de encontrarse otro hiperplano que las separe. Este espacio de rasgos se construye por medio de lo que se conoce como «Kernel Trick» o truco de Kernel.

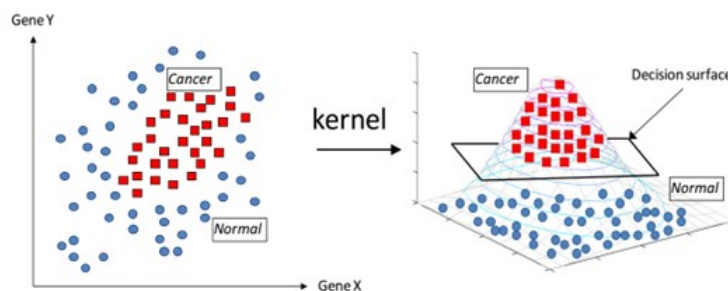


Figura 15. Efecto del mapeo de los datos a un espacio de una dimensión mayor usando el truco del Kernel.

Hiperplanos

La matemática detrás de este método de clasificación es trivial y de alguna manera es preciso que introduzcamos algunos conceptos vitales para entenderlo y el de Hiperplano es uno de ellos.

En un espacio de dos dimensiones el hiperplano es una recta. Pero, un hiperplano se puede definir para un espacio de p dimensiones. La ecuación general del hiperplano para un espacio de p dimensiones es:

Tema 3. Algoritmos de aprendizaje automático supervisado

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p = 0$$

donde el vector:

Se llama vector normal, y es un vector unitario en el cual la suma de los cuadrados es 1. Este vector apunta a una dirección ortogonal a la superficie del hiperplano (vector de color rojo de la figura 1.).

Si proyectamos cada uno de los puntos sobre el vector normal, los puntos que caen sobre el hiperplano tienen valor 0 al proyectarse. Los puntos por encima del hiperplano tienen un valor positivo, los puntos por debajo del hiperplano un valor negativo y además estos valores son mayores cuanto más lejanos están del hiperplano. Es decir, el valor que se obtiene al proyectar los puntos sobre el vector normal es proporcional a la distancia de los puntos al hiperplano. Esta característica geométrica hace posible el uso de las máquinas de vector de apoyo para buscar los patrones necesarios.

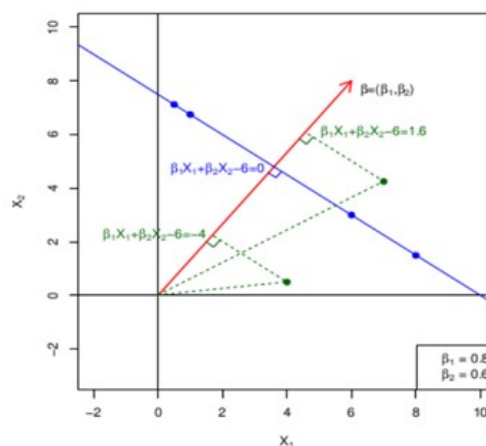


Figura 16. Ejemplo de un hiperplano

La cuestión es que el hiperplano busca separar dos conjuntos de puntos en dos regiones distintas. Por ejemplo, en un problema de clasificación el hiperplano puede separar aquellos puntos que corresponden a personas con un tumor determinado de aquellas personas sanas.

Sin embargo, para un conjunto de puntos determinados existen múltiples hiperplanos posibles que separan los puntos en dos regiones.

Tema 3. Algoritmos de aprendizaje automático supervisado

Separando por hiperplanos

Dado una serie de puntos en un espacio geométrico que se desea clasificar, se puede establecer que aquellos puntos que al proyectarse sobre el vector normal a un hiperplano determinado sean > 0 , correspondan a una clase y aquellos que sean < 0 , a otra clase.

Es decir, de forma matemática:

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p$$

$(X) > 0$ establece los puntos en un lado del hiperplano

$(X) < 0$ en el otro

Sin embargo, lo habitual es encontrarse en la situación en la cual existen múltiples hiperplanos posibles, y por tanto es necesario decidir ¿cuál de ellos establecer?

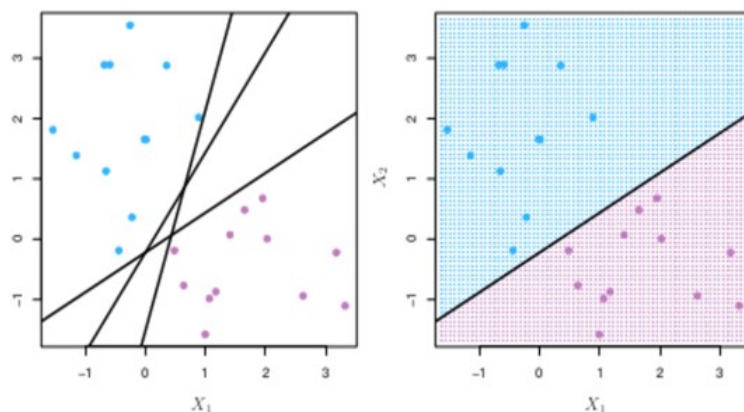


Figura 17. Ejemplo de tres hiperplanos posibles para separar dos conjuntos de clase (izquierda). En el caso de elegir un hiperplano los puntos se clasifican en función de cada una de las regiones sombreadas de rojo y azul.

Clasificador de Margen Máximo

En el caso de un problema de clasificación binaria, de todos los hiperplanos posibles, es necesario buscar aquel que nos proporciona la mayor diferencia entre las dos clases, lo cual se traduce en la mayor distancia entre los puntos que pertenecen a una clase y a otra. La hipótesis que hay detrás de esto, es porque suponemos que este hiperplano será el que tendrá una mayor distancia en el conjunto de test y en las predicciones futuras.

Esta situación puede modelarse como un problema de optimización con restricciones donde es necesario

Tema 3. Algoritmos de aprendizaje automático supervisado

maximizar el margen y, matemáticamente, se define:

Maximizar M

$$\beta_1, \beta_2, \dots, \beta_p$$

Sujeto a las siguientes restricciones

$$\sum_{i=1}^p \beta_i^2 = 1$$

$$y_i (\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p) \geq M$$

Para todo $i=1, \dots, N$

En la siguiente imagen se muestra de forma gráfica el concepto donde la línea continua resaltada es el Margen Máximo, y se muestran dos bandas con líneas discontinúas que contienen la distancia del hiperplano a los primeros puntos de cada una de las clases, el objetivo es maximizar la distancia de estas dos bandas.

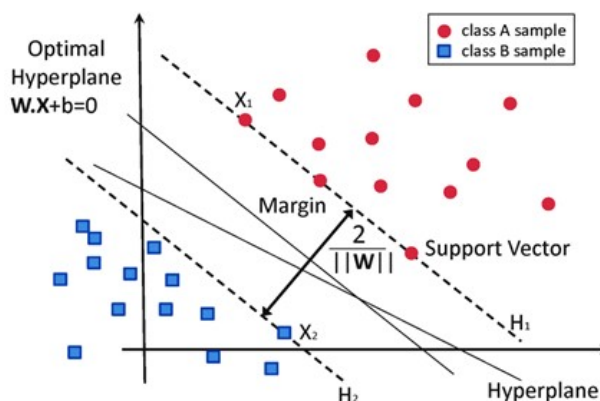


Figura 18. Ejemplo de Hiperplano separante con Margen Máximo.

El principal problema con esta separación óptima del hiperplano es que los datos habitualmente no son linealmente separables con una recta en el caso de un espacio de dos dimensiones, como se observa en la siguiente gráfica, donde es imposible separar los puntos de una clase de los de otra.

Tema 3. Algoritmos de aprendizaje automático supervisado

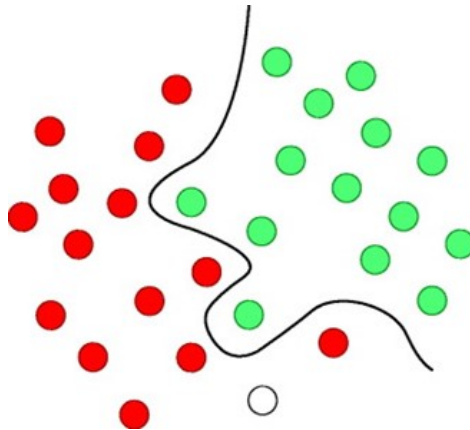


Figura 19. Ejemplo de dos clases que no son linealmente separables.

Esta situación produce una solución pobre para el clasificador de Margen Máximo. Por tanto, el clasificador con vectores de apoyo maximiza un Margen Suave.

Clasificador con Margen Suave

La solución al problema anterior en el cual los puntos no son linealmente separables es maximizar un llamado Margen Suave. Matemáticamente el problema se define de la siguiente forma:

Maximizar M

$$\beta_1, \beta_2, \dots, \beta_p, \zeta_1, \zeta_2, \dots, \zeta_n$$

Sujeto a las siguientes restricciones

$$\sum_{i=1}^p \beta_i^2 = 1$$

$$y_i (\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots \beta_p x_p) \geq M (1 - \zeta_i)$$

$$\zeta_i \geq 0, \sum_{i=1}^n \zeta_i \leq C$$

De forma gráfica, en la siguiente figura se observa:

Tema 3. Algoritmos de aprendizaje automático supervisado

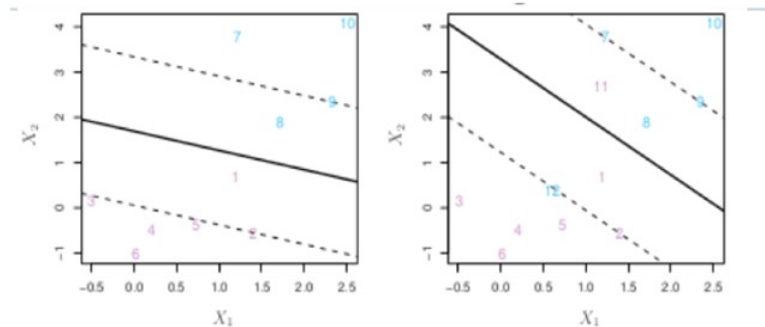


Figura 20. Ajuste de un clasificador con vectores de apoyo a pocos datos. El hiperplano se muestra con una línea sólida y los márgenes con una punteada. La imagen de la derecha muestra el efecto cuando aparecen dos nuevas observaciones (11 y 12) las cuales están en el lado incorrecto del hiperplano y de los márgenes.

Modificando el parámetro C de la ecuación anterior, el cual se conoce como función de coste se puede hacer el margen más grande o pequeño. En concreto un valor de C más grande hace el margen más pequeño y a la inversa un valor C más pequeño hace el margen más grande.

En la siguiente figura se muestra un hiperplano y los márgenes correspondientes en función de diferente valor de C .

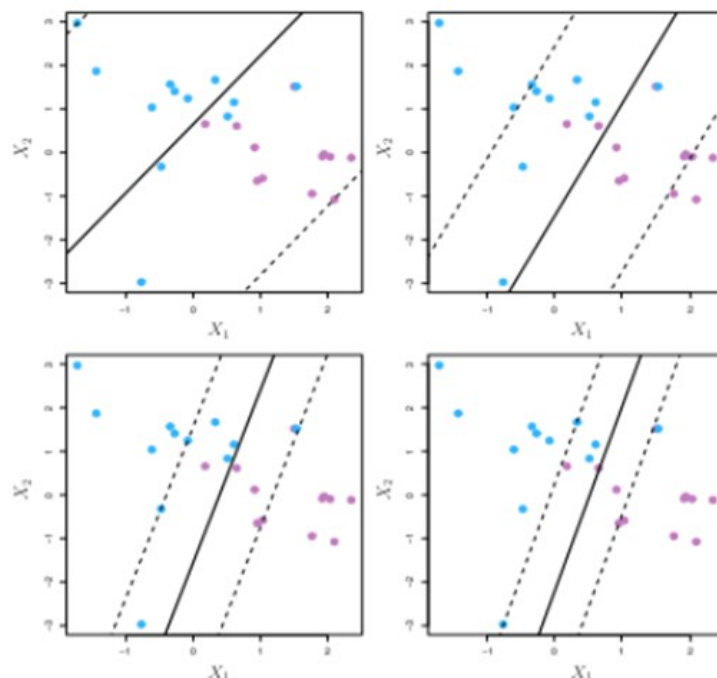


Figura 21. Diferentes fronteras de decisión en los márgenes de una soft-margin en función de diferentes valores de C y utilizando los mismos puntos y el mismo hiperplano. El margen más grande es el de la gráfica superior izquierda y el más pequeño el de la inferior derecha.

Tema 3. Algoritmos de aprendizaje automático supervisado

Problema con las fronteras lineales

Muchas veces las fronteras lineales siguen sin funcionar, independientemente del valor de coste C que se utilice. Por ejemplo, en la siguiente gráfica se muestra un problema donde la separación necesaria entre los puntos de una clase y la otra es no lineal.

Expansión de variables

Para solucionar este problema donde no es posible realizar una separación entre dos clases, una opción es incrementar el espacio de las variables por medio de transformaciones, es decir pasar de un espacio p -dimensional a un espacio de D dimensiones $D > p$. Una vez realizada esta transformación, se debería ajustar un clasificador con vectores de apoyo en este nuevo espacio. El objetivo es obtener fronteras de decisión no lineales sobre el espacio original.

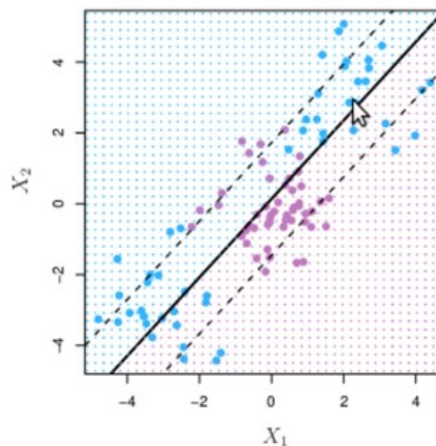


Figura 22. Ejemplo donde no es posible una separación lineal entre dos clases.

Por ejemplo, si tenemos datos de entrada en dos dimensiones es posible utilizar el siguiente espacio de 6 dimensiones: $(X_1, X_2, X_1^2, X_2^2, X_1X_2, 1)$ siendo por tanto la frontera de decisión:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 = 0$$

Esta transformación conlleva fronteras de decisión no-lineales en el espacio original.

Tema 3. Algoritmos de aprendizaje automático supervisado

Polinomios cúbicos

Utilizando una transformación en forma de polinomios cúbicos se puede pasar de 2 a 9 variables. De esta forma el clasificador en el nuevo espacio soluciona el problema de buscar los patrones en el espacio con menor dimensiones.

En la siguiente gráfica se muestra el resultado de las fronteras de separación obtenidas por medio de una transformación en polinomios cúbicos., resultado del cálculo de la siguiente ecuación:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \beta_6 X_1^3 + \beta_7 X_2^3 + \beta_8 X_1 X_2^2 + \beta_9 X_1^2 X_2 = 0$$

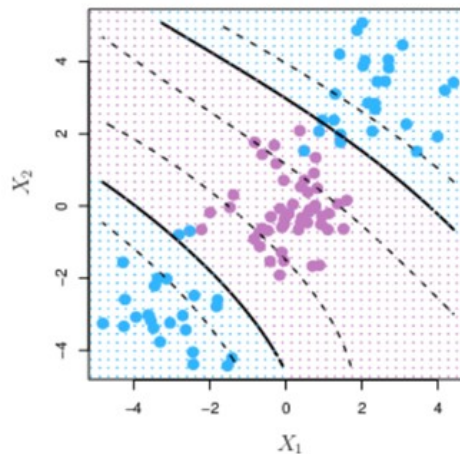


Figura 13. Ejemplo de las fronteras de decisión no lineales obtenidas por medio de una transformación obtenida con polinomios cúbicos.

El truco del Kernel (Kernel trick)

Las expansiones de las variables con polinomios, especialmente aquellos con grandes dimensiones, son computacionalmente costosos. Existe una solución más elegante y controlada de introducir no-linealidad que es utilizado en las máquinas con vectores de apoyo por medio del uso de kernels. El kernel de un operador A denotado por $\ker$ es el conjunto de todos los vectores cuya imagen sea el vector nulo:

$$\text{Ker} A = \{ \vec{v} \in V : A\vec{v} = \vec{0} \}$$

Los kernels se apoyan en concepto del producto vectorial de los vectores de apoyo. Se trata de funciones que reciben dos vectores como parámetros. Uno de los kernels más utilizados es el de base radial que asume que el feature space es de altas dimensiones y se define con la siguiente función matemática:

Tema 3. Algoritmos de aprendizaje automático supervisado

$$Ker(x_i, x'_i) = \exp \left(-\gamma \sum_{i=1}^p (x_{ij} - x'_{ij})^2 \right)$$

$$yi(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M$$

El uso de kernels permite obtener fronteras de decisión no lineales por medio de transformaciones matemáticas sin necesidad de tener que realizar transformaciones con polinomios. En la siguiente figura se muestran las fronteras de decisión no lineales obtenidas por medio del uso de un kernel de base radial.

Está más que claro que una comprensión completa de los principios e interpretación de los conceptos anteriores requerirá mucho más que una lectura a este material. Por eso sugerimos que para profundizar en este tema consulten: A Gentle Introduction to Support Vector Machines in Biomedicine, de los autores Alexander Statnikov, Douglas Hardin, Isabelle Guyon, Constantin F. Aliferis. Así como Support Vector Machines Succinctly de Alexandre Kowalczyk. Aunque hay mucha información disponible sobre el tema.

Hasta aquí el recorrido por la ideas básicas detrás de las Maquinas con Vectores de Apoyo o SVM. Veamos cómo se usan en la práctica y los detalles de los parámetros que son necesario ajustar.

Como a lo largo de estas lecciones, usaremos la implementación de SVM del módulo scikit-learn de Python, con el fin de explicar el uso práctico de este método para generar modelos de aprendizaje automáticos de clasificación.

Como ya mencionamos al principio de esta sección SVM se puede usar tanto para regresión como para clasificación y clustering. Este módulo, implementa tres versiones de SVM para clasificación:

- ▶ SVC (C-SVM)
- ▶ NuSVC (v-SVM)
- ▶ LinearSVC (SVM con kernel Lineal)

Todos los métodos soportan la clasificación binaria o multiclase. NuSVC es una reparametrización de C-SVC implementada en SVC y por tanto matemáticamente equivalentes, pero se incluye un parámetro v en vez de C.

LinearSVC es una implementación de SVM con kernel lineal.

Veremos los parámetros más importantes para cada uno de las implementaciones y luego un ejemplo.

Parámetros de SVC:

Tema 3. Algoritmos de aprendizaje automático supervisado

- ▶ C: toma valores de punto flotante. El valor por omisión es 1. Es un parámetro de regularización y controla la penalización. El efecto de la regularización es inversamente proporcional a C. Debe ser siempre positivo.
- ▶ Kernel: Define el tipo de kernel a usar por SVM y pueden ser los siguientes:
 - Lineal : $\langle x, x' \rangle$
 - Polinomial (polynomial): $(\gamma \langle x, x' \rangle + r)^d$ donde d es especificado por el parámetro degree.
 - Rbf (Función de Base Radial) : $\exp(-\gamma \|x - x'\|^2)$ donde γ es representado por el parámetro gamma. Debe ser mayor que cero.

Cuando se usa este tipo de kernel hay que tener en cuenta dos parámetros, C y gamma. El parámetro C, que es común a todos los kernel, controla el compromiso entre la calidad de la clasificación y la simplicidad de la superficie de decisión. Cuando C es bajo la superficie de decisión es suave. Cuando C es alta se busca clasificar de forma correcta todos los ejemplos del conjunto de entrenamiento y puede conducir a overfitting. Mientras que gamma define cuanto influencia puede tener un ejemplo de entrenamiento.

Elegir de forma adecuada el C y gamma es vital para SVM. Lo ideal es usar Validación Cruzada (GridSearchCV) espaciándolos de manera exponencial.

- ▶ Sigmoide (sigmoid): $(\gamma \langle x, x' \rangle + r)^d$
- ▶ degree: Toma valores enteros y define el grado en el kernel polinomial. El valor por omisión es 3. Es ignorado por el resto de los kernels.
- ▶ gamma: Toma valores float. Es el coeficiente de los kernels rbf, poly y sigmoid. También se le puede pasar los siguientes valores:
 - scale: si gamma='scale' entonces se usa como valor $1/(n \text{ de rasgos} * x.\text{var}())$ como su valor.
 - Si se le pasa gamma='auto' entonces usa $1/(n \text{ de rasgos})$ como valor. Para el resto de los parámetros y sus valores, ver la documentación del método.

En la imagen siguiente mostramos el uso de SVC para clasificar el iris dataset, comparando los resultados contra un clasificador basado en Gradiente descendente estocástico.

En este ejemplo se usa la estrategia Uno contra el resto One-vs-Rest de forma explícita, pero estos métodos lo hacen de manera interna y lo mostramos aquí solo a modo de ejemplo.

Tema 3. Algoritmos de aprendizaje automático supervisado

```
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import SGDClassifier
#=====
iris = datasets.load_iris(as_frame=True)
#=====
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size = 0.25, random_state = 0)
#=====
caler = MinMaxScaler()
X_train = scaler.fit_transform(x_train)
X_test = scaler.transform(x_test)
#=====
ovr_clf = OneVsRestClassifier(SVC(kernel='rbf', gamma='auto'))
ovr_clf.fit(x_train, y_train)
y_pred=ovr_clf.predict(x_test)
print("Datos para SVM sobre el Test Set")
print(classification_report(y_test,y_pred))
#=====
sgd_clf = SGDClassifier(max_iter=1000, tol=1e-3, random_state=42)
sgd_clf.fit(x_train, y_train)
y_pred=sgd_clf.predict(x_test)
print("Datos para SGD sobre el Test Set")
print(classification_report(y_test,y_pred))
```

Figura 24. Ejemplo de uso de SVC con Kernel=rbf.

Los resultados son los siguientes:

Datos para SVM sobre el Test Set

precision recall f1-score support

0 1.00 1.00 1.00 13

1 1.00 0.94 0.97 16

2 0.90 1.00 0.95 9

accuracy 0.97 38

macro avg 0.97 0.98 0.97 38

weighted avg 0.98 0.97 0.97 38

Datos para SGD sobre el Test Set

precision recall f1-score support

0 1.00 1.00 1.00 13

Tema 3. Algoritmos de aprendizaje automático supervisado

```
1 1.00 0.38 0.55 16
```

```
2 0.47 1.00 0.64 9
```

```
accuracy 0.74 38
```

```
macro avg 0.82 0.79 0.73 38
```

```
weighted avg 0.88 0.74 0.72 38
```

El accuracy de SVM es del 97% sobre el conjunto de pruebas, que sin dudas es muy buen resultado.

Con esto terminamos nuestro recorrido por los métodos SVM.

En las dos secciones anteriores hemos visto dos métodos de clasificación que esencialmente usan enfoques muy diferentes para atacar el problema.

El que veremos a continuación es quizás el enfoque clásico, por decirlo de alguna forma y es el de clasificación probabilista.

Clasificación con Naives Bayes

En este tema se va a introducir el clasificador Naive Bayes, el cual está basado en el teorema de Bayes para calcular las probabilidades a posteriori de los eventos a predecir.

Vamos a ver los siguientes puntos:

- ▶ En primer lugar, veremos el teorema de Bayes.
- ▶ A continuación, se describe la forma de calcular las tablas de probabilidad condicionada.
- ▶ Posteriormente se verá por qué es importante asumir independencia condicional en el clasificador Naive Bayes.
- ▶ El clasificador Naive Bayes y finalmente cómo se puede utilizar con variables numéricas.

Teorema de Bayes

El Teorema de Bayes es una proposición planteada por el filósofo inglés Thomas Bayes (1702-1761) en el año 1763 en su artículo «An Essay towards solving a Problem in the Doctrine of Chances» publicado en la revista Philosophical Transactions of the Royal Society of London.

Tema 3. Algoritmos de aprendizaje automático supervisado

Este teorema expresa la probabilidad condicional de un evento aleatorio A dado B en términos de la distribución de probabilidad condicional del evento B dado A y la distribución de probabilidad marginal de solo A.

El teorema de Bayes es bastante relevante porque relaciona la probabilidad de dos eventos A y B utilizando la dependencia condicional de uno de ellos. Es decir, relaciona la probabilidad de que ocurra el evento A y sabemos de antemano que ha ocurrido B, utilizando la probabilidad de que ocurra el evento B sabiendo que ha ocurrido A.

Este teorema permite relacionar, entre otras cosas, síntomas y enfermedades. Por ejemplo, sabiendo la probabilidad de tener dolor de garganta dado que se conoce que se tiene gripe, se puede obtener la probabilidad de tener gripe dado que se tiene dolor de garganta.

El teorema de Bayes relaciona la comprensión de la probabilidad de aspectos causa-efecto dados los eventos dependientes observados. Un evento dependiente es aquel cuyo resultado se ve afectado por el resultado de otro evento o serie de eventos. Los eventos dependientes son la base del modelado predictivo puesto que se busca obtener la probabilidad de que ocurra un suceso teniendo en cuenta la existencia de una serie de eventos dependientes.

En el caso de dos eventos dependientes A y B, el teorema de Bayes describe su relación de la siguiente manera:

$$P(A \vee B) = \frac{P(B \vee A) P(A)}{P(B)}$$

Es decir, la probabilidad de A dado que ocurre B, es igual a la probabilidad de que ocurra B dado que ha ocurrido A, multiplicado por la probabilidad de que ocurra A, dividido por la probabilidad de que ocurra B.

Para poner esto en contexto de la clasificación transformemos la llamada regla de Bayes a una que nos permita clasificar un conjunto de casos.

Entonces tenemos:

$$P(C \vee X) = \frac{P(X \vee C) P(C)}{P(X)}$$

En lenguaje simple y para meros mortales, esto significa que la probabilidad de que el caso X pertenezca a la clase C, está dada por la probabilidad de X dado la clase C multiplicada por la probabilidad de la clase dividida por la probabilidad del predictor X.

Tema 3. Algoritmos de aprendizaje automático supervisado

Dónde:

- ▶ $P(C)$ son las probabilidades a priori de las clases.
- ▶ $P(X \vee C)$ Se conoce como Likelihood la cual es la probabilidad del predictor dada la Clase.
- ▶ $P(A_i \vee B)$ son las probabilidades a posteriori.
- ▶ $P(B)$ es la probabilidad a priori del predictor.

La esencia de este método es que toma los datos del dataset, calcula primero las probabilidades a priori, construyendo tablas de frecuencias y luego tablas de likelihood a partir de ellas. Después calcula las probabilidades a posteriori aplicando la regla que acabamos de ver y elige la clase que más alta probabilidad a posteriori arroje como clase para el caso X .

Ese, de manera general, es el algoritmo del método y para ejemplificar tomemos un pequeño dataset que por su simplicidad se presta para esto. Usaremos el Weather Dataset.

Este dataset permite entrenar generar un modelo que nos prediga si podemos jugar Golf a partir de cuatro variables o rasgos que definen el clima y lo asocian con la posibilidad de que se pueda jugar o no.

Las variables son:

- ▶ Outlook: toma los siguientes valores: Rainy, Sunny, Overcast.
- ▶ Temp: Toma valores: Hot, Mild, Cool.
- ▶ Humidity: Toma los valores: High, Normal.
- ▶ Windy: Toma solo los valores: True o False.
- ▶ Play Golf: Solo toma valores: Yes, No.

El dataset se muestra en la siguiente imagen:

Tema 3. Algoritmos de aprendizaje automático supervisado

Outlook	Temp	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

Figura 25. Dataset Weather.

Construyamos, las tablas de frecuencia y las de Likelihood para poder entender cómo se calculan las probabilidades a priori de cada uno de los rasgos y luego como calculamos las de las clases y la posteriori.

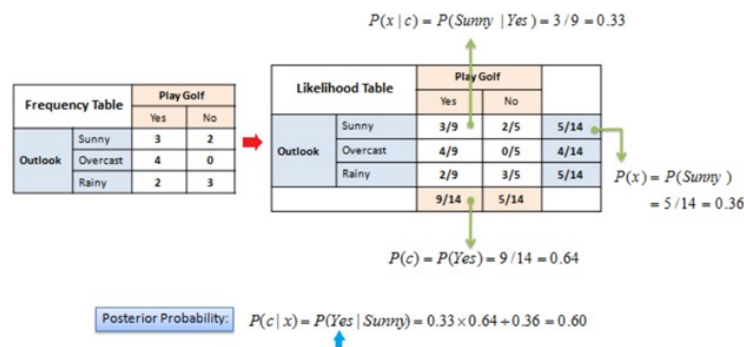


Figura 26. Tabla de frecuencia y Likelihood del rasgo OutLook contra la clase Yes.

Estas tablas se hacen contando las veces que coincide, por ejemplo, el valor sunny del rasgo Outlok con el valor yes de la variable independiente Play Golf. En la imagen anterior se muestran los cálculos para la clase Yes, veamos los de la clase NO.

Tema 3. Algoritmos de aprendizaje automático supervisado

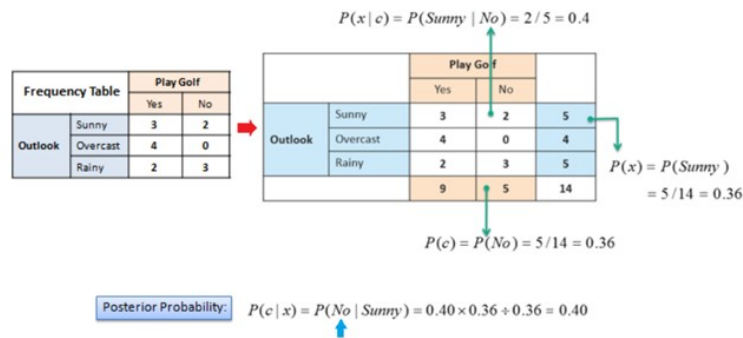


Figura 27. Tabla de frecuencia y Likelihood del rasgo Outlook contra la clase No.

Veamos las tablas del resto de los rasgos.

		Play Golf		
		Yes	No	
Humidity	High	3	4	7
	Normal	6	1	7
		9	5	14

		Play Golf		
		Yes	No	
Temp.	Hot	2	2	4
	Mild	4	2	6
	Cool	3	1	4
		9	5	14

		Play Golf		
		Yes	No	
Windy	False	6	2	8
	True	3	3	6
		9	5	14

Figura 28. Tabla de frecuencia y Likelihood de los rasgos contra la clase.

Con estos datos a mano ya podemos hacer un ejemplo para clasificar un caso.

Veamos, tenemos el siguiente caso y la pregunta es:

Si, Outlook = rainy, Temp = cool, Humidity= High y Windy=true, ¿podemos jugar Golf?

Si lo formulamos por medio del algoritmo Naives Bayes, tenemos que calcular las probabilidades a posteriori de la siguiente forma:

$$P(\text{yes} \vee \text{rainy}, \text{cool}, \text{high}, \text{true}) = \frac{P(\text{rainy}, \text{cool}, \text{high}, \text{true} \vee \text{yes}) P(\text{yes})}{P(\text{rainy}, \text{cool}, \text{high}, \text{true})}$$

Calculamos primero los numeradores de la siguiente forma:

$$P(\text{yes} \vee \text{rainy}, \text{cool}, \text{high}, \text{true}) = P(\text{rainy} \vee \text{yes}) * P(\text{cool} \vee \text{yes}) * P(\text{high} \vee \text{yes}) * P(\text{true} \vee \text{yes}) * P(\text{yes})$$

Tema 3. Algoritmos de aprendizaje automático supervisado

Lo iremos haciendo paso a paso:

$$P(\text{Outlook}=\text{rainy}|\text{yes})=2/9=0,22 \quad P(\text{Temp}=\text{cool}|\text{yes})=3/9=0,33$$

$$P(\text{Humidity}=\text{high}|\text{yes})=3/9=0,33 \quad P(\text{Windy}=\text{true}|\text{yes})=3/9=0,33$$

$$P(\text{yes})=9/14=0,64$$

$$P(\text{rainy}, \text{cool}, \text{high}, \text{true} \vee \text{yes}) P(\text{yes}) = 0,22 * 0,33 * 0,33 * 0,33 * 0,64 = 0,005059$$

Ahora hacemos lo mismo para la clase no:

$$P(\text{Outlook}=\text{rainy}|\text{no})=3/5=0,6 \quad P(\text{Temp}=\text{cool}|\text{no})=1/5=0,2$$

$$P(\text{Humidity}=\text{high}|\text{no})=4/5=0,8 \quad P(\text{Windy}=\text{true}|\text{no})=3/5=0,6$$

$$P(\text{no})=5/14=0,35$$

Por tanto

$$P(\text{rainy}, \text{cool}, \text{high}, \text{true} \vee \text{no}) P(\text{no}) = 0,6 * 0,2 * 0,8 * 0,6 * 0,35 = 0,02016$$

Ahora calculamos las probabilidades a posterioris de la siguiente manera

$$P(\text{yes} \vee \text{rainy}, \text{cool}, \text{high}, \text{true}) = \frac{0,005059}{0,005059 + 0,02016} = 0,2$$

$$P(\text{no} \vee \text{rainy}, \text{cool}, \text{high}, \text{true}) = \frac{0,02016}{0,02016 + 0,005059} = 0,8$$

Ya calculamos las dos probabilidades y ahora escogemos la que mayor probabilidad nos arroja y es la correspondiente a Play Golf = No.

Y sería consistente, pues, si el día está lluvioso, la temperatura fría, la humedad relativa alta y hace mucho viento, bueno lo lógico es que no se pueda jugar golf.

Si los rasgos fueran numéricos, habría que transformar los rasgos antes de construir las tablas de frecuencias o usar las distribuciones de las variables numéricas para estimar las probabilidades; pero no lo vamos a revisar en esta lección.

De esta forma terminamos de dar un recorrido por las ideas básicas detrás de este tipo de clasificador y veremos ahora algunos detalles de como aplicarlo en la práctica usando el módulo scikit-learn de Python.

Tema 3. Algoritmos de aprendizaje automático supervisado

Naives Bayes en scikit-learn incluye varios algoritmos de aprendizaje supervisado. Cada uno con asunciones diferentes. Estos métodos son:

- ▶ Naives Bayes Gausiano. (Gaussian Naives Bayes)
- ▶ Naives Bayes Multinomial. (Multinomial Naive Bayes)
- ▶ Complement Naive Bayes
- ▶ Bernoulli Naive Bayes.
- ▶ Categorical Naive Bayes.
- ▶ Out-of-core naive Bayes model fitting.

Gaussian Naives Bayes

Este método asume que el likelihood de los datos es Gaussiano, es decir, presupone que:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} e^{-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}}$$

En la siguiente imagen se muestra un ejemplo.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=1)
gnb = GaussianNB()
y_pred = gnb.fit(X_train, y_train).predict(X_test)
print("Número de puntos incorrectamente clasificados del total %d punto : %d"
      % (X_test.shape[0], (y_test != y_pred).sum()))
```

Número de puntos incorrectamente clasificados del total 75 punto : 4

Figura 29. Ejemplo de uso de Naives Bayes Gaussiano.

Los detalles del resto de los modelos, por favor, revisarlos en la documentación de SciKit-Learn.

Clasificación con árboles de decisión

En esta sección veremos los modelos de clasificación y regresión basados en los árboles de decisión.

- ▶ En primer lugar, veremos una introducción a los árboles de decisión.
- ▶ A continuación, se hará énfasis en cómo se decide el mejor corte, cubriendo para ello los conceptos de índice Gini y ganancia de información.

Tema 3. Algoritmos de aprendizaje automático supervisado

- ▶ Más adelante, se describe la necesidad de realizar la poda de los árboles.
- ▶ Posteriormente, se describen brevemente las diferencias con los árboles utilizados para clasificación.

Arboles de Decisión

Los árboles de decisión son una de las técnicas más populares dentro del campo del aprendizaje automático. Se llevan utilizando durante muchos años en el área de la minería de datos. Se trata de métodos simples y fáciles de interpretar.

Estos modelos dividen o segmentan el espacio de las variables predictoras en una serie de regiones. Una vez creado el árbol de decisión es utilizado para predecir observaciones futuras. Para este propósito se utiliza la moda en el caso de que la variable a predecir sea categórica o bien la media en el caso de que sea numérica.

Como el conjunto de las reglas para separar las variables predictoras se pueden resumir en forma de árbol, a estos métodos se les conoce popularmente con el nombre de árboles de decisión.

Los árboles de decisión dividen el espacio en rectángulos que minimizan el error de la predicción. Debido a que es computacionalmente imposible considerar cualquier posible partición del espacio de entrada se utiliza un enfoque top-down greedy conocido como recursive binary splitting.

En la siguiente imagen se muestra un árbol de decisión sencillo para obtener el logaritmo del salario de los jugadores de béisbol. Este valor se obtiene por medio de las variables de años en la liga (years) y de hits y utilizando unas reglas de decisión en base a los umbrales de estas variables. Así, por tanto, aquellos jugadores con más de 4,5 años de experiencia y más de 117,5 hits estarían en 6,74, mientras que si tuvieran menos de 117,5 hits estarían en 6,0.

Tema 3. Algoritmos de aprendizaje automático supervisado

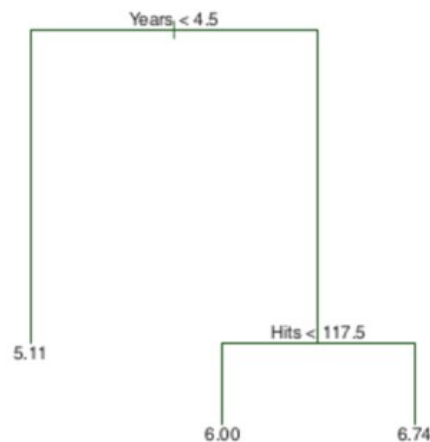


Figura 30. Árbol de Decisión sencillo para estimar el salario de un jugador de béisbol en función de los años y los hits.

El árbol de decisión anterior se obtiene por medio de la representación en un espacio de dos dimensiones de los puntos donde están el eje de años (years) y el de los hits. A cada uno de esos puntos le corresponde un salario determinado y el objetivo es agrupar aquellos puntos que tienen un salario similar utilizando la información de las otras dos variables. (Hits y años de experiencia).

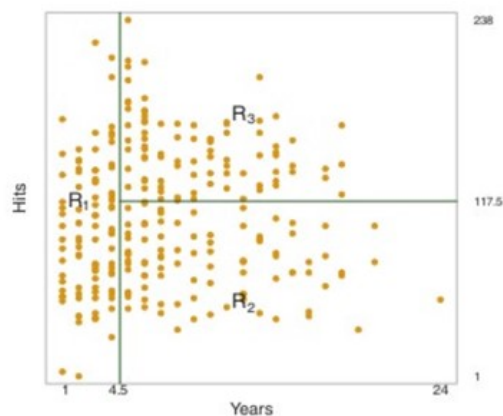


Figura 31. Representación bidimensional de la distribución de Hits y años de los jugadores de la liga.

El árbol de decisión se obtiene por medio de un algoritmo que elige primero aquella variable que es más predictiva de la variable objetivo. A continuación, los ejemplos de entrenamiento se dividen en grupos con distintos valores para las clases de esta primera variable. El algoritmo continúa dividiendo los nodos con la elección de la mejor variable en cada iteración hasta que se alcance el criterio de parada. En cada iteración el algoritmo elige aquella variable que mejor predice la variable objetivo, por tanto, se trata de un

Tema 3. Algoritmos de aprendizaje automático supervisado

algoritmo de tipo greedy. El criterio de parada puede venir dado por algunas de estas situaciones:

- ▶ Todos, o casi todos, los ejemplos del nodo son de la misma clase.
- ▶ No existen variables para distinguir entre los ejemplos.
- ▶ El árbol ha alcanzado un tamaño predefinido.

Existen numerosas implementaciones de los árboles de decisión disponibles en el mercado, no obstante, las más famosas son:

- ▶ C.5.0 desarrollado por Ross Quinlan.
- ▶ C 4.5.
- ▶ ID3 (Iterative Dichotomiser 3).
- ▶ J48, una alternativa basada en Java open source del C.4.5.

De todos ellos el algoritmo C.5.0 está considerado como el estándar en la industria.

Best split: entropía, Gini index, ganancia de información

Cada una de las divisiones del árbol da como resultado dos grupos. Es decir, los datos resultantes de la división por la variable elegida y con el punto de corte determinado están en uno de los dos grupos. Cuando los segmentos de una división contienen valores de una única clase se considera que es una división pura.

A la hora de identificar los criterios para realizar el split o corte existen varias métricas de pureza. Muchos algoritmos, por ejemplo, el C.5.0 utilizan el concepto de entropía.

En este caso un valor de 0 indica que la muestra es completamente homogénea, mientras que un valor de 1 indica desorden completo.

La entropía se define con la siguiente fórmula matemática:

$$Entropia(S) = \sum_{i=1}^n -p_i \log_2(p_i)$$

Por ejemplo, si se ha realizado una partición utilizando una variable y umbral determinado y los datos de dos clases se dividen en un 60 % en un lado de la rama y en un 40 % en otro lado de la rama, tendríamos el siguiente valor de entropía:

Tema 3. Algoritmos de aprendizaje automático supervisado

$$Entropia(s) = -0,60 * \log_2(0,60) - 0,40 * \log_2(0,40) = 0,9709506$$

Utilizando la medida de pureza el algoritmo tiene que decidir con que variable hacer el corte. Una opción es utilizar la entropía para calcular el cambio resultante de hacer el corte en esa variable.

A continuación, se calcula la ganancia de información (information gain o IG) que es la diferencia entre la entropía en el segmento antes de hacer el corte (S_1) y la partición resultante de hacer el corte (S_2), es decir:

Donde IG es la Ganancia de Información.

Después de un corte los datos se pueden dividir en más de una partición, por tanto, se considera la entropía a lo largo de las N particiones, ponderando la entropía de cada partición con el número de instancias de esa partición:

$$InfoGain(f) = entropia(s_1) - entropia(s_2)$$

Poda de los árboles

El proceso anterior de generación de cortes recursivos denominado top-down greedy puede generar buenas predicciones en el conjunto de entrenamiento, pero también sufre el problema del sobre ajuste.

Esto se debe a que el árbol resultante puede llegar a ser muy complejo y ajustarse demasiado bien a los datos de entrenamiento. Un árbol mucho más pequeño puede dar lugar a una menor varianza y mejor interpretación con el coste de un pequeño sesgo.

La estrategia para generar estos árboles más pequeños y con una menor varianza suele ser generar un árbol muy grande y después podarlo para obtener un sub-árbol (post-pruning).

Ahora bien, la cuestión es, ¿cómo seleccionamos el sub-árbol? Una buena solución es utilizar aquel sub-árbol que proporcione un menor error de test en validación cruzada (cross validation) o bien en el conjunto de validación.

Por ejemplo, en la gráfica 21 se muestra la obtención de un árbol sin podar.

Árboles para clasificación

Los árboles se pueden utilizar para realizar una predicción numérica en el caso de regresión o bien para realizar una clasificación. Los árboles de clasificación (classification trees) son similares a los regression trees pero predicen la respuesta de una variable categórica.

Tema 3. Algoritmos de aprendizaje automático supervisado

En regression trees la predicción corresponde a la media de las observaciones de ese nodo terminal. En classification trees se trata de la clase que tiene un mayor número de ocurrencias en ese nodo terminal.

En el caso de clasificación, el criterio que se suele utilizar para realizar los cortes o splits es el Gini index, que se define:

$$G = \sum_{k=1}^k \widehat{pmk} (1 - \widehat{pmk})$$

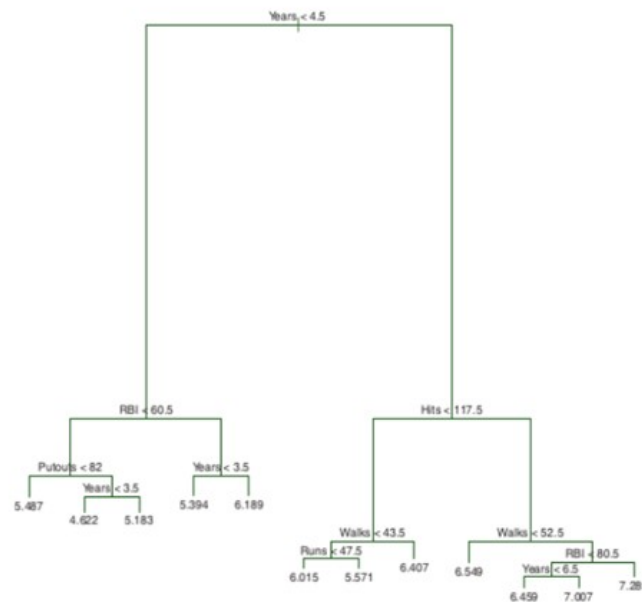


Figura 32. Ejemplo de un árbol sin podar.

O bien la entropía cruzada, la cual se define como:

$$D = - \sum_{k=1}^k \widehat{pmk} \log \widehat{pmk}$$

Árboles vs. Modelos lineales

Una regresión lineal asume un modelo que tiene la siguiente forma:

$$f(x) = \beta_0 + \sum_{j=1}^p X_j \beta_j$$

Mientras que un árbol de regresión, asume un modelo:

Tema 3. Algoritmos de aprendizaje automático supervisado

$$f(x) = \beta_0 + \sum_{j=1}^p X_j \beta_j$$

¿Qué modelo es mejor? Depende de lo que se esté modelando. Si las relaciones entre las variables de entrada y la variable respuesta se pueden aproximar por un modelo lineal, una regresión lineal funciona bien y supera a un árbol de regresión. Esto se debe a que un árbol de regresión no es capaz de modelar esta estructura.

Sin embargo, si la naturaleza del problema es no lineal y con relaciones complejas entre las variables, los árboles funcionan mejor.

Hasta aquí hemos hecho un breve recorrido por los conceptos básicos de los árboles de decisión y como en todos los casos ahora examinaremos como se emplean en la práctica, usando el módulo Scikit-Learn de Python.

En scikit-learn la clase que permite implementar un árbol de decisión para clasificación multiclase es `DecisionTreeClassifier`.

Los parámetros que este permite son los siguientes:

- ▶ `criterion`: Se le pueden pasar dos valores “gini” o “entropia”. El valor por omisión es gini. Estos son las medidas de la calidad de split.
- ▶ `Splitter`: Los valores que permite son “best” y “random”. El valor por omisión es “best”.
- ▶ `max_depth`: Acepta valores enteros y el valor por omisión es “None”. Este parámetro define la máxima profundidad del árbol. Si se define None o no se le pasa otro valor, entonces los nodos serán expandidos hasta que todas las hojas sean puras o hasta que todas las hojas contengan menos que `min_samples_split` muestras.

Para el resto de los parámetros de esta clase por favor revise la documentación.

A continuación le mostramos un ejemplo de cómo usar `DecisionTreeClassifier`, para clasificar el iris Dataset.

Tema 3. Algoritmos de aprendizaje automático supervisado

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.datasets import load_iris
#*****
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
Dtree = DecisionTreeClassifier()
y_pred = Dtree.fit(X_train, y_train).predict(X_test)
print("Datos para DT sobre el Test Set")
print(classification_report(y_test, y_pred))
plot_tree(Dtree)
```

Datos para DT sobre el Test Set					
	precision	recall	f1-score	support	
0	1.00	1.00	1.00	21	
1	0.94	0.97	0.95	30	
2	0.96	0.92	0.94	24	
accuracy			0.96	75	
macro avg	0.96	0.96	0.96	75	
weighted avg	0.96	0.96	0.96	75	

Figura 33. Ejemplo de uso de DecisionTreeClassifier, con el iris Dataset.

Clasificación con random forests

Random forests es un método basado en ensembles de árboles de decisión que fue inventado por Leo Breiman y Adele Cutler en el año 2001.

En este tema se describe como los modelos de random forests explotan la diversidad y son capaces de generalizar y predecir mejor que un árbol de decisión. Esta diversidad se consigue por medio de entrenar modelos con el método bagging y de realizar una selección de variables aleatoria.

A continuación, se discute la interpretación del out-of-bag error. Posteriormente nos enfocamos en determinar el número de árboles y en evaluar la importancia de las variables.

Explotando la diversidad: bagging y selección de variables

Uno de los inconvenientes principales de los árboles de decisión es su baja capacidad predictiva. Este inconveniente se puede solventar por medio de utilizar un ensemble o combinación de modelos de árboles de decisión. El modelo random forests es en esencia un ensemble de árboles de decisión.

Los ensembles de árboles se pueden combinar utilizando los métodos de bagging o boosting. Los modelos de random forests se basan en combinar los árboles por medio de los métodos de bagging.

En el caso de problemas de clasificación en cada observación de test se almacena la clase predicha por cada uno de los B árboles y la clase final se obtiene por medio del voto de la mayoría. Es decir, la predicción global es la clase que más veces ocurre a lo largo de las B predicciones. Por otro lado, en el caso de los problemas de regresión la predicción global es la media de las predicciones de cada uno de los árboles.

Tema 3. Algoritmos de aprendizaje automático supervisado

El modelo de random forests combina los principios de bagging con selección de variables aleatorias para añadir diversidad a los árboles de decisión. Una vez es generado el ensemble de árboles (forest) el modelo utiliza el mecanismo de votación o la media para generar las predicciones.

Se trata de un modelo que combina versatilidad y potencia en un enfoque. A la hora de construir cada uno de los árboles se utiliza una porción pequeña y aleatoria de las variables de entrada disponibles, por lo general, este número se define como $\sqrt{p}$ siendo p el número de variables de entrada.

Al ser un modelo que genera cada árbol con un subconjunto de los registros de entrada y una selección aleatoria de las variables, puede trabajar con conjuntos de datos bastantes grandes y no se encuentra afectado por los problemas de curse of dimensionality. Además, debido a que los árboles son modelos con mucho ruido y muy inestables se pueden beneficiar de forma notoria de ser promediados.

En cada uno de los cortes de cada uno de los árboles se elige una serie de variables candidatas de entre todas las posibles. El valor por defecto, $\sqrt{p}$ variables aleatorias de entre todas las posibles, suele dar buenos resultados.

Este modelo se basa en la utilización de un gran número de árboles. La razón de utilizar un gran número de árboles es para que cada variable, de entre todas las posibles, tenga la oportunidad de aparecer en varios modelos.

La ventaja principal de los modelos de random forests frente a un modelo bagged de árboles es debido a que las variables de cada split de los árboles se obtienen por medio de seleccionar un subconjunto de todas las variables de forma aleatoria. De esta forma, se consigue decorrelacionar los árboles generados. Además, se reduce la varianza porque el resultado se obtiene a partir de calcular el promedio de los árboles y se controla el sobreajuste.

Por tanto, cada árbol de decisión se construye utilizando bootstrapped training samples y en cada split se utiliza una variable a partir de una selección aleatoria de m predictores del total p . Finalmente, los distintos árboles se combinan utilizando el voto de la mayoría para clasificación y la media para regresión.

En los random forests debido a la selección de variables aleatorias sobre el conjunto de entrada completo, el sesgo de los árboles se suele incrementar (con respecto al sesgo un único árbol de decisión); no obstante, debido a promediar los árboles la varianza se reduce y, por lo general, en una proporción mayor que el incremento del sesgo dando lugar a un mejor modelo por lo general. En muchos problemas el rendimiento de los random forests es similar a los modelos boosting a pesar de ser más sencillos de entrenar. Como consecuencia de este hecho, los random forests se han convertido en una técnica muy

Tema 3. Algoritmos de aprendizaje automático supervisado

popular.

Interpretación de out-of-bag error

Una característica importante de los modelos random forests es el error out-of-bag. El out-of-bag error es una forma sencilla de estimar el error de test en un modelo bagged. Se puede demostrar que de media cada árbol construido con un modelo bagged utiliza $2/3$ de las observaciones del conjunto de entrenamiento. El $1/3$ restante de las observaciones de entrenamiento no se utilizan para generar el árbol bagged y son llamadas out-of-bag.

Para cada observación se puede obtener la respuesta de los bagged trees, donde esta observación era out-of-bag. Esto proporciona alrededor de $B/3$ predicciones para cada una de las observaciones.

Si el valor de B (número de árboles) es grande esta estimación es equivalente a un modelo de tipo leave-one-out de validación cruzada. Este modelo de leave-one-out es la forma más rigurosa y extrema de evaluar un modelo de aprendizaje automático.

Un modelo random forests se puede entrenar de forma secuencial, donde no es necesario utilizar validación cruzada, pues las muestras out-of-bag proporcionan una estimación similar. Para ello, es necesario observar la evolución del out-of-bag error y una vez que este error se haya estabilizado el algoritmo puede parar el entrenamiento.

Si el valor de B (número de árboles) es grande esta estimación es equivalente a un modelo de tipo leave-one-out de validación cruzada. Este modelo de leave-one-out es la forma más rigurosa y extrema de evaluar un modelo de aprendizaje automático.

Un modelo random forests se puede entrenar de forma secuencial, donde no es necesario utilizar validación cruzada, pues las muestras out-of-bag proporcionan una estimación similar. Para ello, es necesario observar la evolución del out-of-bag error y una vez que este error se haya estabilizado el algoritmo puede parar el entrenamiento.

Evolución del número de árboles e importancia de variables

Los modelos basados en random forests principalmente tienen dos parámetros para optimizar: el número de árboles y el número de variables que se evalúan en cada tirada. Existen otros parámetros como la profundidad de los árboles, pero en la práctica no presentan muchos cambios.

En la siguiente gráfica se observa la evolución del error en el conjunto de test en función del número de árboles y del número de variables que se prueban en cada split. Se observa que el valor de $\sqrt{p}$

Tema 3. Algoritmos de aprendizaje automático supervisado

proporciona un error menor que los otros valores.

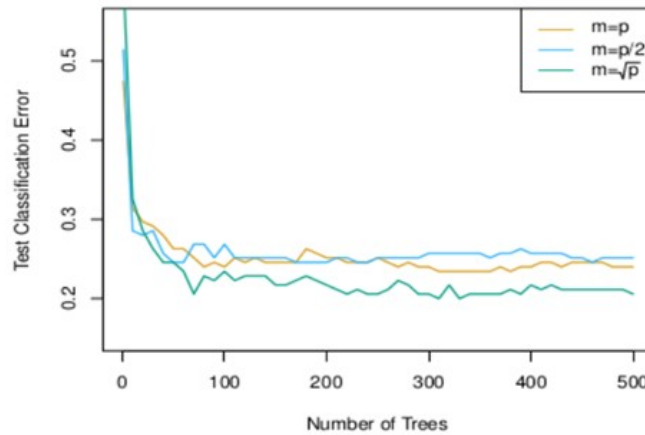


Figura 34. Resultado de tres random forest, en un problema de clasificación con 15 clases y 500 predictores.

En principio se puede pensar que cuanto más grande sea el número de árboles mejor. Sin embargo, por lo general existe un punto óptimo donde a pesar de añadir más árboles el error no se reduce de forma significativa.

Una de las ventajas de los random forests es que pueden generar buenos resultados sin necesidad de muchos ajustes manuales. Además, permiten construir de forma sencilla gráficos de importancia de variables.

En cada decisión de corte de cada árbol la mejora en el criterio de corte es la medida de importancia que se atribuye al corte de esa variable y se agrega para todos los árboles del forest para cada variable.

Random forest también utiliza las muestras fuera del bag (OOB) para construir una medida de importancia de variables que mide la capacidad predictiva de cada una de las variables.

En SciKit-Learn la clase `RandomForestClassifier` del módulo `ensemble`, es la implementación de este método.

Los parámetros más importantes son:

- ▶ `n_estimators`: Espera un entero. Determina la cantidad de árboles en el bosque. El valor por omisión es 100.
- ▶ `criterion`: Puede ser establecido a «Gini» o «entropía». El valor por defecto es «Gini».

Tema 3. Algoritmos de aprendizaje automático supervisado

- ▶ `max_depth`: Espera valores enteros y el valor por defecto es `None`. Establece la profundidad máxima del árbol. Si el valor pasado es `None`, o no se le establece valor, los nodos son expandidos hasta que todas las hojas sean puras o hasta que todas las hojas contengan una cantidad menor de muestras que las especificadas en `min_samples_split`.
- ▶ `min_samples_split`: Espera un valor entero. El valor por defecto es 2. Especifica el número mínimo de casos o muestras requeridos para dividir un nodo interno.

Si este número contiene un valor entero, entonces `min_samples_split` es el mínimo. Si por el contrario este contiene un valor `float`, entonces `min_samples_split` es una fracción y `ceil(min_samples_split*n_samples)` son los números mínimos de muestras.

Este parámetro y el anterior es importante que se tengan en cuenta para optimizar el uso de la memoria, cuando esto es importante.

Ahora veremos un ejemplo de cómo se usa en Python.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

#Lee el dataset de diabetes que está en formato csv y retorna un DataFrame
df = pd.read_csv("diabetes.csv")
dataset = df
columns_incorrect = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
#Sustituye todos los ceros en las variables
for column in columns_incorrect:
    #Separa el dataset en los casos que se saben que son diabéticos de los que no lo son
    df1 = dataset.loc[dataset[column] == 0]
    df2 = dataset.loc[dataset[column] != 0]
    #Reemplaza todos los ceros de las variables con las medianas no con las medias
    df1 = df1.replace({column:0}, np.median(df1[column]))
    df2 = df2.replace({column:0}, np.median(df2[column]))
    dataframe = [df1, df2]
    dataset = pd.concat(dataframe)
#En esta parte tratamos los Outliers
#Ahora utilizaremos un escalador para normalizar los valores de las variables
Y = dataset.Outcome
X = dataset.drop("Outcome", axis = 1)
columns = X.columns
scaler = StandardScaler()
X = scaler.fit_transform(X)
data = pd.DataFrame(X, columns = columns)
#Ahora separamos el dataset en dos partes, un conjunto de entrenamiento y otro de prueba
#El conjunto de prueba es de 15% del total.
x_train, x_test, y_train, y_test = train_test_split(data, Y, test_size = 0.15, random_state = 45)
#Como es evidente que los datos son inválidos, se usa la técnica SMOTE para balancearlos
#Esta técnica genera datos a partir de los existentes, pero solo en el conjunto de entrenamiento
smt = SMOTE()
x_train, y_train = smt.fit_resample(x_train, y_train)
#Ahora aplicamos Random Forest
Forest = RandomForestClassifier()
Forest.fit(x_train, y_train)
y_pred = Forest.predict(x_test)
print("Accurace de RF", classification_report(y_test, y_pred))
```

Figura 35. Ejemplo de aplicación de Random Forest al dataset Pima Indian Diabetes.

Con este ejemplo terminamos lo referente a Random Forest e iniciamos con aquellos métodos se forman por medio de combinaciones de otros métodos.

Tema 3. Algoritmos de aprendizaje automático supervisado

Clasificación con ensembles de clasificadores: bootstrapping, bagging y boosting

En esta sección se van a estudiar los métodos de ensemble. Estos métodos combinan las predicciones de varios estimadores base, que han sido construidos con un algoritmo de aprendizaje automático, para mejorar la generalización/robustez de un único modelo.

En primer lugar, veremos lo correspondiente a bootstrapping. A continuación, se describe el método bagging, el cual se apoya en la técnica de bootstrapping para ensamblar modelos.

Finalmente, veremos el método de boosting, uno de los métodos más populares para llevar a cabo la combinación de modelos débiles para construir un modelo más robusto y con mayor precisión.

Por lo general los métodos de ensemble o combinación de clasificadores se pueden dividir en dos grandes familias:

- ▶ **Averaging methods:** los cuales se basan en construir varios estimadores de forma independiente y luego promediar las predicciones. En media, el estimador combinado se suele comportar mejor que cualquier estimador individual porque la varianza se reduce. Los métodos de bagging funcionan de esta forma.
- ▶ **Boosting methods:** los estimadores base se construyen de forma secuencial con el objetivo de reducir el sesgo del estimador combinado. La motivación es combinar varios modelos débiles para producir un modelo combinado potente.

Técnica de bootstrapping

Bootstrapping es una técnica estadística que consiste en cualquier test o métrica sobre muestreo aleatorio con reemplazo. Bootstrapping permite asignar medidas de precisión definidas en términos de sesgo, varianza, intervalos de confianza o cualquier otra métrica sobre estimaciones muestrales. Esta técnica permite la estimación de la distribución de la muestra de cualquier estadístico utilizando métodos de muestreo. Generalmente, se suele clasificar como una clase de método de muestreo.

Bootstrapping es la práctica de estimar las propiedades de un estimador (así como su varianza) por medio de medir las propiedades al muestrear una distribución aproximada. Una elección habitual para aproximar la distribución es utilizar la función de distribución empírica de los datos observados. En el caso de que las observaciones se puedan asumir que se generan a partir de una población independiente e idénticamente distribuidas, pueden ser obtenidas por medio de muestreos con reemplazo del conjunto de datos observados.

Tema 3. Algoritmos de aprendizaje automático supervisado

La idea sobre la que se basa la técnica de bootstrapping es que la inferencia sobre una población a partir de una muestra de datos puede ser modelada por medio de re-muestrear los datos de la muestra y llevar a cabo inferencias sobre un remuestro de la muestra. Como la población es desconocida el error verdadero en una muestra estadística sobre el valor de la población es desconocido. En el remuestreo en bootstrapping, la población es en realidad la muestra, la cual es conocida. Por tanto, la calidad de la inferencia de la muestra verdadera a partir de los datos muestreados es medible.

Como ejemplo, supongamos que nos interesa conocer la media de la altura de la población mundial. Debido a que es complejo medir la altura de todas las personas en el mundo, en su lugar podemos muestrear una parte de ellas y medir la altura en ellas. Supongamos que el tamaño de esta muestra es N , entonces tenemos la altura de N personas. Con esta muestra solo podemos tener una estimación de la altura media de la población. Para obtener una mejor imagen de la población necesitamos obtener algún tipo de variabilidad sobre los datos obtenidos. El método de bootstrap es el más sencillo para obtener esta variabilidad y consiste en muestrear con remplazo una muestra de tamaño N . Por ejemplo, si muestreamos sobre $[1,2,3,4,5]$ podríamos obtener $[2,5,3,3,1]$. Cuando el tamaño N es suficientemente grande para todos los efectos prácticos existe una probabilidad casi cero de que sea una muestra idéntica a la muestra inicial. Si se repite este proceso miles de veces y para cada muestra se obtiene la media, se obtiene un histograma de medias obtenido con bootstrap.

El método bagging, descrito a continuación, consiste en un meta-algoritmo para ponderar los resultados de múltiples muestras de bootstrapping.

Método bagging

El método de bagging es una clase de algoritmos de ensemble o combinación de clasificadores. El término bagging proviene de la contracción de bootstrap aggregation. Se trata de un procedimiento general que permite reducir la varianza de un método de machine learning. Es un método para combinar varias instancias de estimadores de caja negra que se han construido sobre muestras aleatorias del conjunto de entrenamiento original y que agregan las predicciones individuales para obtener una predicción única.

Dado un conjunto de n observaciones independientes $Z_1 \dots Z_n$, cada una con una varianza σ^2 , la varianza de la media Z de las observaciones es σ^2/n .

Es decir, la media de un conjunto de observaciones reduce la varianza. Sin embargo, como lo habitual es no tener muchas observaciones se suele hacer bootstrap tomando muestras repetidas del data set de entrenamiento.

Tema 3. Algoritmos de aprendizaje automático supervisado

Para ello, se entrenan B conjuntos de entrenamiento distintos y se obtiene la media de las predicciones:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

Estos métodos se utilizan como una forma para reducir la varianza de un estimador base (ejemplo: árboles de decisión) por medio de introducir aleatoriedad en el procedimiento de construcción y a continuación construir el ensemble.

En muchos casos, los métodos de bagging son un método sencillo de mejorar, un único modelo sin la necesidad de tener que modificar el algoritmo de entrenamiento base. Como se trata de un método para reducir el overfitting, los métodos de bagging funcionan mejor cuando se utilizan con modelos complejos (ejemplo: árboles de decisión profundos), en contraste con los métodos de boosting que funcionan mejor con modelos simples. (ejemplo: shallow decisión trees)

Existen diferentes variantes de los métodos de bagging, pero dependen principalmente de la forma en que obtienen las muestras del conjunto de entrenamiento:

- ▶ Cuando se obtienen muestras aleatorias del conjunto de datos como subconjuntos aleatorios de las muestras, el algoritmo se conoce como pasting (Breiman, 1999).
- ▶ Cuando se obtienen los conjuntos para entrenar los algoritmos por medio de muestreo con remplazo del conjunto de entrenamiento, el método se conoce como bagging (Breiman, 1998).
- ▶ Cuando se obtienen subconjuntos aleatorios como subconjuntos del espacio de las variables, el método se conoce como random subspaces (Kam, 1998).
- ▶ Por último, cuando los estimadores base se construyen con subconjuntos de muestras y variables, el método se conoce como random patches (Louppe. Y Geurts, 2012).

Out-of-bag (OOB) error

Existe una forma sencilla de estimar el error de prueba en un modelo bagged, por medio del conjunto de datos que se queda fuera de la muestra. Este conjunto de datos fuera de la muestra, por lo general, suele ser 1/3 del conjunto de datos total. Por tanto, para este 1/3 del conjunto total se puede predecir la respuesta que se hubiera obtenido. Cuando el valor B (número de conjuntos de entrenamiento distintos) es grande, la estimación del out-of-bag error es equivalente al leave-one-out cross validation.

Tema 3. Algoritmos de aprendizaje automático supervisado

Método de boosting

Al igual que bagging, se trata de un método de combinación de modelos que se puede aplicar a los modelos de regresión y de clasificación.

En bagging se crean múltiples copias del conjunto de entrenamiento original utilizando el muestreo bootstrap y entrenando un modelo en cada copia, para posteriormente combinar todos los modelos en uno solo. En bagging cada modelo se construye en un bootstrap data set independiente de los otros. Sin embargo, en el caso de boosting los modelos se generan de forma secuencial, es decir, cada modelo utiliza información de los modelos anteriores.

Boosting es la técnica seguida por los modelos de generalized boosted models (GBMs). En estos modelos, en lugar de entrenar un gran árbol de decisión sobre los datos, lo cual es complicado y puede dar lugar a overfitting, se utiliza el enfoque boosting para aprender poco a poco. Cada uno de los árboles puede ser bastante pequeño, con pocos nodos terminales, pero todos ellos se combinan.

Por tanto, el método boosting puede verse como un meta-algoritmo de ensemble para reducir sesgo y varianza el cual se aplica sobre algoritmos de machine learning supervisados con el objetivo de convertir modelos simples en modelos más precisos.

Este método se basa en una pregunta formulada por Kearns y Valiant (año 1989): ¿Puede un conjunto de modelos débiles crear un modelo fuerte?

En este contexto un modelo débil se define como un clasificador que está poco correlacionado con la clase real. Se trata de ejemplos ligeramente mejor que el acierto aleatorio. Por otro lado, un modelo fuerte se define como un clasificador que está bien correlacionado con la clase verdadera de la clasificación.

La respuesta a la pregunta de Kearns y Valiant ha tenido una gran repercusión en el mundo de machine learning dando lugar al desarrollo de los modelos de boosting.

Cuando se introdujo esta hipótesis de problemas boosting dio lugar a que los algoritmos que consiguieran mejorar por medio de la combinación de modelos débiles fueran llamados boosting.

A pesar de que los algoritmos boosting no tienen ninguna restricción algorítmica, la mayoría de estos algoritmos consisten en ir aprendiendo modelos débiles iterativamente con respecto a una distribución. Cada vez que se añade un nuevo modelo débil, los datos son re-calibrados: los ejemplos que estaban correctamente clasificados pierden peso y los ejemplos incorrectamente clasificados ganan peso.

Tipos de algoritmos boosting

Tema 3. Algoritmos de aprendizaje automático supervisado

Existen diversos algoritmos de tipo boosting. El algoritmo original fue propuesto por Yoav Freund y Robert Schapire en 1997, el cual era no adaptativo y no explotaba de forma completa la ventaja de los modelos débiles. Posteriormente, Freund y Schapire desarrollaron el algoritmo AdaBoost, un algoritmo de boosting adaptativo que ganó el prestigioso premio Gödel.

La principal variación entre los algoritmos de boosting es el método de ponderar los datos de entrenamiento y las hipótesis. AdaBoost es muy popular e históricamente el primero que fue capaz de adaptarse a los modelos débiles. Sin embargo, hay más algoritmos de boosting como: el LPBoost, BrownBoost, TotalBoost, LogitBoost y xgboost.

Hemos hecho hasta aquí, un recorrido por los principales enfoques y sus algoritmos en lo referente al aprendizaje automático supervisado, sin embargo a cualquiera medianamente vinculado con el área se daría cuenta de la ausencia de uno de los iconos de estos tiempos, la estrella del gran show del aprendizaje automático, me refiero a las redes neuronales. Bueno en esta lección no la vamos a tocar por la simple razón que hemos preparado dos lecciones para tratar las redes neuronales clásicas y las redes de aprendizaje profundo.

Conclusiones

Para concluir, solo recomendarles profundizar todo lo que puedan en los temas de álgebra si así lo consideran.

Hemos tocado los principales enfoques, aunque el tema es basto y se han desarrollado cientos de algoritmos y nos es imposible cubrir todo lo que pensamos que es importante. Aun así creemos que hay material para que inicien este fascinante viaje por el mundo de aprendizaje automático.

Tema 4. Algoritmos de aprendizaje automático no supervisado

4.1. Algoritmos de aprendizaje automático no supervisado I

Esta unidad formativa está compuesta por dos lecciones que tratan el tema del aprendizaje automático no supervisado.

En la primera lección se trata de manera inicial las métricas de distancia, concepto que es la piedra angular de los métodos de aprendizajes no supervisados.

Luego se tratan los algoritmos de clustering jerárquicos para continuar con métodos como K-Means, Affinity Propagation, Mean Shift, Spectral Clustering, DBSCAN y por último BIRCH.

La segunda lección la dedicamos exclusivamente a las métricas de validez de los resultados de los algoritmos de aprendizaje automático no supervisados, como son: Calinski-Harabasz Index, Silhouette Coefficient y Davies-Bouldin Index conocidos como criterios de validez interna y luego se tratan los conocidos como criterios externos: Rand Index, Medidas basadas en Información Mutua, Homogeneidad, Completitud y V-Measure, ¡Error! Marcador no definido.

En esta lección trataremos todo lo referente a los métodos de aprendizaje automático no supervisado. En la lección anterior tratamos las métricas de evaluación de los métodos de aprendizaje automático supervisados de regresión y los métodos supervisados para clasificación y en esta, siguiendo ese mismo estilo, inicialmente veremos las métricas de evaluación y a continuación los algoritmos.

El objetivo es, que al final de estas dos lecciones, usted esté en condiciones de aplicar los métodos de aprendizaje no supervisado a problemas prácticos y domine los conceptos esenciales que les permita profundizar luego en la teoría y práctica de estos métodos.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Los objetivos que se persiguen son:

- ▶ Conocer y aplicar las principales métricas de distancia.
- ▶ Conocer y aplicar métodos de agrupamientos como K-Means, Affinity Propagation, Mean Shift, Spectral Clustering, DBSCAN y por ultimo BIRCH.
- ▶ Conocer y aplicar los criterios de evaluación de los resultados de los algoritmos de agrupamientos.

Introducción

En el aprendizaje supervisado dada una serie de etiquetas se “aprendía” un mapeo para obtenerlas. En el caso del aprendizaje no supervisado el problema consiste en probar y determinar la estructura existente en los datos, pero sin utilizar una etiqueta previa. Estos algoritmos, también se conocen con el nombre de algoritmos de agrupamiento o clustering, puesto que agrupan instancias de los datos en función de las variables de los conjuntos de datos. Es decir, este tipo de algoritmos buscan patrones en los datos con el objetivo de encontrar agrupaciones o clúster.

Estas técnicas se utilizan cuando se desconoce la estructura de los datos puesto que no se tiene la variable objetivo de los datos. Por ejemplo, cuando se desconoce cuántos grupos de usuarios similares existen.

Estos métodos dividen los datos en grupos similares. Pero esta división se lleva a cabo sin la necesidad de indicar las características de cada uno de estos grupos. Para este objetivo, las instancias de un grupo deben de ser muy similares entre sí, pero muy distintas a las de otros grupos. Por tanto, para establecer esta distinción entre casos y grupos es imprescindible que definamos unas métricas de similaridad entre objetos y cómo usarla en el contexto de la solución del problema.

La clave de estos algoritmos consiste en buscar buenas variables capaces de distinguir entre las diferentes instancias o registros.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Los algoritmos de agrupamiento se pueden utilizar para diversos objetivos, tales como:

- ▶ **El aprendizaje estadístico semisupervisado:** es aquel que compagina los problemas de clasificación estadística junto con el análisis de grupos.
- ▶ **Filtrado colaborativo:** El agrupamiento proporciona un resumen de usuarios con una mentalidad parecida. Las calificaciones dadas por cada individuo para los demás son usadas para realizar dicho proceso, obteniéndose así sistemas de recomendación.
- ▶ **Segmentación de clientes para agrupamiento de atributos arbitrarios:** Se establece como objetivo para adecuar las características de los productos, a las necesidades de los clientes, utilizándose en el Marketing, la segmentación de mercados, el diseño de producto y la toma de decisiones sobre la tecnología de fabricación.
- ▶ **Minería de datos:** Extrae conocimiento de grandes bases de datos ya sean relacionales o no estructuradas, como la información de las páginas web. Puede detectar tendencias y patrones, así como la detección de datos atípicos.
- ▶ **Resumen de datos y reducción de la dimensionalidad:** El primero permite crear representaciones de datos compactos que se procesan e interpretan más fácilmente. El segundo elimina datos irrelevantes y atributos redundantes. La extracción de características proyecta los datos en un espacio de dimensionalidad más reducida, como ejemplos están el Análisis de Componentes Principales, la Descomposición del Valor y el Análisis Lineal Discriminante. La selección de características selecciona un subconjunto reducido de aquellas, que minimiza la redundancia y maximiza la relevancia del atributo. Destacan entre otros la divergencia de Kullback-Leibler y el Lasso.

Tema 4. Algoritmos de aprendizaje automático no supervisado

- ▶ **Detección de tendencias dinámicas para datos disponibles en streaming:** Con la creciente utilización de toma de datos en tiempo real a través de sensores, cámaras y datos tomados de internet, pueden detectarse tendencias que se produzcan en el tráfico o climatología, por ejemplo. El algoritmo ha de ser adaptable y de fácil escaneo.
- ▶ **Análisis de datos multimedia:** Las imágenes de resonancia magnética permiten conocer la estructura interna de objetos y organismos biológicos. Por ejemplo, determinar la existencia de segmentos similares en las imágenes y realizar la segmentación de las mismas. Esto puede hacerse, por ejemplo mediante un agrupamiento jerárquico. También pueden emplearse las k medias, o también algoritmos borrosos o Fuzzy.
- ▶ **Agrupamiento de documentos, basado en su contenido:** Se utiliza en el almacenamiento y búsqueda automática de documentos. Separa los documentos en grupos que reflejan el contenido de cada documento. Estos algoritmos, han de ser computacionalmente eficientes, dado el elevado volumen de datos manejado.
- ▶ **Análisis de datos biológicos:** Se ha aplicado en la secuenciación del genoma humano. También se han agrupado secuencias lineales de genes y proteínas con niveles similares de expresiones, estableciendo taxonomías biológicas.
- ▶ **Análisis de las redes sociales:** Se utiliza la estructura de la red social para determinar las comunidades importantes de la red subyacente. Esto permite una buena comprensión de la estructura de la comunidad en la red. El agrupamiento también permite resumir las características de la red social.

Los algoritmos de aprendizaje no supervisado se pueden dividir de muchas maneras y es un área de investigación activa, sin embargo de manera general existe consenso en que estos se pueden agrupar en:

- ▶ **Particionales:** tienen definidos de antemano un número de grupos. Son algoritmos iterativos que comienzan con una asignación inicial y se van modificando siguiendo

Tema 4. Algoritmos de aprendizaje automático no supervisado

un criterio de optimización.

- ▶ **Jerárquicos:** En cada iteración solo un objeto cambia de grupo y los grupos están anidados en los de los pasos anteriores. Si un objeto ha sido asignado a un grupo ya no vuelve a cambiar.
- ▶ **Bayesianos:** No los veremos en esta lección.

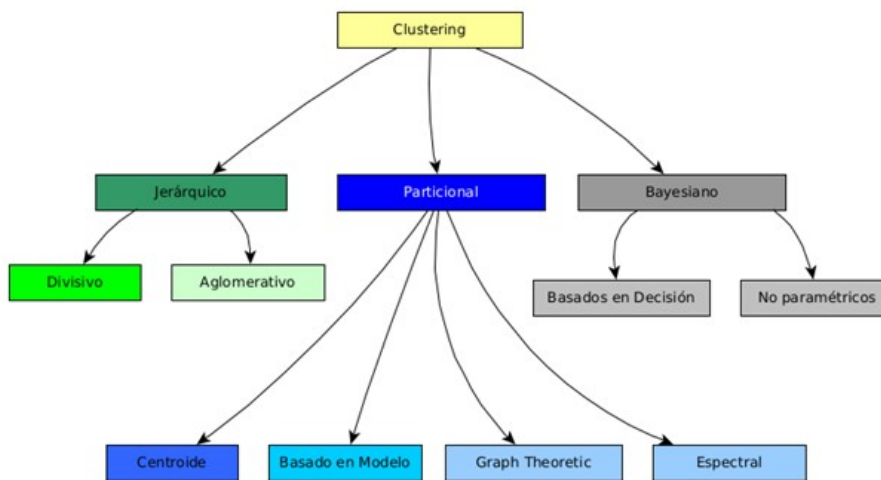


Figura 1. Taxonomía del área de Clustering.

En esta lección y en la que sigue, solo tocaremos los métodos que están en colores en la figura anterior, no tocaremos los que están en color gris.

Aunque esta taxonomía no es completa y hay muchos métodos nuevos, creemos que es importante en una lección de introducción al tema, tocar aquellos métodos que son considerados el núcleo del área de clustering.

Matemáticamente podemos definir el problema de Clustering de la siguiente manera:

Dado un conjunto de datos X de tamaño n y un número positivo k , tal que:

$$k \leq n$$

Encontrar una partición de X en k subconjuntos tal que:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$S_1 \cup S_2 \dots \cup S_k = X$$

Cumpléndose que:

$$k = 1 \quad S_k = \emptyset$$

Es decir que los casos solo deben pertenecer a un solo subconjunto de la partición. A este tipo de modelo se les llama crisp, sin embargo los modelos basados en lógica difusa, como Fuzzy C-Mean, los casos pueden pertenecer a más de un conjunto con cierto grado de pertenencia.

En muchas ocasiones este número k es imposible de conocer a priori y no se tiene información sobre la cantidad de grupos que puede contener el conjunto X . La única forma en que podemos atacar el problema del agrupamiento es definiendo alguna medida que nos permita comparar cada uno de los casos en X con la esperanza de encontrar grupos que nos expliquen la estructura de X .

Para eso, se han desarrollado métricas, sobre las que descansan los métodos de agrupamientos desarrollados.

Métricas de distancia y similaridad

Entonces, sabemos que para poder encontrar grupos o determinar la estructura de un conjunto de datos X , es necesario definir de manera clara, la forma en que comparamos cada $x \in X$. De manera general podemos definir una métrica sobre el conjunto X como $d: X * X \rightarrow R^+$, tal que d cumpla las siguientes propiedades:

- ▶ No negatividad: $d(p, q) \geq 0, \forall p, q \in X$ y $\forall p \in X: d(p, p) = 0$
- ▶ Sea simétrica: $d(p, q) = d(q, p), \forall p, q \in X$
- ▶ Desigualdad Triangular: $d(p, r) \leq d(p, q) + d(q, r), \forall p, q, r \in X$

Entre las medidas de similaridad más comunes tenemos:

Tema 4. Algoritmos de aprendizaje automático no supervisado

Distancia Euclidiana: Es la raíz cuadrada de la suma de las diferencias al cuadrado entre los componentes de los dos vectores o casos que pertenecen a X.

$$d(p,q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Distancia Manhattan: Es la suma de los cuadrados de los valores absolutos entre las diferencias de los componentes del vector o caso que pertenecen a X.

$$d(p,q) = \sum_{i=1}^n |p_i - q_i|^2$$

Distancia máxima:

$$d(p,q) = \max |p_i - q_i|$$

Distancia Minkowski: Esta distancia es una generalización de la distancia euclidiana y depende de un parámetro r, que es un real positivo. Si r=2, entonces esta distancia es equivalente a la euclidiana. Si r=1 entonces esta métrica es equivalente a la distancia Manhattan.

$$d(p,q) = \left(\sum_{k=1}^n |p_k - q_k|^r \right)^{1/r}$$

Distancia Coseno: Básicamente esta métrica es el producto escalar de dos vectores, normalizada con el producto de sus módulos que se resta de 1. Sería 1 menos la similitud coseno es la distancia coseno.

$$d(p,q) = 1 - \frac{p \cdot q}{\|p\| \|q\|} = 1 - \frac{\sum_{i=1}^n (p_i \cdot q_i)}{\sqrt{\sum_{i=1}^n p_i^2} \sqrt{\sum_{i=1}^n q_i^2}} = 1 - \cos \theta$$

El coseno del ángulo entre dos vectores p y q, es una medida de similitud. Por tanto para convertirla en una medida de distancia se hace 1-cos(p,q).

Tema 4. Algoritmos de aprendizaje automático no supervisado

Estas medidas de distancia, asumen que los componentes de cada uno de los vectores son cuantitativos, es decir, cada caso está compuesto por rasgos numéricos. Sin embargo, esto no siempre es así y podemos encontrarnos con muchos conjuntos de datos que tienen rasgos que son más como etiquetas o con rasgos mezclados.

En el caso de datos con rasgos cualitativos, podemos usar la medida de disimilaridad como la medida de distancia de Hamming generalizada igual al número de estos rasgos cuyos valores para los casos comparados son diferentes.

Cuando tenemos que tratar con datos mezclados, es decir, donde tenemos rasgos cualitativos y cuantitativos se puede usar el coeficiente de Gower.

Un análisis completo de las medidas de distancia, similaridad o disimilaridad, se nos escapa, por ser esta una lección de introducción al tema del clustering.

En la práctica no hay un criterio establecido de cuando escoger una medida de distancia u otra. Y la variedad es grande y sigue creciendo. Una de las medidas más usadas en la Euclidiana, aunque es muy sensible a ruidos y tiene dificultades cuando se usa en conjuntos de datos con una alta dimensionalidad.

Métodos de Clustering Jerárquicos

Estos métodos están entre los métodos tradicionales de clustering y consisten en agregaciones o separaciones sucesivas de los casos. El resultado de este tipo de método es una estructura de tipo árbol que se le denomina dendograma.

En el caso de las técnicas aglomerativas, inician viendo cada uno de los casos como un clúster y sucesivamente va encontrando los más cercanos y los va agrupando hasta que no queda más que agrupar.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Algoritmo Clustering Aglomerativo:

Entrada: $X = (x_1, x_2, \dots, x_n)^T$

Salida: $C = \{c_1, c_2, \dots, c_{2n-1}\}$ Dendograma.

1. Inicialización: Establecer a cada caso n de X como un clúster y medir la distancia entre cada uno de ellos. Construir la matriz de distancia $D = d[x_{ij}]$
2. Encontrar un par de C_i, C_j que sean los más cercanos de todos.
3. Formar un nuevo clúster $C_k = C_i \cup C_j$. Esto corresponde a añadir un nuevo nodo en el dendograma y conectarlo con los otros nodos correspondientes a C_i, C_j .
4. Actualizar la matriz de distancias, calculando ahora la distancia de C_k al resto de los nodos, excepto a C_i, C_j .
5. Eliminar de la matriz de distancia las filas y columnas de C_i, C_j y añadir las correspondientes a C_k .
6. Repetir los pasos del 2 al 5 hasta que quede un solo clúster.

Este pseudocódigo nos permite definir a muy groso modo como se procede con este tipo de método, sin embargo el paso 3 esconde detalles especiales y es en donde se concentran las principales diferencias entre los algoritmos que pertenecen a esta clase.

Este paso 3, es implementado de diferentes formas, veamos:

- ▶ En el método conocido como de enlace simple (single Linkage) la distancia entre dos clúster es igual a la distancia entre los dos elementos más cercanos que pertenezcan a los clúster. (este método se conoce como de los vecinos cercanos)
- ▶ En el método conocido como enlace completo (Complete Linkage) o método de los vecinos lejanos, la distancia entre dos clúster se toma como la distancia entre los dos casos más lejanos pertenecientes a clúster diferentes.
- ▶ En el método de enlace promedio (Average Linkage) la distancia entre dos clúster se toma como la distancia promedio de todos los casos que pertenecen a ambos clusters.
- ▶ En el método Promedio ponderado del par-grupo (weighted pair-group average, WPGA) la distancia es calculada similar a la anterior; pero los cálculos se hacen con los pesos igual al número de casos en cada clúster.

Tema 4. Algoritmos de aprendizaje automático no supervisado

- ▶ En el método de los Centroides (unweighted pair-group centroid, UPGC) la distancia es igual a la distancia entre sus centroides o centros de gravedad.
- ▶ En el método conocido como de centroides ponderados (weighted pair-group centroid, WPGC) la distancia se calcula desde los centroides de los clúster aunque en este se usa el número de casos en cada clúster como peso.
- ▶ Este método conocido como el de mínima varianza, se minimiza la suma de los cuadrados entre el objeto y el centro del clúster al que pertenece.

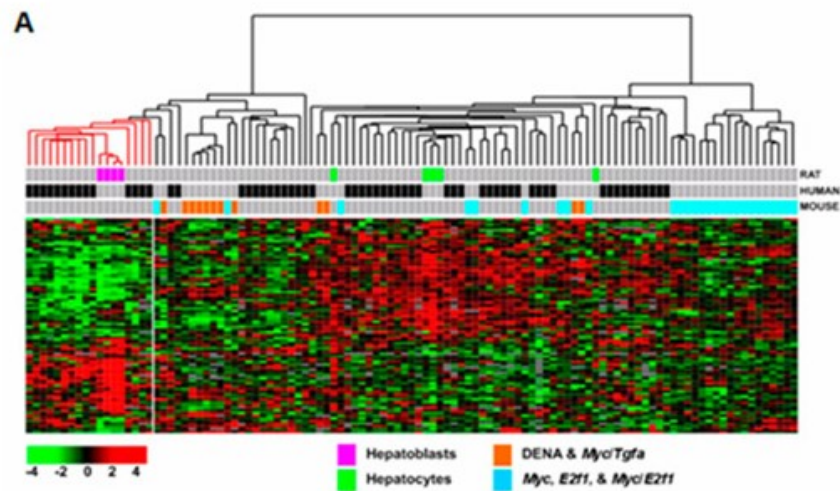


Figura 2. Ejemplo de dendrograma y su mapa de calor para un particionamiento jerárquico.

Tema 4. Algoritmos de aprendizaje automático no supervisado

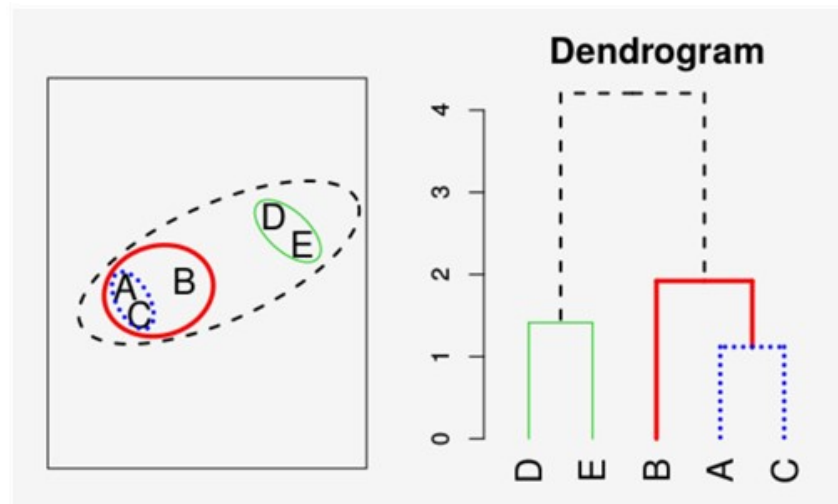


Figura 3. Ejemplo de un dendrograma y su correspondiente agrupamiento.

A diferencia del método anterior, las técnicas divisivas el análisis comienza con un solo clúster formado por todos los casos de X , el cual se va dividiendo sucesivamente hasta que cada caso está contenido en su propio clúster o se llega al criterio de parada establecido.

Algoritmo de Clustering Divisivo

Entrada: $X = (x_1, x_2, \dots, x_n)^T$

Salida: $S_1, S_2, \dots, S_n$

1. Calcular la matriz de distancia basado en una métrica.
2. Dividir el clúster en dos, encontrando el caso más disimilar en cuanto a disimilaridad promedio y poniéndolo en el nuevo clúster, luego se recalcula las métricas de disimilaridad promedio y aquellos casos con valores positivos de mueven también.
3. Se calcula el diámetro de los clusters y se selecciona el de mayor valor para dividirlo.
4. Se repiten los pasos del 2 al 3 hasta que cada clúster contiene un solo caso.

Este pseudocódigo pertenece al algoritmo DIANA o Divisive ANALysis. Estos métodos son muy poco tratados en la literatura y cuando la encuentras el algoritmo no está bien detallado o con ejemplo. Con vistas a ayudar a aclarar cómo se hace este tipo de procesamiento veremos una explicación más o menos detallada basándonos en el ejemplo dado por LEONARD KAUFMAN y PETER J. ROUSSEAU en el libro

Tema 4. Algoritmos de aprendizaje automático no supervisado

donde introducen el método: Finding Groups in Data: An Introduction to Cluster Analysis.

El ejemplo que pondremos es el mismo del libro antes mencionado y por tanto reproducimos aquí los detalles.

EL ejemplo comienza con una matriz de disimilaridad entre los casos {a,b,c,d,e}.

La matriz es la que sigue:

	a	b	c	d	e
a	0.0	2.0	6.0	10.0	9.0
b	2.0	0.0	5.0	9.0	8.0
c	6.0	5.0	0.0	4.0	5.0
d	10.0	9.0	4.0	0.0	3.0
e	9.0	8.0	5.0	3.0	0.0

Como se trata de algoritmo divisivo, asume que todos los datos están inicialmente en el mismo clúster {a,b,c,d,e}.

En el primer paso, como vimos en la descripción del método, el algoritmo tiene que dividir el clúster en dos. Para hacerlo, se busca el caso con la mayor disimilaridad promedio. Cuando hay dos o más objetos con la misma, toma uno al azar. Para poder hacerlo es necesario calcular las disimilaridad promedio entre todos los casos.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Disimilaridad uno vs el resto	
a	$(2.0+6.0+10.0+9.0)/4 = 6.75$
b	$(2.0+5.0+9.0+8.0)/4 = 6.0$
c	$(6.0+5.0+4.0+5.0)/4 = 5.0$
d	$(10.0+9.0+4.0+3.0)/4 = 6.50$
e	$(9.0+8.0+5.0+3.0)/4 = 6.25$

Es evidente que el caso con la disimilaridad promedio más alta es a, entonces se añade a un grupo que llaman Splinter. Por tanto en esta etapa ya tenemos los dos primeros grupos: {a} y {b,c,d,e}. Ahora hay que seguir añadiendo elementos al nuevo conjunto. Para ello se calcula de nuevo la disimilaridad promedio de los objetos del grupo que contiene más elementos y se compara con las disimilaridad del grupo Splinter, que aún es {a}. Veamos cómo queda la tabla.

Objeto	Disimilaridad promedio	Disimilaridad promedio del grupo. Splinter	Diferencia
b	$(5.0+9.0+8.0)/3=7.33$	2.0	5.33
c	$(5.0+4.0+5.0)/3=4.67$	6.0	-1.33
d	$(9.0+4.0+3.0)/3=5.33$	10.0	-4.67
e	$(8.0+5.0+3.0)/3=5.33$	9.0	-3.67

Teniendo en cuenta los nuevos valores recalculados son el caso a y comparado con las disimilaridades de la tabla original, entonces el único que tiene un valor positivo es b, por tanto lo añadimos al Splinter. Ahora nos queda dos grupos: {a,b} y {c,d,e}.

El próximo paso es encontrar cuál de los grupos hay que separar de nuevo y para eso se calcula el diámetro de cada clúster. El diámetro de un conjunto es igual a la

Tema 4. Algoritmos de aprendizaje automático no supervisado

disimilaridad más grande entre dos elementos del clúster. Teniendo en cuenta esto:

$$D(\{a,b\}) = 2.0$$

$$D(\{c,d,e\}) = 5.0$$

¿De dónde sale esto?, pues de las disimilaridades de la matriz de disimilaridad comparando {c,d,e}.

Esto nos dice que el clúster a dividir es {c,d,e}. De nuevo se repite el paso anterior.

Es decir, se escoge el caso con la mayor disimilaridad promedio entre c,d,e:

Objeto	Disimilaridad Promedio
c	$(4.0+5.0)/2=4.5$
d	$(4.0+3.0)/2=3.5$
e	$(5.0+3.0)/2=4.0$

Teniendo en cuenta el resultado del cálculo de la disimilaridad promedio entonces el caso c, es el de mayor disimilaridad, entonces se hace otro clúster con él y el Splinter nos quedaría: {a,b},{c,d,e},{c}. Pero aún no terminamos, hay que ver que otros casos pueden añadirse a este nuevo clúster.

Para hacerlo, como hicimos en el paso anterior, se comparan las disimilaridades de los casos del clúster con los del Splinter.

Objeto	Disimilaridad promedio	Disimilaridad promedio del grupo. Splinter	Diferencia
d	3.0	4.0	-1.0
e	3.0	5.0	-2.0

Estas medidas salen de comparar d y e en la matriz de disimilaridad y contra los casos en el Splinter.

Como todos los casos tienen diferencia negativa, entonces no se puede hacer nada y

Tema 4. Algoritmos de aprendizaje automático no supervisado

terminamos con el proceso y nos quedas los clúster de la siguiente forma: $\{a,b\}$, $\{c\}$ y $\{d,e\}$.

En el nuevo paso, chequeamos si podemos dividir de nuevo, los clúster, de la misma forma que lo habíamos hecho en los pasos anteriores. Buscamos el clúster con el diámetro mayor, veamos cómo lo hacemos:

Obviamente el clúster $\{c\}$, no puede dividirse, mientras que el clúster $\{a,b\}$ tiene diámetro 2, mientras que $\{d,e\}$ tiene diámetro 3. Por tanto hay que dividirlo.

La matriz de similitud de estos dos casos es:

	d	e
d	0.0	3.0
e	3.0	0.0

Como las disimilaridades son las mismas podemos escoger cualquiera de los casos. Por tanto podemos escoger el caso d para formar el otro clúster, entonces ahora el Splinter queda compuesto por: $\{a,b\}$, $\{c\}$, $\{d\}$ y $\{e\}$.

Por tanto, en el próximo paso, de nuevo calculamos los diámetros de los clúster y como tres de ellos son singletons, nos queda solo dividir $\{a,b\}$ en dos clúster singletons más, y al terminar este paso nos quedamos con el Splinter igual a: $\{a\}$, $\{b\}$, $\{c\}$, $\{d\}$ y $\{e\}$.

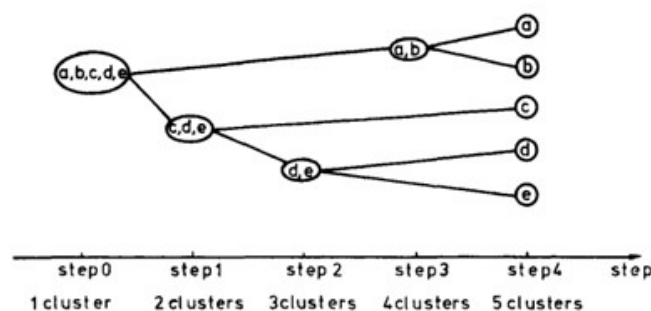


Figura 4. Dendrograma para el ejemplo anterior.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Es importante destacar que el ejemplo que acabamos de exponer pertenece a los llamados métodos top-down y que AGNES otro algoritmo que puede considerarse gemelo de DIANA pero que trabaja de la forma opuesta, es considerado como un modelo Botton-up.

AGNES es sinónimo de HAC, mientras que DIANA los HDC. Que significan Hierarchical Agglomerative clustering y Hierarchical Divisive Clustering.

Algoritmos de Clustering

En esta sección iremos explicando, aunque de manera muy superficial el funcionamiento de algunos de los algoritmos y veremos ejemplos de su uso en Python.

Los métodos que veremos en esta sección son los siguientes:

- ▶ K-Means.
- ▶ Affinity Propagation.
- ▶ Mean Shift.
- ▶ Spectral clustering.
- ▶ DBSCAN.
- ▶ BIRCH.

Comenzaremos con el análisis de estos algoritmos y en la próxima lección veremos los métodos de validación de los resultados de estos.

K-Means

Este es uno de los algoritmos de clustering más usado y del cual abunda documentación, por eso lo tocamos primero.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Este algoritmo se puede clasificar como basado en centroide. La idea detrás de este método es minimizar la suma cuadrada de las distancias de los casos dentro del clúster.

Esto sería: Dado un conjunto de casos $X = (x_1, x_2, \dots, x_n)^T$ encontrar una partición $S_1, S_2, \dots, S_n$ tal que $s_n S_{n-1} = \emptyset$ que minimice la inercia.

Entonces el papel del algoritmo es encontrar la distribución de todos los casos entre los clúster que minimice la suma de los cuadrados de las distancias entre cada caso y centroide del clúster al que pertenece.

Digámoslo de otro modo. K-Mean divide el conjunto de N casos en K clúster disjuntos C_k , cada uno descrito por la media m_j de los casos en el clúster. Estas medias son conocidas como «centroides»; Es importante notar que estos centroides no son casos concretos de X.

Entonces, K-mean elige los centroides de forma que minimicen la «Inercia» o criterio de la suma cuadrada intra-clúster.

El algoritmo consiste en asignar cada uno de los, n casos a uno de los k clusters, donde k es un número definido previamente. El objetivo es minimizar las diferencias entre los grupos de cada clúster y maximizar las diferencias entre clusters.

A menos que k y n sean extremadamente pequeños no es factible calcular los grupos óptimos entre todas las combinaciones posibles de ejemplos. En su lugar, el algoritmo utiliza un proceso heurístico para calcular la solución óptima.

El algoritmo comprende dos fases:

- ▶ Asigna ejemplos a un conjunto inicial de k clusters.
- ▶ Después actualiza las asignaciones ajustando los límites de los grupos de acuerdo con los ejemplos de cada clúster.

Tema 4. Algoritmos de aprendizaje automático no supervisado

- Este proceso de asignación y actualización ocurre varias veces hasta que los cambios no proporcionan mejoras en los clusters.

Debido a la naturaleza heurística del algoritmo los resultados pueden ser distintos en función de la inicialización del algoritmo. Sin embargo, si los resultados difieren mucho los unos de los otros puede indicar un problema. Por ejemplo, puede ocurrir que los datos no se puedan agrupar bien en k clusters.

El algoritmo k -medias considera que los valores de las variables son coordenadas en un espacio multidimensional.

Ejemplo de ejecución

El algoritmo empieza eligiendo los k puntos iniciales.

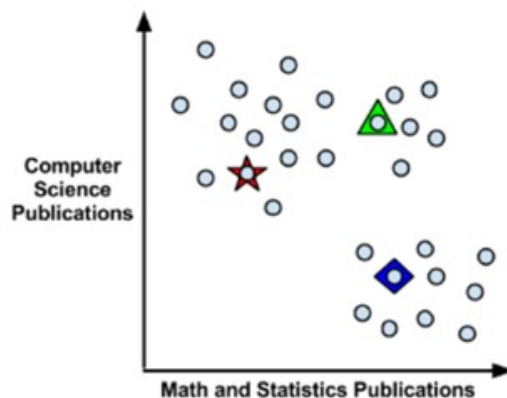


Figura 5. Elección inicial de tres puntos como centroides.

Los puntos iniciales se suelen elegir al azar, en este caso se eligen tres ejemplos al azar. Otras opciones son elegir puntos que pueden ocurrir en cualquier intervalo del espacio de las variables de entrada. Otra opción es asignar inicialmente de forma aleatoria cada punto a un clúster.

Después de elegir los puntos iniciales, los otros ejemplos se asignan al centroide del clúster más cercano de acuerdo a la función de distancia, esta función suele ser la

Tema 4. Algoritmos de aprendizaje automático no supervisado

distancia euclidiana.

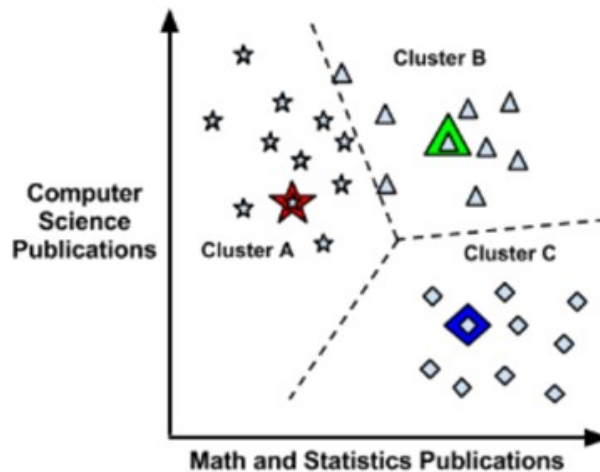


Figura 6. Asignación del resto de los puntos a los centroides.

Hay que tener en cuenta que, debido a que estamos usando distancias, todos los datos deben de ser numéricos y normalizados antes de utilizarlos. El primer paso de la actualización conlleva a reubicar los centroides iniciales a una nueva posición calculada como la media de los puntos asignados al clúster. Como los límites de los centroides se han modificado es muy posible que haya que reasignar instancias a otros clusters.

Tema 4. Algoritmos de aprendizaje automático no supervisado

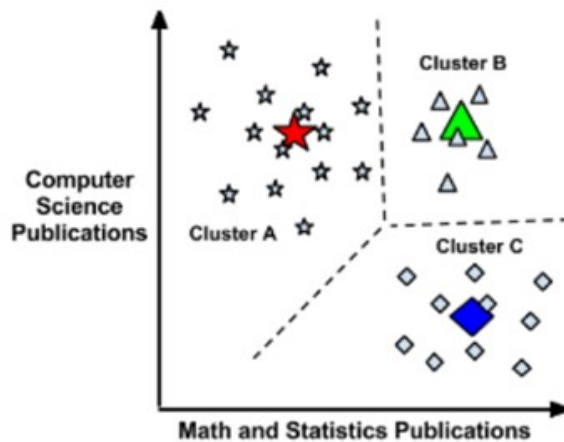


Figura 7. Reajuste de los centroides en función de la asignación del resto de puntos.

Elegir el valor de K

Elegir un buen valor de K requiere de cierto balance. Un número muy grande mejora la homogeneidad, pero a la vez sobre-ajusta los datos.

Idealmente existe un conocimiento a priori sobre el número de grupos apropiado. Por ejemplo, si estamos agrupando películas se puede elegir el valor de k que concuerde con los géneros existentes. Otras veces el número de clusters viene dado por los requisitos de negocio.

Sin conocimiento a priori se suele elegir un valor de $k = \sqrt{(n/2)}$ También se puede usar métricas para medir la homogeneidad versus la heterogeneidad.

A continuación mostramos un ejemplo de cómo se usa KMeans en Python usando Scikit-Learn. Aunque en `scipy.cluster.vq` también hay una implementación de este.

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering con K-Means
from numpy import unique
from numpy import where
from sklearn.cluster import KMeans
from matplotlib import pyplot
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf= datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# el número de clústers que queremos
model = KMeans(3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
pyplot.show()
```

Figura 8. Ejemplo de uso de K-Means en Python, con Scikit-Learn.

El resultado de este programa se da en la siguiente imagen.

```
Cluster:0 cantidad de casos:50
Cluster:1 cantidad de casos:53
Cluster:2 cantidad de casos:47
```

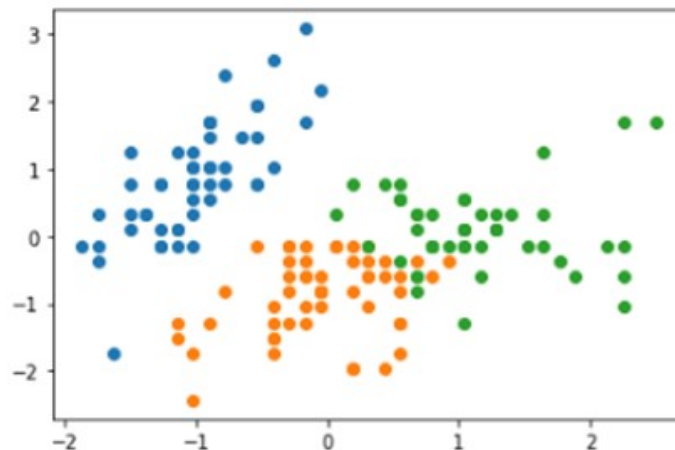


Figura 9. Resultados de la corrida de K-Means sobre Iris DataSet.

Como puede verse, este método se acerca bastante bien a la realidad, aunque hay que pasarle el número de clúster. Esta es una de las desventajas de usar este tipo

Tema 4. Algoritmos de aprendizaje automático no supervisado

de método, pues la mayoría de las veces no conocemos esa cantidad.

También en este mismo módulo se implementa una variante de KMeans llamada MiniBatchKMeans. Revisar la documentación para los detalles.

Affinity Propagation

En este algoritmo, el conjunto de datos puede verse como una red, donde los casos se mandan mensajes entre unos y otros. La idea detrás de esta comunicación es tratar de encontrar aquellos casos que tienen las características adecuadas para representar al resto de los casos de un posible clúster como ejemplar. Los ejemplares, son casos que pueden representar mejor a otros puntos y son más significativos para el grupo. Esta ejemplaridad se evalúa por medio de dos criterios, la responsabilidad y la disponibilidad (*availability*).

Estos criterios se obtienen por medio de los mensajes que se pasan entre todos los casos y el resultado se almacenan en dos matrices:

- ▶ La matriz de responsabilidad R: En esta matriz se almacena un valor $r(i,k)$ que dice cuan capacitado está el caso k para representar, como ejemplar, al caso i.
- ▶ La matriz de «availability» A: Este criterio, $a(i,k)$, explica cuan apropiado para i, sería, elegir a k como ejemplar.

Estos criterios se calculan a partir de una medida de similaridad. En este caso los desarrolladores del algoritmo decidieron usar una variante de la distancia euclidiana.

Específicamente la distancia euclidiana negativa al cuadrado.

$$s(i,k) = -\|x_i - x_k\|^2$$

Entonces la responsabilidad se puede calcular como:

$$r(i,k) = s(i,k) - \max \{a(i,k') + s(i,k')\}$$

Tema 4. Algoritmos de aprendizaje automático no supervisado

Mientras más alto es el valor de r , mejor preparado está k para representar a i como ejemplar.

La disponibilidad o «availability» se calcula de la siguiente forma:

$$a(i,k) \leftarrow \min \left(0, r(k,k) + \sum_{i' \notin \{i,k\}} \max(0, r(i',k)) \right) \quad \forall i \neq k$$
$$a(k,k) \leftarrow \sum_{i' \neq k} \max(0, r(i',k))$$

De esta forma, un caso k , es electo como ejemplar de un clúster basado en: que es suficientemente similar a muchos casos y es elegido por muchos casos como ejemplar.

Una de las ventajas de usar este método es que no hay que especificar de antemano la cantidad de clúster en el dataset, sin embargo es necesario especificar la preferencia, por medio del parámetro “preference” y este está ligado a la cantidad de ejemplares que se escogen y por tanto a la cantidad de grupos que retorna en método.

Veamos un ejemplo de cómo se usa en Python y los resultados que se obtiene sobre el dataset iris.

En la figura 11 se puede observar el resultado de la aplicación de Affinity Propagation al Iris Dataset.

En la figura 11, se puede ver claramente cuantos casos se han asignado a cada uno de los clústers. Si recordamos que el iris dataset tiene 150 casos, tres clases de flores y 50 casos en cada clase, entonces mirando la cantidad de casos que asignó a cada una de las clúster, es un buen síntoma. Sin embargo, no veremos las métricas de evaluación de estos métodos hasta la próxima lección.

Es momento de que veamos los parámetros o argumentos que toma el método a la

Tema 4. Algoritmos de aprendizaje automático no supervisado

hora de construirlo.

```
#Clustering con Propagación por afinidad(AfinitPropagation)
from sklearn.cluster import AffinityPropagation
import matplotlib.pyplot as plt
from sklearn import datasets
from itertools import cycle
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# preference y damping
model = AffinityPropagation(preference=-50,damping=0.9,random_state=0)
af = model.fit(X)
cluster_centers_indices = af.cluster_centers_indices_
labels = af.labels_
n_clusters = len(cluster_centers_indices)
plt.close("all")
plt.figure(1)
plt.clf()
colors = cycle("bgrcmykbgrcmykbgrcmykbgrcmyk")
for k, col in zip(range(n_clusters), colors):
    class_members = labels == k
    cluster_center = X[cluster_centers_indices[k]]
    plt.plot(X[class_members, 0], X[class_members, 1], col + ".")
    plt.plot(
        cluster_center[0],
        cluster_center[1],
        "o",
        markerfacecolor=col,
        markeredgecolor="k",
        markersize=14,
    )
    for x in X[class_members]:
        plt.plot([cluster_center[0], x[0]], [cluster_center[1], x[1]], col)
plt.title("Número estimado de clusters: %d" % n_clusters)
plt.show()
```

Figura 10. Ejemplo de uso de clustering con Affinity Propagation en Python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

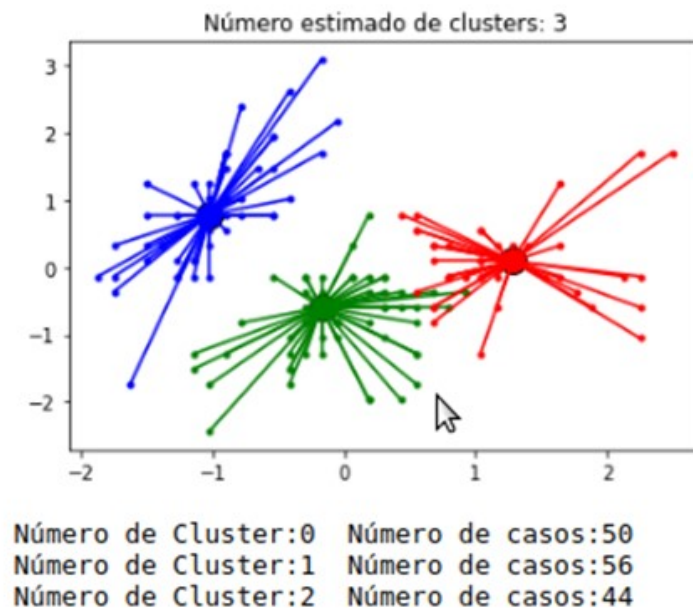


Figura 11. Resultados del uso de clustering con Affinity Propagation en Python usando Iris Dataset.

Parámetros:

- ▶ **damping:** Este parámetro es de tipo float y su valor debe estar entre 0.5 y 1. Se denomina factor de damping, su función es evitar las oscilaciones numéricas y su valor por defecto es 0.5.
- ▶ **max_iter:** Especifica el número máximo de iteraciones, espera valores enteros y el valor por defecto es 200.
- ▶ **preference:** Puede tomar forma de arreglo de `n_samples`. Cuando se pasa como preferencia un arreglo de valores, estos son interpretados como los valores de preferencia para cada uno de los casos. Entre más grande es el valor para un individuo, más seleccionable como ejemplar es. También puede ser un float y el valor por defecto es `None`. Cuando se pasa un único valor es equivalente a pasar el mismo valor de preferencia a todos los casos. Mientras más alto es este valor, más clúster genera este algoritmo.

Yo propongo que para ajustar este valor se use validación cruzada. Sin embargo

Tema 4. Algoritmos de aprendizaje automático no supervisado

cuando el dataset no es muy grande se pueden probar algunas combinaciones, aunque su valor depende del conjunto de datos.

Affinity: Este parámetro define el tipo de medida de similaridad a usar, los valores que se pueden usar son: “euclidean” o “precomputed”. El valor por defecto es “euclidean” y especifica que se usará la distancia euclidiana negativa cuadrada.

Estos son los parámetros más significativos, veremos ahora que retorna este algoritmo luego de ser ajustado a los datos.

- ▶ `cluster_centers_indices_`: Luego de usar el método `fit()`, esta variable contiene un arreglo de tamaño igual al número de clústers que contiene los valores correspondientes a los índices de los centros de cada clúster.
- ▶ `cluster_centers_`: Contiene el valor de los centros de los clústers, si la distancia usada no es “precomputed”.
- ▶ `labels_`: Retorna un ndarray con las etiquetas asignadas a cada caso.

Para conocer detalles adicionales de la implementación de este método, es necesario consultar la documentación de scikit-learn.

Mean Shift

Este algoritmo puede considerarse dentro de la familia de los algoritmos basados en centroide. También se en la literatura se le clasifica dentro de los métodos de los algoritmos de búsqueda de la moda de un conjunto de datos.

Este método, no hace asunción alguna sobre la naturaleza de la distribución estadística de los datos y por tanto también puede clasificarse como no paramétrico. Tampoco pone restricciones sobre la forma de los clúster.

Otra de las ventajas de este es que no es necesario pasarle de antemano la cantidad de clústers en que se pretende agrupar nuestros datos.

Tema 4. Algoritmos de aprendizaje automático no supervisado

La idea detrás de este algoritmo es tratar al conjunto de datos como una muestra de alguna distribución de probabilidad y aproximar su función de densidad probabilística. Luego ejecutar un proceso de gradiente ascendente hasta encontrar las modas de la función.

La aproximación de la función de densidad probabilística se hace por medio de la Estimación de densidad Kernel o KDE. (Kernel Density Estimation)

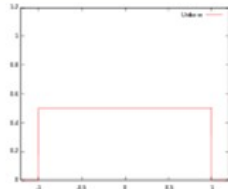
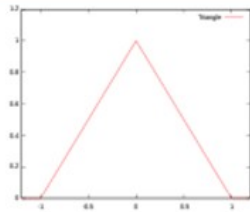
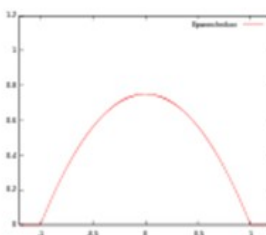
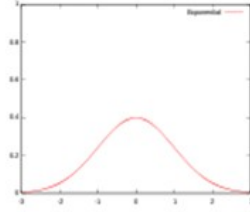
Aunque no es nuestra intención hacer un estudio detallado de la matemática detrás del algoritmo, hagamos algunas introducciones para que se entienda su funcionamiento.

¿Qué es un Kernel? Un Kernel, es una función que toma valores reales, es integrable y no negativa. Además debe satisfacer los siguientes requerimientos:

- ▶ Normalización: $\int_{-\infty}^{+\infty} K(u) du = 1$
- ▶ Simetría: $K(u) = K(-u) \forall u$

Hay varios Kernels muy usados, por ejemplo:

Tema 4. Algoritmos de aprendizaje automático no supervisado

Kernel	Forma del Kernel
Kernel uniforme: $K(u) = \frac{1}{2}$, para $ u \leq 1$	
Kernel Triangular: $K(u) = (1 - u)$, para $ u \leq 1$	
Kernel Epanechnikov: $K(u) = \frac{3}{4}(1 - u^2)$ (parabolic)	
Kernel Gaussian: $K(u) = \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}$	

Aunque muchos Kernel, nos interesa, para explicar el funcionamiento de este método los que mostramos en la tabla anterior.

Sabiendo que es un Kernel y cómo lucen, tanto matemáticamente como desde la forma de la función, estamos en condiciones de entender la manera que se encuentra la función de densidad probabilística a partir de nuestro conjunto de datos.

Para esto, supongamos que nuestro dataset está representado por $\{x_1, x_2, \dots, x_n\}$, entonces la función de densidad probabilística se calcula basada en estos casos, de la siguiente forma:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$f(x) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right) (1)$$

Donde h es llamado bandwidth.

Para explicar esto y desarrollar la intuición de que significa esto, veamos la siguiente imagen:

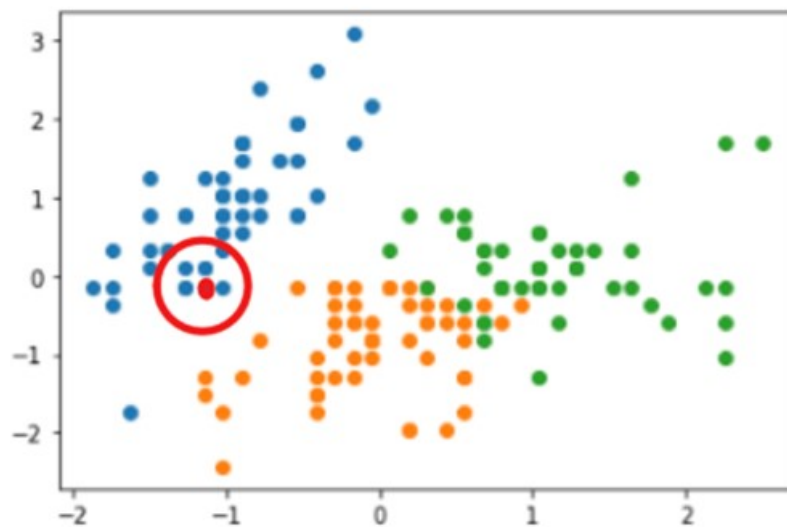


Figura 12. Ejemplo de la interpretación geométrica de la aplicación de un Kernel a un conjunto de datos.

En la figura 12, se muestra un conjunto de datos representados en el plano bidimensional. El círculo rojo representa, aproximadamente, como debe imaginarse la aplicación de la función (1) a n conjunto. El diámetro del círculo está determinado por el parámetro h. Y se le llama vecindad del punto x.

Esto quiere decir que para el cálculo de la función de densidad, basado en nuestro dataset, se aplica un círculo como este y aplica usando los casos que caen dentro de este. Luego se desplaza al otro punto y se calcula de nuevo. La suma de todos los Kernel da como resultado la función de densidad.

Tema 4. Algoritmos de aprendizaje automático no supervisado

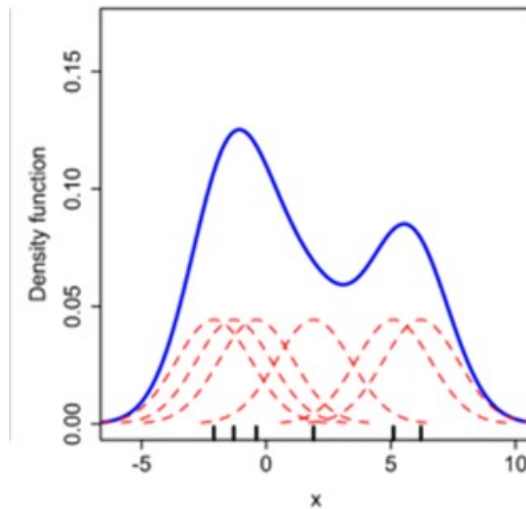


Fig.13. Función de densidad probabilística obtenida por KDE. Usando 6 datos. Cada una de las campanas discontinuas pertenece a la aplicación de un Kernel a un punto.

En la figura 13 se da otra perspectiva de cada uno de los Kernel independientes, distinguidos por líneas discontinuas rojas. Es importante notar que esta curva gaussiana tiene media en el dato x . La función de densidad, entonces, se obtiene sumando las contribuciones individuales.

En la siguiente imagen, se muestra una interpretación geométrica de cómo debe, intuitivamente imaginarse una función de densidad en un espacio tridimensional.

Tema 4. Algoritmos de aprendizaje automático no supervisado

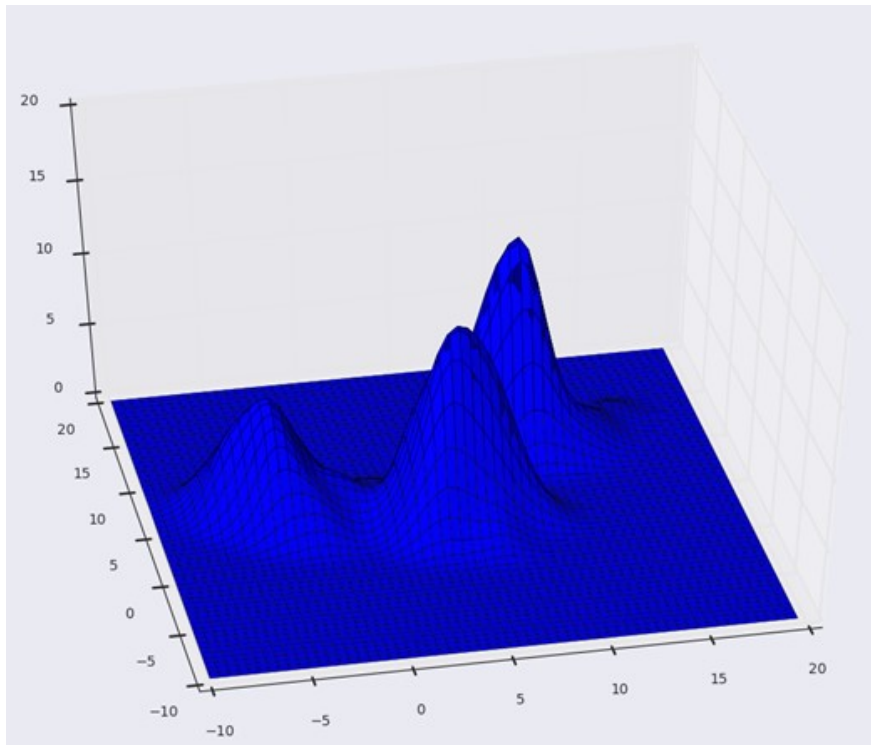


Figura 14. Ejemplo de una Función de Densidad Probabilística, con tres modas o máximos.

Es en la cresta de estos picos donde se da la máxima densidad de casos, es decir, los casos están más agrupados alrededor de estos puntos.

El cálculo de esta función es solo el primer paso en el funcionamiento de este método. Ahora, lo que se hace, es que se calcula el gradiente de cada uno de los puntos o casos, basado en la función de densidad, de la siguiente forma:

$$\nabla f(x) = \frac{2C_{k,d}}{nh^{d+2}} \left[\sum_{i=1}^n g\left(\left\|\frac{x-x_i}{h}\right\|^2\right) \right] \left[\frac{\sum_{i=1}^n x_i g\left(\left\|\frac{x-x_i}{h}\right\|^2\right)}{\sum_{i=1}^n g\left(\left\|\frac{x-x_i}{h}\right\|^2\right)} - x \right]$$

Donde, h es el ancho de banda o bandwidth que especifica el ancho de la ventana del Kernel, d es el número de rasgos de nuestro dataset, g es $-K'(x)$ y es la derivada del Kernel. El primer término, si se dan cuenta, es proporcional al estimado de la función de densidad en el punto x , mientras que el segundo es el vector de la media, que siempre apunta en la dirección de máximo incremento del gradiente y es

Tema 4. Algoritmos de aprendizaje automático no supervisado

denotado por m .

Pues el método MeanShift, ahora lo que hace es:

- ▶ Se elige un punto y se calcula el vector de la media $m(x)$.
- ▶ Se traslada la ventana de estimación de la densidad para otro punto x donde el gradiente sea mayor y se calcula de nuevo $m(x)$.
- ▶ Se repite hasta que se encuentra un punto x para el cual el gradiente es cero. En ese punto se dice que el algoritmo converge.

Veamos cómo se aplica en la práctica, usando el módulo Scikit-learn de Python, y luego veremos los parámetros que acepta.

```
#Clustering con Mean Shift
from sklearn.cluster import MeanShift
from sklearn.cluster import estimate_bandwidth
import matplotlib.pyplot as plt
import statistics as stat
from numpy import unique
from numpy import where
from sklearn import datasets
from itertools import cycle
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# bandwidth y bin_seeding
bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=150)
model = MeanShift(bandwidth=bandwidth, bin_seeding=True)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
plt.show()
```

Figura 15. Ejemplo de uso de MeanShift, calculando el Bandwidth explícitamente. Esto no es estrictamente necesario, pues de no pasarle el parámetro o pasarle None, la misma implementación lo hace.

El resultado de la corrida de este ejemplo, se muestra en la siguiente imagen.

Tema 4. Algoritmos de aprendizaje automático no supervisado

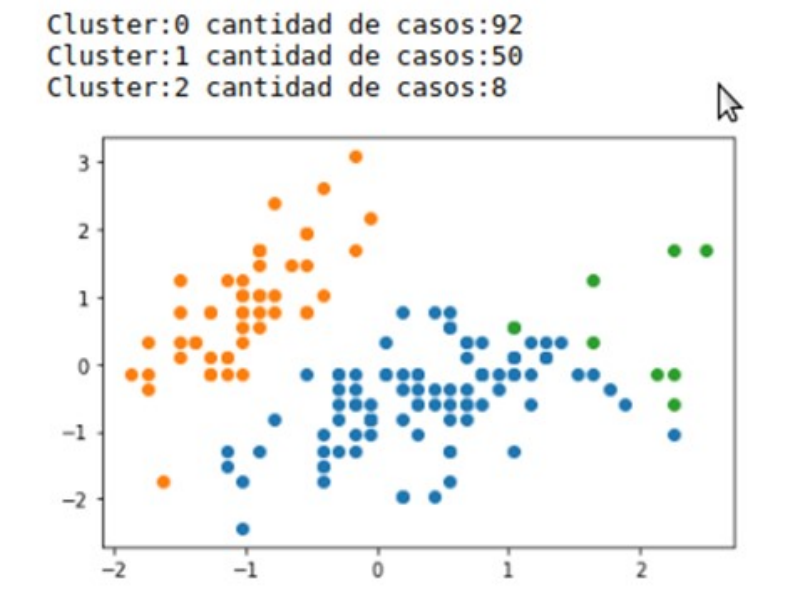


Figura 16. Resultados de la corrida de Mean Shift, sobre el Iris Dataset.

por cada una de las clases y que la corrida de MeanShift distribuye 92 casos a un solo clúster ya nos habla de que necesitamos reconsiderar los parámetros que les estamos pasando.

Veamos los parámetros necesarios para correr MeanShift y su significado.

Parámetros:

bandwidth: Espera un valor de tipo float y el valor por defecto en None. Este parámetro define el tamaño de la ventana para la estimación de KDE. Si se pasa como valor None o no se pasa ningún valor, se usa `sklearn.cluster.estimate_bandwidth` para calcularlo.

Este es el parámetro que define el funcionamiento del algoritmo, por tanto no veremos otros y es necesario que consulten la documentación.

Esta implementación retorna:

- **cluster_centers_:** Es un ndarray con forma (n_clusters, n_features) y contiene las

Tema 4. Algoritmos de aprendizaje automático no supervisado

coordenadas de los centros de los clústers.

- `labels_`: contiene un ndarray, con las etiquetas asignadas a cada uno de los casos.

Spectral clustering

Este tipo de método ha tenido un significativo éxito en la práctica al ser capaz de tener buenos resultados donde otros fallan. La idea detrás de este tipo de método de aprendizaje es ver al conjunto de datos como un grafo, donde cada caso es visto como un vértice que están unidos por medio de vértices que denotan su similaridad o distancia.

Por tanto, dado un conjunto de datos $\{x_1, x_2, \dots, x_n\}$ y una métrica de afinidad o similaridad, como puede ser la norma, L1, L2 o euclidiana, entonces podemos construir un grafo G , tal que:

$$G = \{V, E\}$$

G es un conjunto ordenado de vértices y de aristas o arcos, donde los vértices corresponden a casos en el conjunto de casos y los arcos son las similaridades entre ellos. En la Fig. 17, se muestra como se vería un grafo de este tipo.

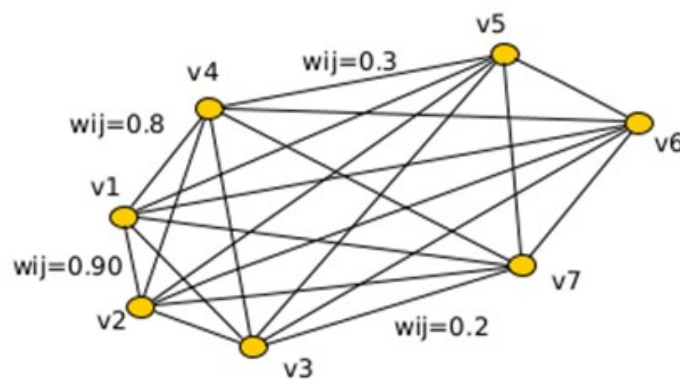


Figura 17. Ejemplo de Grafo completamente conectado de 7 vértices. Los w_{ij} representa la similaridad entre vértices.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Entonces:

$V = \{x_1, x_2, \dots, x_n\}$ es el conjunto de vértices que representan los datos.

$E = \{m_{ij}\}$ es el conjunto de todos los pesos de los arcos que conectan dos vértices.

Asumiremos que el grafo es ponderado y que los pesos de los arcos que unen los vértices, tienen pesos no negativos.

El grado de un vértice, es la suma de los pesos de los arcos que lo tienen como un extremo. De esta forma, el grado d del vértice i , es:

$$d_i = \sum_{j=0}^n w_{ij}$$

Hay varias maneras en las que podemos construir un grafo de similitud. Las más tratadas en la literatura son:

- ▶ Grafo ε -vecindad o ε -neighborhood: En este tipo de grafo, solo se incluyen los vértices y cuyos arcos tienen peso menor o mayor que ε .
- ▶ Grafos de vecindad de los k -vecinos cercanos: En este se incluyen los k vértices que son vecinos cercanos de del vértice v_i .
- ▶ Grafo completamente conectado: En este tipo de grafo se incluyen todos los vértices y los pesos que reflejan su similitud o distancia.

Un grafo, tiene asociada una matriz de afinidad, distancia o similitud que contiene las similitudes entre cada uno de los vértices. Normalmente se representa como W y luce como el Figura 18.

Tema 4. Algoritmos de aprendizaje automático no supervisado

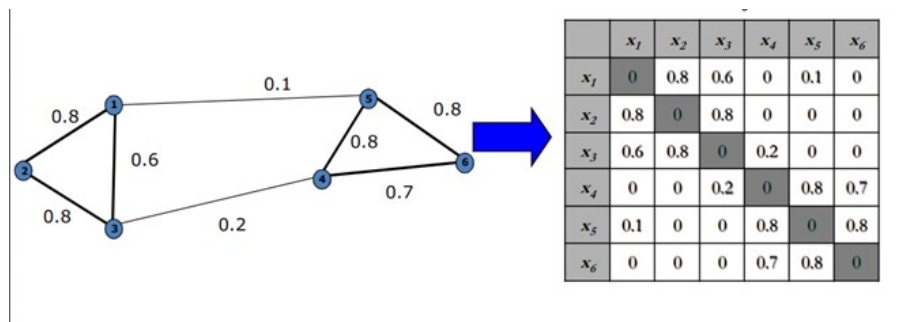


Figura 18. Matriz de similaridad de un Grafo.

También, tiene asociada una Matriz llamada, matriz de grado (degree) y denotada por D , en la cual los valores de la diagonal son los grados de cada uno de los vértices del grafo G . Un ejemplo de esta matriz, se muestra en la figura 19.

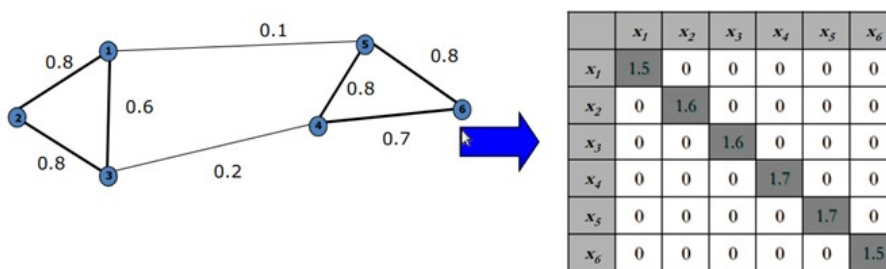


Figura 19. Matriz de grado, del grafo G .

Usando estas dos matrices anteriores se construye otra, llamada Laplaciano o matriz Laplaciana del grafo G y se denota por L . La matriz L se obtiene de la siguiente forma:

$$L = D - W$$

En la fig. 20 mostramos el Laplaciano de las matrices que hemos usado en las dos imágenes anteriores.

Tema 4. Algoritmos de aprendizaje automático no supervisado

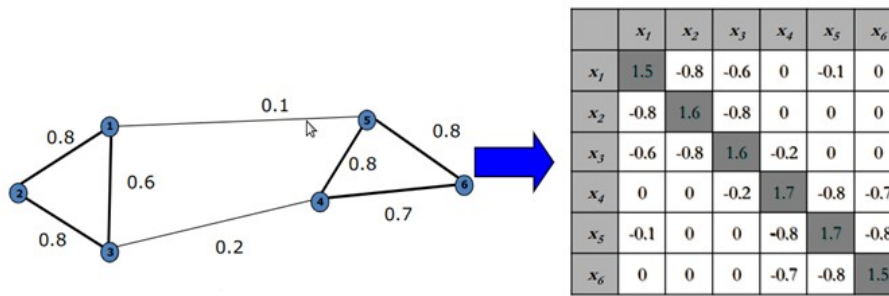


Figura 20. Ejemplo de un Laplaciano, calculado a partir de las dos matrices de la imagen 18 y 19. W y D.

Cada Matriz, tiene unos valores y unos vectores asociados a estas que la identifican de manera única. A estos valores y vectores se les llaman valores o vectores propios. Normalmente en la literatura se les llaman Eigenvalues y Eigenvector y el término «Eigen» se puede entender, aproximadamente como «propio». Por tanto a estos, también, se les llama valores o vectores propios de la matriz.

Los valores propios de una matriz, son aquellos que satisfacen la ecuación característica de ella.

Si tenemos una matriz $A \in \mathbb{R}^{n \times n}$, entonces, al conjunto de raíces de la ecuación:

$$\det(A - \lambda I) = 0$$

Se llaman valores propios de A.

Para cada valor propio de una matriz A hay, asociado con este, un vector que se le llama vector propio de A si,

$$A\vec{x} = \lambda\vec{x}$$

Donde $\vec{x}$ es un vector propio de A y λ el valor propio asociado con él.

De esta forma, al conjunto de vectores propios del Laplaciano del grafo G se le denomina Espectro de G. Hay todo un campo de investigación llamado Teoría

Tema 4. Algoritmos de aprendizaje automático no supervisado

espectral de grafos que se dedica al estudio de estos valores y los métodos para su obtención.

Con todo lo anterior explicado, podemos ahora, a grandes rasgos, claro está describir en que consiste el algoritmo de clustering espectral.

Spectral Clustering:

Entrada: $\{x_1, x_2, \dots, x_n\}$ y el número k de clúster.

- ▶ Construir una matriz de similitud a partir de la entrada, S .
- ▶ Encontrar la matriz Laplaciana de S , llamada L .
- ▶ Encontrar, los k valores y vectores propios de L .
- ▶ Construir una nueva matriz, usando los vectores propios como columnas.
- ▶ Interpretando las filas de la nueva matriz, como vectores en un nuevo espacio vectorial $R^{n \times k}$ aplicar un algoritmo de clustering K-Means.

Lo más interesante aquí, la intuición detrás del funcionamiento de este método es que proyecta nuestro conjunto de datos a un nuevo espacio vectorial donde puede realizarse la separación de los clúster de una manera más simple.

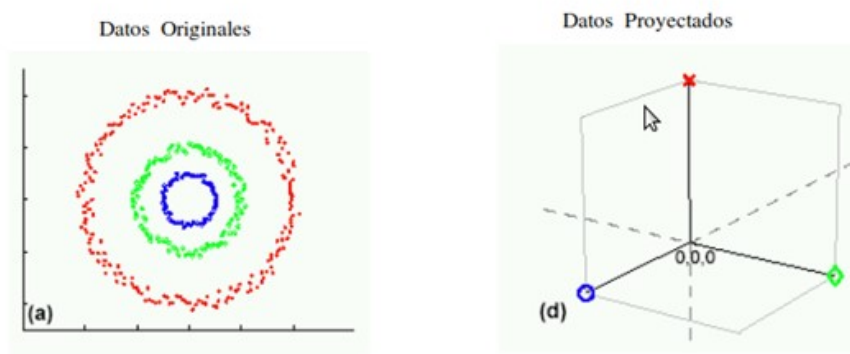


Figura 21. Este método proyecta los datos originales y luego aplica el clustering.

Tema 4. Algoritmos de aprendizaje automático no supervisado

A grandes rasgos, esta es la idea básica detrás de este método de clustering. Veamos ahora como se usa la implementación hecha en Python y que resultados arroja al aplicarlo al dataset iris. Es importante recordar que este dataset, contiene 150 casos, divididos en tres clases y 50 casos en cada una de las clases. Eso nos da una medida, de manera intuitiva de cuan bien puede separar un modelo, los casos del dataset, hasta que en la próxima lección tratemos los métodos de evaluar los resultados de estos métodos.

```
#Clustering con Clustering Espectral
from sklearn.cluster import SpectralClustering
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# el número de clúster que sabemos de antemano que es 3
model = SpectralClustering(3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los lusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
plt.show()
```

Figura 22. Ejemplo de aplicación de Clustering Espectral, sobre el iris Dataset.

Los resultados de la aplicación de este método al iris dataset se muestran en la siguiente figura:

Tema 4. Algoritmos de aprendizaje automático no supervisado

Cluster:0 cantidad de casos:50
Cluster:1 cantidad de casos:63
Cluster:2 cantidad de casos:37

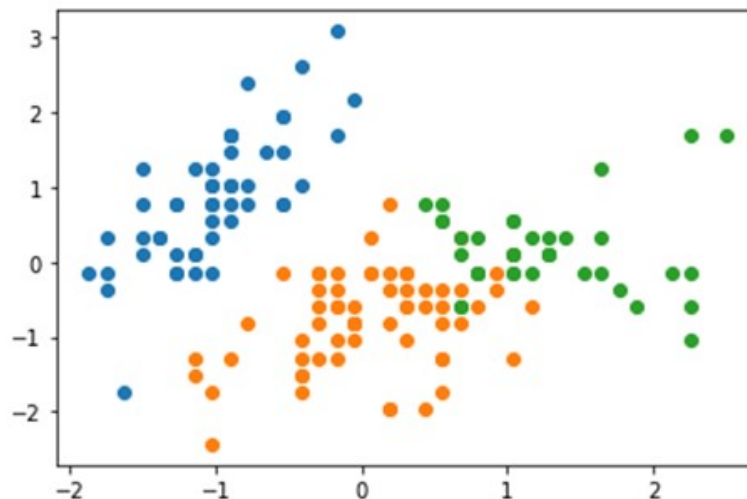


Figura 22. Resultados de la aplicación de Spectral Clustering al iris Dataset.

Veamos los parámetros que esta implementación toma.

Parámetros:

- ▶ **n_clusters:** Este parámetro define la cantidad de clústers en los que se dividirán los datos de entrada y es equivalente a la dimensión del subespacio de proyección. Es decir, especifica la cantidad de dimensiones que tendrá el subespacio. Espera valores enteros y el valor por defecto es 8.
- ▶ **eigen_solver:** Especifica la estrategia para obtener los vectores propios. Este parámetro acepta tres valores: `arpack`, `lobpcg`, `amg`. El valor por defecto es `None` que usa `arpack`, como solver.
- ▶ **n_components:** Especifica la cantidad de vectores propios usar para el subespacio de proyección. Cuando no se especifica valor se usa `n_clusters`.
- ▶ **affinity:** Define el método por el cual se construye la matriz de afinidad o similaridad.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Los valores que acepta son:

- `nearest_neighbors`: Construye la matriz de afinidad a partir de un grafo de los k -vecinos cercanos.
- `rbf`: Construye la matriz de afinidad usando un Kernel de Base Radial. (RBF) Toma como coeficiente el valor del parámetro `gamma`.
- `precomputed`: Interpreta los datos de entrada `X` como una matriz de afinidad ya construida, donde los valores mayores significan similitudes más altas entre los casos.
- `precomputed_nearest_neighbors`: Interpreta los datos de entrada `X` como un grafo disperso de distancias calculadas de antemano y construye una matriz de afinidad binaria a partir del valor del parámetro `n_neighbors` de los vecinos más cercanos a cada instancia.
- ▶ `n_neighbors`: Espera valores enteros y por defecto toma el valor 10. Este parámetro especifica la cantidad de vecinos cercanos a usar cuando se usa `nearest_neighbors` como valor para el parámetro `affinity`. Es decir cuando se pretende construir la matriz de afinidad por el método ϵ -neighborhood.

Como hemos estado haciendo con los métodos anteriores, solo describimos aquellos parámetros que son los que definen la conducta del método, el resto de los parámetros deben consultarse en la documentación de la implementación.

Este método retorna los siguientes valores:

- ▶ `affinity_matrix_`: Después que se ejecuta el método `fix()`, esta variable contiene la matriz de afinidad o matriz de similitud.
- ▶ `labels_`: Contiene las etiquetas de cada uno de los casos. Su tamaño es igual al número de casos que contiene el dataset.

Hasta aquí hemos visto los elementos esenciales y la idea detrás del clustering

Tema 4. Algoritmos de aprendizaje automático no supervisado

espectral, en la siguiente sección veremos otros métodos de clustering o métodos de aprendizaje no supervisado.

DBSCAN

Este método de aprendizaje automático se clasifica dentro de los métodos basados en densidad. La idea detrás de este método es identificar un clúster con un área densa que está separada de otros por un área menos densa.

Su nombre «Density-based spatial clustering of applications with noise», puede traducirse, más o menos, como «Agrupamiento basado en densidad espacial para aplicaciones con ruido» pero no dice mucho de las ideas que tiene detrás.

Este método ve la densidad como la cantidad de casos o puntos en un área determinada, donde la distancia entre ellos es menos que a la del resto del clúster. Este tipo de método no asume ninguna forma en el grupo de casos.

La densidad de cada punto se estima contando el número de puntos, que se encuentran a una distancia no mayor que Eps del punto. Dependiendo de dicho valor, cada punto es clasificado como central (core), frontera (border) o ruido (noise). Eps, significa épsilon.

Tema 4. Algoritmos de aprendizaje automático no supervisado

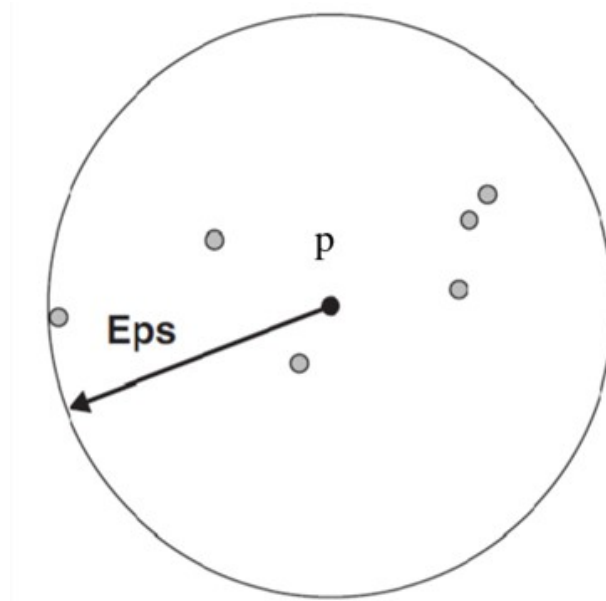


Figura 23. Eps-vecindad de un punto p.

A este círculo con centro p y radio eps se le llama vecindad del punto p. Esto nos lleva a la primera definición.

Definición #1: Eps-vecindad: Se llama eps-vecindad de un punto y denotada por $N_{eps}(p)$ y definida como sigue:

$$N_{eps}(p) = \{q \in D \mid \text{dis}(p, q) \leq \text{eps}\}$$

Esto se lee, la eps-vecindad de un punto p, son todos los puntos que pertenecen a D, el conjunto de datos, tal que la distancia entre p y q es menor que eps.

Sin embargo esto no explica completamente la manera en que un clúster queda definido, pues los puntos del borde tienen una eps-vecindad mucho menor que los del centro y por tanto habría que definir una cantidad mínima de puntos que defina la densidad de cada punto.

Definición #2: Directamente densidad-alcanzable: Un punto p es directamente densidad-alcanzable si:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$\forall p \in C, \exists q \vee p \in N_{eps}(q) \wedge N_{eps}(q) \vee \geq MinP$$

Esto se lee, el punto p es directamente densidad-alcanzable, si pertenece al clúster C y existe un punto q, tal que p pertenece a su eps-vecindad y esta tiene al menos MinP puntos.

Este tipo de relación es simétrica para los puntos centrales o core, del clúster, sin embargo no lo es si uno de los puntos es de frontera.

Teniendo en cuenta este MinP ahora podemos definir los tipos de puntos:

- ▶ **Punto central (core):** Un punto se llama central o core si su eps-vecindad contiene más de MinP puntos.
- ▶ **Punto de frontera:** Un punto es de frontera si no es un punto central; pero hay al menos un punto central dentro de su eps-vecindad.
- ▶ **Punto Ruido:** Un punto se considera Ruido si no es ni central ni frontera.

Definición #3: densidad-alcanzable: Se dice que un punto p es densidad-alcanzable desde un punto q si existe una cadena de puntos $p_1, p_2, \dots, p_n$ donde $p_1 = q; p_n = p$ tal que p_{i+1} es directamente densidad-alcanzable desde p_i .

Definición #4: densidad-conectado: Un punto p es densidad-conectado a un punto q, si existe un punto o tal que, tanto p como q sean densidad-alcanzables desde o.

Ahora, con los conceptos anteriores se puede definir la noción de clúster basado en densidad.

Definición #5: Clúster: Si denotamos por D al conjunto de datos o dataset, un clúster C en D, es un subconjunto no vacío de D que satisface las siguientes propiedades:

- ▶ $\forall p, q$ si $p \in C$ y q es densidad-alcanzable desde p, entonces $q \in C$.
- ▶ $\forall p, q \in C$: p es densidad-conectado a q.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Definición #6: Ruido: Hagamos que $C=\{C_1, C_2, \dots, C_k\}$ sea el conjunto de clúster del dataset D , atendiendo a los parámetros ϵ y MinP , entonces se puede definir el ruido como el conjunto de puntos en D que no pertenecen a ningún C_k clúster.

Es decir, el ruido puede definirse como:

$$\text{ruido} = \{p \in D \mid p \notin C_k\}$$

Esto, en palabras llanas significa que se considera ruido al conjunto de puntos que no pertenecen a ningún clúster.

Algoritmo DBSCAN

- ▶ Etiquetar todos los puntos como central, frontera o ruido.
- ▶ Eliminar los puntos ruido.
- ▶ Unir los puntos centrales que estén a una distancia menor que ϵ .
- ▶ Cada grupo de puntos centrales conectados forma un clúster.
- ▶ Asignar cada punto frontera al clúster de uno de sus puntos centrales.

De manera general, hemos descrito cómo se realiza el proceso de agrupamiento por medio de DBSCAN. Claro, no hemos hablado de la manera de escoger los valores de ϵ y MinP ; sin embargo este tema no lo veremos en esta lección.

En la siguiente imagen veremos un ejemplo de aplicación de DBSCAN al Iris Dataset, usando la implementación de este en scikit-learn de Python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering DBSCAN
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# eps y min samples
model = DBSCAN(eps=0.68, min_samples=15)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
plt.show()
```

Figura 24. Ejemplo de aplicación de DBSCAN al Iris Dataset.

```
Cluster:0 cantidad de casos:44
Cluster:1 cantidad de casos:45
Cluster:2 cantidad de casos:61
```

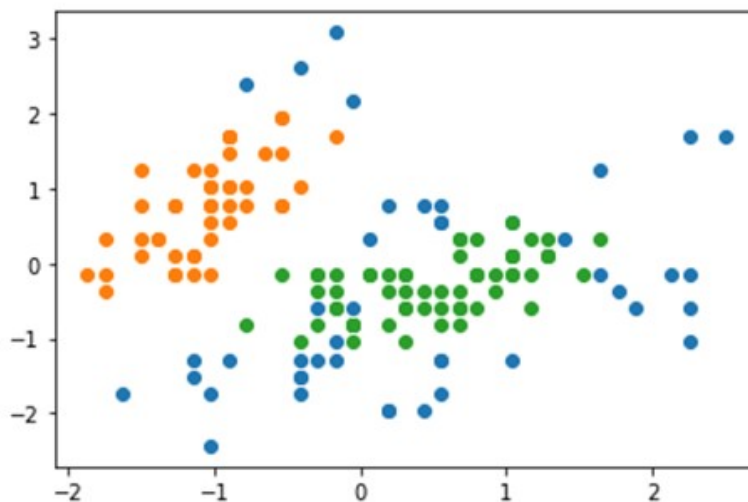


Figura 25. Resultados de la aplicación de DBSCAN al Iris Dataset.

Es importante que tengamos en cuenta que los parámetros son escogidos de manera empírica y no siguiendo ninguno de los métodos propuestos para ello.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Veremos ahora los parámetros o argumentos necesarios para utilizar el algoritmo. Como siempre, en esta sección solo veremos aquellos parámetros que son más importantes para la operación del algoritmo, dejando el resto para que sean consultados en la documentación del método en caso de ser necesario.

Parámetros:

- ▶ **eps:** Este parámetro espera un valor de tipo float y el valor por defecto es 0.5. Este es el principal parámetro de DBSCAN y define el tamaño de la ε -vecindad. Puede interpretarse como la distancia máxima entre dos casos o puntos para que una se considere en la vecindad de la otra.
- ▶ **min_samples:** Espera valores enteros y el valor por defecto es 5. Representa el MinP que hemos descrito antes y define el número mínimo de casos o puntos en la vecindad para que un punto sea considerado como central o core.
- ▶ **metric:** Espera un valor de cadena o string y el valor por defecto es 'euclidean'. Define la métrica que se va a usar para calcular la distancia entre puntos. Puede tomar los valores siguientes:
 - **scikit-learn:** 'cityblock', 'cosine', 'euclidean', 'l1', 'l2', 'manhattan'.
 - **scipy.spatial.distance:** 'braycurtis', 'canberra', 'chebyshev', 'correlation', 'dice', 'hamming', 'jaccard', 'kulsinski', 'mahalanobis', 'minkowski', 'rogerstanimoto', 'russellrao', 'seuclidean', 'sokalmichener', 'sokalsneath', 'sqeuclidean', 'yule'

Retorna:

- ▶ **core_sample_indices_:** Contiene los índices de los puntos core, que se encontraron.
- ▶ **labels_:** Contiene las etiquetas de todos los puntos o casos, asignadas por DBSCAN.
- ▶ Hasta aquí hemos revisado, aunque de manera muy general el método de aprendizaje no supervisado DBSCAN, no entramos en detalles sobre el algoritmo;

Tema 4. Algoritmos de aprendizaje automático no supervisado

pero, si se necesita entender más profundamente es necesario estudiar con detalles la documentación disponible que es abundante.

La mayoría de los métodos de aprendizaje no supervisado, tal como K-means y otros, no son muy eficientes a la hora de tratar con grandes conjuntos de datos. Estos no escalan bien en términos de tiempo de corrida y uso de memoria en la medida que los conjuntos de datos aumentan su tamaño y aquí es donde entra en escena el algoritmo BIRCH, que estudiaremos brevemente en la siguiente sección.

BIRCH

Estas siglas significan «Balanced Iterative Reducing and Clustering using Hierarchies» y podría traducirse, más o menos como «Agrupamiento y reducción iterativa equilibrada usando jerarquías». Aunque el nombre no arroja mucha luz sobre la forma en que se ejecuta el clustering, lo cierto es que da algunas pistas.

Una de las características interesantes y únicas de este método es que no hace el agrupamiento directamente sobre el conjunto de datos, sino que hace una especie de sumario de este, sobre el que luego puede aplicarse cualquiera de los algoritmos ya estudiado para realizar el clustering. Esto es especialmente importante cuando se está trabajando un dataset grande, tanto en número de casos como de rasgos.

Este, se puede categorizar como un algoritmo de clustering jerárquico y solo necesita un paso sobre el dataset para encontrar buenos clústers.

La idea detrás de este método es capturar la estructura del dataset y sumarla en un árbol que se da en denominar árbol CF. La estructura central sobre la que descansa la construcción de dicho árbol, es llamada CF o Clustering Feature y contendrá información sobre un determinado grupo de puntos. Así que es importante que definamos que se entiende por **Clustering Feature**.

Definición #1: Clustering Feature (CF):

Tema 4. Algoritmos de aprendizaje automático no supervisado

Si X es un conjunto de datos tal que $X = \{x_1, x_2, \dots, x_n\}$ entonces Clustering Feature (CF) es una estructura triple:

$$CF = (N, \overrightarrow{LS}, SS)$$

Donde N , es el número de puntos de datos en el clúster que trata de caracterizar el CF.

LS , por otra parte, es la suma lineal de todos los N puntos de X , calculada como sigue:

$$\overrightarrow{LS} = \sum_{i=1}^N \overrightarrow{x_i}$$

Y SS , sería la suma del cuadrado de los x de X , tal como mostramos a continuación:

$$SS = \sum_{i=1}^N (\overrightarrow{x_i})^2$$

Cada CF, caracteriza a un clúster del conjunto de datos X .

Habiendo definido en que consiste la estructura central que soporta la idea básica de este algoritmo, es necesario que definamos las maneras en que se miden las distancias y se calculan los elementos y luego que exploremos la manera en que se construye el árbol y por tanto la manera en que se ejecuta el agrupamiento.

Para entender la forma en que procede este algoritmo hay que definir algunos conceptos como el del Centroide, el Radio y el Diámetro de un Clúster, entonces:

Definición #2: Centroide de un Clúster $\overrightarrow{X_0}$: Se considera el punto central de un clúster y se obtiene de la siguiente forma:

$$\overrightarrow{X_0} = \frac{1}{N} \sum_{i=1}^N \overrightarrow{x_i}$$

Tema 4. Algoritmos de aprendizaje automático no supervisado

Definición #3 Radio de un Clúster R: Se puede considerar como la distancia media entre los puntos que pertenecen al clúster y el centroide $\vec{X}_0$ y se puede obtener por medio de la siguiente expresión:

$$R = \left[\frac{1}{N} \sum_{i=1}^N (\vec{x}_i - \vec{x}_0)^2 \right]^{\frac{1}{2}}$$

Definición #4 Diámetro de un Clúster D: Esta medida puede considerarse como la suma de las distancias, tomados dos a dos, entre todos los puntos de un clúster y se puede definir como sigue:

$$D = \left[\frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=1}^N (\vec{x}_i - \vec{x}_j)^2 \right]^{\frac{1}{2}}$$

Definición #5 Distancia Euclidiana entre dos centroides $\vec{X}_0$ y $\vec{X}_1$, que llamaremos D_0 :

$$D_0 =$$

Definición #6 Distancia Manhattan entre dos centroides $\vec{X}_0$ y $\vec{X}_1$, que llamaremos D_1 :

$$D_1 = |\vec{x}_0 - \vec{x}_1|$$

También es necesario que definamos otro grupo de medidas que son necesarias para entender cómo se construye el árbol CF de este método.

Para ello supongamos que tenemos un par de conjuntos de puntos o clústers $\vec{X}_i$ y $\vec{X}_j$, donde $i = 1, 2, \dots, N_1$ y $j = N_1 + 1, N_1 + 2, \dots, N_1 + N_2$, entonces podemos definir:

Definición #7: Distancia media inter-clúster, que llamaremos D_2 :

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$D_2 = \left[\frac{1}{N_1 N_2} \sum_{i=1}^{N_1} \sum_{j=N_1+1}^{N_1+N_2} (\vec{x}_i - \vec{x}_j)^2 \right]^{\frac{1}{2}}$$

Definición #8: Distancia media intra-clúster, que llamaremos D3:

$$D_3 =$$

Definición #8: Distancia Incremento de la Varianza, que llamaremos D4:

$$D_4 = \sum_{k=1}^{N_1+N_2}$$

Si consideremos cada CF como un vector, entonces, se puede definir la forma en que estos se pueden sumar.

Teorema #1: Aditividad de los CF: Si asumimos que $CF_1 = (N_1, \overrightarrow{LS_1}, SS_1)$ y $CF_2 = (N_2, \overrightarrow{LS_2}, SS_2)$ son vectores de dos clústers disjuntos, entonces el vector resultante de la suma de estos vectores es:

$$CF_1 + CF_2 = (N_1 + N_2, \overrightarrow{LS_1} + \overrightarrow{LS_2}, SS_1 + SS_2)$$

Veamos ahora que aspecto tendría un árbol CF y cómo se construye y de paso, vemos la manera que se hace el agrupamiento.

Bueno, lo primero es que este árbol, tiene asociado dos parámetros que definen la forma de este y son: El factor de ramificación, que llamaremos B y el umbral, al cual llamaremos T.

Teniendo en cuenta esto, es preciso que definamos lo que llamaremos Regla T y que de alguna forma define el tamaño del árbol.

Definición #9: Regla T: Llamaremos Regla T al hecho de que ninguna de las entradas en una hoja puede tener radio o diámetro mayor que T. Donde T es el

Tema 4. Algoritmos de aprendizaje automático no supervisado

parámetro T o umbral.

Cuanto T es menor, el diámetro o radio de las entradas es menor y por tanto absorben menos puntos y son necesarias más entradas, por tanto el árbol crece.

Un árbol CF, tiene un nodo raíz, que contendrá uno o varios CF. También tendrá uno o varios nodos ramas o noleaf y uno o varios nodos hojas. En la Fig. 26, se muestra una representación de cómo luciría un árbol CF. Lo que está por encima de la línea es el dataset y lo que está por debajo de ella es la abstracción que hace el algoritmo de los datos reales.

Es importante que quede claro que el algoritmo no trae a la memoria los datos reales, sino sus características que sumaria de manera implícita en cada uno de los CF.

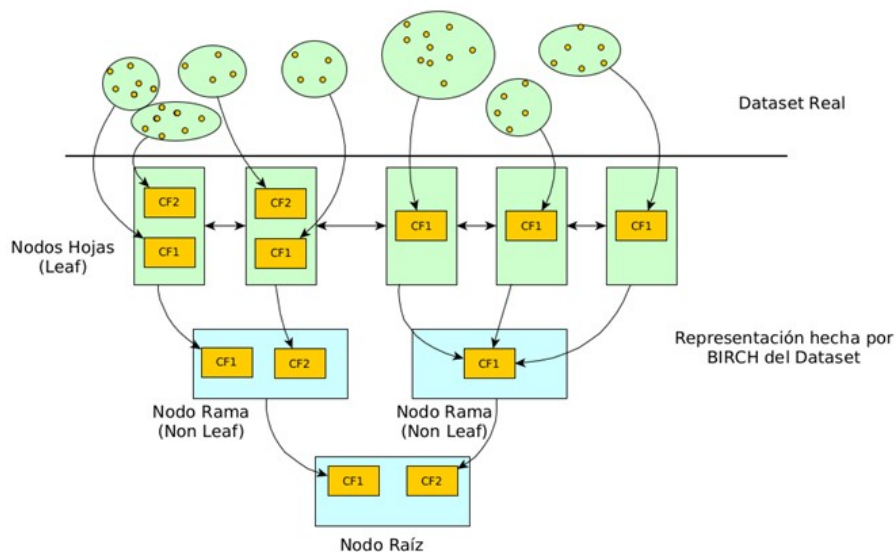


Figura 26. Representación de como luciría un árbol CF construido por BIRCH.

El nodo raíz y los nodos ramas, no pueden contener más de B, entradas, recordar que B es el factor de ramificación y es un parámetro de entrada al algoritmo. Los nodos hojas no pueden contener más de L entradas y estas deben cumplir la Regla T. L es el tamaño de las hojas o nodos hojas.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Es muy sencillo calcular las características de un clúster a partir de la información contenida en cada CF, tal que:

Dado un $CF = (N, \overrightarrow{LS}, SS)$, entonces el Centroide, el Radio y el Distancia media intra-clúster serían:

$$C = \frac{\overrightarrow{LS}}{N}$$

$$R = \sqrt{\frac{SS}{N} - \left(\frac{\overrightarrow{LS}}{N}\right)^2}$$

$$D_2 = \sqrt{\frac{N_1 SS_2 + N_2 * SS_1 - 2 * \overrightarrow{LS}_1 * \overrightarrow{LS}_2}{N_1 N_2}}$$

Veamos cómo sería el algoritmo general que usa este método para realizar el agrupamiento.

$$D_2 = \sqrt{\frac{N_1 SS_2 + N_2 * SS_1 - 2 * \overrightarrow{LS}_1 * \overrightarrow{LS}_2}{N_1 N_2}}$$

Algoritmo BIRCH:

- ▶ Fase 1. Construir el árbol CF.
- ▶ Fase 2. Condensar el árbol a un rango requerido. (opcional)
- ▶ Fase 3. Agrupamiento Global.
- ▶ Fase 4. Refinamiento del agrupamiento. (opcional)

Fase 1: Construir el árbol CF:

- ▶ Crea e inicializa un árbol t_0 en memoria con un pequeño valor T.

Tema 4. Algoritmos de aprendizaje automático no supervisado

- ▶ Comienza a insertar puntos en el árbol.(Ver el algoritmo de inserción)
- ▶ Si la memoria se desborda antes de que todos los puntos sean insertados, se crea un nuevo árbol t_{-1} con un T mayor. Se reinseran todos los puntos que estaban en el anterior y se continúan insertando los nuevos.
- ▶ Opcionalmente se pueden chequear outliers mientras se construye el árbol:
 - Si la entrada de la hoja, contiene mucho menos puntos que la media se escriben a disco en vez de añadirlas al árbol.
 - Si el espacio en disco se acaba, se exploran todos los outliers y se reabsorben aquellos que no aumenten el tamaño del árbol.
- ▶ Si la memoria no se desborda, se ha terminado el proceso y se pasa a la Fase siguiente.

Fase 2: Condensar el árbol a un rango requerido: Aunque esta etapa es opcional, también puede ser muy útil, pues se puede utilizar como un puente entre la salida de la primera fase y la tercera. Pues serviría para adaptar dicha salida a los requerimientos particulares de los algoritmos de agrupamiento externos que quisiéramos usar.

Los desarrolladores de este método se dieron cuenta que hay métodos de aprendizaje que trabajan mejor en determinados rangos del tamaño de la entrada y usan esta etapa para adaptar la salida de la primera fase a estos rangos de tamaño. Sin embargo como ya se ha dicho, esta etapa es opcional.

Fase 3: Agrupamiento Global: En esta fase se puede someter los datos de salida de la fase 1, directamente a un algoritmo de clustering como K-means o puede usarse la salida de la Fase 2. Para la aplicación de un algoritmo externo hay varias posibilidades, entre las que se encuentra: aplicar el algoritmo de forma directa y sin modificar sobre los CF o modificar uno de los algoritmos para aplicarlo a este tipo de

Tema 4. Algoritmos de aprendizaje automático no supervisado

entrada.

Fase 4: Esta fase es opcional, pero es usada para refinar el conjunto de salida, eliminando posibles outliers y terminando de afinar los clústers.

Inserción de una entrada en el árbol CF:

Dada una entrada «ent»:

- ▶ Identificar la hoja apropiada: Comenzando desde la raíz, descender de forma recursiva el árbol, eligiendo el nodo hijo más cercano de acuerdo a las medidas D_0, D_1, D_2, D_3, D_4 .
- ▶ Modificando la Hoja elegida: Cuando se alcanza el nodo hoja, encontrar la entrada más cercana y probar si puede “absorber” la nueva entrada sin violar la Regla T. Si se viola, el vector CF se actualiza, de lo contrario se añade la nueva entrada. Si no hubiera espacio para añadir más entradas en la hoja, entonces hay que dividirlo. Esta división es realizada, usando como semilla el par de entradas más alejadas y redistribuyendo las restantes entradas, basado en el criterio de las más cercanas.
- ▶ Modificando el camino hasta la hoja: Luego de insertar la entrada en la hoja, hay que actualizar la información en todos los CF de los nodos de las ramas en todo el camino desde la hoja hasta el nodo raíz. En caso de que no haya división de los CF, solo implica añadir vectores a un CF recalculando N, LS y SS. Sin embargo en caso de que haya que dividir el CF, pues hay que añadirlos para reflejar la nueva hoja.

Hasta aquí hemos visto las ideas generales que están detrás del método de aprendizaje automático conocido como BIRCH, ahora veremos un ejemplo de cómo usarlo sobre el Iris Dataset, como lo hemos estado haciendo con los métodos anteriores.

Recordamos que usamos la implementación de este método en Python y el módulo scikit-learn.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Siempre recordamos, como lo hemos hecho anteriormente, que el Iris Dataset, tiene un total de 150 casos, distribuidos en tres clases y 50 casos en cada una de ellas. Todos los rasgos son numéricos.

En este caso, hemos usado un factor de ramificación igual 8 y un umbral de 0,3; pero no se ha usado ningún método específico para su selección.

```
#Clustering BIRCH
from sklearn.cluster import Birch
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
# Definimos el modelo, le pasamos como parámetro
# B y T. (B=branching_factor y T= threshold)
model = Birch(branching_factor=8, threshold=0.3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
```

Fig. 27. Ejemplo de implementación del método BIRCH sobre el Iris Dataset.

En la siguiente imagen, se muestran los resultados de la corrida de este modelo.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Cluster:0 cantidad de casos:64
Cluster:1 cantidad de casos:50
Cluster:2 cantidad de casos:36

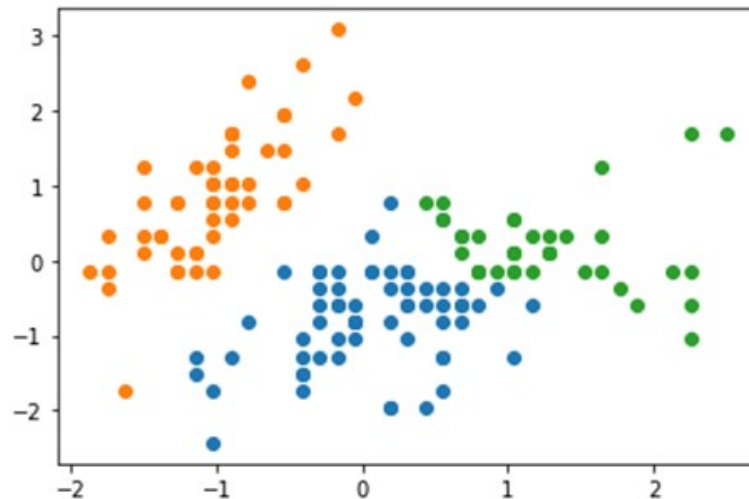


Figura 28. Resultados de la corrida del método BIRCH sobre el Iris Dataset.

Veremos ahora los parámetros que son necesarios para utilizar este modelo y la salida que nos arroja después del entrenamiento.

Parámetros:

- ▶ **threshold:** Este es el parámetro T que ya explicamos anteriormente y define el tamaño del árbol de forma conjunta con B. Este parámetro espera valores de tipo float y el valor defecto es 0.5.
- ▶ **branching_factor:** Este es el factor de ramificación y es el parámetro B del que hablamos cuando describimos el método. Este determina el número máximo de CF en cada nodo. Este parámetro espera valores enteros y su valor por defecto es 50.
- ▶ **n_clusters:** Este parámetro espera valores enteros y especifica el número de clústers que en los que se desea agrupar el conjunto de datos. El valor por defecto es 3. Si se le pasa None, como valor, no se ejecuta la fase de clustering y los subclusters son retornados de la manera en que los agrupa el modelo. También se le puede

Tema 4. Algoritmos de aprendizaje automático no supervisado

pasar como valor uno de los métodos contenidos en el paquete `sklearn.cluster`, en cuyo caso el modelo es ajustado tratando los subclusters como muestras nuevas y el dato inicial es mapeado a la etiqueta del subclúster más cercano. Cuando se le pasa un valor entero el modelo se ajusta usando `AgglomerativeClustering` y `n_clusters` es igual al entero pasado.

Valores que retorna BIRCH:

- ▶ `root_`: Contiene el nodo raíz del árbol CF construido a partir del dataset.
- ▶ `dummy_leaf_`: Contiene el puntero inicial que permite recorrer todos los nodos hojas del árbol.
- ▶ `subcluster_centers_`: Contiene los centroides de todos los subclusters.
- ▶ `labels_`: Contiene las etiquetas asignadas a cada uno de los casos del dataset.

Claro que estos no son todos los parámetros y los valores que retorna la implementación de este método en `scikit-learn`, pero como siempre comento, para más detalles es necesario consultar la documentación.

Hasta aquí lo correspondiente al método BIRCH, aunque es necesario que destaquemos que solo hemos tocado la idea central del método y que este requiere un estudio mucho más profundo y la literatura sobre él no es del todo buena. Por eso sugerimos la lectura del artículo original: BIRCH: An Efficient Data Clustering Method for Very Large Databases, de los autores: Tian Zhang; Raghu Ramakrishnan y Miron Livny. Este artículo está disponible de forma gratuita en Internet.

Conclusiones

Hasta aquí hemos estado tratando lo concerniente a los métodos de aprendizaje no supervisado, está claro que no podemos ni siquiera imaginar el tema agotado y hemos solo mostrado las ideas básicas que soportan algunos de los métodos. La idea es, como con los métodos supervisados, abrir las puertas al mundo del

Tema 4. Algoritmos de aprendizaje automático no supervisado

aprendizaje automático mostrando el funcionamiento de algunos de sus métodos; pero es un campo basto y complejo y que requerirá de mucho, mucho esfuerzo por dominar.

En esta lección, sin embargo, no hemos tocado lo concerniente a la evaluación de los métodos no supervisados pues lo haremos en la próxima lección, a diferencia de los métodos supervisados que lo hicimos al inicio de la primera.

Tema 4. Algoritmos de aprendizaje automático no supervisado

4.2. Algoritmos de aprendizaje automático no supervisado II

En la lección anterior vimos lo concerniente a los modelos de aprendizaje no supervisado, sin embargo, no tocamos los métodos de evaluar los resultados de su ajuste. El objetivo de esta, es describir y poner ejemplos de cómo se aplican estas medidas, así como abordar su clasificación.

Cuando termine esta lección, estará en condiciones de aplicar los métodos de aprendizaje no supervisado y evaluar los resultados.

En la lección anterior explicamos que son los métodos de aprendizaje no supervisado y revisamos las ideas que hay detrás de algunos de esos métodos. Sin embargo, no tocamos el tema de cómo evaluar sus resultados.

Evaluar los resultados de los métodos no supervisados es considerablemente más difícil que evaluar los resultados de los métodos supervisados, pues, normalmente no se cuenta con información acerca de la estructura de los datos. En los métodos no supervisados al no contar con información que nos diga cuán bien o mal lo estamos haciendo nos obliga a buscar tanto dentro de los mismos datos como fuera, algún criterio objetivo y mejor aún cuantitativo de la calidad del trabajo hecho por el método.

Otra de las causas que complican la evaluación de los métodos no supervisados es la subjetividad que es inherente a la actividad de agrupamiento de objetos o cosas.

La cantidad de métodos que se han desarrollado hablan de la complejidad del tema, sin embargo, de manera general, los métodos de evaluación de la validez de los resultados del agrupamiento arrojados por un método no supervisado se pueden agrupar en dos grandes grupos:

Tema 4. Algoritmos de aprendizaje automático no supervisado

Tipos de criterios de validez de los clústers:

- ▶ Criterios Internos.
- ▶ Criterios Externos.

Cuando hablamos de Criterios Internos, hacemos referencia a un grupo de métricas que evalúan la validez de los agrupamientos usando solamente la información aportada por el propio agrupamiento.

Por el contrario, los criterios externos, usan información conocida a priori sobre el particionamiento o agrupamiento del conjunto de datos. Un ejemplo es el uso de conjuntos de datos usados para clasificación, donde se conoce de antemano a qué clase pertenece cada caso. Esta información puede provenir de anotadores humanos que son expertos en el campo. Este tipo de métrica puede usarse para evaluar y escoger métodos con los mejores resultados sobre un dataset determinado. Sin embargo, esta información, externa, no siempre está disponible.

Estás métricas, además de ayudarnos a evaluar la calidad de un agrupamiento basado, digamos en un conjunto de parámetros, también nos ayuda a escogerlos. Un caso típico es el uso de ellos para escoger el número óptimo de clústers.

La validación de Clustering, viene acompañada, básicamente por tres tareas: Evaluación del Clustering, que busca valorar la calidad de un agrupamiento. Estabilidad del clustering, que busca comprender la sensibilidad de un modelo a las variaciones de los distintos parámetros de los modelos y la Tendencia al clustering, que pretende evaluar lo adecuado de los datos para aplicar sobre ellos técnicas de agrupamiento, es decir, evaluar cuando los datos tienen de forma inherente una estructura de grupo.

A continuación, exploraremos algunos de los criterios más usados y aquellos implementados en el módulo scikit-learn de python. Esta decisión está basada en la

Tema 4. Algoritmos de aprendizaje automático no supervisado

disponibilidad de documentación, la popularidad alcanzada por el framework y su facilidad de uso. Esto implica que puede comenzar a usarlos en sus proyectos de forma inmediata.

Criterios de validez Internos

Debido a la naturaleza misma de la tarea de agrupamiento, que trata de poner los casos más parecidos en grupos y que pretende que la distancia entre estos grupos sea máxima, se impone evaluar, básicamente, dos aspectos de cada agrupamiento: La cohesión de los grupos y la separación entre ellos.

La mayoría de las medidas o criterios de evaluación de los agrupamientos, miden alguno de estos dos aspectos.

La cohesión evalúa cuán parecidos o cercanos son los miembros de cada clúster, mientras que las medidas de separación permiten evaluar la distancia entre los grupos.

Una de las maneras más elementales de evaluar la cohesión y que aflora de manera natural cuando se razona sobre el tema es la Suma de Cuadrados Intra-clúster. (Sum of Squares Within)

$$SSW = \sum_{i=1}^C \sum_{j=1}^{|C_i|} dist^2(m_i, x_j)$$

Donde C es la cantidad de clúster, $|C_i|$ es el número de puntos o casos en el clúster i, m_i es el centroide del clúster i y x_j es el caso j del clúster i.

Usando la misma idea se puede evaluar la separación:

$$SSB = \sum_{j=1}^K n_j dist^2(c_j - \bar{x})$$

Donde, K es el número de clúster, n_j es el número de casos en el clúster j, c_j es el

Tema 4. Algoritmos de aprendizaje automático no supervisado

centroide el clúster j y $\underline{x}$ es la media del dataset. A esta medida se le llama Suma cuadrada Inter-clúster. (SSB)

Es muy sencillo darse cuenta que los valores de estas medidas dependen del valor de n , es decir de la cantidad de casos en el dataset, lo que las hace difícil de interpretar cuando se comparan dos métodos.

Se han propuesto varios Índices que se basan en uno o los dos anteriores, que listamos abajo; pero que no todos los veremos en más detalles.

Algunos índices basados en SSW y SSB:

Ball y Hall (1965):

$$BH = \frac{SSW}{K}$$

Donde K es el número de clústers.

Calinski y Harabasz (1974):

$$CH = \frac{SSB / (k - 1)}{SSW / (n - k)}$$

Donde K es el número de clústers y n el número de casos.

Hartigan (1975):

$$Har = \log \left(\frac{SSB}{SSW} \right)$$

Xu (1997):

$$xu = d * \log \left(\sqrt{\frac{SSW}{dn^2}} \right) + \log(k)$$

Donde d es la dimensión o cantidad de rasgos del dataset.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Veamos con más detalles algunas de estas medidas.

Calinski-Harabasz Index:

Este índice es la tasa entre la suma de la dispersión inter-clúster y la dispersión intra-clúster para todos los clústers. La dispersión se entiende aquí como la suma de la distancia cuadrada.

Por tanto, para un conjunto de datos E , con tamaño n_E el cual ha sido agrupado en k clústers, entonces:

$$s = \frac{\frac{tr(B_k)}{tr(W_k)} * n_E - k}{k - 1}$$

Donde $tr()$ es la función Trace. Y W_k y B_k son definidos como:

$$W_k = \sum_{q=1}^k \sum_{x \in C_q} (x - C_q)(x - C_q)^T$$

$$B_k = \sum_{q=1}^k n_q (c_q - c_E)(c_q - c_E)^T$$

Una de las ventajas de usar este índice es su facilidad de interpretación, pues un valor alto de este implica clústers densos y bien separados. Sin embargo, este índice suele ser un poco más alto para clústers convexos.

En la siguiente tabla, mostramos los valores de este índice, para evaluar el resultado de la aplicación de varios métodos al Iris Dataset, usando la implementación de este en el módulo scikit-learn de python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	Calinski-Harabasz Index
K-Means	241.90
Affinity Propagation	239.75
Mean Shift	152.47
Spectral Clustering	239.23
DBSCAN	84.26
BIRCH	237.65

En la siguiente imagen mostramos la manera en que se usa este índice de forma conjunta con el método de agrupamiento.

```
#Clustering con K-Means
from numpy import unique
from numpy import where
from sklearn.cluster import KMeans
from matplotlib import pyplot
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import calinski_harabasz_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# el número de clústers que queremos
model = KMeans(3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
pyplot.show()
# Calcula las métricas de validez
print("calinski_harabasz_score:", calinski_harabasz_score(X, model.labels_))
```

Fig. 1. Evaluando la validez del agrupamiento de un método K-Means sobre el Iris Dataset por medio del índice Calinski-Harabasz.

Este índice toma como parámetros el conjunto de datos y la asignación que ha hecho el método aplicado y retorna un valor de punto flotante.

Silhouette Coefficient

Este método de evaluación de la calidad del agrupamiento, combina la manera de evaluar la cohesión y la separación en un solo índice. Ve la cohesión como la

Tema 4. Algoritmos de aprendizaje automático no supervisado

distancia promedio de x hasta todos los demás puntos en el mismo clúster y la separación como la distancia promedio de x a todos los puntos del clúster más cercano.

Matemáticamente el coeficiente Silhouette para un punto, puede expresarse como:

$$s = \frac{b - a}{\max(a, b)}$$

Dónde:

a: Es la distancia promedio entre el punto x y el resto de los puntos dentro del clúster.

b: Es la distancia promedio entre el punto x y todos los puntos del clúster más cercano.

El coeficiente para todo el agrupamiento es:

$$S = \frac{1}{N} \sum_{i=1}^N s(x)$$

Esta métrica toma valores entre -1 y 1, donde valores cercanos a -1 indican un agrupamiento incorrecto y valores cercanos a 1, clústers más densos. Valores cercanos a cero pueden indicar solapamiento en los clústers.

Como en la métrica anterior, mostramos a continuación una tabla con los valores del coeficiente para diferentes métodos de aprendizaje no supervisado que evalúa la calidad del agrupamiento sobre el Iris Dataset.

Método	Silhouette Coefficient
K-Means	0.46
Affinity Propagation	0.46
Mean Shift	0.40
Spectral Clustering	0.46
DBSCAN	0.31
BIRCH	0.46

Tema 4. Algoritmos de aprendizaje automático no supervisado

En la siguiente imagen mostramos la manera de usar esta métrica junto a un método de aprendizaje automático no supervisado.

```
#Clustering DBSCAN
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# eps y min_samples
model = DBSCAN(eps=0.68, min_samples=15)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula las métricas de validez
print("silhouette_score:",silhouette_score(X,model.labels_))
```

Fig. 2. Evaluando la validez del agrupamiento de un método DBSCAN sobre el Iris Dataset por medio del Coeficiente Silhouette.

La métrica `silhouette_score`, toma los siguientes parámetros:

- ▶ `X`: Representa el conjunto de datos al que se le aplica el agrupamiento.
- ▶ `labels`: el etiquetado que retorna el método de aprendizaje.
- ▶ `metric`: La métrica utilizada para el cálculo del índice. El valor por defecto es “euclidean”. Ahora bien, si `X` es una matriz de distancias entonces este parámetro debe tomar como valor ‘precomputed’.

Retorna un valor float.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Davies-Bouldin Index:

Este índice se basa en la idea de la similaridad promedio entre los clústers. Aquí la similaridad se entiende como una medida que compara la distancia entre clústers con el tamaño de los clústers en sí mismo.

Matemáticamente, este índice puede expresarse como sigue:

$$DB = \frac{1}{k} \sum_{i=1}^k R_{ij}$$

Dónde:

$$R_{ij} = \frac{s_i + s_j}{d_{ij}}$$

s_i : distancia media entre cada punto del clúster i y el centroide del clúster.

d_{ij} : distancia entre los centroides i y j .

En este índice los valores cercanos a cero son indicadores de una mejor partición. Por tanto, el agrupamiento que minimice el valor de este, se considera óptimo.

Veamos en la tabla, como se comporta a la hora de evaluar un agrupamiento sobre el Iris Dataset. Para eso usamos la implementación de scikit-learn de python.

Método	Davies-Bouldin Index
K-Means	0.83
Affinity Propagation	0.84
Mean Shift	0.79
Spectral Clustering	0.83
DBSCAN	7.71
BIRCH	0.84

Veamos cómo se usa de conjunto con uno de los métodos anteriores:

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering BIRCH
from sklearn.cluster import Birch
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import davies_bouldin_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# B y T. (B=branching factor y T= threshold)
model = Birch(branching_factor=8, threshold=0.3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula las métricas de validez
print("davies_bouldin_score:", davies_bouldin_score(X, model.labels_))
```

Fig. 3. Evaluando la validez del agrupamiento de un método BIRCH sobre el Iris Dataset por medio del Índice Davies-Bouldin.

`davies_bouldin_score()` toma como parámetros:

X: el conjunto de datos sobre el que se ejecuta el proceso de agrupamiento.

labels: el etiquetado producido por la aplicación del método.

Retorna un float.

Criterios de validez Externos

Lo que caracteriza a este tipo de criterio es el hecho de necesitar de un etiquetado de referencia externo, ya sea aportado por el conjunto de datos o por expertos humanos en el campo.

Rand Index.

Este es un índice que pertenece al conjunto de los criterios externos. Y la idea detrás

Tema 4. Algoritmos de aprendizaje automático no supervisado

de este, es medir la similaridad entre las etiquetas que el método de aprendizaje asignó con aquellas que son conocidas de antemano. En realidad, se puede usar para medir la similaridad entre los resultados de dos agrupamientos.

Supongamos un conjunto S de n elementos tal que:

$$S = \{s_0, s_1, \dots, s_n\}$$

Y tengamos dos particiones de S , X en r subconjuntos y Y en s subconjuntos tal que:

$$X = \{X_1, X_2, \dots, X_r\}$$

Y

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

Entonces podemos definir el Rand Index o Índice de Rand de la forma siguiente:

$$RI = \frac{a + b}{n(n-1)/2}$$

Dónde:

- ▶ a es el número de pares de elementos en S que están en el mismo subconjunto en X y en el mismo subconjunto en Y .
- ▶ b es el número de pares de elementos en S que están en subconjuntos diferentes en X y en subconjuntos diferentes en Y .
- ▶ $n(n-1)/2$ es el número total de pares en S .

Este índice toma valores entre 0 y 1, indicando el cero que no hay acuerdo entre los dos agrupamientos, mientras que 1 es el perfecto acuerdo.

Sin embargo, este índice no asegura que para un etiquetado aleatorio el valor sea exactamente cero, y por esto se usa el índice de Rand ajustado. Adjusted Rand Index.

Tema 4. Algoritmos de aprendizaje automático no supervisado

En la siguiente tabla, mostramos los resultados arrojados de este índice y el ajustado, usado para evaluar varios métodos de aprendizaje no supervisado. Para elaborar la tabla usamos la implementación de este índice en scikit-learn de python y las medidas se tomaron sobre el Iris dataset.

Método	RI Ajustado	RI no Ajustado
K-Means	0.63	0.84
Affinity Propagation	0.63	0.84
Mean Shift	0.60	0.83
Spectral Clustering	0.58	0.81
DBSCAN	0.41	0.74
BIRCH	0.61	0.83

En la siguiente imagen mostramos un ejemplo de cómo usarlo en scikit-learn.

```
#Clustering BIRCH
from sklearn.cluster import Birch
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import rand_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf= datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels=idf.target
# Definimos el modelo, le pasamos como parámetro
# B y T. (B=branching_factor y T= threshold)
model = Birch(branching_factor=8, threshold=0.3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
Labels = model.labels_
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula el índice de Rand
Rand = rand_score(true_labels, Labels)
print("Rand Index:",Rand)
```

Fig. 4. Evaluando la validez del agrupamiento de un método BIRCH sobre el Iris Dataset por medio del índice de Rand.

Es importante notar que, en esta implementación, el índice de Rand nos permite

Tema 4. Algoritmos de aprendizaje automático no supervisado

comparar el etiquetado conocido del Iris dataset con el etiquetado resultado del ajuste del modelo BIRCH. Esta es precisamente, una de las desventajas de usar este tipo de índice, pues esta información no siempre está disponible.

Medidas basadas en Información Mutua

La información mutua trata de medir el acuerdo entre dos agrupamientos ignorando las permutaciones. En otras palabras, esta medida trata de medir la cantidad de información compartida entre el agrupamiento y un particionamiento y es definida como sigue:

Dado un conjunto S de N elementos, tal que:

$$S = \{S_1, S_2, \dots, S_N\}$$

Y dos particiones en este, que llamaremos X y Y tal que estas sean:

$$X = \{X_1, X_2, \dots, X_r\}$$

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

Suponiendo que: $X_i \cap X_j = \emptyset \forall i \neq j$ y que $Y_i \cap Y_j = \emptyset \forall i \neq j$, es decir que los clústers son duros y que además son completos, es decir:

$$\bigcup_{i=1}^r X_i = \bigcup_{i=1}^s Y_i = S$$

Entonces, la Información mutua entre X y Y vendría dada por:

$$MI(X, Y) = \sum_{i=1}^r \sum_{j=1}^s P_{XY}(i, j) * \log \frac{P_{XY}(i, j)}{P_X(i) P_Y(j)}$$

Donde la probabilidad $P_{XY}(i, j)$ es la probabilidad de que un caso o punto pertenezca tanto a un clúster X_i de X como a un clúster Y_j de Y, se puede calcular de la siguiente forma:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$P_{XY}(i,j) = \frac{|X_i \cap Y_j|}{N}$$

La información mutua, expresada como cardinalidad de conjuntos puede verse de la siguiente manera:

$$MI(X,Y) = \sum_{i=1}^{|X|} \sum_{j=1}^{|Y|} \frac{|X_i \cap Y_j|}{N} * \log \left(\frac{N |X_i \cap Y_j|}{|X_i| |Y_j|} \right)$$

Dónde:

$|X|$ y $|Y|$ son las cantidades de clúster que contiene cada una de las respectivas particiones.

$|X_i|$ y $|Y_j|$ son las cantidades de puntos o casos que contiene el clúster i y j de cada partición.

N es el número de casos o puntos en S .

La información mutua, cuando X y Y son independientes, toma el valor cero; pero no tiene una cota superior. Por eso, aparece una versión de esta, llamada Información Mutua Normalizada y se puede expresar como:

$$NMI(X,Y) = \frac{MI(X,Y)}{mean}$$

Donde $H(X)$ es la entropía de X y $H(Y)$ la entropía de Y , y pueden expresarse como:

$$H(X) = - \sum_{i=1}^{|X|} P(i) * \log(P(i))$$

$$H(Y) = - \sum_{j=1}^{|Y|} P(j) * \log(P(j))$$

La Información Mutua Ajustada, por su parte, puede expresarse de la siguiente manera:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$AMI(X,Y) = \frac{MI(X,Y) - E[MI(X,Y)]}{mean(H(X)H(Y)) - E[MI(X,Y)]}$$

Donde $E[MI(X,Y)]$ es el valor esperado de la Información Mutua.

Como ya comentamos estas métricas de evaluación de la calidad del agrupamiento llamadas Información Mutua, toman valores entre 0 y 1, en la tabla siguiente hemos recopilado, como en el caso anterior una comparación de sus valores al aplicarla sobre el Iris Dataset usando varios métodos de aprendizaje no supervisado.

Método	MI	AMI	NMI
K-Means	0.72	0.65	0.66
Affinity Propagation	0.73	0.66	0.67
Mean Shift	0.75	0.63	0.64
Spectral Clustering	0.68	0.61	0.62
DBSCAN	0.53	0.48	0.49
BIRCH	0.72	0.66	0.67

MI: Información Mutua. (Mutual Information)

AMI: Información Mutua Ajustada. (Adjusted Mutual Information)

NMI: Información Mutua Normalizada. (Normalized Mutual Information)

En la siguiente imagen mostramos la manera de cómo calcular estos índices usando scikit-learn de python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering DBSCAN
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.metrics import mutual_info_score
from sklearn.metrics import adjusted_mutual_info_score
from sklearn.metrics import normalized_mutual_info_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf= datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# eps y min_samples
model = DBSCAN(eps=0.68, min_samples=15)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
MI5 = mutual_info_score(true_labels, model.labels_)
print("MI:",MI5)
AMI5 = adjusted_mutual_info_score(true_labels, model.labels_)
print("AMI:",AMI5)
NMI5 = normalized_mutual_info_score(true_labels, model.labels_)
print("NMI:",NMI5)
Silhouette = silhouette_score(X, model.labels_, metric='euclidean')
print("Coeficiente Silhouette:",Silhouette)
```

Fig. 5. Evaluando la validez del agrupamiento de un método DBSCAN sobre el Iris Dataset por medio de Información Mutua, Ajustada y Normalizada.

Estos índices, comparan el etiquetado verdadero (`true_labels`), por decirlo de alguna manera, con el asignado por el método de aprendizaje sin información alguna sobre la estructura de los datos. (`model.labels_`)

Homogeneidad, Completitud y V-Measure

Estas son otras medidas consideradas dentro del grupo de los criterios de evaluación externos y los tres están estrechamente ligados.

Con el propósito de explicar las ideas detrás de estas medidas de validez de un

Tema 4. Algoritmos de aprendizaje automático no supervisado

agrupamiento, debemos hacer las mismas suposiciones que hemos estado haciendo en las medidas anteriores.

Dado un conjunto S de N elementos, tal que:

$$S = \{S_1, S_2, \dots, S_N\}$$

Y dos particiones en este, que llamaremos X y Y tal que estas sean:

$$X = \{X_1, X_2, \dots, X_r\}$$

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

$$\bigcup_{i=1}^r X_i = \bigcup_{i=1}^s Y_i = S$$

Entonces podemos comenzar a definir lo que se entiende por Homogeneidad, Completitud y V-Measure.

La Homogeneidad y la Completitud de un agrupamiento dado como resultado de la aplicación de un método de aprendizaje no supervisado a un conjunto de datos son dos medidas complementarias que se combinan para crear V-measure,

Para explicar estos conceptos es necesario que aclaremos que llamaremos a cada $X_i \in X$ una clase y a cada $Y_i \in Y$ un clúster. Y veremos a X como una plantilla contra la cual compararemos cuán bien el método de aprendizaje asignó cada punto de datos de S a Y sin información ninguna de la estructura dada por X .

Un agrupamiento, digamos Y , satisface el criterio de Homogeneidad, si todos los clústers en Y contienen puntos de datos de un mismo X_i , es decir de una misma clase en X . Por el contrario, un agrupamiento Y satisface el criterio de Completitud si un clúster $Y_i \in Y$ tiene entre sus puntos elementos de todas las clases $X_i \in X$.

Esto implica que la homogeneidad de un clúster es mayor mientras menos diverso es, el caso ideal, es decir, donde se maximizará esta medida es en un clúster con elementos de una sola clase.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Por otro lado, la Completitud sería mayor mientras más diverso es el clúster, es decir mientras más elementos de este correspondan a más clases de $X_i \in X$.

Estas medidas son simétricas y por tanto se complementan de alguna manera.

La Homogeneidad puede expresarse de la siguiente forma:

$$h = 1 - \frac{H(X \vee Y)}{H(X)}$$

Y la Completitud, se expresaría como sigue:

$$c = 1 - \frac{H(Y \vee X)}{H(Y)}$$

Donde $H(X \vee Y)$ es la entropía condicional y puede expresarse de la siguiente forma:

$$H(Y) = - \sum_{i=1}^{|X|} \sum_{j=1}^{|Y|} \frac{n_{x,y}}{N} * \log \left(\frac{n_{x,y}}{N} \right)$$

Mientras que $H(X)$ es la entropía de la clase y se expresa como:

$$H(X) = - \sum_{i=1}^{|X|} \frac{n_x}{n} * \log \left(\frac{n_x}{n} \right)$$

Donde n_x es el número de casos en la clase o clúster i de X o de Y . Solo mostramos la entropía de X pero la de Y se calcula de la misma forma.

Por otra parte, la V-measure se expresa como la media armónica de estas dos medidas, y se expresa como:

$$v = \frac{2 * h * c}{h + c}$$

Sin embargo, una variante de esta, permite ponderar cada una de las medidas.

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$v = \frac{(1 + \beta) * h * c}{(\beta * h + c)}$$

Cuando $\beta < 1$ se pondera la homogeneidad, es decir se le da mayor importancia.

Cuando $\beta > 1$ se le da más importancia a la completitud.

Sin embargo, uno de los problemas de estas medidas es que no están normalizadas con respecto al etiquetado aleatorio e implica que el resultado para este tipo de etiquetado no siempre sería el mismo ya que dependen del número de muestras, la cantidad de clústers y clases. Este problema puede ignorarse de manera segura si la cantidad de casos es mayor que 100 y la cantidad de clúster menor que 10.

En la tabla siguiente veremos, como hemos hecho con las medidas anteriores, una comparación de sus valores con el propósito de evaluar el agrupamiento arrojado por varios métodos sobre el Iris Dataset.

Método	h	c	v-measure
K-Means	0.66	0.66	0.66
Affinity Propagation	0.67	0.67	0.67
Mean Shift	0.69	0.60	0.64
Spectral Clustering	0.62	0.62	0.62
DBSCAN	0.48	0.49	0.48
BIRCH	0.66	0.68	0.67

Una de las ventajas de este tipo de medidas es que no hace asunción alguna sobre la forma de los clústers, lo que la hace apta para comparar el resultado del agrupamiento arrojado por varios métodos.

Como hemos estado haciendo hasta ahora, veremos la manera en que se aplican estas medidas usando la implementación en scikit-learn de Python. En la siguiente imagen se muestra el código de la aplicación de un método Mean Shift sobre el Iris Dataset y se evalúa por medio de la Homogeneidad, Completitud y v-measure. Los valores son lo que se muestran en la tabla anterior.

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering con Mean Shift
from sklearn.cluster import MeanShift
from sklearn.cluster import estimate_bandwidth
import matplotlib.pyplot as plt
import statistics as stat
from numpy import unique
from numpy import where
from sklearn import datasets
from itertools import cycle
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import homogeneity_score
from sklearn.metrics import completeness_score
from sklearn.metrics import v_measure_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# bandwidth y bin_seeding
#bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=150)
model = MeanShift(bandwidth=0.99)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
Labels = model.labels_
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
plt.show()
# Calcula las métricas de validez
print("Homogeneidad:", homogeneity_score(true_labels, model.labels_))
print("Compleitud:", completeness_score(true_labels, model.labels_))
print("V-Measure", v_measure_score(true_labels, model.labels_))
```

Fig. 6. Evaluando la validez del agrupamiento de un método Mean Shift sobre el Iris Dataset por medio de Homogeneidad, Compleitud y v-measure.

Índice Fowlkes-Mallows

Este es otro de los índices o criterios de validez externa, como en las anteriores medidas, para usar esta, se necesita conocer el verdadero etiquetado de los casos del conjunto de datos.

El índice de Fowlkes-Mallows, puede ser interpretada como la media geométrica de la Precisión y el Recall y puede expresarse matemáticamente como:

$$FMI = \frac{TP}{\sqrt{(TP + FP)(TP + FN)}}$$

Donde TP, es el valor de los Casos Positive (TP), es decir, el número de pares de puntos que pertenecen al mismo clúster tanto en el etiquetado verdadero y el

Tema 4. Algoritmos de aprendizaje automático no supervisado

etiquetado producido por el método aplicado. FP, es el número de Falsos Positivos (FP), es decir el número de pares de puntos que pertenecen al mismo clúster en el etiquetado verdadero, pero no así en el etiquetado arrojado por el método de aprendizaje. FN, son los llamados Falsos Negativos (FN), que son los pares de puntos en el etiquetado producido por el método de aprendizaje, pero no en el etiquetado verdadero.

Este tipo de medida tiene un rango entre 0 y 1. Valores altos son indicativos de una alta similitud entre las clases predichas y las reales.

Entre las ventajas de este método está que para un etiquetado aleatorio arroja valores cercanos a cero para cualquier cantidad de clústers y casos. Otra de las ventajas es su interpretabilidad, pues valores cercanos a cero indican que dos asignaciones o etiquetados son prácticamente independientes, mientras que valores cercanos a uno, indican un significativo acuerdo entre las dos. Además de un valor cero, indica dos etiquetados completamente diferentes o independientes y un valor exacto de uno dos etiquetados exactamente iguales.

Este índice, tampoco hace asunciones sobre la forma de los clústers y por tanto puede usarse para comparar diferentes métodos de aprendizaje automático.

Conclusiones

Hasta aquí hemos hecho un breve recorrido por algunos de los métodos de evaluación de los resultados de los métodos de aprendizaje no supervisado. Sin ánimo de cubrir todo el campo, pues de hecho es muy basto. Literalmente hay cientos de métodos y esto refleja la complejidad del asunto.

Es importante para entender a cabalidad este tópico que se busque información adicional y se profundice en el tema.

En alguna literatura, a los criterios de evaluación, internos y externos se les suma una categoría adicional llamada criterios relativos que no hemos tocado en esta

Tema 4. Algoritmos de aprendizaje automático no supervisado

lección con vistas a no complicarla más de lo necesario.

Tampoco hemos tocado otras métricas que no están implementadas en scikit-learn, pues como meta tenemos que se puedan usar de forma inmediata los conocimientos que adquieran en estas lecciones. Sin embargo, se pueden encontrar en la literatura y la mayoría son muy fáciles de implementar.

Tema 4. Algoritmos de aprendizaje automático no supervisado

4.3. Aprendizaje automático

En la lección anterior vimos lo concerniente a los modelos de aprendizaje no supervisado, sin embargo, no tocamos los métodos de evaluar los resultados de su ajuste. El objetivo de esta, es describir y poner ejemplos de cómo se aplican estas medidas, así como abordar su clasificación.

Cuando termine esta lección, estará en condiciones de aplicar los métodos de aprendizaje no supervisado y evaluar los resultados.

Introducción

En la lección anterior explicamos que son los métodos de aprendizaje no supervisado y revisamos las ideas que hay detrás de algunos de esos métodos. Sin embargo, no tocamos el tema de cómo evaluar sus resultados.

Evaluar los resultados de los métodos no supervisados es considerablemente más difícil que evaluar los resultados de los métodos supervisados, pues, normalmente no se cuenta con información acerca de la estructura de los datos. En los métodos no supervisados al no contar con información que nos diga cuán bien o mal lo estamos haciendo nos obliga a buscar tanto dentro de los mismos datos como fuera, algún criterio objetivo y mejor aún cuantitativo de la calidad del trabajo hecho por el método.

Otra de las causas que complican la evaluación de los métodos no supervisados es la subjetividad que es inherente a la actividad de agrupamiento de objetos o cosas.

La cantidad de métodos que se han desarrollado hablan de la complejidad del tema, sin embargo, de manera general, los métodos de evaluación de la validez de los resultados del agrupamiento arrojados por un método no supervisado se pueden agrupar en dos grandes grupos:

Tema 4. Algoritmos de aprendizaje automático no supervisado

Tipos de criterios de validez de los clústers:

- ▶ Criterios Internos.
- ▶ Criterios Externos.

Cuando hablamos de Criterios Internos, hacemos referencia a un grupo de métricas que evalúan la validez de los agrupamientos usando solamente la información aportada por el propio agrupamiento.

Por el contrario, los criterios externos, usan información conocida a priori sobre el particionamiento o agrupamiento del conjunto de datos. Un ejemplo es el uso de conjuntos de datos usados para clasificación, donde se conoce de antemano a qué clase pertenece cada caso. Esta información puede provenir de anotadores humanos que son expertos en el campo. Este tipo de métrica puede usarse para evaluar y escoger métodos con los mejores resultados sobre un dataset determinado. Sin embargo, esta información, externa, no siempre está disponible.

Estás métricas, además de ayudarnos a evaluar la calidad de un agrupamiento basado, digamos en un conjunto de parámetros, también nos ayuda a escogerlos. Un caso típico es el uso de ellos para escoger el número óptimo de clústers.

La validación de Clustering, viene acompañada, básicamente por tres tareas: Evaluación del Clustering, que busca valorar la calidad de un agrupamiento. Estabilidad del clustering, que busca comprender la sensibilidad de un modelo a las variaciones de los distintos parámetros de los modelos y la Tendencia al clustering, que pretende evaluar lo adecuado de los datos para aplicar sobre ellos técnicas de agrupamiento, es decir, evaluar cuando los datos tienen de forma inherente una estructura de grupo.

A continuación, exploraremos algunos de los criterios más usados y aquellos implementados en el módulo scikit-learn de python. Esta decisión está basada en la

Tema 4. Algoritmos de aprendizaje automático no supervisado

disponibilidad de documentación, la popularidad alcanzada por el framework y su facilidad de uso. Esto implica que puede comenzar a usarlos en sus proyectos de forma inmediata.

Criterios de validez Internos

Debido a la naturaleza misma de la tarea de agrupamiento, que trata de poner los casos más parecidos en grupos y que pretende que la distancia entre estos grupos sea máxima, se impone evaluar, básicamente, dos aspectos de cada agrupamiento: La cohesión de los grupos y la separación entre ellos.

La mayoría de las medidas o criterios de evaluación de los agrupamientos, miden alguno de estos dos aspectos.

La cohesión evalúa cuán parecidos o cercanos son los miembros de cada clúster, mientras que las medidas de separación permiten evaluar la distancia entre los grupos.

Una de las maneras más elementales de evaluar la cohesión y que aflora de manera natural cuando se razona sobre el tema es la Suma de Cuadrados Intra-clúster. (Sum of Squares Within)

$$SSW = \sum_{i=1}^C \sum_{j=1}^{|C_i|} dist^2(m_i, x_j)$$

Donde C es la cantidad de clúster, $|C_i|$ es el número de puntos o casos en el clúster i, m_i es el centroide del clúster i y x_j es el caso j del clúster i.

Usando la misma idea se puede evaluar la separación:

$$SSB = \sum_{j=1}^K n_j dist^2(c_j - \bar{x})$$

Tema 4. Algoritmos de aprendizaje automático no supervisado

Donde, K es el número de clúster, n_j es el número de casos en el clúster j , c_j es el centroide el clúster j y $\underline{x}$ es la media del dataset. A esta medida se le llama Suma cuadrada Inter-clúster. (SSB)

Es muy sencillo darse cuenta que los valores de estas medidas dependen del valor de n , es decir de la cantidad de casos en el dataset, lo que las hace difícil de interpretar cuando se comparan dos métodos.

Se han propuesto varios Índices que se basan en uno o los dos anteriores, que listamos abajo; pero que no todos los veremos en más detalles.

Algunos índices basados en SSW y SSB:

Ball y Hall (1965):

$$BH = \frac{SSW}{K}$$

Donde K es el número de clústers.

Calinski y Harabasz (1974):

$$CH = \frac{SSB / (k - 1)}{SSW / (n - k)}$$

Donde K es el número de clústers y n el número de casos.

Hartigan (1975):

$$Har = \log \left(\frac{SSB}{SSW} \right)$$

Xu (1997):

$$xu = d * \log \left(\sqrt{\frac{SSW}{dn^2}} \right) + \log(k)$$

Tema 4. Algoritmos de aprendizaje automático no supervisado

Donde d es la dimensión o cantidad de rasgos del dataset.

Veamos con más detalles algunas de estas medidas.

Calinski-Harabasz Index.

Este índice es la tasa entre la suma de la dispersión inter-clúster y la dispersión intra-clúster para todos los clústers. La dispersión se entiende aquí como la suma de la distancia cuadrada.

Por tanto, para un conjunto de datos E , con tamaño n_E el cual ha sido agrupado en k clústers, entonces:

$$s = \frac{\frac{tr(B_k)}{tr(W_k)} * n_E - k}{k - 1}$$

Donde $tr()$ es la función Trace. Y W_k y B_k son definidos como:

$$W_k = \sum_{q=1}^k \sum_{x \in C_q} (x - C_q)(x - C_q)^T$$

$$B_k = \sum_{q=1}^k n_q (c_q - c_E)(c_q - c_E)^T$$

Una de las ventajas de usar este índice es su facilidad de interpretación, pues un valor alto de este implica clústers densos y bien separados. Sin embargo, este índice suele ser un poco más alto para clústers convexos.

En la siguiente tabla, mostramos los valores de este índice, para evaluar el resultado de la aplicación de varios métodos al Iris Dataset, usando la implementación de este en el módulo scikit-learn de python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	Calinski-Harabasz Index
K-Means	241.90
Affinity Propagation	239.75
Mean Shift	152.47
Spectral Clustering	239.23
DBSCAN	84.26
BIRCH	237.65

En la siguiente imagen mostramos la manera en que se usa este índice de forma conjunta con el método de agrupamiento.

```
#Clustering con K-Means
from numpy import unique
from numpy import where
from sklearn.cluster import KMeans
from matplotlib import pyplot
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import calinski_harabasz_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# el número de clústers que queremos
model = KMeans(3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    pyplot.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
pyplot.show()
# Calcula las métricas de validez
print("calinski_harabasz_score:", calinski_harabasz_score(X, model.labels_))
```

Fig. 1. Evaluando la validez del agrupamiento de un método K-Means sobre el Iris Dataset por medio del índice Calinski-Harabasz.

Este índice toma como parámetros el conjunto de datos y la asignación que ha hecho el método aplicado y retorna un valor de punto flotante.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Silhouette Coefficient:

Este método de evaluación de la calidad del agrupamiento, combina la manera de evaluar la cohesión y la separación en un solo índice. Ve la cohesión como la distancia promedio de x hasta todos los demás puntos en el mismo clúster y la separación como la distancia promedio de x a todos los puntos del clúster más cercano.

Matemáticamente el coeficiente Silhouette para un punto, puede expresarse como:

$$s = \frac{b - a}{\max(a, b)}$$

Dónde:

a: Es la distancia promedio entre el punto x y el resto de los puntos dentro del clúster.

b: Es la distancia promedio entre el punto x y todos los puntos del clúster más cercano.

El coeficiente para todo el agrupamiento es:

$$S = \frac{1}{N} \sum_{i=1}^N s(x)$$

Esta métrica toma valores entre -1 y 1, donde valores cercanos a -1 indican un agrupamiento incorrecto y valores cercanos a 1, clústers más densos. Valores cercanos a cero pueden indicar solapamiento en los clústers.

Como en la métrica anterior, mostramos a continuación una tabla con los valores del coeficiente para diferentes métodos de aprendizaje no supervisado que evalúa la calidad del agrupamiento sobre el Iris Dataset.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	Silhouette Coefficient
K-Means	0.46
Affinity Propagation	0.46
Mean Shift	0.40
Spectral Clustering	0.46
DBSCAN	0.31
BIRCH	0.46

En la siguiente imagen mostramos la manera de usar esta métrica junto a un método de aprendizaje automático no supervisado.

```
#Clustering DBSCAN
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# eps y min_samples
model = DBSCAN(eps=0.68, min_samples=15)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula las métricas de validez
print("silhouette_score:",silhouette_score(X,model.labels_))
```

Fig. 2. Evaluando la validez del agrupamiento de un método DBSCAN sobre el Iris Dataset por medio del Coeficiente Silhouette.

Tema 4. Algoritmos de aprendizaje automático no supervisado

La métrica `silhouette_score`, toma los siguientes parámetros:

- ▶ `X`: Representa el conjunto de datos al que se le aplica el agrupamiento.
- ▶ `labels`: el etiquetado que retorna el método de aprendizaje.
- ▶ `metric`: La métrica utilizada para el cálculo del índice. El valor por defecto es «euclidean». Ahora bien, si `X` es una matriz de distancias entonces este parámetro debe tomar como valor 'precomputed'.

Retorna un valor float.

Davies-Bouldin Index:

Este índice se basa en la idea de la similaridad promedio entre los clústers. Aquí la similaridad se entiende como una medida que compara la distancia entre clústers con el tamaño de los clústers en sí mismo.

Matemáticamente, este índice puede expresarse como sigue:

$$DB = \frac{1}{k} \sum_{i=1}^k R_{ij}$$

Dónde:

$$R_{ij} = \frac{s_i + s_j}{d_{ij}}$$

s_i : distancia media entre cada punto del clúster i y el centroide del clúster.

d_{ij} : distancia entre los centroides i y j .

En este índice los valores cercanos a cero son indicadores de una mejor partición. Por tanto, el agrupamiento que minimice el valor de este, se considera óptimo.

Veamos en la tabla, como se comporta a la hora de evaluar un agrupamiento sobre el Iris Dataset. Para eso usamos la implementación de `scikit-learn` de python.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	Davies-Bouldin Index
K-Means	0.83
Affinity Propagation	0.84
Mean Shift	0.79
Spectral Clustering	0.83
DBSCAN	7.71
BIRCH	0.84

Veamos cómo se usa de conjunto con uno de los métodos anteriores:

```
#Clustering BIRCH
from sklearn.cluster import Birch
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import davies_bouldin_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# B y T. (B=branching factor y T= threshold)
model = Birch(branching_factor=8, threshold=0.3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i,len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula las métricas de validez
print("davies_bouldin_score:",davies_bouldin_score(X,model.labels_))
```

Fig. 3. Evaluando la validez del agrupamiento de un método BIRCH sobre el Iris Dataset por medio del Índice Davies-Bouldin.

`davies_bouldin_score()` toma como parámetros:

- ▶ X: el conjunto de datos sobre el que se ejecuta el proceso de agrupamiento.
- ▶ labels: el etiquetado producido por la aplicación del método.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Retorna un float.

Criterios de validez externos

Lo que caracteriza a este tipo de criterio es el hecho de necesitar de un etiquetado de referencia externo, ya sea aportado por el conjunto de datos o por expertos humanos en el campo.

Rand Index

Este es un índice que pertenece al conjunto de los criterios externos. Y la idea detrás de este, es medir la similitud entre las etiquetas que el método de aprendizaje asignó con aquellas que son conocidas de antemano. En realidad, se puede usar para medir la similitud entre los resultados de dos agrupamientos.

Supongamos un conjunto S de n elementos tal que:

$$S = \{s_0, s_1, \dots, s_n\}$$

Y tengamos dos particiones de S , X en r subconjuntos y Y en s subconjuntos tal que:

$$X = \{X_1, X_2, \dots, X_r\}$$

Y

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

Entonces podemos definir el Rand Index o Índice de Rand de la forma siguiente:

$$RI = \frac{a + b}{n(n-1)/2}$$

Dónde:

- ▶ a es el número de pares de elementos en S que están en el mismo subconjunto en X y en el mismo subconjunto en Y .

Tema 4. Algoritmos de aprendizaje automático no supervisado

- ▶ b es el número de pares de elementos en S que están en subconjuntos diferentes en X y en subconjuntos diferentes en Y .
- ▶ $n(n - 1) / 2$ es el número total de pares en S .

Este índice toma valores entre 0 y 1, indicando el cero que no hay acuerdo entre los dos agrupamientos, mientras que 1 es el perfecto acuerdo.

Sin embargo, este índice no asegura que para un etiquetado aleatorio el valor sea exactamente cero, y por esto se usa el índice de Rand ajustado. Adjusted Rand Index.

En la siguiente tabla, mostramos los resultados arrojados de este índice y el ajustado, usado para evaluar varios métodos de aprendizaje no supervisado. Para elaborar la tabla usamos la implementación de este índice en scikit-learn de python y las medidas se tomaron sobre el Iris dataset.

Método	RI Ajustado	RI no Ajustado
K-Means	0.63	0.84
Affinity Propagation	0.63	0.84
Mean Shift	0.60	0.83
Spectral Clustering	0.58	0.81
DBSCAN	0.41	0.74
BIRCH	0.61	0.83

En la siguiente imagen mostramos un ejemplo de cómo usarlo en scikit-learn.

Tema 4. Algoritmos de aprendizaje automático no supervisado

```
#Clustering BIRCH
from sklearn.cluster import Birch
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import rand_score

# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target

# Definimos el modelo, le pasamos como parámetro
# B y T. (B=branching_factor y T= threshold)
model = Birch(branching_factor=8, threshold=0.3)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
Labels = model.labels_

# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
# Calcula el índice de Rand
Rand = rand_score(true_labels, Labels)
print("Rand Index:", Rand)
```

Fig. 4. Evaluando la validez del agrupamiento de un método BIRCH sobre el Iris Dataset por medio del índice de Rand.

Es importante notar que, en esta implementación, el índice de Rand nos permite comparar el etiquetado conocido del Iris dataset con el etiquetado resultado del ajuste del modelo BIRCH. Esta es precisamente, una de las desventajas de usar este tipo de índice, pues esta información no siempre está disponible.

Medidas basadas en Información Mutua

La información mutua trata de medir el acuerdo entre dos agrupamientos ignorando las permutaciones. En otras palabras, esta medida trata de medir la cantidad de información compartida entre el agrupamiento y un particionamiento y es definida como sigue:

Dado un conjunto S de N elementos, tal que:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$S = \{S_1, S_2, \dots, S_N\}$$

Y dos particiones en este, que llamaremos X y Y tal que estas sean:

$$X = \{X_1, X_2, \dots, X_r\}$$

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

Suponiendo que: $X_i \cap X_j = \emptyset \forall i \neq j$ y que $Y_i \cap Y_j = \emptyset \forall i \neq j$, es decir que los clústers son duros y que además son completos, es decir:

$$\bigcup_{i=1}^r X_i = \bigcup_{i=1}^s Y_i = S$$

Entonces, la Información mutua entre X y Y vendría dada por:

$$MI(X, Y) = \sum_{i=1}^r \sum_{j=1}^s P_{XY}(i, j) * \log \frac{P_{XY}(i, j)}{P_X(i) P_Y(j)}$$

Donde la probabilidad $P_{XY}(i, j)$ es la probabilidad de que un caso o punto pertenezca tanto a un clúster X_i de X como a un clúster Y_j de Y, se puede calcular de la siguiente forma:

$$P_{XY}(i, j) = \frac{|X_i \cap Y_j|}{N}$$

La información mutua, expresada como cardinalidad de conjuntos puede verse de la siguiente manera:

$$MI(X, Y) = \sum_{i=1}^{|X|} \sum_{j=1}^{|Y|} \frac{|X_i \cap Y_j|}{N} * \log \left(\frac{N |X_i \cap Y_j|}{|X_i| |Y_j|} \right)$$

Dónde:

$|X|$ y $|Y|$ son las cantidades de clúster que contiene cada una de las respectivas particiones.

$|X_i|$ y $|Y_j|$ son las cantidades de puntos o casos que contiene el clúster i y j de cada

Tema 4. Algoritmos de aprendizaje automático no supervisado

partición.

N es el número de casos o puntos en S.

La información mutua, cuando X y Y son independientes, toma el valor cero; pero no tiene una cota superior. Por eso, aparece una versión de esta, llamada Información Mutua Normalizada y se puede expresar como:

$$NMI(X,Y) = \frac{MI(X,Y)}{mean}$$

Donde H(X) es la entropía de X y H(Y) la entropía de Y, y pueden expresarse como:

$$H(X) = - \sum_{i=1}^{|X|} P(i) * \log(P(i))$$

$$H(Y) = - \sum_{j=1}^{|Y|} P(j) * \log(P(j))$$

La Información Mutua Ajustada, por su parte, puede expresarse de la siguiente manera:

$$AMI(X,Y) = \frac{MI(X,Y) - E[MI(X,Y)]}{mean(H(X)H(Y)) - E[MI(X,Y)]}$$

Donde $E[MI(X,Y)]$ es el valor esperado de la Información Mutua.

Como ya comentamos estas métricas de evaluación de la calidad del agrupamiento llamadas Información Mutua, toman valores entre 0 y 1, en la tabla siguiente hemos recopilado, como en el caso anterior una comparación de sus valores al aplicarla sobre el Iris Dataset usando varios métodos de aprendizaje no supervisado.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	MI	AMI	NMI
K-Means	0.72	0.65	0.66
Affinity Propagation	0.73	0.66	0.67
Mean Shift	0.75	0.63	0.64
Spectral Clustering	0.68	0.61	0.62
DBSCAN	0.53	0.48	0.49
BIRCH	0.72	0.66	0.67

MI: Información Mutua. (Mutual Information)

AMI: Información Mutua Ajustada. (Adjusted Mutual Information)

NMI: Información Mutua Normalizada. (Normalized Mutual Information)

En la siguiente imagen mostramos la manera de cómo calcular estos índices usando scikit-learn de python.

```
#Clustering DBSCAN
from sklearn.cluster import DBSCAN
import numpy as np
import matplotlib.pyplot as plt
from numpy import unique
from numpy import where
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.metrics import mutual_info_score
from sklearn.metrics import adjusted_mutual_info_score
from sklearn.metrics import normalized_mutual_info_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# eps y min samples
model = DBSCAN(eps=0.68, min_samples=15)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
i=0
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(i, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
    i+=1
# show the plot
plt.show()
MI5 = mutual_info_score(true_labels, model.labels_)
print("MI:",MI5)
AMI5 = adjusted_mutual_info_score(true_labels, model.labels_)
print("AMI:",AMI5)
NMIS = normalized_mutual_info_score(true_labels, model.labels_)
print("NMI:",NMIS)
Silhouette = silhouette_score(X, model.labels_, metric='euclidean')
print("Coeficiente Silhouette:",Silhouette)
```

Fig. 5. Evaluando la validez del agrupamiento de un método DBSCAN sobre el Iris Dataset por medio de

Tema 4. Algoritmos de aprendizaje automático no supervisado

Información Mutua, Ajustada y Normalizada.

Estos índices, comparan el etiquetado verdadero (`true_labels`), por decirlo de alguna manera, con el asignado por el método de aprendizaje sin información alguna sobre la estructura de los datos. (`model.labels_`).

Homogeneidad, Completitud y V-Measure

Estas son otras medidas consideradas dentro del grupo de los criterios de evaluación externos y los tres están estrechamente ligados.

Con el propósito de explicar las ideas detrás de estas medidas de validez de un agrupamiento, debemos hacer las mismas suposiciones que hemos estado haciendo en las medidas anteriores.

Dado un conjunto S de N elementos, tal que:

$$S = \{S_1, S_2, \dots, S_N\}$$

Y dos particiones en este, que llamaremos X y Y tal que estas sean:

$$X = \{X_1, X_2, \dots, X_r\}$$

$$Y = \{Y_1, Y_2, \dots, Y_s\}$$

Suponiendo que: $X_i \cap X_j = \emptyset \forall i \neq j$ y que $Y_i \cap Y_j = \emptyset \forall i \neq j$, es decir que los clústers son duros y que además son completos, es decir:

$$\bigcup_{i=1}^r X_i = \bigcup_{i=1}^s Y_i = S$$

Entonces podemos comenzar a definir lo que se entiende por Homogeneidad, Completitud y V-Measure.

La Homogeneidad y la Completitud de un agrupamiento dado como resultado de la aplicación de un método de aprendizaje no supervisado a un conjunto de datos son

Tema 4. Algoritmos de aprendizaje automático no supervisado

dos medidas complementarias que se combinan para crear V-measure.

Para explicar estos conceptos es necesario que aclaremos que llamaremos a cada $X_i \in X$ una clase y a cada $Y_i \in Y$ un clúster. Y veremos a X como una plantilla contra la cual compararemos cuán bien el método de aprendizaje asignó cada punto de datos de S a Y sin información ninguna de la estructura dada por X .

Un agrupamiento, digamos Y , satisface el criterio de Homogeneidad, si todos los clústers en Y contienen puntos de datos de un mismo X_i , es decir de una misma clase en X . Por el contrario, un agrupamiento Y satisface el criterio de Completitud si un clúster $Y_i \in Y$ tiene entre sus puntos elementos de todas las clases $X_i \in X$.

Esto implica que la homogeneidad de un clúster es mayor mientras menos diverso es, el caso ideal, es decir, donde se maximizará esta medida es en un clúster con elementos de una sola clase.

Por otro lado, la Completitud sería mayor mientras más diverso es el clúster, es decir mientras más elementos de este correspondan a más clases de $X_i \in X$.

Estas medidas son simétricas y por tanto se complementan de alguna manera.

La Homogeneidad puede expresarse de la siguiente forma:

$$h = 1 - \frac{H(X \vee Y)}{H(X)}$$

Y la Completitud, se expresaría como sigue:

$$c = 1 - \frac{H(Y \vee X)}{H(Y)}$$

Donde $H(X \vee Y)$ es la entropía condicional y puede expresarse de la siguiente forma:

Tema 4. Algoritmos de aprendizaje automático no supervisado

$$H(Y) = - \sum_{i=1}^{|X|} \sum_{j=1}^{|Y|} \frac{n_{x,y}}{N} * \log \left(\frac{n_{x,y}}{N} \right)$$

Mientras que $H(X)$ es la entropía de la clase y se expresa como:

$$H(X) = - \sum_{i=1}^{|X|} \frac{n_x}{n} * \log \left(\frac{n_x}{n} \right)$$

Donde n_x es el número de casos en la clase o clúster i de X o de Y . Solo mostramos la entropía de X pero la de Y se calcula de la misma forma.

Por otra parte, la V-measure se expresa como la media armónica de estas dos medidas, y se expresa como:

$$v = \frac{2 * h * c}{h + c}$$

Sin embargo, una variante de esta, permite ponderar cada una de las medidas.

$$v = \frac{(1 + \beta) * h * c}{(\beta * h + c)}$$

Cundo $\beta < 1$ se pondera la homogeneidad, es decir se le da mayor importancia. Cuando $\beta > 1$ se le da más importancia a la completitud.

Sin embargo, uno de los problemas de estas medidas es que no están normalizadas con respecto al etiquetado aleatorio e implica que el resultado para este tipo de etiquetado no siempre sería el mismo ya que dependen del número de muestras, la cantidad de clústers y clases. Este problema puede ignorarse de manera segura si la cantidad de casos es mayor que 100 y la cantidad de clúster menor que 10.

En la tabla siguiente veremos, como hemos hecho con las medidas anteriores, una comparación de sus valores con el propósito de evaluar el agrupamiento arrojado por varios métodos sobre el Iris Dataset.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Método	h	c	v-measure
K-Means	0.66	0.66	0.66
Affinity Propagation	0.67	0.67	0.67
Mean Shift	0.69	0.60	0.64
Spectral Clustering	0.62	0.62	0.62
DBSCAN	0.48	0.49	0.48
BIRCH	0.66	0.68	0.67

Una de las ventajas de este tipo de medidas es que no hace asunción alguna sobre la forma de los clústers, lo que la hace apta para comparar el resultado del agrupamiento arrojado por varios métodos.

Como hemos estado haciendo hasta ahora, veremos la manera en que se aplican estas medidas usando la implementación en scikit-learn de Python. En la siguiente imagen se muestra el código de la aplicación de un método Mean Shift sobre el Iris Dataset y se evalúa por medio de la Homogeneidad, Completitud y v-measure. Los valores son lo que se muestran en la tabla anterior.

```
#Clustering con Mean Shift
from sklearn.cluster import MeanShift
from sklearn.cluster import estimate_bandwidth
import matplotlib.pyplot as plt
import statistics as stat
from numpy import unique
from numpy import where
from sklearn import datasets
from itertools import cycle
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import homogeneity_score
from sklearn.metrics import completeness_score
from sklearn.metrics import v_measure_score
# Cargamos el Iris Dataset. Tres clases, 150 casos
# 50 casos por clase, todos los atributos numéricos
idf = datasets.load_iris()
sc = StandardScaler()
X = sc.fit_transform(idf.data)
true_labels = idf.target
# Definimos el modelo, le pasamos como parámetro
# bandwidth y bin_seeding
#bandwidth = estimate_bandwidth(X, quantile=0.2, n_samples=150)
model = MeanShift(bandwidth=0.99)
# Hacemos el clustering y retornamos el clúster
# asignado a cada caso
yhat = model.fit_predict(X)
Labels = model.labels_
# Recuperamos los clusters
clusters = unique(yhat)
# Vamos a mostrar un Scatterplot para cada clúster
for cluster in clusters:
    print("Cluster:{0} cantidad de casos:{1}".format(cluster, len(yhat[yhat==cluster])))
    row_ix = where(yhat == cluster)
    plt.scatter(X[row_ix, 0], X[row_ix, 1])
# show the plot
plt.show()
# Calcula las métricas de validez
print("Homogeneidad:", homogeneity_score(true_labels, model.labels_))
print("Completitud:", completeness_score(true_labels, model.labels_))
print("V-Measure", v_measure_score(true_labels, model.labels_))
```

Fig. 6. Evaluando la validez del agrupamiento de un método Mean Shift sobre el Iris Dataset por medio de Homogeneidad, Completitud y v-measure.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Índice Fowlkes-Mallows

Este es otro de los índices o criterios de validez externa, como en las anteriores medidas, para usar esta, se necesita conocer el verdadero etiquetado de los casos del conjunto de datos.

El índice de Fowlkes-Mallows, puede ser interpretada como la media geométrica de la Precisión y el Recall y puede expresarse matemáticamente como:

$$FMI = \frac{TP}{\sqrt{(TP + FP)(TP + FN)}}$$

Donde TP, es el valor de los Casos Positivo (TP), es decir, el número de pares de puntos que pertenecen al mismo clúster tanto en el etiquetado verdadero y el etiquetado producido por el método aplicado. FP, es el número de Falsos Positivos (FP), es decir el número de pares de puntos que pertenecen al mismo clúster en el etiquetado verdadero, pero no así en el etiquetado arrojado por el método de aprendizaje. FN, son los llamados Falsos Negativos (FN), que son los pares de puntos en el etiquetado producido por el método de aprendizaje, pero no en el etiquetado verdadero.

Este tipo de medida tiene un rango entre 0 y 1. Valores altos son indicativos de una alta similitud entre las clases predichas y las reales.

Entre las ventajas de este método está que para un etiquetado aleatorio arroja valores cercanos a cero para cualquier cantidad de clústers y casos. Otra de las ventajas es su interpretabilidad, pues valores cercanos a cero indican que dos asignaciones o etiquetados son prácticamente independientes, mientras que valores cercanos a uno, indican un significativo acuerdo entre las dos. Además de un valor cero, indica dos etiquetados completamente diferentes o independientes y un valor exacto de uno dos etiquetados exactamente iguales.

Tema 4. Algoritmos de aprendizaje automático no supervisado

Este índice, tampoco hace asunciones sobre la forma de los clústers y por tanto puede usarse para comparar diferentes métodos de aprendizaje automático.

Conclusiones

Hasta aquí hemos hecho un breve recorrido por algunos de los métodos de evaluación de los resultados de los métodos de aprendizaje no supervisado. Sin ánimo de cubrir todo el campo, pues de hecho es muy basto. Literalmente hay cientos de métodos y esto refleja la complejidad del asunto.

Es importante para entender a cabalidad este tópico que se busque información adicional y se profundice en el tema.

En alguna literatura, a los criterios de evaluación, internos y externos se les suma una categoría adicional llamada criterios relativos que no hemos tocado en esta lección con vistas a no complicarla más de lo necesario.

Tampoco hemos tocado otras métricas que no están implementadas en scikit-learn, pues como meta tenemos que se puedan usar de forma inmediata los conocimientos que adquieran en estas lecciones. Sin embargo, se pueden encontrar en la literatura y la mayoría son muy fáciles de implementar.

Tema 5. Redes neuronales y deep learning

5.1. Introducción y objetivos

En esta unidad formativa se trata el tema de las redes neuronales, el aprendizaje automático profundo y el aprendizaje con reforzamiento. Las redes neuronales son modelos que pueden aplicarse tanto a tareas de clasificación como de regresión, sin embargo, por su importancia en el contexto actual, se tratan de manera independiente al resto de los modelos de aprendizaje.

El aprendizaje profundo, se basa en modelos de redes neuronales que hacen uso masivo de los componentes de las redes como las neuronas y las capas, usando funciones de activación mejoradas y optimizadoras.

Como las anteriores unidades funcionales, esta, está dividida en dos lecciones, en la primera se tratan las bases de las redes neuronales iniciando por una breve historia para luego introducir el concepto de neurona artificial y redes de neuronas artificiales.

Luego tratamos lo que se conoce como perceptrón, la más simple de las redes que pueden ser útiles y consideradas dentro de la categoría FeedForward, entonces estudiamos el perceptrón multicapa y sus algoritmos de entrenamiento que forman la base del ulterior desarrollo para terminar estudiando otro modelo de red, conocidas como redes neuronales recurrentes. En esta lección hacemos una introducción a modelos recurrentes como Hopfield, Elman y Jordan.

El estudio de los modelos recurrentes se continúa en la siguiente lección, estudiando modelos más avanzados como LSTM y GRU.

En esta segunda lección se hace una introducción a las redes neuronales convolucionales que solo se tratan de manera superficial y no porque no sean importantes sino por cuestiones de la complejidad aparejada a una lección introductoria al tema de las redes neuronales.

Tema 5. Redes neuronales y deep learning

En esta segunda lección se trata de manera también bastante superficial las principales plataformas de desarrollo de modelos usando aprendizaje profundo (Deep Learning) y terminamos la lección tocando lo referente al aprendizaje con reforzamiento, que, aunque merece una lección aparte puede formar parte de esta al estar relacionada directamente con el uso de redes neuronales para aprender las funciones de valor y políticas de estos métodos.

En las lecciones anteriores hemos tratado los métodos de aprendizaje automáticos supervisados y no supervisados, sin embargo uno de los métodos más importantes, las redes neuronales, las habíamos pasado por alto. El motivo, es simple, este tema necesita un apartado por sí solo, pues a día de hoy constituye el motor de los éxitos que ha alcanzado el campo del aprendizaje automático.

En estas dos lecciones, tocaremos las bases conceptuales de las redes neuronales y su uso tanto en aprendizaje supervisado como no supervisado; pero haremos hincapié en su uso para el aprendizaje profundo.

Por tanto, al finalizar estas dos lecciones estará usted equipado con el conocimiento necesario para comenzar a usar, aunque a nivel introductorio las técnicas de aprendizaje profundo con redes neuronales.

Los objetivos que se persiguen son:

- ▶ Conocer que son las redes neuronales artificiales y sus clasificaciones.
- ▶ Conocer que es un perceptrón multicapa y su algoritmo de entrenamiento. (BackPropagation)
- ▶ Conocer y aplicar las redes neuronales recurrentes y sus principales modelos.
- ▶ Conocer y aplicar las redes neuronales convolucionales.
- ▶ Conocer las principales plataformas para el desarrollo de modelos de aprendizaje profundo, como Tensorflow y Pytorch.

Tema 5. Redes neuronales y deep learning

- Conocer que es y como se usa el aprendizaje con reforzamiento.

Introducción

Las redes neuronales, forman hoy, bajo el término paraguas «Deep Learning» una de las principales tendencias dentro de la Inteligencia Artificial y especialmente el Aprendizaje Automático. Si se le ocurre hacer una búsqueda en Google usando el término «Deep Learning» le sorprenderá que arroje 2,080,000,000 de elementos encontrados, contra 880,000,000 elementos encontrados con el término «Artificial Intelligence». Aunque son métricas que solo muestran la popularidad del término, dan una idea más o menos clara de su éxito.

Esta popularidad se explica en gran medida por el éxito en la aplicación de las técnicas de Aprendizaje automático a problemas que antes eran impensables. Esta, ha sido catalizada por eventos tales como la victoria de una Inteligencia Artificial basada en Deep Learning ante el varias veces campeón mundial de go, los éxitos en su aplicación a la conducción autónoma de coches, la calidad de las traducciones del traductor de google, a los incontables éxitos en reconocimiento de imágenes y de manera general a todos los éxitos en la mayoría de las esferas de la vida cotidiana. Según IDC, la inversión en tecnología IA, superará los 110 mil millones en el 2024.

Este impresionante éxito ha sido propiciado por varios ingredientes, en primer lugar los avances en la solución de los problemas que aquejaban el entrenamiento de redes neuronales con muchas capas ocultas, la disponibilidad de grandes conjuntos de datos gracias a internet, el uso intensivo de GPU en el entrenamiento de modelos de IA, la aparición de frameworks de código abierto como Theano, Pytorch, TensorFlow y wrappers como Keras, por solo mencionar algunos.

Si el éxito a gran escala de la IA, en los últimos años, ha dependido de Deep Learning, esta depende de las redes neuronales artificiales, que forman su núcleo y que no son nuevas, pues fueron introducidas alrededor de 1943. Hagamos algo de

Tema 5. Redes neuronales y deep learning

historia.

Breve reseña histórica

En 1888, el Español Santiago Ramón y Cajal, descubre los mecanismos que gobiernan la morfología y los procesos conectivos de las células nerviosas de la materia gris del sistema nervioso cerebroespinal. Su teoría fue aceptada en 1889 en el Congreso de la Sociedad Anatómica Alemana, celebrado en Berlín. Su esquema estructural del sistema nervioso como un aglomerado de unidades independientes y definidas, pasó a conocerse como “doctrina de la neurona” y en ella destaca la ley de la polarización dinámica, modelo capaz de explicar la transmisión unidireccional del impulso nervioso. Su trabajo y su aportación a la neurociencia se verían reconocidos, finalmente, en 1906, con la concesión del Premio Nobel de Fisiología y Medicina.

En 1943, Warren McCulloch y Walter Pitts introducen las redes neuronales, aunque no de forma algorítmica, sino por medio de una red con circuitos eléctricos. Lo interesante es que por primera vez se modelaba de forma práctica el funcionamiento de una neurona biológica. Los resultados de sus modelos fueron funciones lógicas elementales tales como “a or b” y “a and b”.

A finales de los 40, exactamente en 1949, D. O. Hebb, en su libro “The Organization of Behavior”, basado en la idea de Neuroplasticidad, explica el proceso de aprendizaje y propone una hipótesis de aprendizaje conocida como regla de Hebb y el aprendizaje basado en esta se conoce como “aprendizaje Hebbiano”.

En 1954, Farley y Wesley A. Clark fueron los primeros en usar máquinas computacionales entonces conocidas como “calculators”, para simular una red Hebbiana.

En 1958, Frank Rosenblatt desarrolla el Perceptron, la primera red neuronal. Financiada por la Office of Naval Research.

En 1960, Widrow y Hoff, de la Universidad de Stanford, desarrollan ADALINE

Tema 5. Redes neuronales y deep learning

(ADAPtive Llinear Element). El ADALINE fue un dispositivo electrónico analógico hecho de componentes simples, con un método de aprendizaje diferente al del Perceptron, empleando la regla de aprendizaje basada en mínimos cuadrados. (LMS. Least Mean Square).

En 1965, se publica el trabajo correspondiente a la primera red neuronal multicapa, desarrollado por Ivakhnenko y Lapa, como el Group Method of Data Handling. (GMDH)

Tema 5. Redes neuronales y deep learning

En 1960, se sentaron las bases para el método “Backpropagation” continuo a partir de la teoría del control, con aportes sustanciales de Henry J. Kelley, que fue profesor de ingeniería aeroespacial del Instituto Politécnico de Virginia. Otro de los aportes a «Backpropagation» llegó en 1961 de parte del profesor Arthur Earl Bryson Jr, de la universidad de Stanford, considerado el padre de la teoría del control óptimo moderna. Resolvía el problema de «Backpropagation» con Programación Dinámica.

En 1969, Minsky y Papert presentaron un trabajo, en cual se recalcaron las limitaciones del Perceptrón para solucionar problemas complejos, esencialmente aquellos que no son linealmente separables. Estos autores, además generalizaron las limitaciones de un Perceptrón monocapa a las redes neuronales multicapa, cuando plantearon: «...nuestro intuitivo juicio es que la extensión (a sistemas multicapa) es una tarea estéril». El resultado de las afirmaciones de este trabajo fue disminuir el interés de los investigadores en la simulación de redes neuronales; lo que generó un desencanto de los investigadores en el área y trajo como resultado un periodo en el cual prácticamente se dejó de hacer nuevas propuestas.

En 1970, Seppo Linnainmaa publicó un método general para la diferenciación automática de redes discretas conectadas de funciones diferenciables anidadas.

Tema 5. Redes neuronales y deep learning

En 1973, Dreyfus usó «backpropagation» para adaptar los parámetros de un controlador en proporción al error del gradiente.

En 1974, Paul Werbos, fue el primero en proponer en los Estados Unidos el uso de «backpropagation» para el entrenamiento de redes neuronales.

En 1977, Stephen Grossberg, desarrolla la Teoría de Resonancia Adaptada, que es una arquitectura de red que se diferencia de todas las demás previamente inventadas. La misma simula otras habilidades del cerebro, como memoria a largo y corto plazo.

En 1985, John Hopfield provocó el renacimiento de las redes neuronales con su libro: *Computación neuronal de decisiones en problemas de optimización*.

En 1986, David Rumelhart y G. Hinton. Redescubrieron el algoritmo backpropagation. En este mismo año Rumelhart, Hinton y Williams mostraron que el algoritmo backpropagation aprendía representaciones internas interesantes de palabras como vectores de rasgos cuando se entrenaban para predecir la próxima palabra en una secuencia. Este algoritmo fue el que permitió entrenar redes multicapas de manera supervisada.

En 1989, Yann LeCun introducen las Redes Neuronales Convolucionales, inspiradas en las propiedades del cortex visual de los animales.

En 1997, se introducen las LSTM y las redes neuronales recurrentes. LSTM, significa *Long Short Term Memory*, estas memorias permiten capas que se conectan hacia atrás entre las capas lo que las hace ideales para el procesamiento de datos de series de tiempo.

En 2006, Geoffrey Hinton et al. Proponen el aprendizaje de representaciones de alto nivel usando capas sucesivas llamadas variables latentes, binarias o de valor real, usando Máquinas de Boltzman Restringidas, para modelar dichas capas.

Tema 5. Redes neuronales y deep learning

En 2012, Andrew Y. Ng y Jeff Dean crearon una red que reconocía conceptos de alto nivel, tales como gatos, solo observando imágenes no etiquetadas. Con esto nacía lo que es conocido como *Deep Learning*.

En el 2014, aparecen las GAN, *Generative Adversarial Networks*.

Hasta aquí hemos hecho un recorrido por los hechos más relevantes de la historia del desarrollo de las redes neuronales hasta lo que es hoy. En este periodo, desde su aparición hasta la fecha hay tres periodos claramente definidos: El comprendido entre el 40 y el 70, desde los 80 hasta el 2006 y a partir del 2006.

No es nuestra intención hacer una evaluación de los distintos periodos y solo los mencionamos para poner todo el contenido que a continuación expondremos, en contexto.

Queda completamente claro, de esta primera parte, que el concepto de red neuronal artificial se desarrolló a partir de la neurona biológica que había descubierto Cajal en el 1888. En la próxima sección veremos en algunos detalles la estructura y funcionamiento de la neurona biológica.

La neurona biológica

La neurona biológica, es una célula viva y cuenta con los mismos elementos que el resto de ellas. Como puede verse en la imagen de la fig. 1. Una neurona cuenta con un cuerpo, llamado soma con apéndices llamados dendritas y uno llamado axón.

La función de estos apéndices es conectarlas unas entre otras, por medio de lo que se conoce como sinapsis.

Tema 5. Redes neuronales y deep learning

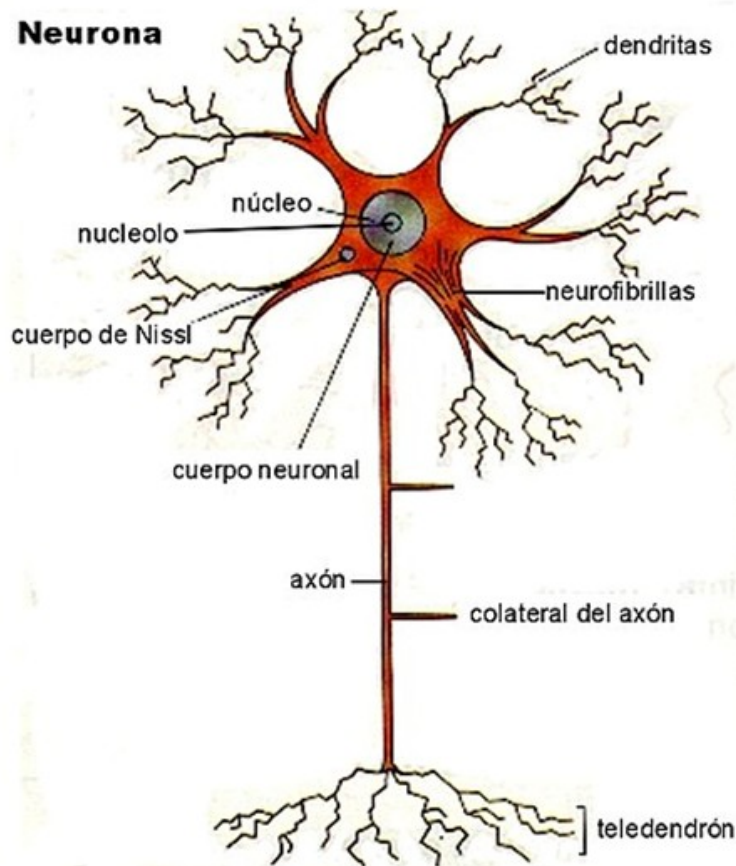


Fig. 1. Representación de una neurona biológica. (Wikipedia)

Las neuronas reciben impulsos eléctricos de otras neuronas, por medio de las dendritas, que son integrados por el núcleo para luego ser transmitidos, a otras, por medio del axón. Cada neurona se conecta con cientos de otras, por medio de las dendritas y axones. Se piensa que el número de conexiones en un cerebro es del orden de 10^{15} .

Toda la base del funcionamiento del cerebro es electroquímica. Las neuronas de forma interna transmiten impulsos eléctricos, que son transferidos entre ellas por medio de una acción química en las sinapsis.

La conexión entre una neurona y otra no se hace de forma directa, se hace por medio de lo que se conoce como sinapsis.

Tema 5. Redes neuronales y deep learning

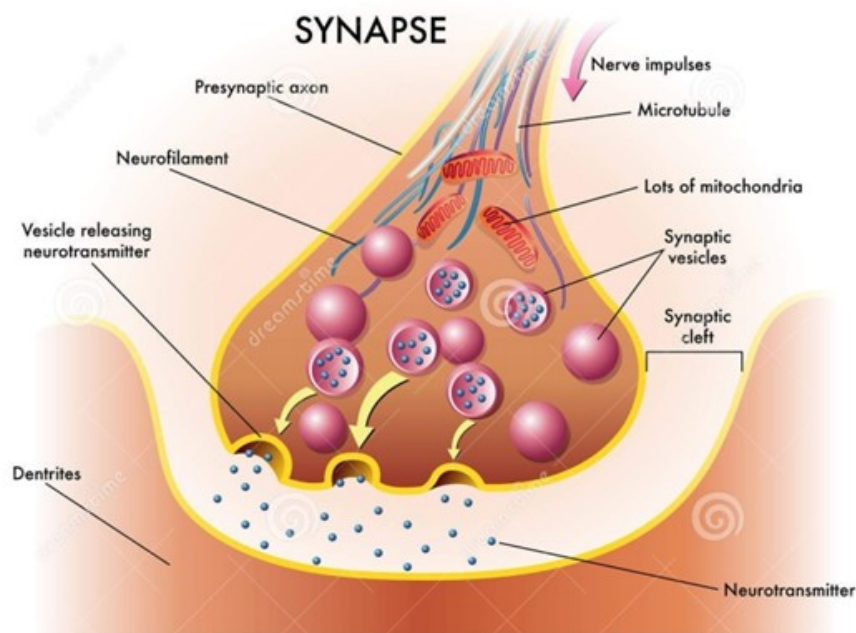


Fig. 2. Imagen de una Sinapsis.

En la imagen de la figura 2. Puede observarse una representación de una sinapsis, mientras que en la Fig. 3. Se muestran los componentes de esta.

La sinapsis, está compuesta por la neurona pre sináptica, el espacio sináptico y la neurona post sináptica. Los impulsos eléctricos que viajan por el axón, llegan a esta zona y provocan descargas de neurotransmisores que cruzando el espacio sináptico llegan a las dendritas de la neurona post sináptica. El efecto de estas descargas puede ser excitatorias, inhibitorias o modulador.

Las descargas de tipo excitatorias, propicia que las neuronas post sinápticas alcancen el potencial de acción (disparo de un impulso eléctrico por medio del axón), mientras que las inhibitorias las reducen.

Cada sinapsis, se puede caracterizar por su fuerza o peso sináptico, que puede definirse como la cantidad promedio de corriente o la amplitud del cambio de voltaje producido en la neurona post sináptica por un potencial de acción en la neurona pre

Tema 5. Redes neuronales y deep learning

sináptica. Es decir, el peso sináptico define el efecto que tiene un impulso nervioso producido en una neurona, sobre otra, conectada con la primera.

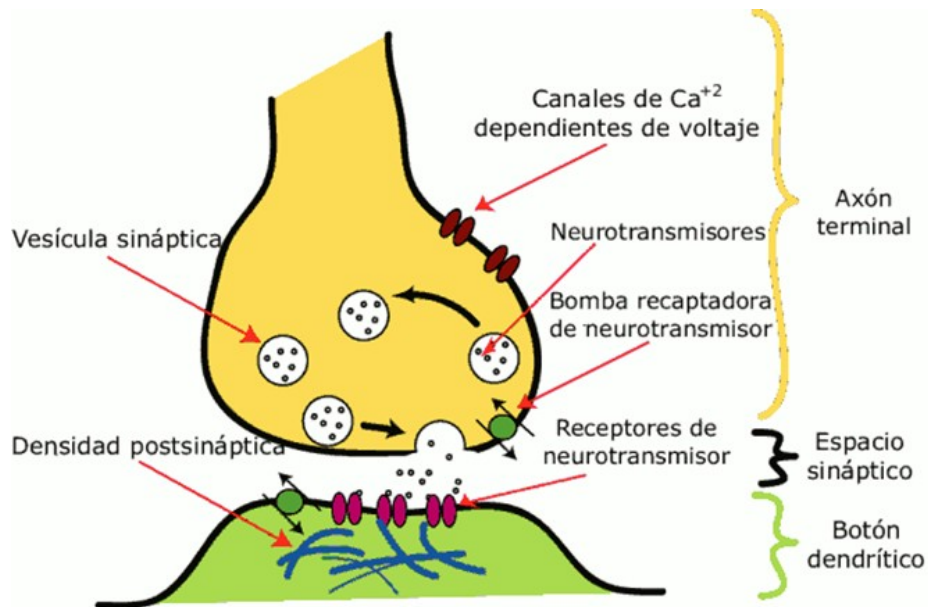


Fig. 3. Componentes de una sinapsis. (Wikipedia)

La neurona artificial, introducida por Warren McCulloch y Walter Pitts, es una abstracción de la neurona biológica y se basa, esencialmente, en los elementos que hemos introducido anteriormente: Las dendritas, el soma o cuerpo de la neurona, el axón, los pesos sinápticos y la forma en que se combinan las señales de las dendritas para provocar el disparo de un impulso eléctrico por el axón.

Básicamente, la idea detrás de esta abstracción es que las señales entrantes por las dendritas, moduladas por el efecto del peso sináptico, se combinan en el soma de la neurona hasta que alcanza el potencial de acción que viaja a lo largo del axón hasta otras neuronas.

Con estos elementos a mano podemos explicar en qué consiste la idea de neurona artificial y que veremos en las secciones siguientes.

Tema 5. Redes neuronales y deep learning

La neurona artificial

La neurona artificial está compuesta por n entradas, cada una haciendo las veces de dendritas y asociada a un peso w , que recuerda al peso o fuerza sináptica. Estas entradas son combinadas de forma lineal en el soma de la neurona y luego se determina la salida por medio de una función no lineal. Dicha salida puede ir conectada a otras neuronas o usada de manera independiente.

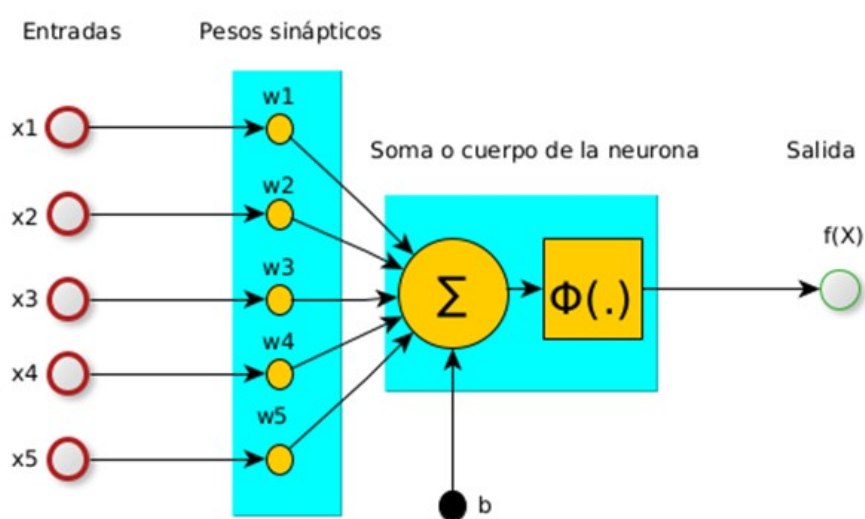


Fig. 4. Esquema de una neurona artificial.

En la Fig. 4. Se muestra una representación de una neurona artificial, los círculos con bordes rojos son las entradas, en este caso se muestran 5 pero pueden ser n . Con círculos de color naranja sobre un rectángulo y con etiquetas w , se muestran los pesos asociados a cada entrada.

Matemáticamente, el proceso interno para derivar la salida $f(X)$ de esta neurona artificial es la siguiente:

$$f(X) = \Phi \left(\sum_{i=1}^n x_i w_i + b \right)$$

Tema 5. Redes neuronales y deep learning

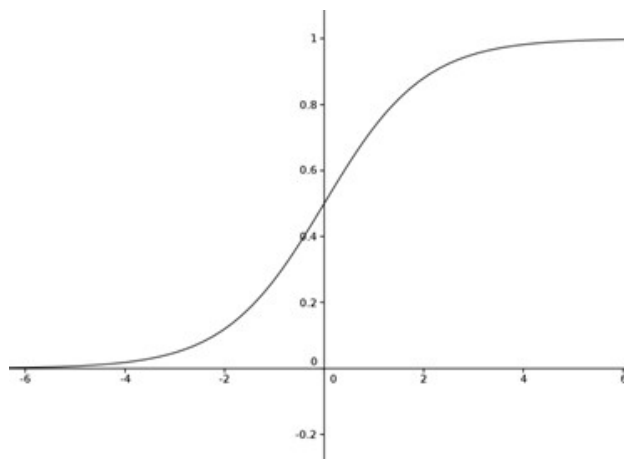
Para $X = \{x_1, x_2, x_3, \dots, x_n\}$ y b . Donde a b se le denomina bias y a la función $\Phi()$ se le llama función de activación.

Esta función de activación puede tomar muchas formas, por ejemplo:

Función Sigmoide:

$$\Phi(x) = \frac{1}{1 + e^{-x}}$$

El gráfico de la función es el siguiente:

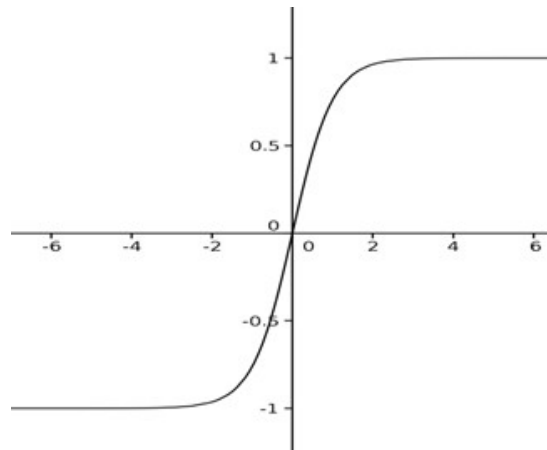


Función Tangente Hiperbólica:

$$\Phi(x) = \frac{\sinh(x)}{\cosh(x)} = \tanh(x)$$

El gráfico de la función es el siguiente:

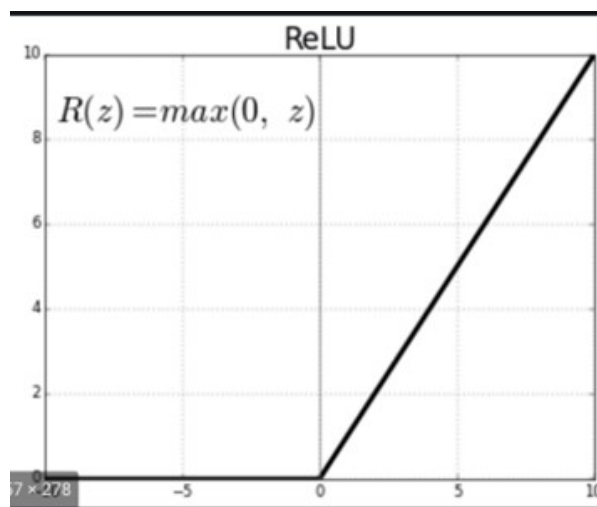
Tema 5. Redes neuronales y deep learning



Función RELU:

$$\Phi(x) = \max(0, x)$$

El gráfico de la función es el siguiente:



En la siguiente figura mostramos a modo de resumen, algunas de las funciones de activación para neuronas, aunque no son las únicas.

Tema 5. Redes neuronales y deep learning

Funciones de Activación

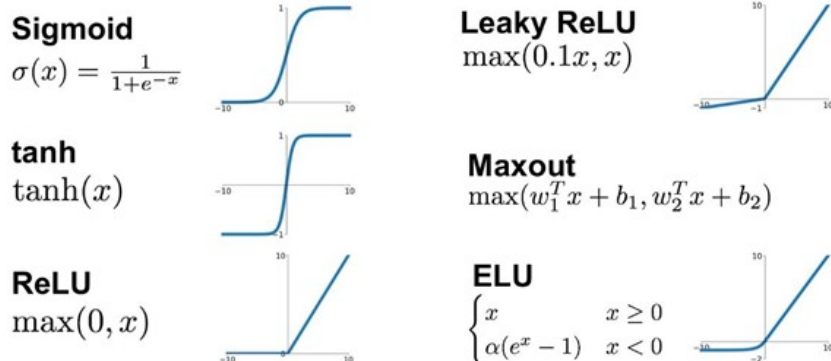


Fig. 5. Algunas de las Funciones de activación usadas.

A modo de resumen, tenemos que la neurona artificial, tiene x entradas, n pesos y una función de activación. Sin embargo, estas neuronas, por si solas no tienen mucha aplicabilidad, sin embargo cuando estas son combinadas para formar redes más complejas su utilidad comienza a ser significativa.

Antes de pasar el próximo tema, es importante que expliquemos, que para encontrar la función de salida $f(X)$ en una neurona, es preciso encontrar un conjunto de pesos W y b . Es decir, los parámetros a estimar a partir de los x para encontrar y son los w y los b . Es sumamente importante que lo entiendan desde ahora, pues todo el entrenamiento y predicción de las redes neuronales se basa en dicha estimación y uso.

Es a esta combinación de neuronas individuales a lo que se le llama Redes Neuronales. En la siguiente sección estudiaremos algunas de las formas de combinarlas y las características de estas.

Redes Neuronales Artificiales

Las redes neuronales son, conjuntos de neuronas organizados de una determinada manera, en donde la información se pasa desde la entrada a la salida. A esta organización de las neuronas se le llama arquitectura y pueden irse clasificando

Tema 5. Redes neuronales y deep learning

según algunos criterios. Para obtener una idea, a groso modo de lo que se conoce como Red Neuronal, veamos una imagen de alguna y eso ayudará a formar una intuición inicial.

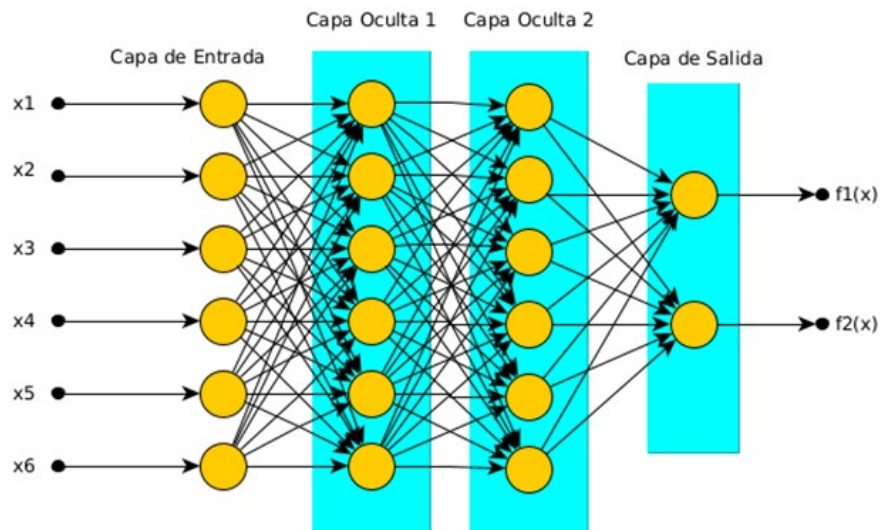


Fig. 6. Representación de una red neuronal multicapa feedforward, con 6 neuronas en la capa de entrada, 2 capas ocultas de 6 neuronas y una capa de salida con dos neuronas.

En esta imagen, las neuronas están representadas por círculos naranjas. Como puede apreciarse en la imagen, las neuronas se pueden agrupar en capas. Siempre una red constará de una capa de entrada y una de salida y cualquier cantidad de las capas intermedias que se les llama capas ocultas.

Es importante subrayar, que en la capa de entrada no se efectúan operaciones y su función es acoplar cada rasgo del dataset a las neuronas de las capas ocultas.

Si tenemos en cuenta la cantidad de capas de una red, estas se pueden clasificar en:

- ▶ Monocapa: Tienen una sola capa.
- ▶ Multicapas: Tienen más de una capa.

Si se toma en cuenta la forma en que fluye la información, pues se pueden clasificar

Tema 5. Redes neuronales y deep learning

en:

- ▶ Redes Feedforward: La información fluye desde la capa de entrada hasta la capa de salida. En una sola dirección.
- ▶ Redes Recurrentes: En determinadas capas se permiten conexiones hacia atrás.

Redes Neuronales Monocapa

En esta arquitectura la red neuronal se forma a partir de un grupo de neuronas artificiales cuyo número coincide con el número de rasgos del conjunto de datos. Como en la capa de entrada no tiene lugar ningún procesamiento, todo el aprendizaje tiene lugar en la misma capa de salida.

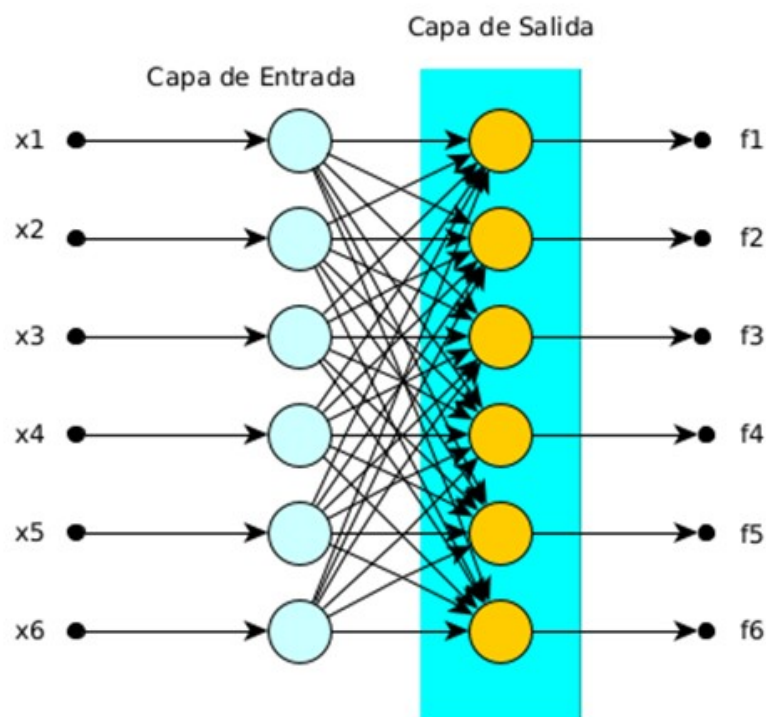


Fig. 7. Red neuronal monocapa.

Este tipo de red no es muy utilizada, pero pueden ser perfectamente funcionales para

Tema 5. Redes neuronales y deep learning

determinado problema.

5.1.7. Redes Neuronales Multicapa

Este tipo de red, está compuesta por una capa de entrada con n neuronas y una capa de salida con p neuronas. Entre ellas pueden aparecer x capas llamadas ocultas con y neuronas. Las capas ocultas pueden tener cualquier cantidad de neuronas y sus funciones de activación no tienen que ser las mismas.

Una representación de una red neuronal multicapa se ha dado ya en la fig. 6. En el caso de esta red, tiene dos capas ocultas, con 6 neuronas en cada una y una capa de salida con dos neuronas.

5.1.8. Redes Neuronales Feedforward

Se le llama de esta forma a una red neuronal donde las salidas de todas las neuronas en la capa de entrada se conectan a las entradas de las neuronas de la capa siguiente y así sucesivamente hasta la capa de salida. Es decir la información de entrada es procesada de forma secuencial y de forma unidireccional, desde la entrada a la salida.

Este es el caso de la red neuronal mostrada en la fig. 6. Cada neurona en la capa de entrada es conectada a cada una de las entradas en las neuronas de la capa siguiente y así hasta la capa de salida.

5.1.9. Redes Neuronales Recurrentes

A diferencia de las redes neuronales Feedforward, en este tipo de red las neuronas pueden conectarse a neuronas en las capas anteriores o a sus propias entradas por medio de lazos de realimentación.

En la siguiente imagen mostramos un ejemplo de red neuronal recurrente.

Tema 5. Redes neuronales y deep learning

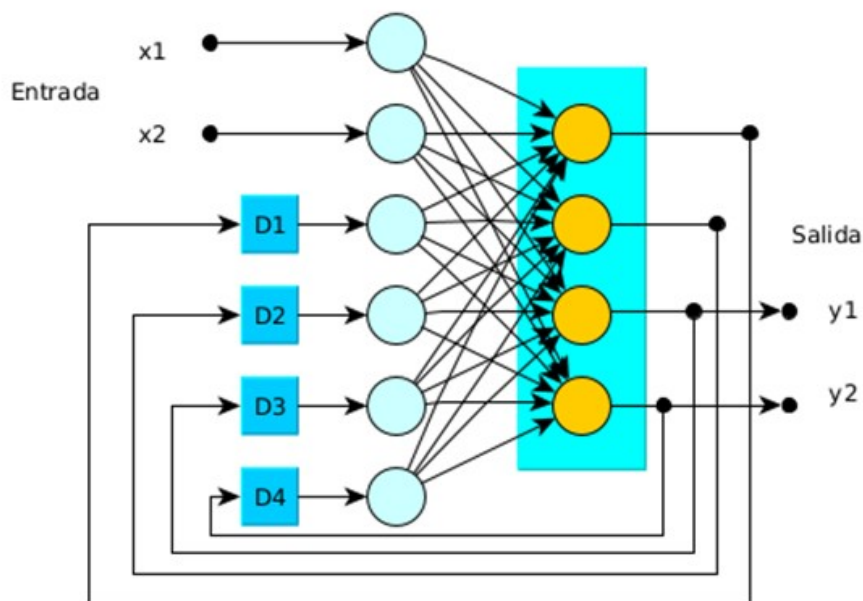


Fig.8. Red Neuronal Recurrente.

En la imagen de la Fig. 8, se ve claramente el detalle distintivo de las redes recurrentes, el lazo de realimentación que se hace por medio de los operadores de demora denotados aquí con el D1 a D4. No profundizaremos aquí en este tipo de red, pues en secciones siguientes nos detendremos en los detalles de su funcionamiento.

Aunque esta clasificación que hemos dado, puede que no sea completa es suficiente para introducir los conceptos más importantes del campo de las redes neuronales y el aprendizaje automático profundo. (Deep Learning) Por tanto no haremos hincapié sobre el tema y seguiremos avanzando.

5.1.10. El Perceptrón

El Perceptrón es sin duda la más simple de las redes neuronales y fue introducida por Frank Rosenblatt en 1958. Al ser una neurona sencilla, es muy útil para introducir los elementos esenciales que permiten luego entender redes mucho más complejas.

Tema 5. Redes neuronales y deep learning

En esta sección desarrollaremos la intuición del funcionamiento interno de esta neurona. Para ello diseñaremos una neurona artificial de tipo Perceptrón, con dos entradas y una salida. El esquema de dicha neurona se muestra en la siguiente figura.

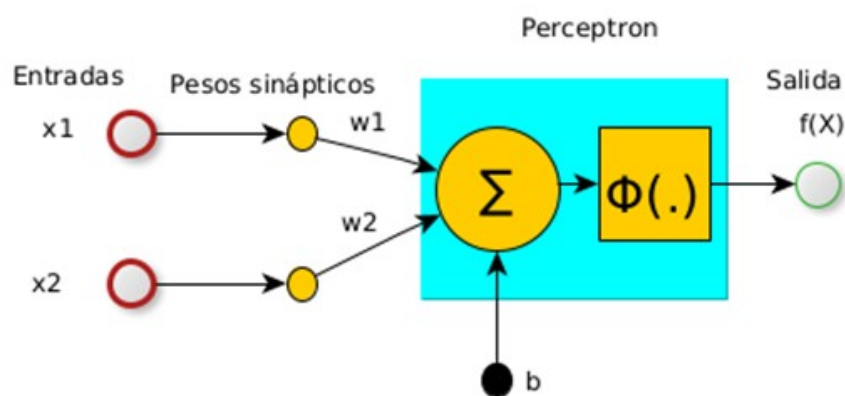


Fig.9. Perceptrón.

Es importante que destaquemos que este tipo de neurona es perfectamente funcional aunque solo da buenos resultados cuando es aplicado a problemas donde las clases son linealmente separables.

Rosenblatt, introdujo el teorema de convergencia del Perceptron y este dice:

Teorema de convergencia del Perceptrón: Dado un Perceptrón de una neurona y un conjunto de entrada $X = \{x_1, x_2, x_3, \dots, x_n\}$ de dos clases linealmente separables. Si le presentamos los datos en una secuencia arbitraria en cada época. Entonces, a partir de un estado inicial arbitrario, el procedimiento de aprendizaje del Perceptrón siempre converge y arroja un hiperplano de decisión para las dos clases en un tiempo finito.

$$f(X) = \Phi \left(\sum_{i=1}^n x_i w_i + b \right)$$

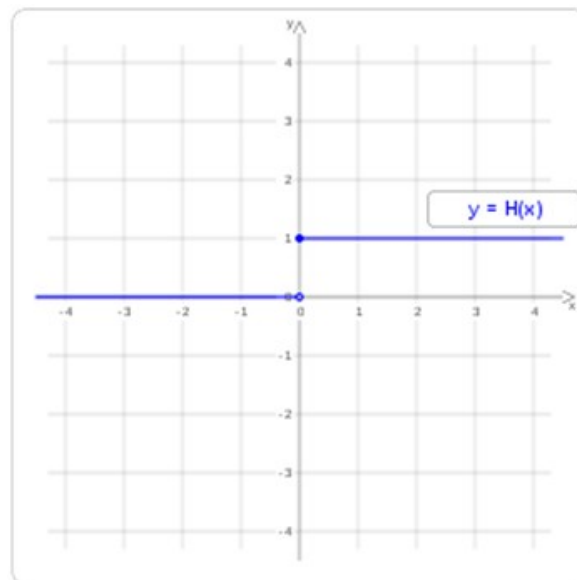
Tema 5. Redes neuronales y deep learning

Por tanto a los efectos de demostrar el funcionamiento del Perceptrón, usaremos la Función Escalón de Heaviside, cuya interpretación matemática es como sigue:

$$\Phi(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ 0 & \text{si } x < 0 \end{cases}$$

Esto significa que cuando x es mayor o igual a cero la función retorna 1 y de lo contrario cero.

El gráfico de esta función se muestra a continuación:



Nos queda por definir la manera que se actualizarán los pesos de nuestra neurona durante el proceso de entrenamiento, en este caso, usaremos la llamada “Regla de actualización del Perceptrón” y tiene el siguiente aspecto:

$$W_{t+1} = W_t + \sigma (y_i - \hat{y}_i) x_i$$

Donde σ se le conoce como tasa de aprendizaje o Learning Rate y determina la velocidad de convergencia del Perceptrón. Sus valores deben estar entre 0 y 1.

Con esto, quedan completamente definidos los elementos necesarios para entrenar una neurona de tipo Perceptrón.

Tema 5. Redes neuronales y deep learning

Los datos que usaremos como ejemplo para el entrenamiento son los siguientes:

```
X=np.array([[0.3,0.84],[0.45,0.9],[0.48,0.82],[0.53,0.73],
[0.54,0.32],[0.76,0.35],[0.75,0.49],[0.69,0.23]])

Y =[1,1,1,1,0,0,0,0]

# Puntos que sirven de conjunto de prueba.

X_test=np.array([[0.07,0.65],[0.06,0.75],[0.14,0.9],[0.34,0.79],
[0.38,0.06],[0.56,0.05],[0.69,0.16],[0.53,0.17]])

Y_test =[1,1,1,1,0,0,0,0]
```

Como en todos los métodos de aprendizaje que hemos visto hasta aquí, los casos del conjunto de datos se consideran vectores en un espacio n dimensional. Esta n se refiere a la cantidad de rasgos que contiene cada caso.

En los datos que utilizaremos para demostrar como aprende una neurona, hemos elegido por simplicidad, dos rasgos para cada caso y cada uno de estos rasgos coincide con la coordenada de un punto en un plano cartesiano.

Con esto claro, podemos decir entonces, que aprender, en este contexto significa encontrar los parámetros que definen un hiperplano que separa las clases del conjunto de datos de forma segura.

En un plano bidimensional, este hiperplano toma forma de una recta cuya ecuación tiene el siguiente aspecto:

$$y = Wx + b$$

Seguramente habrán notado que para encontrar dicha recta, solo tenemos los datos X , por tanto tenemos que estimar los parámetros W y b .

Tema 5. Redes neuronales y deep learning

El procedimiento que usaremos para ello es el siguiente:

Algoritmo de entrenamiento del Perceptrón

Algoritmo:

Entrada: Conjunto de Datos.

Entrada: Taza de aprendizaje. (Learning Rate)

Entrada: Número de repeticiones: épocas.

Inicializar $W=0$ (o de forma aleatoria)

Para cada época, repetir

- ▶ Para cada caso x , repetir
 - Clasifica x usando los parámetros en W
 - Si la clasificación es incorrecta ajustar W

Terminar y retornar W

Como el algoritmo es sumamente sencillo, no creo que lleve mucha explicación, sin embargo los puntos más interesantes son los pasos 1 y 2. Veamos cómo se clasifica un x , es decir, como se le asigna una clase a partir del vector de parámetros w .

Clasifica (x, W)

- ▶ Realizar el producto escalar de los vectores x y w
- ▶ Aplicar la función escalón de Heaviside:
 - Retornar 1.0 si el producto es mayor o igual a cero, si no, cero.

En el caso que intentamos explicar, x es de dos dimensiones y w debe ser de la

Tema 5. Redes neuronales y deep learning

misma cantidad, pues sabemos que w contiene el peso para cada una de las entradas de la neurona y que estas son iguales a la cantidad de dimensiones de los vectores de entrada. Sin embargo, hay que añadir el bias, que se considera como un peso más y por tanto hay que añadirlo también a la entrada para que tengan la misma dimensión, esto es solo algo práctico, pero no interviene a nivel conceptual.

El producto escalar de dos vectores se obtiene de la siguiente forma:

$$\{w_1, w_2\} \otimes \begin{Bmatrix} x_1 \\ x_2 \end{Bmatrix} = \{(w_1 * x_1) + (w_2 * x_2)\}$$

Como podemos ver, esta clasificación es sumamente sencilla, así que veamos cómo se ajustan los pesos en W cuando se ha incurrido en un error de clasificación y esta es la esencia del método de aprendizaje en cualquier tipo de neurona o red.

Al método de ajustar los pesos en W , es el siguiente:

Para cada $w_i \in W$:

$$w_i = w_i + \sigma(y_i - \hat{y}_i) x_i$$

Donde y_i es la etiqueta del punto x y $\hat{y}_i$ es la etiqueta predicha, basada en W .

La componente $\sigma(y_i - \hat{y}_i) x_i$ de esta regla de actualización encierra tanto la magnitud del incremento como la dirección de este. Veamos.

Dado que y_i y $\hat{y}_i$ solo pueden tomar valores de 0 o 1, dado por la función escalón o la función de activación de la neurona, entonces pueden suceder dos cosas:

- ▶ $y_i = 0$ y $\hat{y}_i = 1$
- ▶ $y_i = 1$ y $\hat{y}_i = 0$

En el primer caso el resultado sería un valor negativo, lo cual indica una disminución en los pesos y el segundo caso daría como resultado lo inverso.

Tema 5. Redes neuronales y deep learning

Pongamos como ejemplo el siguiente, supongamos que el learning rate, sea 0,5 y el peso 0,3 y el valor de x , sea 0,7, entonces:

$$w = 0.3 + 0.5 ((1 - 0) 0.7)$$

$$w = 0.3 + 0.35$$

Este es la situación, donde el caso x es de la clase 1 y se le asignó de forma errónea la clase 0 basado en los pesos w . Esto implica aumentar el peso en 0.35. Veamos el caso contrario.

$$w = 0.3 + 0.5 ((0 - 1) 0.7)$$

$$w = 0.3 - 0.35$$

Lo que implica que habría que disminuir el peso w .

$$w_i \leftarrow w_i + \Delta w_i$$

Donde:

$$\Delta w_i = \eta (t - o) x_i$$

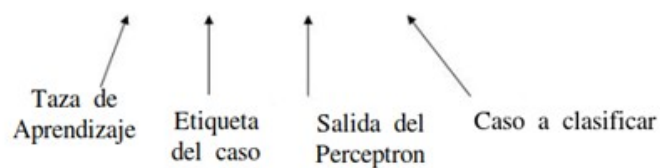


Fig.10. Regla de aprendizaje del Perceptrón. (Regla de Hebb).

En la siguiente imagen mostramos la implementación de este método de aprendizaje para un Perceptrón, solo para clasificación binaria y con clases linealmente separables.

Es importante que se entienda que el resultado del entrenamiento es la matriz de

Tema 5. Redes neuronales y deep learning

pesos W . Luego a la hora de predecir la clase de casos no vistos en el entrenamiento o en las pruebas, se usa siempre esta matriz.

También es importante que se den cuenta que en la predicción solo intervienen dos operaciones, la multiplicación de un vector por una matriz de pesos y aplicación de la función umbral o función de activación de la neurona o la red y el trabajo pesado se hace a la hora del entrenamiento.

Esto quiere decir que una vez que entrenamos una red y obtenemos la matriz de pesos, podemos hacer predicciones con muy poca carga computacional, incluso pueden usarse dispositivos embebidos, para tomar decisiones basados en el resultado de una red neuronal con muy poco costo.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
#*****
def perceptron(X,y,lrate,epocas):
    #Se extrae el número de casos y el número de rasgos
    m, n = X.shape
    # Se inicializan los pesos a cero y se agrega el bias
    W = np.zeros((n+1,1))
    miss_list=[]
    # Esto lo repetimos el número de epocas
    for epoca in range(epocas):
        # Variable donde guardamos el número de errores
        miss_n=0
        for index, x in enumerate(X):
            #inserta el bias
            x = np.insert(x, 0, 1).reshape(-1,1)
            # Intenta clasificar el caso x con los parámetros w
            y_hat = step_func(np.dot(x.T, W))
            #si hay error en la clasificación ajusta los pesos W
            if (y_hat - y[index]) != 0:
                #aplica la regla de actualización lrate*(Actual-predicho)*x
                W += lrate*((y[index] - y_hat)*x)
                miss_n += 1
        miss_list.append(miss_n)
    return W,miss_list
#*****
#Función Step de Heaviside
def step_func(z):
    return 1.0 if(z>=0) else 0.0
#*****
```

Fig.11. Código implementado para el entrenamiento de un Perceptron. (Adaptado a partir del artículo de Suraj Verma, en el blog Towards data Science)

Para examinar el funcionamiento del código anterior se utiliza el código que mostramos en la Fig. 12.

Tema 5. Redes neuronales y deep learning

Es importante que tengamos en cuenta que el código que mostramos aquí es solo ha sido escrito con el propósito de explicar las ideas detrás del entrenamiento de una neurona y por ningún medio pretende ser, ni perfecto ni usable a nivel de producción.

```
#####
def plot_decision_boundary(X, W):
    # en X estan todos los casos
    # W contiene todos los pesos
    # La superficie de decisión es una linea y=mx+c
    # De forma que mx+c = W0.X0 + W1.X1 + W2.X2
    # Vamos a resolver la ecuación
    x1 = [min(X[:,0]), max(X[:,0])]
    m = -W[1]/W[2]
    c = -W[0]/W[2]
    x2 = m*x1 + c

    # Plotting
    fig = plt.figure(figsize=(8,8))
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "b^")
    plt.xlabel("Rasgo 1")
    plt.ylabel("Rasgo 2")
    plt.title('Algoritmo del Perceptron')
    plt.plot(x1, x2, 'y-')
    plt.show()

#####
def Perceptron_predict(X,W):
    n=len(X)
    target=[]
    for index, x in enumerate(X):
        x = np.insert(x, 0, 1).reshape(-1,1)
        target.append(step_func(np.dot(x.T, W)))
    return target

#####
# Puntos que forman el conjunto de entrenamiento
X=np.array([[0.3,0.84],[0.45,0.9],[0.48,0.82],[0.53,0.73],
            [0.54,0.32],[0.76,0.35],[0.75,0.49],[0.69,0.23]])
Y =[1,1,1,1,0,0,0,0]
# Puntos que sirven de conjunto de prueba.
X_test=np.array([[0.07,0.65],[0.06,0.75],[0.14,0.9],[0.34,0.79],
                 [0.38,0.06],[0.56,0.05],[0.69,0.16],[0.53,0.17]])
Y_test =[1,1,1,1,0,0,0,0]
W, miss_l = perceptron(X, Y, 0.6, 10)
X_C=np.concatenate((X,X_test), axis=0)
plot_decision_boundary(X_C, W)
pred_test = Perceptron_predict(X_test,W)
pred_train = Perceptron_predict(X,W)
acc_train= accuracy_score(Y_test,pred_train)
acc_test= accuracy_score(Y_test,pred_test)
print("Acc sobre el conjunto de prueba:",acc_train)
print("Acc sobre el conjunto de prueba:",acc_test)
```

Fig.12. Muestra el código para graficar el hiperplano resultante del aprendizaje del Perceptron y el accuracy de este. (Adaptado a partir del artículo de Suraj Verma, en el blog Towards data Science)

La gráfica y el accuracy resultante del entrenamiento de este Perceptrón con los datos mostrados se muestran en la Figura 12.

Tema 5. Redes neuronales y deep learning

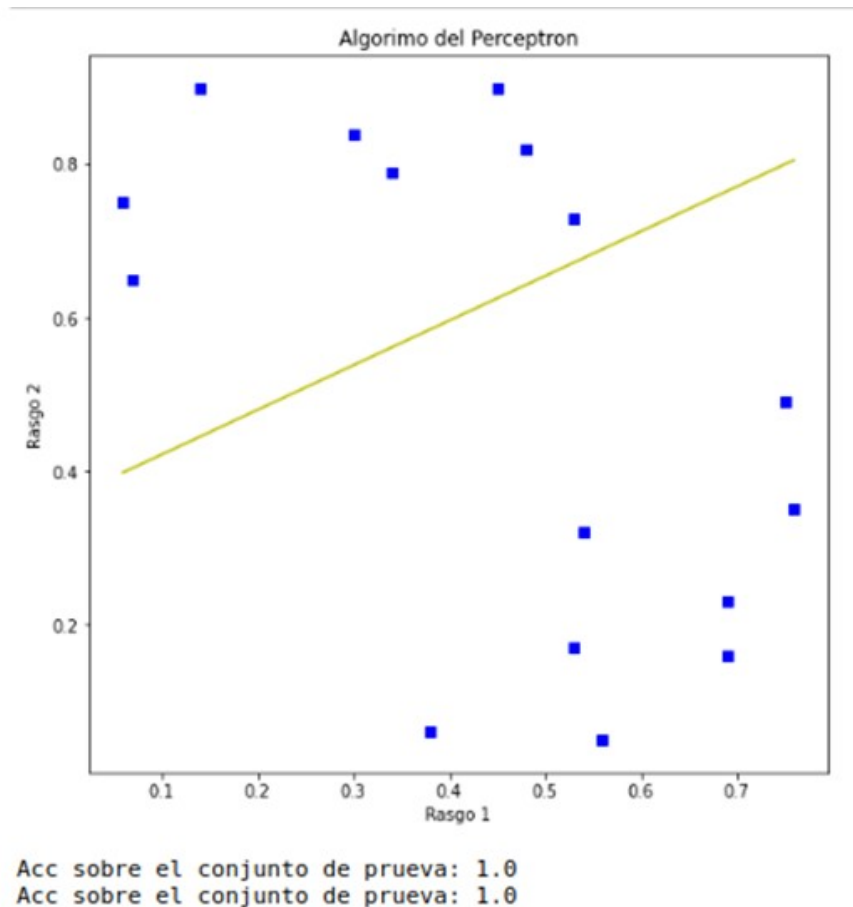


Fig.13. Muestra el hiperplano resultante del aprendizaje del Perceptrón y el accuracy de este.

Como ya hemos comentado, la principal limitación de este modelo es que solo es aplicable a problemas de clasificación binaria donde las clases sean linealmente separables.

Desde su introducción estas ideas se han desarrollado una tras otras resolviendo las limitaciones y tornándose mucho más sofisticadas.

El método que permitió aplicar el modelo de aprendizaje que llamamos Perceptrón a problemas más complejos fue Backpropagation.

Este método permitía entrenar modelos que contenían muchas capas con muchas neuronas de manera relativamente sencilla. A la arquitectura de red que contiene

Tema 5. Redes neuronales y deep learning

varias capas de Perceptrones se les llamó Perceptrón Multicapa y el método que usaba para entrenarse Backpropagation.

5.1.11. El Perceptrón Multicapa y Backpropagation

El Perceptrón Multicapa, como su nombre sugiere, es una red neuronal que se clasifica como Feedforward multicapa. Este tipo de red puede tener cualquier cantidad de capas ocultas con cualquier cantidad de neuronas. En esta red, todas las neuronas son del tipo Perceptron. Normalmente se identifica con las siglas MLP que corresponde a su denominación en inglés: Multilayer Perceptrón.

Estamos analizando una de las redes más populares y aplicadas y su uso data de alrededor de los 80. Esta red y su algoritmo de aprendizaje fueron introducidos en la literatura por Rumelhart en 1986, en un libro titulado *Parallel Distributed Processing*.

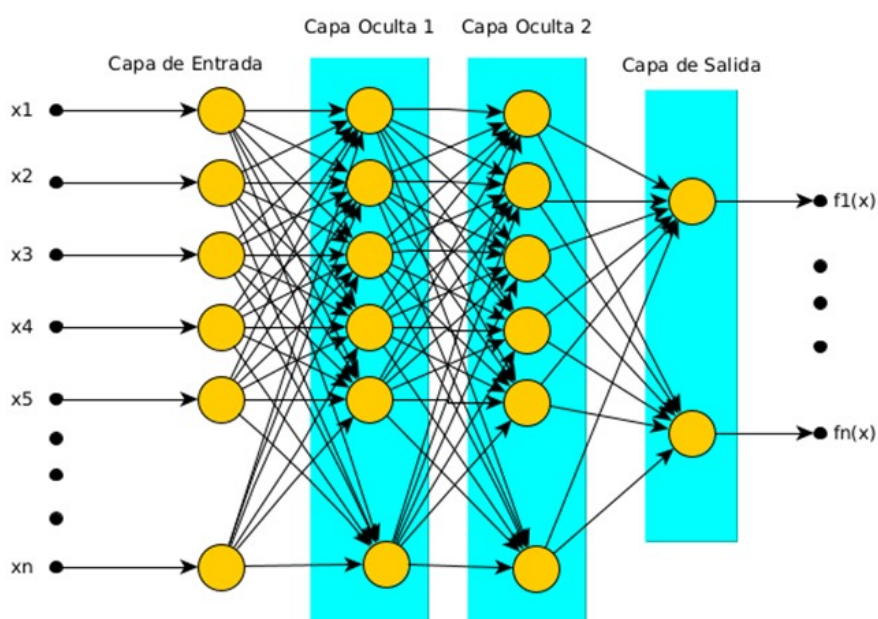


Fig.14. Perceptrón Multicapa.

El Perceptrón Multicapa, puede tener n capas ocultas y cualquier cantidad de neuronas en ellas y también cualquier cantidad de neuronas en la capa de salida,

Tema 5. Redes neuronales y deep learning

solo determinadas por el problema que modelamos. La cantidad de capas y neuronas depende de la complejidad del problema, pero entre más neuronas y capas más complejo se torna su entrenamiento.

El método de entrenamiento del MLP se denomina Backpropagation, que significa aproximadamente retropropagación o propagación hacia atrás del error.

Como ya habíamos comentado anteriormente, la limitación más importante del Perceptrón era su incapacidad de resolver problemas donde las clases no fueran linealmente separables. Con vistas a resolver esto fue necesario sustituir las funciones de activación, como la función escalón de Heaviside, por funciones de activación no lineales, tales como sigmoide o tangente hiperbólica.

Ahora bien, cuando usamos funciones de activación no lineales, cuyo resultado puede tomar cualquier valor entre -1 y 1 o entre 0 y 1, ya no podemos usar la regla Hebb para la actualización de los pesos de un Perceptrón. Recordemos la regla de Hebb.

$$w_{i+1} = w_i + \eta(tr - tp) x_i$$

Donde Δw_i es el peso modificado, w_i es el peso antes de la modificación o actualización, η es el Learning Rate que definía cuán rápido se alcanzaba la convergencia, tr es el target real, es decir la etiqueta asignada en el dataset al caso x , tp , es el target predicho por la neurona y x_i es el caso de entrada.

Es importante que recordemos que esta regla nos daba dos informaciones necesarias para la actualización del peso, nos dice la magnitud del incremento del peso y la dirección de este. Es decir, nos dice si incrementamos o disminuimos el peso. La parte que nos dice la dirección es $(tr - tp)$. Como tp y tr , usando una función como el escalón de Heaviside, solo pueden tomar valor de 0 y 1, $(tr - tp)$ solo podría tomar valores de -1 y 1, especificando cuando aumentar o disminuir los pesos.

Tema 5. Redes neuronales y deep learning

Entonces aflora, de manera natural la pregunta ¿Cómo actualizamos los pesos cuando usamos funciones no lineales? La solución llegó de la mano de los modelos llamados ADALINE/MADALINE y se llama regla Delta.

En estos modelos, a diferencia del Perceptrón, los pesos se ajustan después de calcular una función del error arrojado por la predicción de todos los casos en el conjunto de entrada.

Esta función, es el error medio cuadrático, expresado matemáticamente como:

$$E = \frac{1}{2} \sum_{p=1}^m (d^p - y^p)^2$$

Donde p es el número de casos en el conjunto de casos, d^p es el valor real de la etiqueta de clase del caso y y^p es la clase predicha por la red.

En estos modelos, ADALINE/MADALINE, se procesan todos los casos, se calcula el Error y luego se ajustan los pesos. La idea es encontrar un conjunto de pesos W que haga que el valor de esta función sea mínimo. Con este fin se usa la regla delta.

$$\Delta W = -\eta \nabla E(W) = -\eta \frac{\partial E}{\partial W}$$

Donde $\frac{\partial E}{\partial W}$ es la derivada de la función de error con respecto a los pesos. Por tanto la manera en que actualizamos ahora los pesos es como sigue:

$$w_{t+1} = w_t - \eta \frac{\partial E}{\partial W}$$

A esta regla se le conoce también como Gradiente Descendiente.

Tema 5. Redes neuronales y deep learning

$$S(x) = \frac{1}{1 + e^{-x}}$$

Y daremos, sin más pruebas, que la derivada de esta es:

$$\frac{\partial S(x)}{\partial x} = S(x)(1 - S(x))$$

También daremos sin prueba alguna, que:

$$\frac{\partial E}{\partial w} = \frac{\partial}{\partial w} \left(\frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right) = (y_i - \hat{y}_i)$$

De esta forma ahora podemos formular de manera clara la regla que usaremos para actualizar los pesos cada neurona en un MLP.

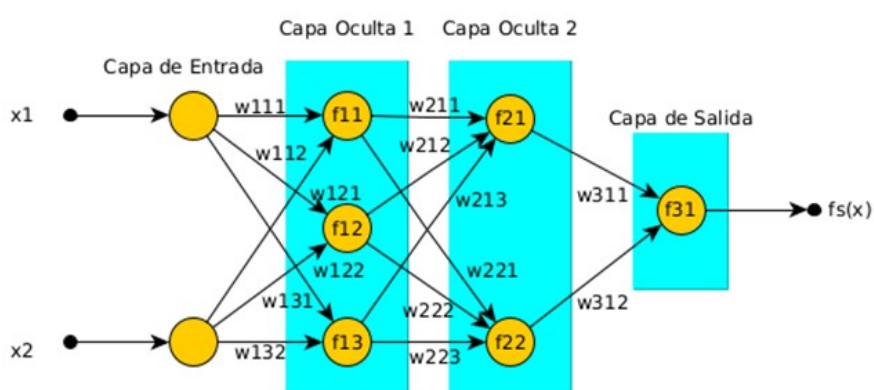


Fig.16. MLP con tres capas y nombres de los pesos.

La notación que hemos usado para denominar los pesos y las neuronas es el siguiente, las primera neurona de la capa 1 es la f11 y la segunda la f12 y así sucesivamente, la primera neurona de la capa 2, es la f21 y así. Por tanto el subíndice, xxx, indica, capa, neurona, entrada. Teniendo en cuenta esto, el peso w223, significa el peso de la entrada 3, de la neurona 2 de la capa 2.

Por simplicidad, no explicaremos la fase forward de backpropagation, pues en esta solo se calculan las salidas de la misma manera que se hace en el Perceptrón

Tema 5. Redes neuronales y deep learning

simple.

Veremos cómo se imputa el error basado en la contribución de cada neurona al error total, que denotaremos δ y la manera en que se actualizan los pesos.

Para esto, comencemos por calcular el error total δ :

$$\delta = (y_i - \hat{y}_i)$$

Este es el error a la salida de la red y donde y_i es la etiqueta asignada al caso x en el dataset, mientras que $\hat{y}_i$ es el estimado que hace nuestra red.

Ahora bien, cada neurona aporta un determinado error y por tanto debemos calcular este, así:

El error que aporta f21 es:

$$\delta_{21} = w_{311} \delta$$

El error en f22 es:

$$\delta_{22} = w_{312} \delta$$

El error en las neuronas de la capa 1, se calculan ahora de la siguiente forma:

El error en las neuronas de la capa 1, son:

$$\delta_{11} = w_{211} \delta_{21} + w_{221} \delta_{22}$$

$$\delta_{12} = w_{212} \delta_{21} + w_{222} \delta_{22}$$

$$\delta_{13} = w_{213} \delta_{21} + w_{223} \delta_{22}$$

Ya que se calcularon los aportes de cada neurona al error total, se puede calcular el ajuste que lleva cada uno de los pesos comenzando desde la neurona de salida. De esta forma:

Tema 5. Redes neuronales y deep learning

$$w_{311} = w_{311} + \eta \delta \frac{df_{31}}{dx} y_{21}$$

$$w_{312} = w_{312} + \eta \delta \frac{df_{31}}{di} y_{22}$$

Donde η es el Learning Rate, δ es el error total a la salida de la red neuronal, $\frac{df_{31}}{dx}$ es la derivada de la función de activación de la neurona f_{31} , que en este caso es una función sigmoide, i representa el argumento de la función. Mientras que y_{21} y y_{22} corresponden a las salidas de las neuronas f_{21} y f_{22} .

De esta misma forma podemos ahora calcular la actualización de los pesos en la capa 2:

$$w_{211} = w_{211} + \eta \delta_{21} \frac{df_{21}}{di} y_{11}$$

$$w_{212} = w_{212} + \eta \delta_{21} \frac{df_{21}}{di} y_{12}$$

$$w_{213} = w_{213} + \eta \delta_{21} \frac{df_{21}}{di} y_{13}$$

$$w_{221} = w_{221} + \eta \delta_{22} \frac{df_{22}}{di} y_{11}$$

$$w_{222} = w_{222} + \eta \delta_{22} \frac{df_{22}}{di} y_{12}$$

$$w_{223} = w_{223} + \eta \delta_{22} \frac{df_{22}}{di} y_{13}$$

Ya hemos calculado los pesos correspondientes a la capa 2, calculemos ahora los nuevos pesos de la capa 1.

$$w_{111} = w_{111} + \eta \delta_{11} \frac{df_{11}}{di} x_1$$

$$w_{112} = w_{112} + \eta \delta_{11} \frac{df_{11}}{di} x_2$$

Tema 5. Redes neuronales y deep learning

$$w_{121} = w_{121} + \eta \delta_{12} \frac{df_{12}}{di} x_1$$

$$w_{122} = w_{122} + \eta \delta_{12} \frac{df_{12}}{di} x_2$$

$$w_{131} = w_{131} + \eta \delta_{13} \frac{df_{13}}{di} x_1$$

$$w_{132} = w_{132} + \eta \delta_{13} \frac{df_{13}}{di} x_2$$

Con esto hemos ajustado todos los pesos a partir de la aplicación de un caso de entrada y el proceso se repite de nuevo hasta que se alcanza la condición de parada. Más que los detalles de la implementación de este método el objetivo es que tengan una idea de cómo se ejecuta el aprendizaje en este tipo de modelo.

De manera general a los métodos que utilizan la técnica de utilizar el gradiente para la actualización de los pesos se les denomina métodos de gradiente descendente.

Con las ideas que hemos explicado hasta el momento estamos en condiciones de ver cómo usamos una red de tipo MLP sobre los datasets que hemos usado en las lecciones anteriores. Mostraremos como se hace, usando la implementación de un MLP en scikit-learn y luego lo haremos usando Keras.

MLP en scikit-learn

En el módulo `sklearn.neural_network`, se implementan dos variantes de MLP, uno para clasificación y el otro para regresión.

`neural_network.MLPClassifier` y `neural_network.MLPRegressor`, en seguida veremos un ejemplo de cómo usar cada uno de ellos en la clasificación de tipos de flores usando Iris Dataset y luego veremos cómo usar `MLPRegressor` para predecir una variable continua.

Tema 5. Redes neuronales y deep learning

```
#Clasificación del Iris Dataset con MLP
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
#*****
iris = datasets.load_iris(as_frame=True)

#*****
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size = 0.25, random_state = 0)
#*****
scaler = MinMaxScaler()
X_train = scaler.fit_transform(x_train)
X_test = scaler.transform(x_test)
#*****
mlp = MLPClassifier(solver='lbfgs', hidden_layer_sizes=(6,3), max_iter=500)
mlp.fit(X_train, y_train)
print('Accuracy de MLP training set: {:.2f}'
      .format(mlp.score(X_train, y_train)))
print('Accuracy de MLP test set: {:.2f}'
      .format(mlp.score(X_test, y_test)))
y_pred=mlp.predict(X_test)
print(classification_report(y_test,y_pred))
```

Fig.16. MLP usando scikit-learn.

Para probar la calidad de la clasificación de este modelo de aprendizaje automático supervisado, hemos usado el Iris dataset, que como recordarán contenía 150 casos, repartidos en tres clases con 50 casos en cada una. Creamos una red de 4 capas, dos de ellas ocultas de 6 y 3 neuronas cada una. Para el entrenamiento se usaron 500 épocas o iteraciones.

Los resultados son los mostrados a continuación:

```
Accuracy de MLP training set: 1.00
Accuracy de MLP test set: 0.97
      precision    recall  f1-score   support

0         1.00      1.00      1.00        13
1         1.00      0.94      0.97        16
2         0.90      1.00      0.95         9

 accuracy
macro avg      0.97      0.98      0.97        38
weighted avg    0.98      0.97      0.97        38
```

Fig.17. Resultados del MLP usando scikit-learn sobre el Iris Dataset.

En la siguiente imagen, mostramos la aplicación de un MLP como regresor, para

Tema 5. Redes neuronales y deep learning

predecir las ventas globales de videojuegos a partir del dataset video games sales.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.preprocessing import MinMaxScaler
#*****
data = pd.read_csv("vgsales.csv")
data = data.dropna()
x = data[["Rank", "NA_Sales", "EU_Sales", "JP_Sales", "Other_Sales"]]
y = data["Global_Sales"]
#*****
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size=0.2, random_state=42)
#*****
scaler = MinMaxScaler()
X_train = scaler.fit_transform(xtrain)
X_test = scaler.transform(xtest)
#*****
LR_model = LinearRegression()
LR_model.fit(X_train, ytrain)
LR_predictions = LR_model.predict(X_test)
print("Error medio absoluto LR:", mean_absolute_error(ytest, LR_predictions))
#*****
MLPR_model = MLPRegressor(hidden_layer_sizes=(15, 8), activation='identity', solver='lbfgs', max_iter=500)
MLPR_model.fit(X_train, ytrain)
MLPR_predictions = MLPR_model.predict(X_test)
print("Error medio absoluto MLP:", mean_absolute_error(ytest, MLPR_predictions))

Error medio absoluto LR: 0.003106141878474775
Error medio absoluto MLP: 0.0030886339979280734
```

Fig.18. Regresión con MLP usando scikit-learn sobre el Videogame Sales Dataset.

Para evaluar la calidad de la predicción se usa el Error medio absoluto, pero se pueden usar varias de las disponibles. Aquí es importante destacar que la función de activación es importante a la hora de usar regresión.

Veremos a continuación los parámetros para construir un MLP, tanto en clasificación como en regresión los parámetros son casi idénticos, veamos.

Parámetros para MLPClassifier y MLPRegressor:

hidden_layer_sizes: Con este parámetro se fija la cantidad de neuronas y la cantidad de capas ocultas de la red. Por ejemplo:

`hidden_layer_sizes=(13,4,4)`

Usar este parámetro con estos valores le dice al constructor de la clase que cree un MLP con tres capas ocultas, la primera con 13, la segunda y la tercera con 4. El valor por defecto de este valor es (100,).

Tema 5. Redes neuronales y deep learning

Es idéntico para la versión de clasificación como de regresión.

Activation: Este parámetro determina el tipo de función de activación de las neuronas de las capas ocultas. Pueden tomar los siguientes valores:

- ▶ Identity: Se denomina función identidad y su expresión matemática es: $f(x) = x$.
- ▶ Logistic: Es el nombre que se le da a la función sigmoide que hemos tratado en las explicaciones anteriores y su expresión matemática es: $f(x) = 1 / (1 + \exp(-x))$.
- ▶ Tanh: Es la tangente hiperbólica y su expresión: $f(x) = \tanh(x)$.
- ▶ Relu: Es la función de activación decide entre x o cero y su expresión es: $f(x) = \max(0, x)$

Solver: Este parámetro determina el tipo de algoritmo usado para el aprendizaje del modelo, es decir, los sustitutos de backpropagation.

Los solvers utilizados son los siguientes:

- ▶ Lbfgs: Es un algoritmo de optimización de la familia de los métodos quasi-Newton.
- ▶ sgd: Método del Gradiente Descendiente Estocástico.
- ▶ adam: Este es un método basado en el gradiente descendiente propuesto por Kingma, Diederik, y Jimmy Ba.

El valor por defecto de este parámetro es 'adam' que trabaja muy bien sobre datasets relativamente grandes, sin embargo, para datasets pequeños lbfgs converge más rápido y tiene mejores resultados.

alpha: Este parámetro la magnitud de la norma L2 que se usa como penalización. El valor por defecto de este parámetro es 0.0001. Pero lo correcto sería encontrarlo por medio de Validación cruzada.

Tema 5. Redes neuronales y deep learning

batch_size: Este parámetro define el tamaño de los lotes de ejemplos que se presentan antes de ajustar los pesos en los algoritmos de optimización estocásticos. Cuando se usa lbfgs, este valor se ignora, pues este no usa batch.

learning_rate: Taza de aprendizaje, determina a la velocidad que converge el aprendizaje cuando se usa el método de aprendizaje gradiente descendiente estocástico, los valores que puede aceptar son los siguientes:

- ▶ **constant:** Usa un Learning Rate constante decidido por 'learning_rate_init'.
- ▶ **Invscaling:** Decrementa el Learning Rate en cada paso según $\text{effective_learning_rate} = \text{learning_rate_init} / \text{pow}(t, \text{power_t})$.
- ▶ **adaptive:** Mantiene el Learning Rate constante e igual a learning_rate_init mientras la función de pérdida sobre el conjunto de entrenamiento decrezca. Cuando dos épocas consecutivas fallan en disminuir las pérdidas de entrenamiento y `early_stopping=on` entonces el Learning Rate es dividido entre 5.

learning_rate_init: Este es el valor para el Learning Rate y su efecto depende del valor escogido para el parámetro learning_rate. Su valor por defecto es 0.001. Solo es usado cuando los algoritmos de aprendizaje son 'sgd' o 'adam'.

Como nota final de este tema, es necesario subrayar la susceptibilidad tanto de MLP como clasificador como regresor a la normalización de los casos del conjunto de datos. Siempre normalizar es necesario y hacerlo tanto en el conjunto de entrenamiento como el de prueba.

Estos son los parámetros más importantes que determinan el funcionamiento de un MLP construido con scikit-learn.

Redes Neuronales Recurrentes

Como hemos visto en las secciones anteriores, que hemos dedicado a las redes Feedforward, las conexiones entre las neuronas, siempre se realizan entre las

Tema 5. Redes neuronales y deep learning

salidas de estas, a las entradas de las neuronas en la siguiente capa. Por tanto, la información siempre viaja desde las entradas hasta la salida de la red. Por el contrario, en las redes neuronales recurrentes, existen lazos de realimentación. En este tipo de red, las salidas de las neuronas de una capa pueden conectarse a las entradas de otras ubicadas en la capa anterior o a su propia entrada.

Este tipo de red, fue introducida en la literatura en 1982, por John Hopfield y a las redes que propone se les denominan Redes de Hopfield.

Las redes recurrentes se han ido desarrollando en función de resolver problemas prácticos relacionados con el reconocimiento de voz y la predicción de series temporales, entre otros. Los impresionantes avances que se han producido en áreas como el reconocimiento del habla, la síntesis de texto a voz, traducción automática, y otras, están asociados al uso de Redes Neuronales Recurrentes.

La propiedad principal de este tipo de red y que las distingue de las Feedforward, son las características del tipo de dato con que opera. Mientras que las redes del tipo Feedforward, presuponen que los datos de entrada son idéntica e independientemente distribuidos, las redes neuronales recurrentes suponen que los datos de entrada son una secuencia donde las componentes tienen dependencias temporales. Esta dependencia temporal, significa por ejemplo, en una conversación, las palabras deben aparecer en un orden y no basta con tomarlas una a una, es necesario mantener dicho orden para conservar el sentido.

En esta sección iniciaremos haciendo una introducción a las ideas básicas de las redes de Hopfield, Jordan, Elman y las más recientes LSTM y GRU. Sin embargo, antes de pasar a revisar cada una de estas arquitecturas, es necesario que nos detengamos un instante a revisar el concepto de secuencia sobre las que operan las redes neuronales recurrentes.

Desde el inicio de este curso, hemos presupuesto que los conjuntos de entrenamientos sobre los que operan nuestros métodos de aprendizaje tienen una

Tema 5. Redes neuronales y deep learning

forma tabular y que cada fila representa un caso que normalmente le denotamos como x_i y que cada uno de ellos está representado por p rasgos. En este tipo de conjunto de datos, se presupone que cada caso x_i , es independiente de otro x_j del mismo conjunto. Esto quiere decir, en palabras llanas, que el conjunto de datos es estático y la red neuronal que opera con ellos realiza un mapeo estático entre los x_i del conjunto a y_i en las salidas.

Por el contrario, en una secuencia, el caso actual, puede que esté relacionado con el caso anterior e incluso, con todos los anteriores.

Una secuencia S , puede representarse como:

$$S = \{x(t), x(t-1), x(t-2), \dots, x(t-n)\}$$

Donde x es un vector con componentes reales.

Con el fin de aclarar el concepto, veamos algunos ejemplos de secuencias: Es una secuencia la frase «Redes Neuronales Recurrentes», ya que cada una de las consonantes y vocales con siguen a la primera, dependen de ella respecto al sentido de la frase y eso da un sentido de temporalidad. Es una secuencia o serie de tiempo, por ejemplo los valores de la demanda eléctrica de la ciudad de Madrid medida hora a hora. En este tipo de serie de tiempo, el valor de la demanda depende de la demanda anterior. Aunque los valores de una serie de demanda eléctrica dependen de muchísimos factores, puede considerarse que son altamente dependientes de la hora del día y del día de la semana.

Es importante que destaquemos, que cuando nos hacemos referencia al subíndice t en la secuencia, no nos estamos refiriendo de forma única a secuencias basadas en tiempo, como el caso de series de tiempo, sino que también lo hacemos a secuencias no temporales como secuencias de ADN. En estos casos, cuando hacemos referencia a t , lo hacemos pensando en el orden de presentación del caso a la red.

Tema 5. Redes neuronales y deep learning

Por tanto, una secuencia de entrada, consiste de n puntos de datos $x(t)$, que arriban a la red en una secuencia discreta de pasos temporales indexados por t .

Las redes neuronales recurrentes son capaces de manejar este tipo de datos de manera natural, aunque eso no significa que las redes neuronales Feedforward no puedan, pero sería necesario adaptar los casos de entrada. Es decir, habría que convertir, la secuencia temporal a estática.

Redes de Hopfield

Como mencionamos en algún momento, las redes de Hopfield, fueron las primeras de su linaje y abrió el camino a los ulteriores desarrollos de las Redes Neuronales Recurrentes.

Básicamente, el modelo de red de Hopfield, implementa un tipo de memoria asociativa, cuyas características básicas iremos definiendo.

Una red de Hopfield, no es más que un conjunto de neuronas organizadas de tal manera que sus salidas están conectadas con las entradas de todas las demás. Esto hace que siempre hayan $n(n-1)$ pesos o conexiones entre las n neuronas.

Una de las características distintivas de este tipo de red es que los pesos W_{ij} que conectan dos neuronas i y j , son simétricos, tal que $w_{ij} = w_{ji}$; pero $W_{ii} = 0$.

Tema 5. Redes neuronales y deep learning

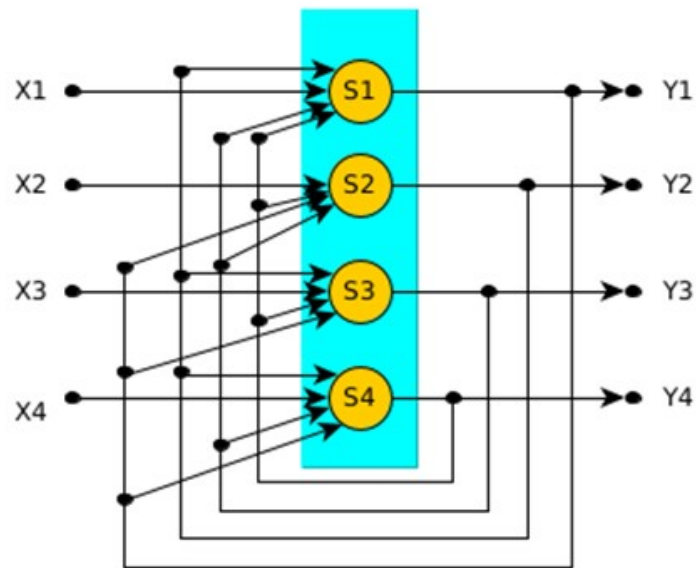


Fig.19. Red Neuronal Recurrente de Hopfield.

El hecho de que $W_{ii} = 0$, indica que no hay lazos de realimentación entre la salida y la entrada de una misma neurona.

Cada neurona en esta red, está asociada a un estado S_i . Y se asume que cada uno de dichos estados es binario. (0,1) aunque pueden usarse $\{-1,1\}$.

El valor del estado depende de si la entrada supera determinado umbral:

$$S_i = 1 \text{ si } \sum_j W_{ij} S_j > U_i$$

$$S_i = -1 \text{ si } \sum_j W_{ij} S_j < U_i$$

Por tanto, un estado S_i se calcula de la siguiente forma:

$$S_i = \text{sign} \left(\sum_j W_{ij} S_j - U_i \right)$$

El umbral se toma como el valor 0.

Tema 5. Redes neuronales y deep learning

Cada estado, corresponde a una neurona y cada una de ellas corresponde a una dimensión del dataset.

Los pesos de cada una de las conexiones, son los parámetros de una red de Hopfield y el aprendizaje tiene lugar por medio de sus ajustes.

Los pesos W_{ij} en una red de Hopfield se actualizan usando la regla de aprendizaje de Hebb.

De manera general, esta regla establece que el aprendizaje tiene lugar por el reforzamiento o debilitamiento de las sinapsis de las conexiones entre las neuronas y matemáticamente se expresa como sigue:

$$W_{ij} = W_{ij} + \sum_{k=1}^L S_k(i) S_k(j)$$

Donde $S_k(i)$ y $S_k(j)$ representan el estado de la neurona i y j en el instante de tiempo k .

Cuando se revisan los pesos de una red como esta, los valores más positivos denotan una alta correlación positiva entre esas neuronas o estados, mientras que pesos con grandes valores negativos son indicativos de una alta correlación negativa.

A diferencia de las redes como el perceptrón multicapa, que minimizan una función de pérdida, las redes de Hopfield, minimizan una función de energía. En la fase de entrenamiento, se aprenden los pesos, de tal forma que se minimice la energía cuando los estados de la red son fijados a los valores de los atributos binarios de los puntos de entrenamiento.

La Energía E , de una red de Hopfield puede expresarse de la siguiente forma:

$$E = - \sum b_i S_i - \sum W_{ij} S_i S_j$$

Tema 5. Redes neuronales y deep learning

En esencia, esta red se entrena para memorizar el dataset. Con el fin de poner un ejemplo de cómo aprende una red de Hopfield, es preciso que recordemos que las redes neuronales tienen dos fases bien definidas, una llamada fase de entrenamiento y la otra llamada fase de generalización.

Para poner este ejemplo, debemos suponer que la matriz de pesos que contiene los pesos iniciales se ha rellenado con ceros. Es decir, en el instante $t=0$, todos los pesos de las conexiones entre neuronas son cero. Para este ejemplo, también tomaremos como cantidad de neuronas 4 y usaremos la imagen de la figura 19, como referencia.

Para el ejemplo, usaremos el siguiente dataset:

X1	1	1	1	-1
X2	-1	1	1	1
X3	1	1	-1	-1
X4	-1	-1	1	1

Si presentamos a la red el primer caso, veamos cómo se calculan los pesos iniciales si ya sabemos que la matriz de pesos inicialmente es cero.

$$x_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} W_{t=0} = \begin{bmatrix} x & 0 & 0 & 0 \\ 0 & x & 0 & 0 \\ 0 & 0 & x & 0 \\ 0 & 0 & 0 & x \end{bmatrix}$$

Como ya sabemos que los pesos se obtienen por medio de la regla de aprendizaje de Hebb, entonces:

$$W_{ij} = W_{ij} + \sum_{k=1}^L S_k(i) S_k(j)$$

Si, tomamos $k=1$ como el instante de tiempo en que se aplica a la entrada de la red el caso x_1 y vamos a calcular el peso correspondiente a las neuronas $i=1$ y $j=2$,

Tema 5. Redes neuronales y deep learning

además sabemos que el estado de la neurona i se calcula de la siguiente forma:

$$S_i = \text{sign} \left(\sum_j W_{ij} S_j - U_i \right)$$

Entonces, para la neurona 1:

$$S_i = \text{sign}((0 + 0 + 0 + 1) * 1 - 0) = 1$$

El estado de la neurona j es igual a 1, también, excepto la última que es -1, pues en el estado inicial, los estados de cada una de las neuronas de hace igual a la entrada.

Esto hace que el peso de la neurona 1 a 2:

$$W_{t=1} = \begin{bmatrix} x & 1 & 0 & 0 \\ 1 & x & 0 & 0 \\ 0 & 0 & x & 0 \\ 0 & 0 & 0 & x \end{bmatrix}$$

Esto se debe a que el peso, ajustado por medio de la regla de Hebb, para la sinapsis entre la neurona 1 y la 2, es:

$$W_{1,2} = S_1 S_2 = 1 * 1 = 1$$

De esta forma la matriz de pesos, después de aplicar el patrón x_1 , es:

$$x_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} W_{t=1} = \begin{bmatrix} x & 1 & 1 & -1 \\ 1 & x & 1 & -1 \\ 1 & 1 & x & -1 \\ -1 & -1 & -1 & x \end{bmatrix}$$

Sería sumamente interesante continuar el proceso de cálculo de la matriz de peso de esta red; pero desde el punto de vista de una introducción a las redes neuronales y por el espacio o longitud de las lecciones, preferimos que este se continúe como actividad independiente indicada en documento aparte.

Es importante que subrayemos que el ejemplo que hemos iniciado corresponde al

Tema 5. Redes neuronales y deep learning

uso de una red de Hopfield como autoasociador, es decir, una especie de memoria direccionable por contenido. Para aclarar este concepto, comparémoslo con una memoria de ordenador. En una memoria de ordenador, se obtiene un dato, después de un tiempo t , llamado latencia, de aplicar una dirección al bus de direcciones y aplicar la señal de control de lectura. En una memoria direccionable por contenido, como la de Hopfield, si suponemos que tenemos un registro compuesto por los datos de un cliente que incluye, nombre, dirección, teléfono, etc. Este se puede recuperar, de forma completa aunque solo apliquemos, a la entrada, una parte de la información del registro, por ejemplo, solo el nombre.

También, necesitamos precisar, que lo que hemos introducido en esta sección es parte de lo que se llaman modelos de Hopfield discretos, pues en otro artículo en 1984, Hopfield introdujo una variante en tiempo continuo.

Redes de Jordan

La propiedad distintiva de este tipo de red, es que las neuronas de la capa de salida se conectan a la entrada de la red por medio de neuronas especiales llamadas unidades de contexto. Es importante entender, que las neuronas que forman esta red son perceptrones.

Tema 5. Redes neuronales y deep learning

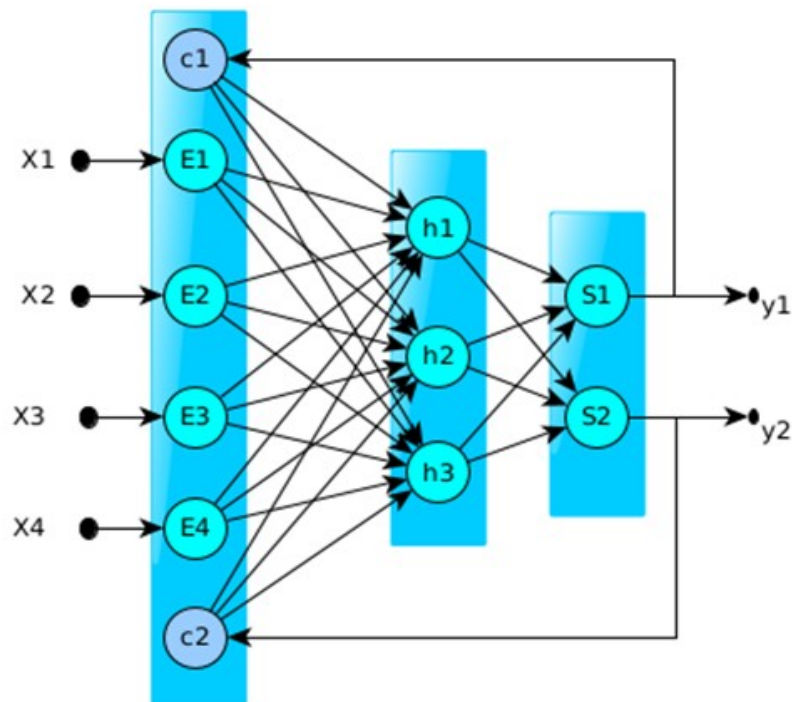


Fig.20. Red Neuronal Recurrente de Jordan. (Aunque no se muestran, las unidades de contexto tienen conexiones recurrentes en ellos).

En la imagen de la Fig. 19 mostramos un gráfico de una red neuronal recurrente de Jordan. Esta red tiene 4 neuronas en la capa de entrada, tres en la capa oculta y dos en la capa de salida. Ahora bien, la diferencia respecto a un Perceptrón Multicapa es que se le han añadido dos neuronas, llamadas unidades de contexto, que representamos con el nombre c1 y c2, que implementan un lazo de realimentación desde la salida y1 y y2 a las entradas. Estas unidades de contexto se han representado con un color diferente al resto, con el fin diferenciarlas.

En este tipo de redes, antes de poder aplicar un caso a la entrada se inicializan los pesos de todas las entradas, incluyendo las de las neuronas de estado o unidades de contexto. Luego, se aplica un caso a la entrada y este se propaga de la manera habitual en un MLP hasta alcanzar la salida. Una vez que se alcanza la salida, los valores de y1 y y2 se almacenan en las neuronas de estado que se usarán como

Tema 5. Redes neuronales y deep learning

entradas a las neuronas de la capa oculta para el cálculo de la activación correspondiente al proximo caso presentado.

Las fórmulas que caracterizan a esta red son las siguientes:

$$h_t = \sigma(W_h x_t + U_h Y_{(t-1)} + b_h)$$

$$Y_t = \sigma(W_y h_t + b)$$

Donde h_t es la salida de las neuronas de la capa oculta, mientras que Y_t es la salida de las neuronas de la capa de salida. $\sigma()$ representa la función de activación. $W_h x_t$ representa el net de la parte correspondiente a las entradas de las neuronas de la capa oculta que no tienen que ver con las unidades de contexto y $U_h Y_{(t-1)}$ es el net correspondiente a las unidades de contexto.

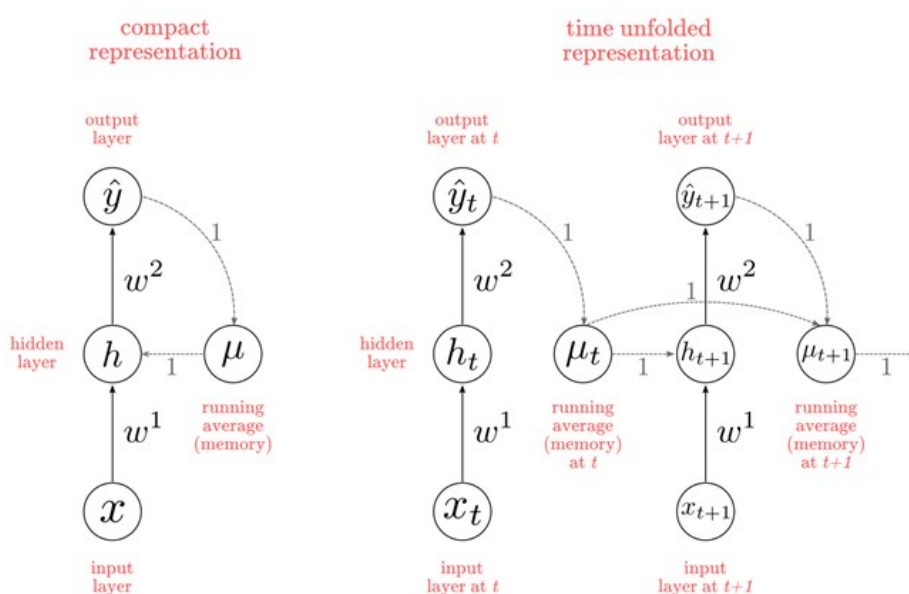


Fig.21. Representación compacta y desplegada en el tiempo de una Red Neuronal Recurrente de Jordan.

Hemos incluido la imagen correspondiente a la figura 20, con el objetivo de que se familiaricen con este tipo de representación que es muy común en la literatura de redes neuronales recurrentes.

Tema 5. Redes neuronales y deep learning

En la parte izquierda se representa JRNN (Jordan Recurrent Neural Network) en su forma compacta y la derecha se despliega en el tiempo.

Redes de Elman

En las redes de este tipo, la salida de la capa oculta es la que se conecta a la entrada por medio de neuronas llamadas unidades de contexto. Al igual que las redes de Jordan, las neuronas que forman esta red son perceptrones. Los pesos de las unidades de contexto, se fijan a 1.

Aunque las redes de Hopfield, son consideradas como redes recurrentes, fueron las redes de Elman las primeras redes neuronales recurrentes exitosas, entrenadas con backpropagation. Estas, se introdujeron en la literatura, en 1990 por Jeffrey Elman, por ese entonces Científico cognitivo en la UC de San Diego.

Para desarrollar las redes de Elman, el autor, se basó en las redes desarrolladas por Jordan en 1986, sin embargo, en estas, son las salidas de la capa oculta la que se realimentan a la entrada de ella por medio de neuronas especiales llamadas unidades de contexto. Los pesos de las entradas correspondientes a estas son fijados a 1. Otro de los elementos de este tipo de red, es que usan función de activación purelin para la capa de salida y tansig, para las neuronas de las capas ocultas. Esta decisión está basada en el hecho de que una red con esas funciones de activación y dos capas puede aproximar cualquier función con una aurracy arbitraria.

Las funciones que caracterizan esta red son las siguientes:

$$h_t = \sigma(W_h x_t + U_h h_{(t-1)} + b_h)$$

$$Y_t = \sigma(W_y h_t + b)$$

Tema 5. Redes neuronales y deep learning

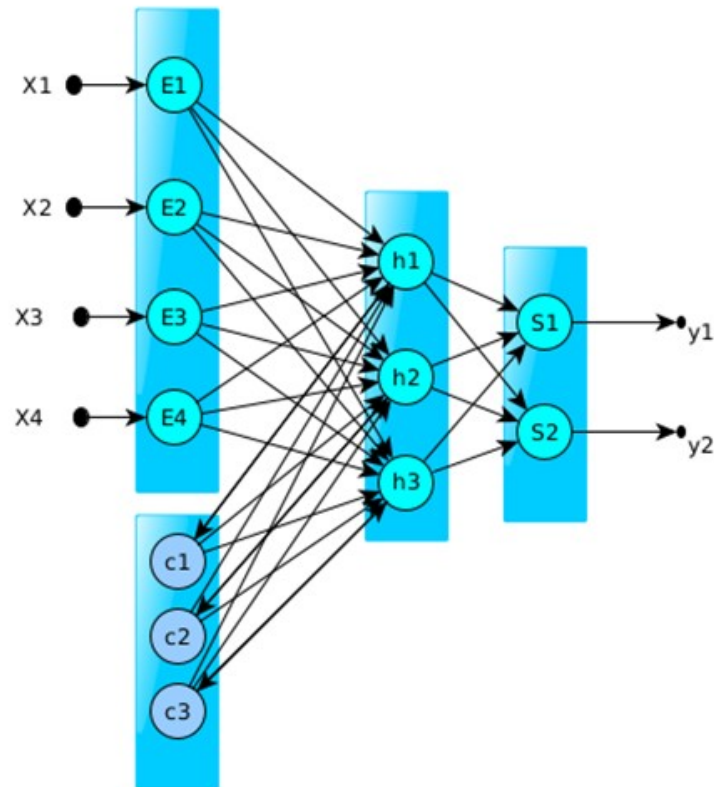
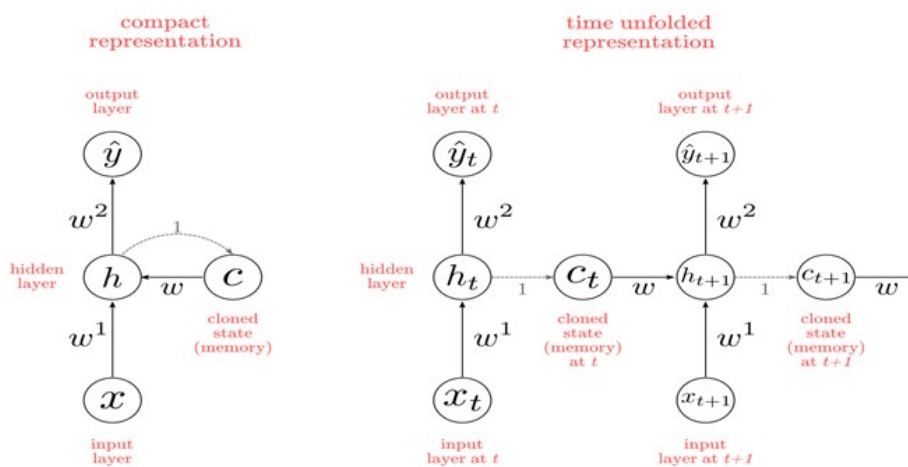


Fig.22. Red Neuronal Recurrente de Elman.

De la misma forma que hicimos con la red de Jordan, mostramos en una imagen la forma compacta de representar una red de Elman y la forma desplegada en el tiempo.



Tema 5. Redes neuronales y deep learning

Fig.23. Representación compacta y desplegada en el tiempo de una red recurrente de Elman.

De las redes neuronales simples, la red de Elman es la más común y de hecho ha servido de base a los desarrollos posteriores que ha tenido el campo de las Redes Neuronales en esa dirección.

Con el fin de ejemplificar el uso de estas redes en la predicción de series temporales, usaremos la implementación de una red de Elman, usando el módulo `neurolab` de `python`.

Este módulo no está instalado de forma nativa en las instalaciones de Anaconda y por tanto es necesario instalarlo, para hacerlo, una de las vías sería ejecutando el siguiente comando:

```
pip install neurolab
```

Este comando puedes ejecutarlo desde una de las celdas de código Jupyter Notebook o Google Colab.

Para nuestro ejemplo usaremos como dataset, la serie de demanda eléctrica de la provincia de Granma, medida hora a hora. Este dataset contiene más de 16 mil mediciones, como las mediciones están hechas hora a hora, solo usaremos dos días para nuestro ejemplo, es decir 48 mediciones.

Tema 5. Redes neuronales y deep learning

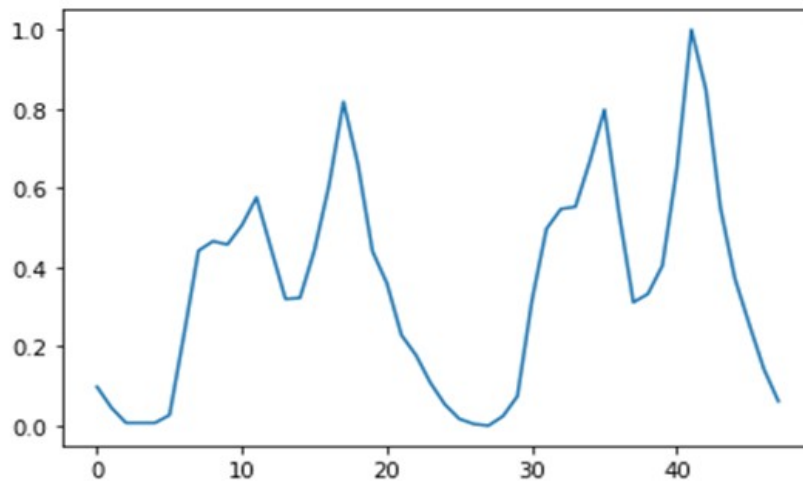


Fig.24. Demanda Eléctrica horaria de la provincia de granma, cuba. Dos días.

En la imagen que mostramos, se ven claramente dos picos, uno correspondiente al medio día y el otro al pico producido sobre las 7 de la noche. Mostramos dos días en la imagen.

A continuación, mostramos una imagen con el código y luego describiremos parte de él y la gráfica resultante.

Tema 5. Redes neuronales y deep learning

```
import neurolab as nl
import numpy as np
import pandas as pd
import pylab as pl
from sklearn.preprocessing import MinMaxScaler

#*****
serie = pd.read_csv('demanda.csv', nrows= 48)
demanda=np.asarray([float(x.replace(',','.')) for x in serie['Valor']]).reshape(-1,1)
scaler = MinMaxScaler()
demanda=scaler.fit_transform(demanda)
target = demanda
#*****
# Creamos una red de Elman con dos capas
net = nl.net.newelm([[0, 1]], [20, 1], [nl.trans.TanSig(), nl.trans.PureLin()])
# Inicializamos los pesos de manera aleatoria
net.layers[0].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
net.layers[1].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
net.init()
# Entrenamos la red
error = net.train(demanda, target, epochs=500, show=100, goal=0.01)
# Hacemos las predicciones
output = net.sim(demanda)
#*****
pl.subplot(211)
pl.plot(error)
pl.xlabel('Número de Epocas')
pl.ylabel('Error de Entrenamiento (MSE)')
pl.subplot(212)
pl.plot(target)
pl.plot(output)
pl.legend(['Valor Real', 'Valor Predicho'])
pl.show()
```

Fig.25. Código que implementa una red de Elman para predecir la demanda eléctrica horaria.(Código adaptado para la predicción de la demanda eléctrica a partir del ejemplo dado en la documentación de neurolab)

Es realmente increíble que se pueda entrenar y utilizar una red neuronal recurrente en tan pocas líneas de código. Explicuemos un poco este ejemplo.

```
serie = pd.read_csv('demanda.csv', nrows= 48)
```

En esta línea de código, se leen 48 datos de la demanda eléctrica desde un fichero en formato csv.

```
demanda=np.asarray([float(x.replace(',','.')) for x in
serie['Valor']]).reshape(-1,1)
```

Los datos almacenados en el fichero csv, usan como separador de valores decimales una coma, por tanto hay que sustituirla por un punto, también filtramos el dataset y solo tomamos el valor de la demanda de potencia activa y la ajustamos a una

Tema 5. Redes neuronales y deep learning

columna.

```
scaler = MinMaxScaler()
```

```
demanda=scaler.fit_transform(demanda)
```

Con estas dos líneas de código, normalizamos los valores de la demanda de potencia a valores en el rango entre 0 y 1.

En el segundo bloque de código es donde se realiza el trabajo pesado del uso de la red de Elman.

```
net = nl.net.newelm([[0, 1]], [20, 1], [nl.trans.TanSig(),  
nl.trans.PureLin()])
```

En la línea anterior es donde se define la red. El primer parámetro dice que solo tendremos una neurona de entrada cuyo rango de valores va desde 0 a 1. El segundo argumento dice que tendremos una capa oculta con 20 neuronas y una capa de salida con 1 neurona. Los siguientes dos, dicen que en la capa oculta las neuronas tendrán una función de activación de tipo TanSig y en la capa de salida PureLin. Los detalles de los parámetros de la función newelm() , están explicados en la documentación de neurolab.

En las siguientes dos líneas se inicializan los pesos de las matrices de pesos correspondientes a las capas de la red.

```
net.layers[0].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
```

```
net.layers[1].initf= nl.init.InitRand([-0.1, 0.1], 'wb')
```

En la siguiente línea se inicializa la red.

```
net.init()
```

Con la siguiente línea de código, se entrena la red de Elman, usando 500 ciclos de

Tema 5. Redes neuronales y deep learning

entrenamiento a los que se les llama épocas. La red terminará su entrenamiento cuando culmine con los ciclos o alcance la meta de un error de 0,01.

```
error = net.train(demanda, target, epochs=500, show=100, goal=0.01)
```

Ya que nuestra red está entrenada, es hora de probar su calidad y lo esto lo hacemos por medio de la siguiente línea de código.

```
output = net.sim(demanda)
```

El resto del código es solo para graficar los resultados, que se mostrarán en la siguiente imagen.

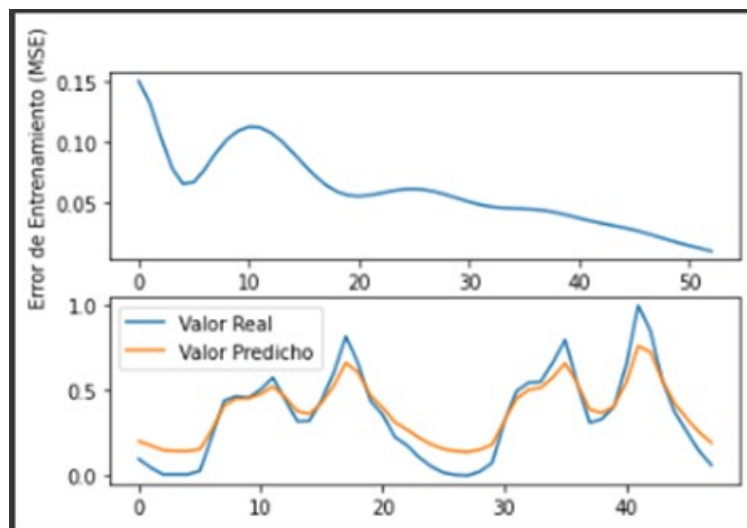


Fig.26. Resultados del entrenamiento de una red de Elman para la predicción de la demanda de potencia activa en la provincia de Granma, cuba.

No haremos un análisis del resultado, pues no es nuestra intención la predicción de la demanda, sino ejemplificar como pueden usarse esas redes en problemas comunes y de la vida cotidiana.

Conclusiones

Hasta aquí hemos hecho una brevísima introducción al campo de las redes

Tema 5. Redes neuronales y deep learning

neuronales en el camino a explicar las más nuevas adiciones al campo y lo que se conoce como Deep Learning. Solo hemos tocado las redes feedforward y las recurrentes en alguna medida. Los conceptos que hemos tocado nos servirán de base para comprender los nuevos que iremos introduciendo en la próxima lección, donde tocaremos las redes neuronales recurrentes LSTM y GRU, consideradas como profundas. Tocaremos, además las redes neuronales convolucionales, entrado así en el campo del Deep Learning.

Tema 5. Redes neuronales y deep learning

5.2. Redes neuronales y Deep learning (continuación)

Introducción

En los últimos años se ha acuñado el término, aprendizaje superficial (Shallow Learning) a los métodos de aprendizaje automático tradicionales, como los que hemos estudiado en las lecciones anteriores, con el fin de diferenciarlos de aquellos métodos que se denominan “Aprendizaje profundo” o Deep Learning. Sin embargo, no queda claro cuál es la frontera en la que una red neuronal se considera profunda.

Lo que sí está, sumamente claro, sin embargo, es que métodos como Support Vector Machines, XGBoost, el perceptrón multicapa y todos los que hemos visto hasta esta lección, se consideran métodos de aprendizaje superficial; pero ¿entonces cuando podemos decir que una red es de aprendizaje profundo? ¿Cuándo tiene 10 capas? ¿Cuándo tiene 100?

No daremos aquí una definición exacta de cuando llamar de una forma u otra a una red, daremos los elementos necesarios y que pueda usted llegar a sus propias conclusiones. Como elementos de juicio y con la esperanza de usted pueda formar su propia opinión, podemos asegurar que las redes consideradas profundas comparten las siguientes características:

- ▶ Pueden contener, literalmente, cientos de millones de parámetros y muchas neuronas.
- ▶ Usan datasets masivos para su entrenamiento.
- ▶ Son entrenadas en clústers de GPU o TPU. (GPU: Unidades de procesamiento Gráfico y TPU: Unidades de procesamiento Tensorial)
- ▶ En Visión por computadoras, usando Deep Learning se elimina la necesidad de

Tema 5. Redes neuronales y deep learning

Ingeniería de rasgos.

- Desde el punto de vista metodológico, usan mejores algoritmos de entrenamiento, capas de neuronas con diferentes arquitecturas, funciones de activación refinadas, etc.

Lo cierto es que las redes neuronales profundas, son una evolución importante de las primeras redes como el MLP y las redes de Hopfield.

Sería imposible cubrir todo el terreno que abarcan los aspectos de las Redes Neuronales profundas en una introducción al tema, como lo es este, por tanto, tocaremos aquellos aspectos que nos parecen esenciales para entender el campo. Para ello, usaremos, como hemos venido haciendo hasta ahora el lenguaje de programación python.

Redes neuronales recurrentes

En la lección anterior, estudiamos, aunque a un nivel muy básico, las redes neuronales recurrentes, por medio de los modelos de Hopfield, Jordan y Elman. Aunque estas redes son interesantes desde el punto de vista teórico, desde la perspectiva práctica no lo son tanto y en la literatura son conocidas como Redes Neuronales Recurrentes Simples. En esta lección estudiaremos, redes más recientes, como LSTM y GRU, consideradas redes con aprendizaje profundo.

¿Por qué se consideran redes con aprendizaje profundo? Para entender esta idea, pongamos como ejemplo el del uso de una red neuronal recurrente para modelar un texto de 50 palabras. Al desplegar, la red, tendríamos una capa para cada palabra y por tanto, tendríamos, también, una red neuronal recurrente de 50 capas.

Entrenar redes neuronales de muchas capas es considerablemente difícil y las recurrentes, son especialmente difícil de entrenar. Dos de los problemas que complican el entrenamiento de las redes neuronales con muchas capas, es el conocido como desvanecimiento/explosión del gradiente. Grosso modo, el

Tema 5. Redes neuronales y deep learning

desvanecimiento, tiene lugar al hacerse muy pequeños los pesos de las conexiones de las neuronas en las capas más alejadas de la salida, lo que hace que los cambios se hagan muy lentos y por tanto dificulta la convergencia de la red. Lo mismo pasa con la explosión, pues los pesos se hacen demasiado grandes.

Las nuevas redes neuronales recurrentes como LSTM han resuelto este problema. Una de las ventajas del uso de estas redes recurrentes es que pueden aplicarse a problemas donde la longitud de las entradas, son desiguales. Estas redes, como ya dijimos, operan sobre secuencias.

Los escenarios posibles pueden ser:

Uno a Uno: En este tipo de escenario, los vectores de entradas son de longitud fija y la red se limita a encontrar un mapeo estático entre la entrada y la salida.

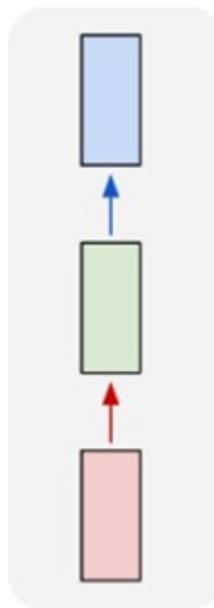


Fig. 1. Escenario de aplicación de las redes neuronales. Uno a Uno. Un vector de entrada y arroja un vector de salida.

Ejemplos de uso de este tipo de escenario hay muchos y uno de ellos por ejemplo es la clasificación de una imagen. La red toma como entrada el vector que contiene la

Tema 5. Redes neuronales y deep learning

imagen y retorna una clase, para la entrada.

Uno a Muchos: En este escenario la red toma como entrada un vector y arroja como salida una secuencia de vectores. Por ejemplo, una red entrenada para describir el contenido de una imagen, llamada captioning.

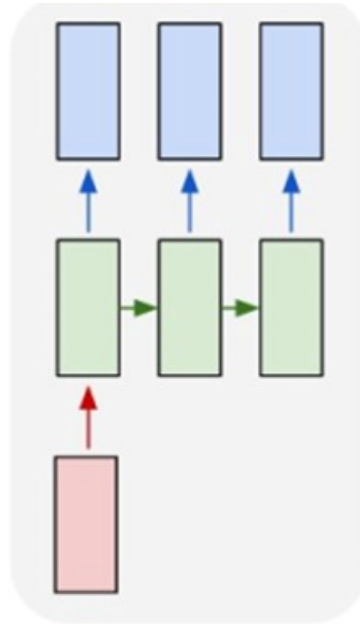


Fig. 2. Escenario de aplicación de las redes neuronales. Uno a Muchos. Un vector de entrada y arroja una secuencia de salida.

En este tipo de escenario, por ejemplo, se da como entrada a la red una imagen o trama de video y la red arroja como salida una secuencia de palabras que describen el contenido de la imagen.

Muchos a Uno: En este escenario, la red toma como entrada una secuencia y arroja un único vector a la salida. Un ejemplo de este tipo de uso, es el de análisis de sentimiento, donde se da como entrada a la red, un tuit, con el fin de clasificar si este es positivo o no.

Tema 5. Redes neuronales y deep learning

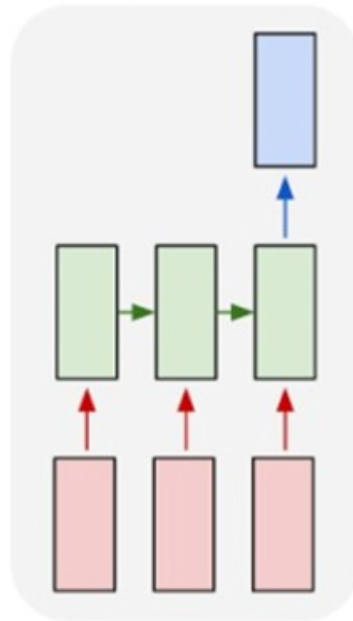


Fig. 3. Escenario de aplicación de las redes neuronales. Muchos a Uno. Una secuencia de entrada y arroja un vector de salida.

Muchos a Muchos: En este tipo de aplicaciones, la red toma como entrada una secuencia de vectores y arroja otra secuencia. De este tipo existen dos variantes, una sincronizada y la otra no.

Ejemplos de aplicación de estos escenarios, son la traducción de textos, donde se toma como entrada una secuencia de longitud n y se retorna otra de longitud m .

La forma sincronizada, se da, por ejemplo, cuando queremos etiquetar en una secuencia de video todas las tramas.

Tema 5. Redes neuronales y deep learning

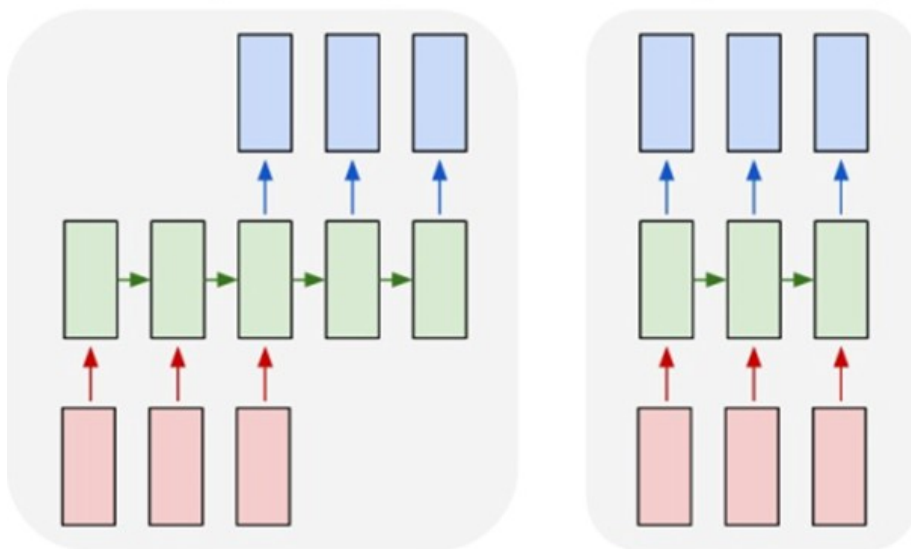


Fig. 4. Escenario de aplicación de las redes neuronales. Muchos a Muchos. Una secuencia de entrada y arroja otra secuencia de salida.

LSTM (Long Short Term Memory)

Este tipo de red fue introducida en 1997 por Hochreiter and Schmidhuber y son de las redes neuronales recurrentes más usadas y en parte, junto a las redes de tipo GRU y los Transformers, son el soporte de las actuales avances en reconocimiento de voz y traducción automática de textos. Sin embargo han dejado de ser el estado del arte en Procesamiento de Lenguaje Natural a favor de redes neuronales llamadas Transformes.

Al igual que las redes neuronales recurrentes simples, como la de Elman y Jordan, estas redes LSTM, tienen que verse como componentes que se despliegan en el tiempo. Esto quiere decir que habrá una celda LSTM, para cada uno de los componentes de la serie que procesamos o del texto que tratamos de traducir.

Las imágenes de las figuras 1 y 2, muestran la estructura interna de una celda LSTM.

Tema 5. Redes neuronales y deep learning

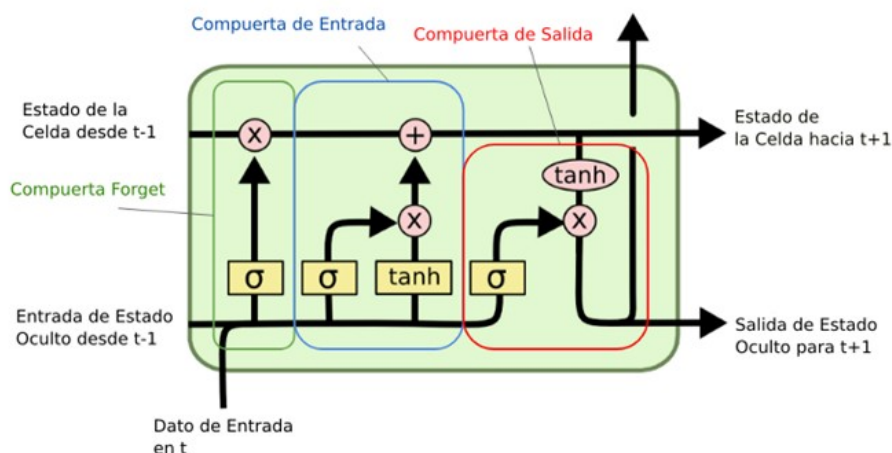


Fig. 5. Celda LSTM. (Imagen tomada de: <https://medium.com/analytics-vidhya/lstms-explained-a-complete-technically-accurate-conceptual-guide-with-keras-2a650327e8f2>)

Una celda LSTM, como se muestra en la imagen de la figura 1, toma como entradas, el estado o contenido almacenado en la memoria de la celda anterior, el estado oculto procedente, también de la celda anterior y un vector de entrada correspondiente al instante de tiempo actual. Como salidas ofrecen a la celda posterior el estado actual del contenido de la memoria de la celda y el estado de las neuronas de la capa oculta.

Estas celdas LSTM, se componen de tres compuertas, llamadas, Forget, entrada y salida, respectivamente. Representadas, en la imagen, con los colores verde, azul y rojo. Estas compuertas funcionan como válvulas que controlan el flujo de la información dentro de la celda y nos detendremos, un instante, a explicar cómo funcionan.

Estas compuertas, a las cuales llamaremos, compuertas sigmoidales, no son otra cosa que redes neuronales con función de activación sigmoide. Es preciso que recordemos que una función sigmoide, comprime los valores de la entrada a un intervalo entre cero y uno.

Tema 5. Redes neuronales y deep learning

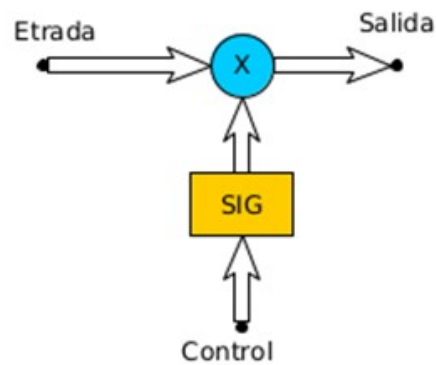


Fig. 6. Compuerta Sigmoidal.

Cuando la señal de control es mayor o igual a cero, el contenido de la entrada puede salir por la compuerta, en caso contrario la salida se hace cero. Y el funcionamiento es sumamente sencillo e intuitivo, cuando la entrada de la compuerta es menor que cero, entonces su salida será cero y en el círculo azul, se multiplica los valores de entrada por la salida de la red sigmoidal, que es cero y por tanto la salida es cero.

Esta, es la idea de las compuertas sigmoideas. Es importante que lo entendamos, pues una celda LSTM contiene tres de estas.

Tema 5. Redes neuronales y deep learning

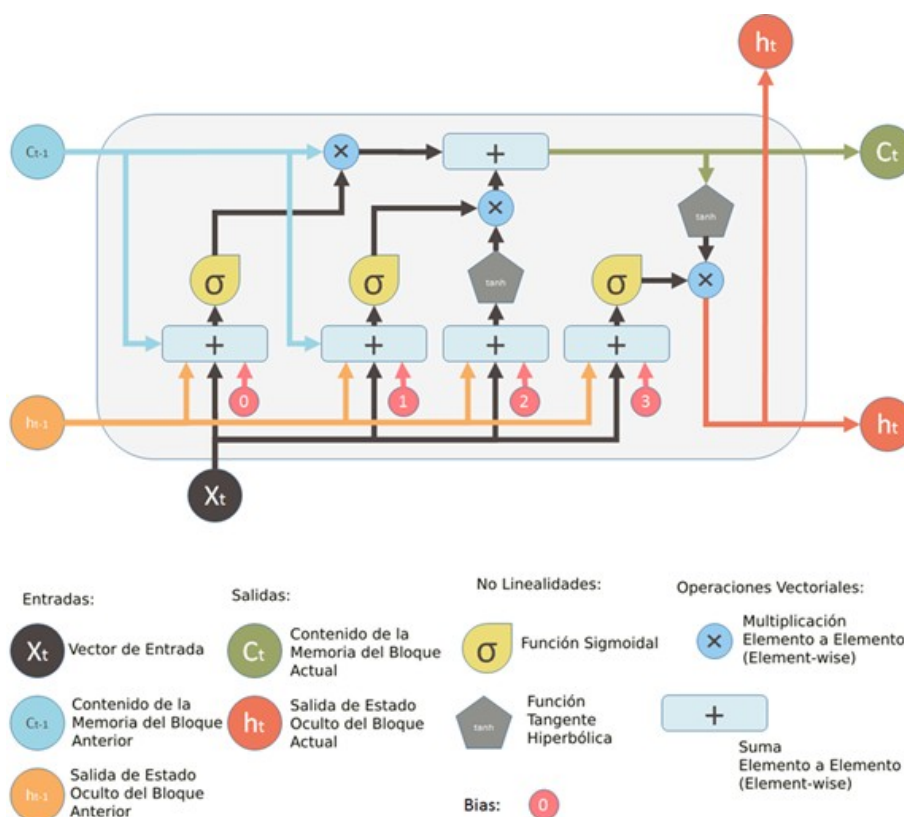


Fig.7. Celda LSTM con detalles internos. (Imagen tomada de: <https://blog.mlreview.com/understanding-lstm-and-its-diagrams-37e2f46f1714>)

Como ya mencionamos, los componentes de una red LSTM son:

- ▶ El Cell State.
- ▶ La compuerta Forget.
- ▶ La compuerta de entrada.
- ▶ La compuerta de salida.

Cell State: El elemento central de toda la arquitectura lo constituye el Cell State, en esencia, es la parte de la celda que funciona como memoria de la red. Este puede verse como una banda transportadora que conecta todos los bloques desplegados en el tiempo. En las figuras de la celda LSTM se representa con el símbolo C y un

Tema 5. Redes neuronales y deep learning

subíndice que indica al instante de tiempo al que pertenece, siendo t el instante actual, $t-1$ el anterior y así sucesivamente.

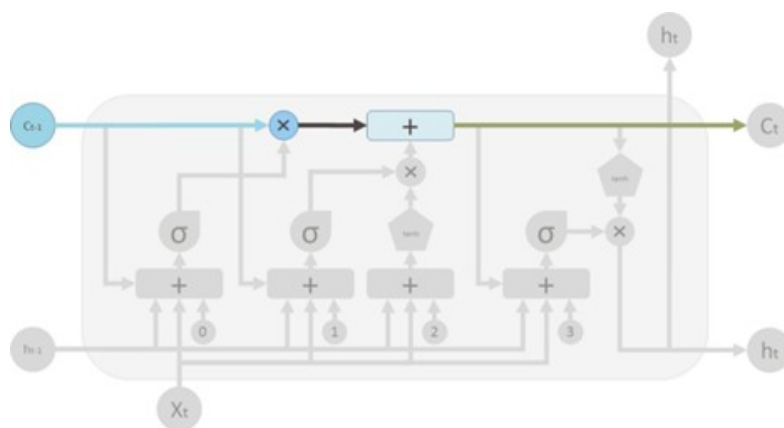


Fig.8. Cell State de una Celda LSTM.

Compuerta Forget: La función de esta compuerta sigmoidea es la de aprender cuando olvidar todo el historial de procesamiento pasado de la red, no permitiendo el paso de los vectores de estado de los bloques anteriores.

Como es una compuerta sigmoidea, está compuesta por una red neuronal con función de activación sigmoidea y una operación de multiplicación elemento a elemento. En la siguiente imagen se muestra la estructura de la compuerta.

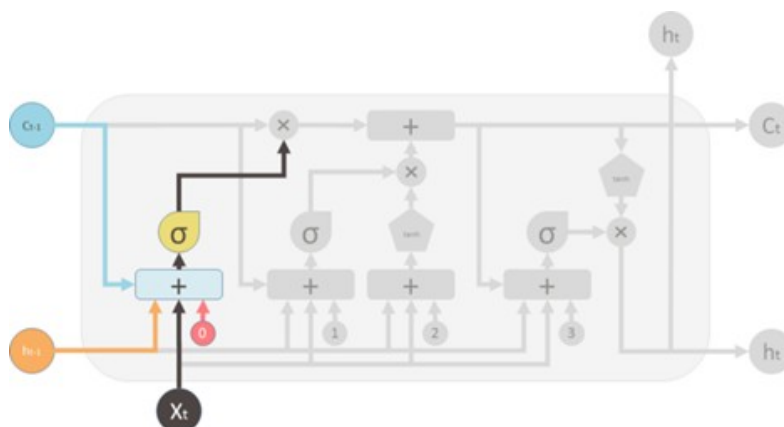


Fig. 9. Estructura de la compuerta Forget de una celda LSTM.

Tema 5. Redes neuronales y deep learning

Esta compuerta toma como entradas, el estado oculto del bloque t-1 y el vector de entrada actual X_t , aunque en esta imagen se muestra como entrada, también, C_{t-1} , esta, es opcional.

Si llamamos f a la salida de la compuerta Forget, entonces:

$$f_t = \sigma(W_f[h_{t-1}, X_t] + b_f)$$

Compuerta de Entrada: Esta compuerta tiene la función de elegir qué información se guardará en el cell state y esto se hace por medio de otra compuerta sigmoideal.

En la próxima figura, mostramos la estructura de la compuerta de entrada. Al igual que la compuerta Forget, toma como entradas a los vectores h_{t-1} y X_t y de forma opcional C_{t-1} .

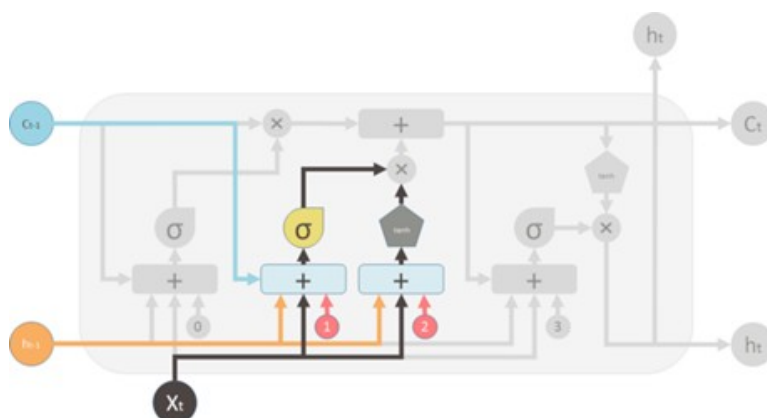


Fig. 10. Estructura de la compuerta de entrada de una celda LSTM.

En el instante de tiempo t , las dos redes neuronales que componen la compuerta, toman como entradas los vectores h_{t-1} y X_t , más los bias y opcionalmente C_{t-1} . Luego de calcular las respectivas salidas, en un caso con una función de activación sigmoideal y la otra por medio de una tangente hiperbólica, se combinan por medio de una operación de multiplicación elemento a elemento. Si la salida de la sigmoide es cero, la salida de la tangente hiperbólica será cero y luego sumado componente a componente también al estado de la red del bloque t-1 para formar así el C_t . El

Tema 5. Redes neuronales y deep learning

efecto de que la salida de la función sigmoide sea cero, es que no se memoriza nada. Si, por el contrario, la salida es 1, tiene el efecto de sumar la salida de la red con tangente hiperbólica como función de activación al estado C_{t-1} , para formar el C_t .

Por tanto, si llamamos i a la salida de la compuerta sigmoideal y $\widetilde{C}_t$ a la salida de la red con función de activación tangencial, entonces:

$$i_t = \sigma(W_i[h_{t-1}, X_t] + b_i)$$

$$\widetilde{C}_t = \tanh(W_c[h_{t-1}, X_t] + b_c)$$

Y por tanto, el estado actual, es decir en tiempo t , de la red, viene dado por:

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C}_t$$

En este proceso es en el que se actualiza el estado de la red.

Compuerta de Salida: Esta compuerta determina, que información va a usarse como salida de la red. La salida está basada en el estado de la red, es decir, en C_t , pero no será este C_t propiamente dicho, será una versión filtrada de esta. Como se muestra en la imagen correspondiente a la compuerta, C_t se hace pasar por una función tangencial hiperbólica para luego decidirse si se convierte o no en el próximo h_t . Esta decisión se toma por medio de la compuerta sigmoideal asociada a la salida, por el mismo mecanismo que hemos explicado.

Tema 5. Redes neuronales y deep learning

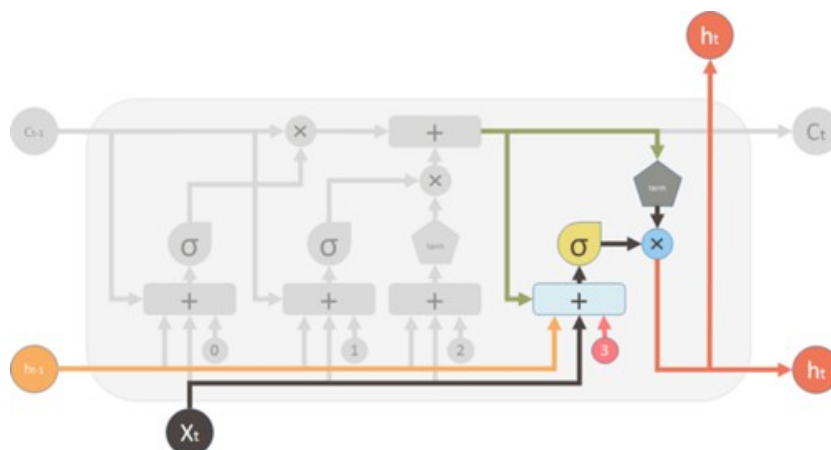


Fig. 7. Estructura de la compuerta de salida de una celda LSTM.

Conociendo la función de cada una de las partes que conforman la estructura de una celda LSTM, podemos esbozar, a groso modo la secuencia de operaciones que tienen lugar en ella.

Lo primero que ocurre cuando se alcanza el estado t , por una celda LSTM, es decidir, luego de recibir las entradas C_{t-1} , h_{t-1} y X_t , es decidir si se acepta como viene del bloque, el vector C_{t-1} . Como ya hemos mencionado, esta decisión se toma por medio de la compuerta Forget. Luego, se decide la manera en que se actualizará el cell state, de la red y la responsabilidad recae sobre la compuerta de entrada. Esta compuerta, como ya hemos visto, está dividida en dos partes, una en la que se decide que vector se usará para actualizar y la otra que forma el vector de actualización en sí. Una vez que se actualizado el estado de la red, se decide que vector del cell state se usará como la salida de la red, para alimentar el bloque correspondiente al estado $t+1$. Esto se hace por medio de la compuerta de salida, que como la anterior está dividida en dos partes, una que decide el vector candidato y la otra que somete el vector C_t a pasar por una función tangencial hiperbólica, antes de convertirse en h_t .

Como la mayoría de las redes neuronales recurrentes, las LSTM, se entrenan por medio de una adaptación de Backpropagation, llamada BPTT o Backpropagation a

Tema 5. Redes neuronales y deep learning

través del tiempo.

Veremos ahora un ejemplo de predicción de una serie de tiempo usando LSTM. La serie, cuestión, es la demanda de potencia activa (Demanda Eléctrica) en la provincia de Granma, Cuba.

Esta serie contiene 16784 mediciones de la demanda de potencia activa de la provincia de Granma en Cuba, desde el 1 de enero del 2012 hasta el 31 de diciembre del 2013. Las mediciones son tomadas hora a hora.

Producto a lo largo del script, lo hemos dividido en varias imágenes; pero el script completo estará disponible para su descarga.

```
#*****
# Predicción de la demanda de Potencia Activa en Granma usando LSTM
# Versión Stateless
#*****
import numpy as np
import pandas as pd
import pylab as pl
import math as mth
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
#*****
serie = pd.read_csv('demanda.csv')
demanda=np.asarray([float(x.replace(',','.')) for x in serie['Valor']]).reshape(-1,1)
scaler = MinMaxScaler(feature_range = (0, 1))
demanda=scaler.fit_transform(demanda)
train_size = int(len(demanda)*0.75)
test_size = len(demanda)-train_size
train, test = demanda[0:train_size,:], demanda[train_size:len(demanda):]
#*****
# Este procedimiento ajusta la forma de la entrada de los datos a la LSTM
def create_dataset(dataset, look_back=1):
    dataX, dataY = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        dataX.append(a)
        dataY.append(dataset[i + look_back, 0])
    return np.array(dataX), np.array(dataY)
#*****
look_back = 1
trainX, trainY = create_dataset(train, look_back)
testX, testY = create_dataset(test, look_back)
```

Fig. 8. Predicción de la demanda de potencia activa (demanda Eléctrica) con LSTM. (parte 1)

El código que aparece en la imagen anterior, está dividido en tres secciones, en la

Tema 5. Redes neuronales y deep learning

primera se importan todas las librerías que usamos dentro del script, en la segunda se importan los datos desde el fichero demanda.csv.

Este fichero, contiene las mediciones de la serie de tiempo y cada fila, en el fichero, contiene una columna con la fecha, la hora y el valor de la demanda a esa hora, separada por coma. Sin embargo, para poder procesarlo, es necesario convertir cada valor a uno de punto flotante y filtrarlo, pues no nos interesa el campo de fecha.

Luego, la serie se normaliza a un intervalo entre 0 y 1, pues invariablemente cuando usamos Redes Neuronales es preciso hacerlo. Esta normalización como en los ejemplos anteriores, se realiza por medio de MinMaxScaler.

En esta sección, también, se divide la serie en una parte para entrenamiento y la otra para prueba. Aquí, escogemos el 75% de las mediciones para entrenamiento y el resto para prueba.

La tercera sección, mostrada en esta imagen, corresponde a un procedimiento que retorna dos arreglos de numpy, cuyo objetivo es servir de dataset para el entrenamiento de la red. En este caso, para predecir el timestep i usamos como entrenamiento $i-1$. En la última parte de la imagen se muestra el procedimiento anterior para producir los conjuntos de entrenamiento y las etiquetas.

Tema 5. Redes neuronales y deep learning

```
# Las LSTM esperan los datos de entrada en la forma [samples, time steps, features]
trainX = np.reshape(trainX, (trainX.shape[0], 1, trainX.shape[1]))
testX = np.reshape(testX, (testX.shape[0], 1, testX.shape[1]))
#*****
# En esta sección se define la RNN LSTM y se entrena
model = Sequential()
model.add(LSTM(10, input_shape=(1,1)))
model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(loss='mean_squared_error',optimizer='adam')
model.fit(trainX, trainY, epochs=100, batch_size=100)
#*****
# Se hacen las predicciones, tanto sobre el conjunto de entrenamiento como
# el conjunto de prueba
trainPredict = model.predict(trainX)
testPredict = model.predict(testX)
# En el inicio se Normalizaron los valores de los datos a un intervalo
# entre 0 y 1, aquí se hace la operación inversa
trainPredict = scaler.inverse_transform(trainPredict)
trainY = scaler.inverse_transform([trainY])
testPredict = scaler.inverse_transform(testPredict)
testY = scaler.inverse_transform([testY])
# En esta sección se calcula el la rais del error medio cuadrático
# es necesario que se den cuenta que las series están desalineadas
trainScore = mth.sqrt(mean_squared_error(trainY[0], trainPredict[:,0]))
print('Train Score: %.2f RMSE' % (trainScore))
testScore = mth.sqrt(mean_squared_error(testY[0], testPredict[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
```

Fig. 9. Predicción de la demanda de potencia activa (demanda Eléctrica) con LSTM. (parte 2)

La imagen corresponde a la parte 2 del script de predicción de la demanda eléctrica con LSTM. Es importante destacar que éste, solo tiene intenciones didácticas y de ninguna manera está optimizado y aun así arroja un RMSE de 5.75.

En la imagen, se muestran tres secciones, en la primera, se adapta el conjunto de entrenamiento al formato que espera una LSTM como entrada.

Es decir, una LSTM, espero como entrada un arreglo de 3 dimensiones, de la forma: [N de muestras, N de timesteps, N de rasgos]. Donde el N de muestras (batch), es la cantidad de casos con los que vamos a entrenar la red y en este caso es 12586, que es el 75% de la cantidad de datos de la serie de demanda. N de timesteps, es la cantidad de pasos en el tiempo, anteriores al que queremos predecir que usaremos para entrenar, mientras que N de rasgos, es el número de rasgos que contiene nuestra secuencia.

Esta es uno de los puntos que más confusión genera en la preparación de los datos

Tema 5. Redes neuronales y deep learning

para el uso de LSTM.

En la sección que sigue, se definen las características de la red LSTM. Lo primero es la definición de la capa de entrada, la cual tiene 10 celdas y la forma de la entrada será de 1 timestep y un rasgo. En el modo Stateless, el tamaño de batch se define en tiempo de corrida por el propio algoritmo.

Luego se le agrega un Dropout de 0,2, que no explicaremos en detalles cuál es su propósito, pero se usa con el fin de mejorar la capacidad de generalización de la red.

Luego se define la salida de la red, como una capa densa de 1 salida, pues lo que queremos es predecir una variable, la demanda.

Las dos últimas líneas de código de esta sección se usan para compilar el modelo LSTM, usando como función de pérdida, el error medio cuadrático y como optimizador "Adam". Luego el modelo se entrena usando el método `fit()` del modelo, usando como conjunto de entrenamiento `trainX` y como etiquetas a predecir `trainY`. Para el entrenamiento se usan 100 épocas, que no es otra cosa que la cantidad de repeticiones en los que se les presentan los datos al algoritmo para poder obtener el mejor modelo. El `batch_size`, establece la cantidad de casos que se presentan en cada época y aquí, lo especificamos igual a 100.

En la siguiente sección se usan los modelos entrenados para hacer predicciones sobre el conjunto de entrenamiento y el de pruebas, con el fin de evaluar la calidad de los modelos.

Esto se hace por medio del método `predict()` del modelo. Ahora bien, como los datos, con el fin del entrenamiento de la red, se habían normalizado, ahora es necesario retornarlos a su estado original y eso se hace por medio del scaler usando una transformación inversa.

Al final de la sección se calcula, una medida de error llamada `Rain` del error medio

Tema 5. Redes neuronales y deep learning

cuadrático. El resto del código lo mostramos en la próxima imagen.

```
#En esta sección se alinean las series de tiempo y se grafican
trainY= trainY.reshape(-1,1)
pl.figure(1,figsize=(10,5))
pl.plot(range(240),trainY[0:240,0])
pl.plot(range(240),trainPredict[0:240,0])
pl.legend(['Valor Real de Entrenamiento','Valor predicho'])
#*****
testY=testY.reshape(-1,1)
#*****
testP = []
for i in range(len(testPredict)):
    if i<len(testPredict)-1:
        testP.append(testPredict[i+1,0])
testP.append(0)
testPP = np.array(testP).reshape(-1,1)
pl.figure(2,figsize=(10,5))
pl.plot(range(240),testY[0:240,0])
pl.plot(range(240),testPP[0:240,0])
pl.legend(['Valor Real de Prueba','Valor predicho'])
pl.show()
testScore = mth.sqrt(mean_squared_error(testY, testPP[:,0]))
print('Test Score: %.2f RMSE' % (testScore))
```

Fig. 10. Predicción de la demanda de potencia activa (demanda Eléctrica) con LSTM. (parte 3)

En esta parte del código, se grafican las series de entrenamiento y la predicción del modelo de esta serie, además de la gráfica de la serie de prueba y su predicción.

Ahora bien, hay un detalle que es importante destacar. Al final de la sección anterior, se calculaba la Raíz del error medio cuadrático, pero con las series desalineadas, pues es importante que recordemos que con el fin de construir nuestro conjunto de datos de entrenamiento y prueba, las etiquetas se corren un timestep, por tanto cuando se calcula el RMSE, este da como resultado un valor más alto. En la segunda parte del código mostrado en la parte 3, se muestra el código para alinear las dos series y el cálculo del RMSE.

En la siguiente imagen, mostramos los resultados de la corrida de este script. Al final de las gráficas se muestra que el valor arrojado por el modelo es de 5.75% lo que quiere decir que nuestro modelo tiene una calidad del 94.25%. Lo cual es muy bueno, aun sin optimizar el modelo. Y eso habla de la habilidad de las redes LSTM

Tema 5. Redes neuronales y deep learning

para manejar este tipo de datos.

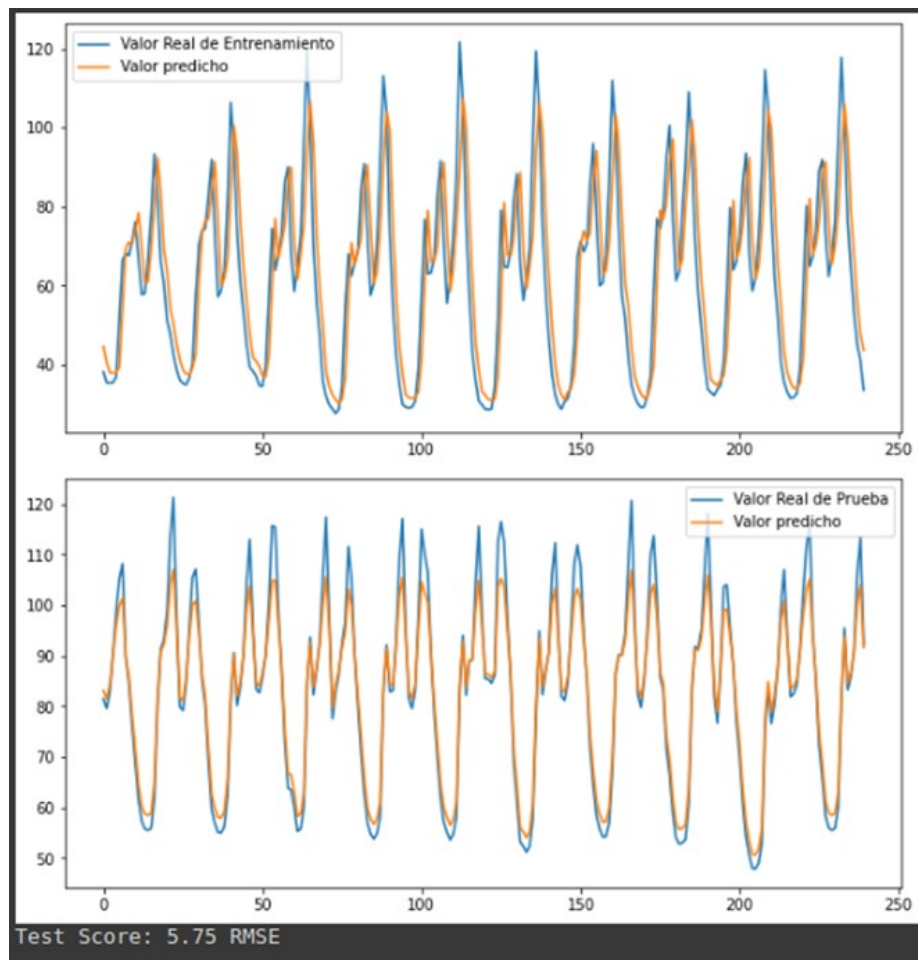


Fig. 11. Predicción de la demanda de potencia activa (demanda Eléctrica) con LSTM. (Resultados)

En la imagen correspondiente a los resultados, puede verse claramente en la figura superior las dos series desalineadas.

GRU (Gated Recurrent Unit)

Este tipo de Red Neuronal Recurrente, puede considerarse como una modificación a LSTM y arroja, al igual que la anterior, muy buenos resultados en la mayoría de las aplicaciones en la que se usa. Esta red, fue introducida en la literatura por Kyunghyun Cho, de la universidad de Montreal, en el 2014.

Tema 5. Redes neuronales y deep learning

Al igual que LSTM, usa el método de las compuertas sigmoidales para controlar el flujo interno de la información en la celda. Estas son, relativamente más nuevas que las LSTM y tienen una arquitectura mucho más simple.

A diferencia de las LSTM, estas redes solo contienen dos compuertas sigmoidales, llamadas compuerta de Reset y compuerta de Update. En este tipo de red prescinden de la celda de memoria de la LSTM o Cell State y usan en su lugar el estado oculto. Como tiene menos componentes, estas memorias se entrenan más rápido.

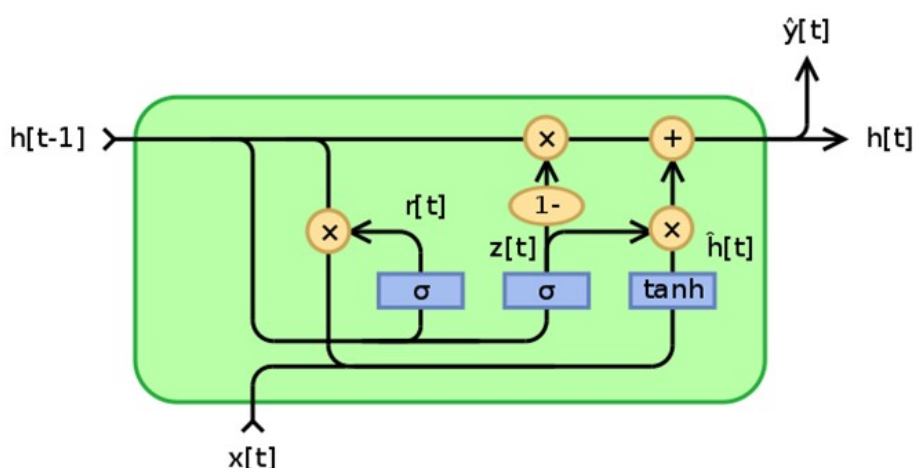


Fig. 12. Celda GRU. (Imagen tomada de Wikipedia)

Justamente como en LSTM, estas compuertas sigmoidales, son entrenadas para determinar selectivamente que parte de la información histórica aportada por las capas anteriores, por medio de h_{t-1} , retener y cual descargar, así como cual combinar con la entrada actual para formar la salida h_t .

La función de esta compuerta sigmoide es determinar qué valores del estado oculto de la capa anterior usamos o si seguimos adelante sin tenerla en cuenta.

Esta compuerta es muy parecida en su función a la compuerta forget de LSTM y desde la perspectiva funcional es idéntica. En la imagen se representa como r_t y su expresión matemática es la siguiente:

Tema 5. Redes neuronales y deep learning

$$r_t = \sigma(W_{rxt} X_t + W_{rht-1} h_{t-1} + b_r)$$

Donde $W_{xt} X_t$ es la entrada actual en el tiempo t y $W_{ht-1} h_{t-1}$ es la entrada correspondiente al bloque anterior o h_{t-1} .

Es importante que recuerden que la salida de esta compuerta es 0 o 1 y que controla una unidad multiplicativa, que hace una operación de multiplicación elemento a elemento con la información de h_{t-1} , determinando cuando se hace cero o no y por tanto si se utiliza en el tiempo t .

Update Gate

La compuerta de Update, de GRU, tiene como función decidir que composición tendrá la salida de la red h_t . Esta decide, si la salida será la el estado oculto h_{t-1} del bloque anterior o será el valor del estado oculto actual.

Su expresión matemática es prácticamente idéntica a la de la Reset Gate, excepto que está configurada con dos salidas mutuamente excluyentes.

$$z_t = \sigma(W_{zxt} X_t + W_{zht-1} h_{t-1} + b_z)$$

Veamos ahora la operación de la red. El primer paso en el funcionamiento de una red de tipo GRU, es determinar un estado oculto $\widehat{h}_t$, llamado estado oculto candidato. La expresión matemática de este es la sigue:

$$\widehat{h}_t = \tanh(W_{ht} X_t + (r_t * h_{t-1}) W_{ht-1})$$

Usamos aquí el $*$, para indicar la operación de multiplica elemento a elemento. Está sumamente claro de la expresión anterior, la función de la compuerta de reset. La salida de dicha compuerta está denotada en la expresión como r_t y por tanto determina, si es cero, que el estado oculto solo contenga la entrada actual, si por el contrario, es 1, entonces este estado estará compuesto, tanto por el valor actual X_t como por el valor que contiene la información histórica h_{t-1} , que proviene de

Tema 5. Redes neuronales y deep learning

las celdas anteriores en el tiempo.

Una vez que se tiene el estado oculto candidato $\widehat{h}_t$, se procede a producir la salida de la red. Esta salida estará determinada por la salida de la compuerta de update y la expresión matemática es la que sigue:

$$h_t = z_t * \widehat{h}_t + (1 - z_t) * h_{t-1}$$

Está claro, a partir de la expresión, que el valor de h_t , depende completamente del valor de la salida de la compuerta de Update z_t . Si $z_t = 0$, entonces la salida h_t solo contendrá la información histórica de la red, no usando el valor de la entrada actual. Si por el contrario es $z_t = 1$, entonces, la salida h_t no contendría información histórica de la red, procedente de las capas anteriores h_{t-1} y usará solamente la del tiempo actual t .

No mostraremos ejemplos de uso de GRU, pues es intercambiable a nivel de código con LSTM y si se desea probar el ejemplo de predicción de una serie de tiempo con GRU, solo hay que cambiar la palabra LSTM por GRU en la definición de la red.

Con la explicación que hemos dado del funcionamiento de una red GRU, terminamos lo correspondiente a las redes neuronales recurrentes para adentrarnos ahora en el mundo de las convolucionales. En la próxima sección comenzaremos con una introducción a este tipo de red.

Redes Neuronales Convolucionales

Las redes neuronales convolucionales, junto a las redes neuronales recurrentes como LSTM y GRU, han ayudado a la popularidad del campo de *Deep Learning* y hacer de este término un slogan que se escucha en cada oficina y se lee en cada blog, periódico o noticia.

En parte, este éxito ha tenido que ver con hitos que han ido imponiendo distintas arquitecturas de estas redes en competencias internacionales y en aplicaciones

Tema 5. Redes neuronales y deep learning

prácticas.

Este tipo de red es inspirada en el funcionamiento del cortex visual. Una de las primeras motivaciones detrás de estas redes, fueron los experimentos de Hubel y Wiesel, sobre el cortex visual de un gato. El cortex visual tiene pequeñas regiones de células que son sensibles a regiones específicas del campo visual. Esto significa que si dichas regiones del campo visual son excitadas, entonces el grupo de células asociadas a dicha región serán activadas. Adicionalmente, las células excitadas dependen de la forma y orientación del objeto presentado en el campo visual. Esto indica que hay neuronas o grupos de ellas que se excitan al poner en el campo visual, patrones de líneas verticales y otras para patrones de líneas horizontales.

Se piensa que el cortex visual de los mamíferos usa una distribución en capas de neuronas para construir una imagen. Donde las capas superiores funcionan a niveles de abstracción más altos. Luego se verá, que las redes convolucionales obedecen este mismo principio.

Basado sobre estas ideas la primera red neuronal que se desarrolló fue el llamado Neocognitrón.

De manera general, las arquitecturas de estas redes han evolucionado de la siguiente forma:

- ▶ **LeNet:** Esta fue la primera red convolucional con aplicabilidad práctica. Usaba backpropagation como método de entrenamiento y fue diseñada para clasificación de dígitos manuscritos. Se usó, con el fin de procesar grandes volúmenes de cheques escritos a mano en estados unidos. Desafortunadamente este enfoque no alcanzó mucho éxito al no escalar bien a problemas mucho más grandes, principalmente debido a que fue entrenado con un dataset de tamaño insuficiente, los ordenadores de la época no eran lo suficientemente rápidos y usaban funciones de activación no lineales que resultaron incorrectas. (Desarrollada por Yan LeCun cuando estaba en IBM)

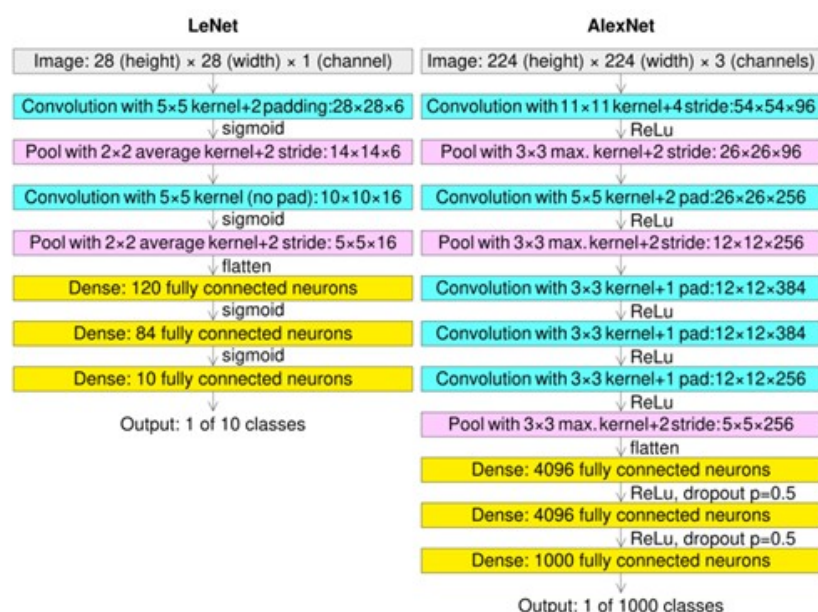
Tema 5. Redes neuronales y deep learning

- **AlexNet:** Fue en el 2012, cuando una red con este nombre ganó la competencia internacional ImageNet ILSVRC de ese año. (ILSVRC: Large-Scale Visual Recognition Competition). Esta arquitectura logró disminuir el error en la clasificación de las imágenes en el dataset de la competencia, desde 26 a 15%, lo que es muy significativo para esa época.

Este éxito, fue en parte, debido a:

- Entrenada con el dataset de ImageNet, que contiene alrededor de 15 millones de imágenes etiquetadas, distribuidas en aproximadamente 22000 categorías.
- El modelo se entrenó usando 2 GPU o tarjetas gráficas Nvidia GTX580, que demoró entre 5 y 6 días.
- Se usó como función de activación, RELU, en vez de otras más lentas como tangente hiperbólica y sigmoide. Otro de las ventajas de usar esta función, es que no experimenta el problema del desvanecimiento del gradiente, que explicamos brevemente en la sección de Redes Recurrentes.

La arquitectura de esta red, esta compuesta por 5 capas convolucionales, 3 capas de pooling y un clasificador softmax de 1000 puntos.



Tema 5. Redes neuronales y deep learning

Fig. 13. Arquitectura en capas de LeNet y AlexNet. (Imagen tomada de Wikipedia)

- ▶ ZFNet: Apareció en 2013. El rasgo distintivo de esta arquitectura, es que disminuye el tamaño del filtro de la primera capa de 11x11 a 7x7 y el stride de 4 a 2. Esta arquitectura ganó la competencia Internacional (ILSVRC: Large-Scale Visual Recognition Competition) en ese año.
- ▶ VGGNet: Fue introducida en 2014. Esta red, cuenta con 19 capas con filtros más pequeños 3x3 y stride de 1 en todas las capas convolucionales y de 2x2 en las capas de pooling.
- ▶ GoogLeNet: Fue desarrollada por Google en el 2015. El modelo tiene 22 capas y fue el ganador de ILSVRC en ese año con una tasa de error de 6.7%. La arquitectura es esencialmente diferente al resto de las redes.
- ▶ ResNet: Introducida por Microsoft Research Asia en 2015 y cuenta con 152 capas. Esta, ganó la competencia ILSVRC del 2015 con una increíble tasa de error del 3.6%. Esta es una tasa mucho mejor que las arrojadas por humanos que rondan entre el 5 y el 10%. En esta arquitectura se introduce el concepto de conexión residual.
- ▶ Inception-ResNet: Es una arquitectura híbrida y se introdujo en 2017. Este modelo, al incorporar conexiones residuales combinadas con capas de tipo inception, logró reducir de forma dramática el tiempo de entrenamiento y mejoró los resultados de la red ResNet pura.

Aunque hay algunas arquitecturas más, no es nuestra intención hacer un listado completo de ellas y si, enumerar los nombres de las más representativas, de forma que incentive el autoestudio y la exploración individual de ellas.

La idea general, pero básica detrás del funcionamiento de una red neuronal convolucional es explotar las características espaciales de la imagen o el dato de

Tema 5. Redes neuronales y deep learning

entrada. De la misma manera en que una red neuronal recurrente explota las dependencias temporales de una secuencia de entrada, las convolucionales lo hacen con las relaciones espaciales de la entrada.

Las redes neuronales convolucionales, se basan en varios conceptos elementales tales como: convolución, Filtro o kernel, stride, padding, pooling. Antes de entender, como se organizan las capas de una red y que función tienen, es necesario dominar estos conceptos que son la piedra angular de su funcionamiento.

Filtros o kernel

Un filtro o kernel, como también se le llama, es una matriz cuadrada de dimensiones $n_k \times n_k$. Las dimensiones de los kernel o filtros más usados son los de 11 x 11, 7 x 7, 5 x 5 y 2 x 2.

Estos filtros o kernels son usados en las operaciones de convolución, como veremos más adelante.

A continuación, mostramos algunos de los kernels más usados:

$$K_h = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Este kernel, permite detectar contornos horizontales.

$$K_v = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Este otro, permite detectar los contornos verticales.

$$K_L = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Este kernel, permite detectar contornos cuando la luminosidad cambia abruptamente.

Tema 5. Redes neuronales y deep learning

$$K_B = \frac{-1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Y finalmente, este kernel, provoca que la imagen se haga borrosa.

Convolución

El término convolución es de hecho, la operación por el cual las redes que estudiamos toman su nombre. Pero; ¿qué son las operaciones de convolución?

Una convolución es una operación, que toma como entrada dos tensores y da como resultado otro tensor.

El concepto de tensor, que no vamos a definir aquí, no es más que una generalización de los conceptos de escalar, vector y matriz. Un escalar, es considerado como un tensor de orden 0, un vector, es considerado como un tensor de orden 1 y una matriz es considerada como un tensor de orden 2.

De esta forma, una convolución se realiza, suponiendo dos tensores, de 3 x 3, y donde uno es un kernel como la siguiente:

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} * \begin{bmatrix} k_1 & k_2 & k_3 \\ k_4 & k_5 & k_6 \\ k_7 & k_8 & k_9 \end{bmatrix} = \sum_{i=1}^9 a_i * k_i$$

Esto, en definitiva, quiere decir que:

$$c = (a_1 * k_1) + (a_2 * k_2) + \dots + (a_9 * k_9)$$

Ahora, normalmente el primer tensor, en redes convolucionales es una imagen y el segundo, es un kernel. En el ejemplo que hemos usado para definir la operación de convolución, el primer tensor es de 3 x 3, como el kernel, veamos qué pasa si necesitamos hacer una operación usando un kernel de 2 x 2.

Tema 5. Redes neuronales y deep learning

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} * \begin{bmatrix} k_1 & k_2 \\ k_3 & k_4 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 \\ c_3 & c_4 \end{bmatrix}$$

Dónde:

$$c_1 = (a_1 * k_1) + (a_2 * k_2) + (a_4 * k_3) + (a_5 * k_4)$$

$$c_2 = (a_2 * k_1) + (a_3 * k_2) + (a_5 * k_3) + (a_6 * k_4)$$

$$c_3 = (a_4 * k_1) + (a_5 * k_2) + (a_7 * k_3) + (a_8 * k_4)$$

$$c_4 = (a_5 * k_1) + (a_6 * k_2) + (a_8 * k_3) + (a_9 * k_4)$$

Esto es equivalente a tomar el kernel y posicionarlo sobre el primer tensor, de forma que k_1 coincida en el lugar de a_1 , es decir la esquina superior izquierda, hacer la operación para obtener c_1 y luego desplazar el kernel a la derecha y así sucesivamente.

Es vital, que nos demos cuenta que estamos moviendo, el kernel, sobre el primer tensor para calcular, primero c_1 , luego movemos el kernel una columna a la derecha y ahora hacemos coincidir k_1 con a_2 , cuando no podemos movernos más a la derecha, entonces nos movemos a la izquierda y descendemos una fila de forma que ahora k_1 coincide con a_4 y cuando terminamos nos movemos otra columna más a la derecha.

Ahora bien, al número de columnas que nos movemos, para realizar la operación es a lo que se conoce como stride y es necesario que lo recordemos.

Si denotamos al stride como s , entonces, se sabe que el tamaño n_b del tensor que resulta de una operación de convolución entre un tensor de tamaño n_a con un kernel de tamaño n_k es:

$$n_b = \lfloor \frac{n_a - n_k}{s} + 1 \rfloor$$

Tema 5. Redes neuronales y deep learning

Donde la operación $\lfloor x \rfloor$ indica que se toma la parte entera del argumento.

Otra de las operaciones que son básicas para las redes convolucionales, es la llamada Pooling, veamos en que consiste.

Pooling

Esta operación es conceptualmente mucho más sencilla de entender que la de convolución. Para explicarla, supongamos que tenemos un tensor, o matriz de dimensión 4 x 4 y tenemos una matriz de tamaño 2 x 2 que llamaremos m.

$$\begin{bmatrix} a_1 & a_2 & a_3 & a_4 \\ a_5 & a_6 & a_7 & a_8 \\ a_9 & a_{10} & a_{11} & a_{12} \\ a_{13} & a_{14} & a_{15} & a_{16} \end{bmatrix} \pi_{+} \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} = \begin{bmatrix} p_1 & p_2 \\ p_3 & p_4 \end{bmatrix}$$

Donde π_{+} denota la operación de max-pooling y la matriz m, cuyos elementos son las m_x nos permite escoger los elementos sobre los que operamos.

Como hicimos en la operación de convolución, situamos nuestra matriz m, en la esquina superior izquierda del primer tensor, es decir, donde m_1 coincide con a_1 y m_4 coincide con a_6 . Ahora, para realizar la operación de max-pooling en esa posición y por tanto calcular p_1 se hace lo siguiente:

$$p_1 = \max(a_1, a_2, a_5, a_6)$$

Es decir, se escoge aquel elemento con mayor valor.

Para calcular p_2 ahora se mueve m, dos columnas a la derecha (stride igual a 2) de forma que el ahora m_1 coincida con a_3 , tal que:

$$p_2 = \max(a_3, a_4, a_7, a_8)$$

Como ya no podemos movernos más a la derecha, pues nos movemos dos filas abajo y dos a la izquierda para comenzar de nuevo y calcular ahora p_3 .

Tema 5. Redes neuronales y deep learning

Para eso, m_1 debe coincidir con a_9 , y entonces:

$$p_3 = \max(a_9, a_{10}, a_{13}, a_{14})$$

De la misma manera, nos movemos, como indica el $\text{stride}=2$, dos columnas a la derecha, para calcular p_4 , de tal forma que m_1 coincida con a_{11} , y entonces:

$$p_4 = \max(a_{11}, a_{12}, a_{15}, a_{16})$$

Existen otros tipos de pooling como el average, pero son menos usados y por tanto no le dedicaremos espacio.

Otro de los conceptos que son claves a la hora de entender el funcionamiento de las redes convolucionales es el de Padding. Veamos en que consiste.

Padding

Es importante notar que a medida que se aplican capas de convoluciones, se produce una pérdida de información en los bordes de la imagen, o en los mapas de rasgos y este efecto indeseable puede ser resuelto por medio de Padding.

Esta pérdida de información se puede observar, por medio de la fórmula que dimos para calcular la dimensión del tensor de salida luego de realizar una operación de convolución. Hagamos un ejemplo:

Sabemos, que por medio de la siguiente expresión podemos calcular la dimensión del tensor resultante de una operación de convolución.

$$n_b = \lfloor \frac{n_a - n_k}{s} + 1 \rfloor$$

Entonces, si nuestra imagen de entrada es de 32×32 y el kernel de 3×3 , y el stride igual a 1.

$$n_b = \lfloor \frac{32 - 3}{1} + 1 \rfloor$$

Tema 5. Redes neuronales y deep learning

$$n_b = 30$$

Por tanto, hemos perdido dos filas y columnas de la imagen de entrada. Para resolver esta pérdida de información se usa el Padding.

Esta técnica consiste en añadir dos filas de píxeles al inicio y final de la imagen o del mapa de rasgos y dos columnas a ambos lados de ellos. Los píxeles se pueden inicializar a cero.

El tamaño final de la imagen se puede calcular por medio de la siguiente expresión:

$$h + 2p - f + 1 \quad w + 2p - f + 1$$

Donde h y w son el alto y el ancho de la imagen de entrada, p es el padding y f el tamaño del filtro.

Ahora bien, si queremos que nuestra imagen de entrada no sufra pérdida, podemos calcular el padding, de la siguiente forma:

$$p = \frac{f - 1}{2}$$

Donde f es el tamaño del filtro.

De esta forma, cuando usamos filtros de 5×5 , es necesario usar un padding igual a 2.

Hemos usado en varias ocasiones el término «mapa de rasgos» sin que hayamos explicado en que consiste. Un mapa de rasgos, es el resultado que se obtiene luego de aplicar una convolución. Este adquiere su nombre, precisamente del hecho de que los kernel o filtros se aplican a las imágenes con el objetivo de detectar los rasgos que conforman los objetos y la imagen en general.

Luego de que hayamos explicado los conceptos de filtro o kernel, que hayamos explicado que es el stride, el padding y los mapas de rasgos, es hora de que

Tema 5. Redes neuronales y deep learning

revisemos en más detalles la composición de las redes convolucionales.

Una red de este tipo, como el resto de las redes, se compone de capas y estas pueden ser de los siguientes tipos:

- ▶ Capas convolucionales.
- ▶ Capas de Pooling. (subsampling)
- ▶ Capas densas completamente conectadas.
- ▶ Capas Dropout.

De manera general, las capas convolucionales se alternan con las capas de pooling y como salida se usan las capas densamente conectadas.

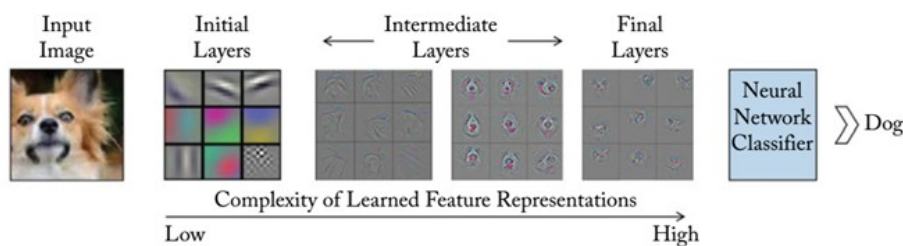


Fig. 14. Estructura general de una red convolucional.

Como puede verse en la imagen anterior, una red convolucional, a grandes rasgos estará dividida en dos partes, la primera realiza todas las operaciones de aprendizaje de los rasgos necesarios para que la segunda parte, el clasificador, pueda hacer su trabajo. La primera etapa se caracteriza por el uso de capas convolucionales y de pooling, mientras que el clasificador se construye usando capas densas

La capa convolucional

Esta capa es el elemento distintivo de las redes que llevan su nombre. Una capa de este tipo, toma como entrada una imagen con una determinada cantidad de pixels de

Tema 5. Redes neuronales y deep learning

alto, otra de ancho y un canal para cada color, RGB. Esto quiere decir que la entrada de una capa convolucional pueden ser una imagen de $1024 \times 768 \times 3$. Donde 1024 es el ancho, 768 el alto de la imagen y tres colores: Rojo, Verde y Azul.

En estas capas se realizan las operaciones de convolución usando los kernels o filtros. Por tanto, a diferencia de las capas de las redes feedforward, donde se realizan multiplicaciones de matrices, en esta se realizan las convoluciones, con los elementos de los kernel. La función de esta capa, por tanto, es detectar los rasgos que se encuentran en las regiones locales de la imagen de entrada que son comunes a todas las imágenes del dataset y mapear su apariencia a los mapas de rasgos.

El área de la imagen de entrada sobre la que se aplica el kernel o filtro se le llama campo local receptivo y tienen el mismo tamaño que este. En una capa convolucional puede haber varios filtros. (kernels).

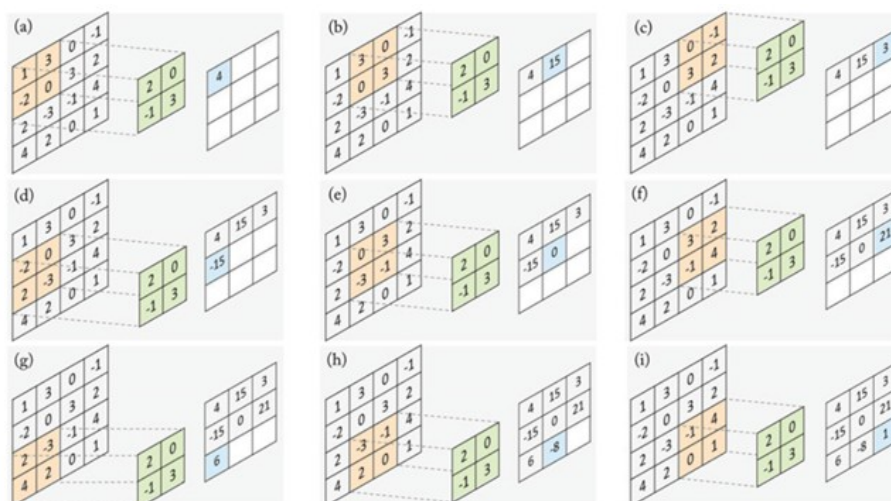


Fig. 15. Ejemplo de convolución con un Kernel de 2×2 . (Tomada de A Guide to Convolutional Neural Networks for Computer Vision).

En la imagen de la Fig 15, se muestra un ejemplo de cómo se realiza una convolución, sin embargo, cuando se trabaja con imágenes a color, estas además del ancho y del alto contienen tres canales, uno para color y por tanto los filtros

Tema 5. Redes neuronales y deep learning

tienen que ser no solo bidimensionales, sino que tienen que aplicarse de forma simultánea a cada canal. Eso implica que ahora el filtro es de $3 \times 3 \times 3$.

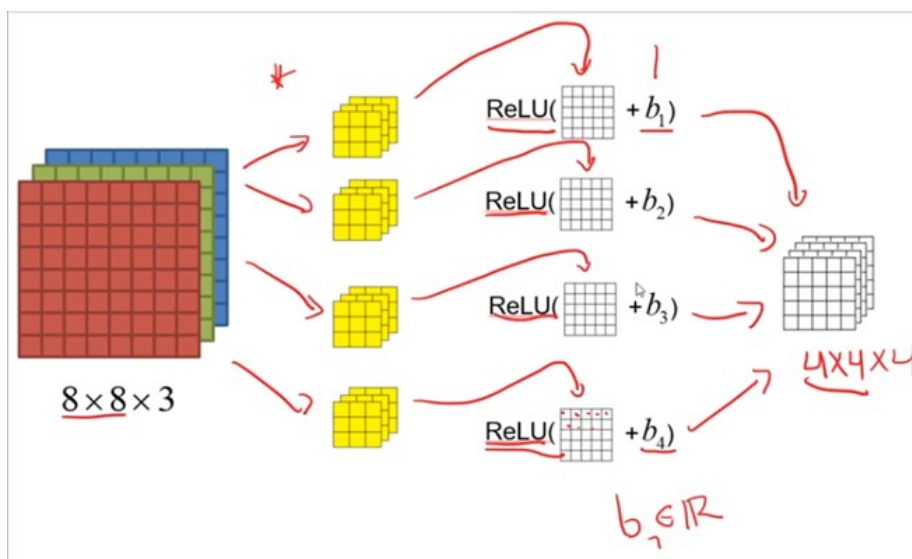


Fig. 16. Acciones que se ejecutan en una capa convolucional.

La idea básica, que es central a todo el tema de redes convolucionales, es que cada filtro, puede interpretarse como una neurona. Y se efectúa sobre los valores de entrada el mismo proceso que en una neurona regular. Sin embargo, ahora en vez de pesos, se ajustan y aprenden, por medio del algoritmo de aprendizaje, los valores de los elementos del filtro. Por tanto, cuando se aplica una imagen de entrada, se aplica inicialmente los valores del filtro a la imagen de entrada, se le añade un bias, también llamado sesgo y luego se le aplica una función de activación para obtener un tensor de salida B.

En la imagen de la Fig.16 se muestra a grandes rasgos esta idea. Para concretar esto, veamos el proceso que ocurre en una capa convolucional, a grandes rasgos:

- ▶ Se aplica la imagen de entrada.
- ▶ Se le aplican los filtros a la imagen de entrada, usualmente 10, 16 o más filtros.

Tema 5. Redes neuronales y deep learning

- ▶ Se le añade un bias al resultado de la convolución.
- ▶ Se aplica a dicho resultado una función de activación, que pueden ser tangente hiperbólica, sigmoide o ReLu.
- ▶ Se obtiene la matriz de salida B.

Ahora bien, la cantidad de parámetros que tiene que aprender una red neuronal convolucional, no depende el tamaño de la imagen de entrada, depende el número y tamaño de los filtros que se pretenden usar. Así, el número de parámetros de una capa convolucional se puede calcular de la siguiente forma:

$$n_p = (n_k^2) n_c + b_c$$

Donde n_p es el número de parámetros de la capa, n_k es la dimensión del filtro, n_c el número de filtros y b_c es el número de bias (Un bias para cada filtro).

Supongamos que tenemos 16 filtros de 3 x 3. Entonces:

$$n_p = (3^2) 16 + 16 = 160$$

Tendríamos 160 parámetros para aprender.

Para configurar, una capa convolucional, entonces tenemos, principalmente 4 hiperparámetros:

- ▶ Tamaño del kernel: Los filtros pueden tener tamaños mayores de 2 x 2 y menor que la imagen de entrada; pero los tamaños más usuales están entre 11 x 11 y 3 x 3.
- ▶ Número de kernels: Puede ser cualquier número razonable de filtros. Por ejemplo AlexNet usa, 96 filtros de 11 x 11.
- ▶ Stride: Como ya explicamos, este parámetro define la cantidad de filas o columnas que se mueve el filtro a lo largo y ancho de la imagen de entrada. Es recomendable no usar ni valores muy grandes, ni muy pequeños.

Tema 5. Redes neuronales y deep learning

- **Padding:** Como también explicamos anteriormente, este hiperparámetro define la cantidad de filas y columnas que se añadirán a la imagen de entrada para controlar el efecto de la pérdida de información producto a las operaciones de convolución.

Capa de Pooling

Esta capa tiene como función, sumarizar la información contenida en la salida de las capas de convolución. La salida de una capa de convolución es un tensor, cuyos elementos son mapas de rasgos, los cuales se combinan por medio de una función de pooling que puede ser el Máximo, como en la imagen de la fig 17, o el average.

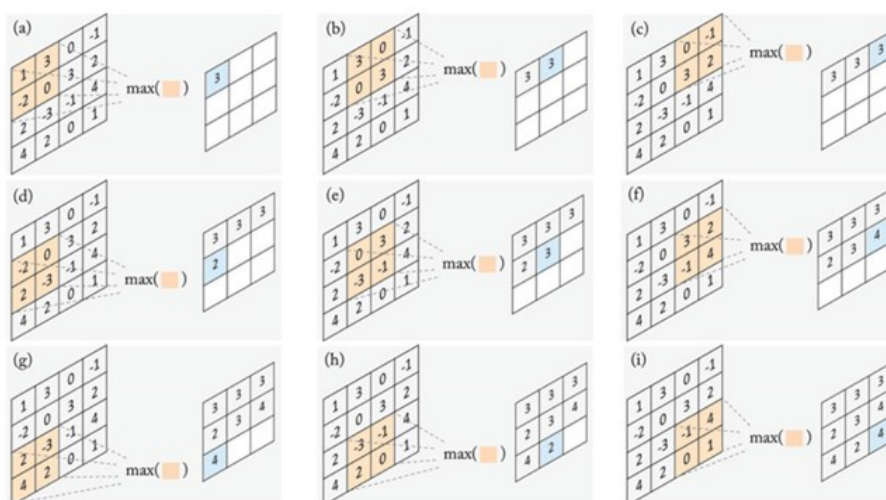


Fig. 17. Ejemplo de MaxPooling. (Tomada de A Guide to Convolutional Neural Networks for Computer Vision).

Esta operación resulta en un muestreo de los mapas de rasgos y es útil para obtener una representación compacta de los rasgos aprendidos en la capa de convolución que son invariantes, en alguna medida a la translación, la escala y la pose.

En este tipo de capa no hay aprendizaje, pero son necesarios un par de hiperparámetros y estos son el tamaño de la ventana de pooling y el stride.

Capas Densas o completamente conectadas

Tema 5. Redes neuronales y deep learning

Estas capas, son básicamente igual a las capas que ya estudiamos en los Perceptrones multicapas. En ellas cada una de las neuronas está conectada a cada una de las de la capa anterior.

Estas capas se usan en las redes neuronales convolucionales para implementar la porción del clasificador de la red, cuando se usa con el fin de clasificar imágenes.

Debido a que estas capas, se conectan con cada una de las capas anteriores, pueden contener una gran cantidad de parámetros.

No estudiaremos aquí, este tipo de capa, pues ya fueron estudiadas en el MLP.

Capas Dropout

Estas capas, cumplen una función de regularización dentro de la arquitectura de las redes neuronales convolucionales. Estas capas evitan el overfitting, eliminando algunas neuronas de forma aleatoria.

Es importante recordar que el fenómeno de overfitting se da, cuando la calidad de la clasificación de la red es superior en el conjunto de entrenamiento que en el conjunto de prueba.

Apilamiento de las capas en una red convolucional

Como vimos al principio de esta introducción a las redes neuronales convolucionales, las redes de este tipo, están compuestas por una pila de capas que se colocan una detrás de otra o si se prefiere, en el argot de las redes neuronales profundas, una encima de las otras.

Lo normal es que se coloquen capas de pooling justo luego de una convolucional, pero no es estrictamente necesario.

Se pueden apilar cualquier cantidad de capas, aunque en esta arquitectura siempre se termina con capas de tipo «completamente conectadas».

Tema 5. Redes neuronales y deep learning

A continuación veremos, a groso modo, como funciona AlexNet, con el fin de atrapar la idea de cómo se organizan las capas en una red completamente funcional y luego presentamos un ejemplo de esta para clasificar imágenes, que ustedes pueden modificar con el fin de mejorar los resultados.

AlexNet

Esta arquitectura fue desarrollada por Alex Krizhevsky y supervisada por Geoffrey E. Hinton que ganó la ILSVRC del 2012. (Large-Scale Visual Recognition Competition). Esta CNN, está formada por 8 capas, las cuales no cuentan las MaxPooling, 5 de ellas convolucionales y 3 completamente conectadas.

En la imagen de la fig. 18 se muestra la arquitectura de esta CNN y los detalles de cada capa, como número de filtros, stride, padding y el tamaño.

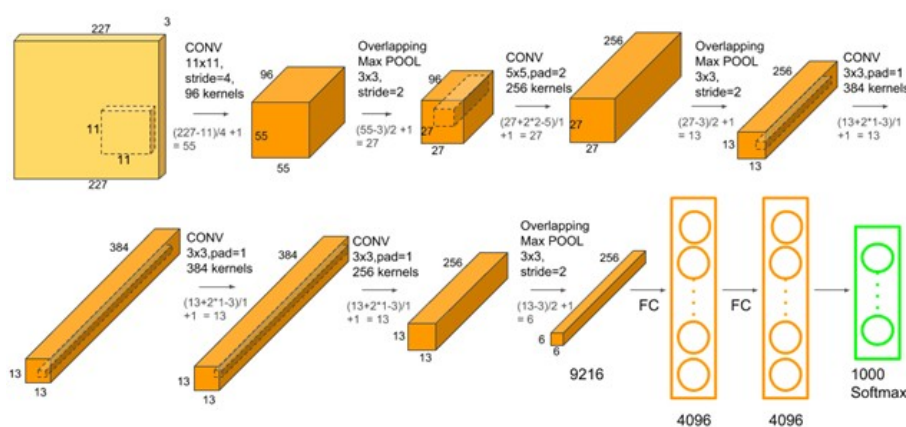


Fig. 18. Arquitectura de la CNN AlexNet original. (Tomada de <https://inblog.in/AlexNet-CNN-architecture-With-Implementation-in-Keras-Q4strWr4iZ>).

Esta red se entrenó, principalmente con el dataset ImageNet, especialmente construido para la competencia ILSVRC, sin embargo, en el ejemplo que le mostraremos, usamos el dataset CIFAR 10.

Tema 5. Redes neuronales y deep learning

El dataset, CIFAR 10, al igual que el CIFAR 100, son subconjuntos del dataset TinyImage Dataset, el cual ha sido puesto fuera de línea, por tener imágenes y nombres que han provocado determinado malestar en algunas personas. TinyImage Dataset, contaba con 80 millones de imágenes, las cuales se descargaron en formato 32 x 32 desde internet, por medio de la búsqueda a partir de nombres en WordNet. Este dataset fue recopilado por Alex Krizhevsky, Vinod Nair, y Geoffrey Hinton.

El CIFAR 10, viene como parte del módulo datasets de keras y es la versión que usamos en nuestro ejemplo. Este dataset, cuenta con 60000 imágenes a color en formato 32 x 32 que pertenecen a 10 clases diferentes. Cada clase, contiene 6000 imágenes. El dataset está dividido en 50000 imágenes para entrenamiento y 10000 para pruebas, con 1000 imágenes aleatoriamente escogidas en cada clase de test.

La arquitectura AlexNet que ganó la ILSVRC del 2012, usaba los pesos de una red pre-entrenada con la base de datos de la competencia. (ImageNet) A esto se le llama transfer learning, la cual no usamos para este ejemplo.

La red que implementamos en nuestro ejemplo, mantiene todos los parámetros de la arquitectura, descrita en el trabajo original, sin transfer Learning. El entrenamiento de esta AlexNet la hicimos en Kaggle usando GPU como acelerador.

```
import keras
from keras.models import Sequential
import tensorflow as tf
from keras.layers import Dense, Activation, Dropout, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.layers import BatchNormalization
import numpy as np
import tensorflow.keras as tk
from keras.datasets import cifar10
from sklearn.model_selection import train_test_split
from sklearn.utils.multiclass import unique_labels
from tensorflow.keras.utils import to_categorical
from keras.preprocessing.image import ImageDataGenerator
from keras.callbacks import ReduceLROnPlateau
```

Fig. 19. Implementación de una AlexNet. Primera Parte.

Tema 5. Redes neuronales y deep learning

En la imagen de la Fig. 19, mostramos la parte correspondiente a las librerías que son necesarias para la implementación de una AlexNet. En la próxima sección hablaremos de las librerías keras y tensorflow, así que por el momento no importa si aún no domina del todo los conceptos que hemos expuesto en esta introducción.

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=.3)
y_train = to_categorical(y_train)
y_val = to_categorical(y_val)
y_test = to_categorical(y_test)

train_generator = ImageDataGenerator(rotation_range=2, horizontal_flip=True, zoom_range=.1)
val_generator = ImageDataGenerator(rotation_range=2, horizontal_flip=True, zoom_range=.1)
test_generator = ImageDataGenerator(rotation_range=2, horizontal_flip=True, zoom_range=.1)

#Fitting the augmentation defined above to the data
train_generator.fit(x_train)
val_generator.fit(x_val)
test_generator.fit(x_test)
#Learning Rate Annealer
lrr = ReduceLROnPlateau(monitor='val_acc', factor=.01, patience=3, min_lr=1e-5)
```

Fig. 20. Implementación de una AlexNet. Segunda Parte.

La segunda parte, de código que usamos para implementar una AlexNet, se muestra en la imagen de la Fig.20.

En la primera línea de código se carga el dataset CIFAR 10, y se divide en un conjunto de entrenamiento y otro de prueba. Luego, el conjunto de entrenamiento se divide, a su vez, en uno de entrenamiento y el otro de validación.

Luego, se definen tres generadores, cuya función es la llamada «Data Augmentation» y tiene como función enriquecer el conjunto de imágenes del dataset, provocando, entre otras, traslaciones o rotaciones de las imágenes originales. Esto se hace por medio de la clase ImageDataGenerator().

Tema 5. Redes neuronales y deep learning

```
#####
# Definición de una AlexNet, con los valores originales      ***
#####
AlexNet = Sequential()

#1era capa Convolutiva
AlexNet.add(Conv2D(filters=96, input_shape=(32,32,3), kernel_size=(11,11), strides=(4,4), padding='same'))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
AlexNet.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='same'))

#2nd capa Convolutiva
AlexNet.add(Conv2D(filters=256, kernel_size=(5, 5), strides=(1,1), padding='same'))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
AlexNet.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='same'))

#3rd capa Convolutiva
AlexNet.add(Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), padding='same'))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))

#4th capa Convolutiva
AlexNet.add(Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), padding='same'))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))

#5th capa Convolutiva
AlexNet.add(Conv2D(filters=256, kernel_size=(3,3), strides=(1,1), padding='same'))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
AlexNet.add(MaxPooling2D(pool_size=(2,2), strides=(2,2), padding='same'))
```

Fig. 21. Implementación de una AlexNet. Tercera Parte.

En la imagen de la Fig. 21 mostramos la definición de las primeras 5 capas de nuestra AlexNet. Estas corresponden a las capas convolucionales y las capas de MaxPooling. En el parámetro padding, el valor que se le pasa (same) significa que este será calculado por la expresión:

$$p = \frac{f - 1}{2}$$

Donde f es el tamaño del filtro.

En la siguiente imagen, en la correspondiente a la Fig. 22, mostramos la definición de las capas completamente conectadas.

La primera de las capas es una capa de aplanamiento, cuya función es conectar la última capa de MaxPooling de la 5 capa convolutiva, con las completamente conectadas.

Luego, se definen tres capas densas, con 4096 neuronas las dos primeras y 1000 la

Tema 5. Redes neuronales y deep learning

tercera más la capa de salida, con función de activación softmax.

```
#Capa de aplanamiento que conecta las capas convolucionales con las Densas
AlexNet.add(Flatten())
# 1era Capa completamente conectada
AlexNet.add(Dense(4096, input_shape=(32,32,3)))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
# añade Dropout para prevenir overfitting
AlexNet.add(Dropout(0.2))

#2 da Capa completamente conectada
AlexNet.add(Dense(4096))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
#capa de Dropout
AlexNet.add(Dropout(0.2))

#3rd Capa completamente conectada
AlexNet.add(Dense(1000))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('relu'))
#Capa de Dropout
AlexNet.add(Dropout(0.2))

#Capa de salida
AlexNet.add(Dense(10))
AlexNet.add(BatchNormalization())
AlexNet.add(Activation('softmax'))

# Muestra el resumen del modelo
AlexNet.summary()
```

Fig. 21. Implementación de una AlexNet. Cuarta Parte.

A cada una de estas capas se le añaden capas de Dropout, con el fin de reducir el overfitting.

Luego de que hemos definido nuestro modelo AlexNet, para el conjunto de datos CIFAR 10, entonces lo entrenamos. Y el código para esto lo mostramos en la imagen correspondiente a la Figura 22.

Tema 5. Redes neuronales y deep learning

```
#Defining the parameters
batch_size= 100
epochs=100
learn_rate=.001
AlexNet.compile(loss = keras.losses.categorical_crossentropy, optimizer= 'adam', metrics=['accuracy'])
#Training the model
AlexNet.fit(train_generator.flow(x_train, y_train, batch_size=batch_size), epochs = epochs,
            steps_per_epoch = x_train.shape[0]//batch_size,
            validation_data = val_generator.flow(x_val, y_val, batch_size=batch_size),
            validation_steps = 250, callbacks = [lrr], verbose=1)
AlexNet.save("./AlexNet.h5")
```

Fig. 22. Implementación de una AlexNet. Quinta Parte.

El entrenamiento del modelo AlexNet, lo haremos usando un tamaño de lote igual a 100 y entrenaremos nuestro modelo en 100 ciclos que llamamos épocas. Y como algoritmo de entrenamiento usaremos Adam, una alternativa más moderna que Backpropagation o SGD. Como función de pérdida usamos Entropía cruzada categórica.

Ahora bien, introducimos en este ejemplo, el modo en que se guarda un modelo después de entrenado para poder usarlo cuando nos apetezca para hacer predicciones. Este modelo de AlexNet, demora más de media hora en entrenar usando este dataset, por tanto, lo guardamos para no tener que estar en frente de la consola esperando que se entrene.

Esta acción de guardar el modelo se hace por medio de:

```
AlexNet.save("Camino a donde se va a guardar/nombre del archivo")
```

Y luego se usa

```
AlexNet = load_model('./AlexNet.h5')
```

Para cargarlo de Nuevo y usarlo. Esto se muestra en la última parte de este ejemplo.

Tema 5. Redes neuronales y deep learning

```
from keras.models import load_model
AlexNet = load_model('./AlexNet.h5')
#Making prediction
y_pred=np.argmax(AlexNet.predict(x_test) ,axis=-1)
y_true=np.argmax(y_test,axis=1)
#Classification accuracy
from sklearn.metrics import accuracy_score
acc_score = accuracy_score(y_true, y_pred)
print('Accuracy = ', acc_score)

Accuracy Score = 0.6569
```

Fig. 23. Implementación de una AlexNet. Sexta Parte.

En la imagen correspondiente a la Fig. 23, mostramos cómo cargamos un modelo que previamente fue salvado y lo usamos para predecir.

La predicción o clasificación de una imagen con este modelo se hace por medio de:

```
AlexNet.predict("imagen")
```

Sin embargo, como usamos una capa de salida con función de activación Softmax, tenemos que usar la función `argmax()` para determinar la clase.

Hasta aquí hemos hecho una introducción a las redes neuronales convolucionales y finalizamos esta sección con la implementación de un modelo AlexNet, entrenado usando el dataset CIFAR 10.

El entrenamiento de este modelo lo hicimos en kaggle, usando como acelerador GPU. El código de este ejemplo estará disponible para que ustedes lo puedan usar y modificar.

Ahora bien, aunque siempre podemos desarrollar nuestros propios modelos usando ambientes de desarrollo como Kaggle, Google Colab e incluso Jupyter Notebooks en nuestros propios ordenadores, que son libres y ofrecen un sinnúmero de posibilidades, hacerlo no siempre es sencillo y lo usual es que consuma algún tiempo

Tema 5. Redes neuronales y deep learning

y recursos. Por eso, hay una variante, que permite que podamos usar redes preentrenadas de forma directa en nuestros proyectos o ajustarlas a nuestras propias necesidades con mucho menos esfuerzo.

A esta técnica de usar modelos preentrenados y ajustarlo a las necesidades de un proyecto, se le llama Transfer Learning y fue introducido en la literatura por Lioron Pratt en 1993. Afortunadamente Keras, que en la próxima sección estudiaremos, nos ofrece la posibilidad de hacerlo. Keras contiene 26 modelos de redes neuronales profundas preentrenadas.

A continuación, mostramos una tabla con estos modelos.

Nombre del Modelo	Tamaño	Top-1 Accuracy	Top-5 Accuracy	Número de Parámetros	Profundidad
Xception	88 MB	0.79	0.945	22,910,480	126
VGG16	528 MB	0.713	0.901	138,357,544	23
VGG19	549 MB	0.713	0.9	143,667,240	26
ResNet50	98 MB	0.749	0.921	25,636,712	-
ResNet101	171 MB	0.764	0.928	44,707,176	-
ResNet152	232 MB	0.766	0.931	60,419,944	-
ResNet50V2	98 MB	0.76	0.93	25,613,800	-
ResNet101V2	171 MB	0.772	0.938	44,675,560	-
ResNet152V2	232 MB	0.78	0.942	60,380,648	-
InceptionV3	92 MB	0.779	0.937	23,851,784	159
InceptionResNetV2	215 MB	0.803	0.953	55,873,736	572
MobileNet	16 MB	0.704	0.895	4,253,864	88
MobileNetV2	14 MB	0.713	0.901	3,538,984	88
DenseNet121	33 MB	0.75	0.923	8,062,504	121
DenseNet169	57 MB	0.762	0.932	14,307,880	169
DenseNet201	80 MB	0.773	0.936	20,242,984	201
NASNetMobile	23 MB	0.744	0.919	5,326,716	-
NASNetLarge	343 MB	0.825	0.96	88,949,818	-
EfficientNetB0	29 MB	0.771	0.933	5,330,571	-
EfficientNetB1	31 MB	0.791	0.944	7,856,239	-
EfficientNetB2	36 MB	0.801	0.949	9,177,569	-
EfficientNetB3	48 MB	0.816	0.957	12,320,535	-
EfficientNetB4	75 MB	0.829	0.964	19,466,823	-
EfficientNetB5	118 MB	0.836	0.967	30,562,527	-
EfficientNetB6	166 MB	0.84	0.968	43,265,143	-
EfficientNetB7	256 MB	0.843	0.97	66,658,687	-

Tabla 1. Modelos preentrenados disponibles en keras.applications. Fuente: elaboración propia.

En el siguiente ejemplo mostramos la manera en que se usa un modelo

Tema 5. Redes neuronales y deep learning

preentrenado VGG16 para clasificar una imagen de un colibrí descargada al azar de internet.



Fig. 24. Imagen de colibrí, usada en el ejemplo de clasificación de imágenes con VGG16.

En el ejemplo que mostramos en la imagen de la Fig. 25, inicialmente se cargan las librerías que nos harán falta, como es usual en todos los ejemplos que hemos presentado. Aquí, lo más importante es que noten que el modelo preentrenado está en la librería `keras.applications` y se importa de la siguiente forma:

```
from keras.applications.vgg16 import VGG16
```

Tema 5. Redes neuronales y deep learning

```
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.applications.vgg16 import preprocess_input
from keras.applications.vgg16 import decode_predictions
from keras.applications.vgg16 import VGG16
# cargar el modelo
model = VGG16()
# cargar imagen desde el archivo
image = load_img('../input/test-image/ave.jpeg', target_size=(224, 224))
# remodelar los datos para el modelo
image = img_to_array(image)
# preparar la imagen para el modelo VGG
image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
# predecir la probabilidad en todas las clases de salida
image = preprocess_input(image)
# convertir las probabilidades en etiquetas de clase
yhat = model.predict(image)
# convert the probabilities to class labels
label = decode_predictions(yhat)
# recuperar el resultado más probable, por ejemplo, el más alto
label = label[0][0]
# imprimir la clasificación
print('%s (%.2f%%)' % (label[1], label[2]*100))

hummingbird (98.95%)
```

Fig. 25. Ejemplo de uso de un modelo preentrenado para clasificar imágenes.

Al final de la imagen, se muestra el resultado de la clasificación de la imagen de ejemplo que usamos. Esto quiere decir que el clasificador nos dice que la imagen pertenece a un colibrí con una probabilidad del 98,95%.

Como puedes darte cuenta a mil kilómetros por horas, si usas este ejemplo, has implementado un clasificador de imágenes con una calidad impresionante en solo 13 líneas de código. Sencillamente espectacular, ¿verdad?

Bueno, no es tan simple, estamos usando una red con 138,357,544 parámetros. Leamos esto despacio, 138 millones, 357 mil, 544 parámetros que la red aprendió usando al menos un millón de imágenes. El modelo solamente, pesa 528 Megabytes y se descarga de internet en el momento de usar el modelo por primera vez y se almacena de forma local.

Hasta este punto, hemos recorrido las redes neuronales recurrentes profundas y

Tema 5. Redes neuronales y deep learning

hemos hecho una breve introducción a las redes neuronales convolucionales. Sin embargo, aún nos queda tocar lo referente a los frameworks más usados para el desarrollo de los modelos de *deep learning*, así como algunos de los ambientes de trabajo. Este breve recorrido por este tema lo haremos en la siguiente sección.

Frameworks para Deep Learning

Aunque metodológicamente, este tema debimos tocarlo al principio de esta lección, desde una perspectiva práctica era más lógico continuar con las redes neuronales recurrentes profundas como LSTM y GRU, pues habíamos finalizado la lección anterior tratando las redes de Hopfield y Elman, consideradas simples y superficiales. Esto nos daba la oportunidad de pasar de las redes neuronales superficiales a las profundas sin un salto drástico, es decir no tratar a las redes LSTM y GRU como algo distinto de las Redes neuronales superficiales, sino verlas como una continuación en su desarrollo.

Uno de los factores, que, sin lugar a duda, ha contribuido de forma importante al auge de las redes neuronales profundas, es la disponibilidad tanto, de ambientes de desarrollo, como de frameworks, la mayoría de ellos libres.

La función de dichos frameworks, es facilitar las tareas de diseño, entrenamiento, validación y despliegue de modelos de deep learning. Con este objetivo como meta, muchos gigantes tecnológicos han desarrollado sus propios frameworks que luego han abierto al público general y otros han sido desarrollados como parte de proyectos abiertos.

A continuación, listamos algunos de los más usados y de ninguna forma es una lista completa, ni el orden en que los mostramos es una medida de su popularidad. Esta sección es más informativa que un análisis de las características de cada uno de ellos.

- ▶ Tensorflow.

Tema 5. Redes neuronales y deep learning

- ▶ Keras.
- ▶ Pytorch.
- ▶ Mxnet.
- ▶ CNTK.
- ▶ DL4J
- ▶ Caffe.



TensorFlow

Este es, probablemente, el framework de deep learning más usado en la actualidad. Desarrollado por el Google Brain Team y liberado el 9 de noviembre de 2015. En el momento en que se escribe esta conferencia la versión estable es la 2.4.1.

Tensorflow, tiene interfaces en C++, Java, Javascript, Go y Julia, pero el lenguaje usado por excelencia es el Python. Este framework soporta los sistemas operativos, linux, windows, macos y android. Está escrito en python, C++ y CUDA.

En tensorflow, los modelos de aprendizaje automático son vistos como grafos computacionales, donde los vértices o nodos, son operaciones y las aristas son los flujos de los datos entre estas operaciones. Cada variable que usamos en este framework tendrá forma de tensor y de ahí su nombre. Como ya hemos explicado en alguna parte de este trabajo, un tensor es una generalización del concepto de matriz.

Tema 5. Redes neuronales y deep learning

Además de la versión local de tensorflow, este tiene una versión distribuida que permite entrenar modelos sobre clusters de GPU.



Keras

Es un framework escrito en Python por Francois Chollet de Google como parte del proyecto ONEIROS (**Open-ended Neuro-Electronic Intelligent Robot Operating System**). Está concebido para simplificar las tareas de desarrollo de modelos de *Deep Learning*, ofreciendo una API simplificada.

Keras funciona como una capa por encima de otros frameworks como Tensorflow, Theano y CNTK. Los cuales, por medio de configuración simple, pueden cambiarse o incluso combinarse.

Una de las ventajas de usar Keras es que soporta múltiples plataformas, los modelos desarrollados en keras pueden desplegarse, por ejemplo, en un Apple CoreML, o en Android usando Keras Tensorflow Android Runtime, se pueden desplegar en Google Cloud, incluso correrlos en un Raspberry Pi.

Otra de las ventajas es el soporte para múltiples GPU, lo que facilita el entrenamiento de modelos grandes, dividiendo el trabajo entre múltiples tarjetas gráficas y sobre muchos servidores.

El mantenimiento de Keras está a cargo de muchos de los gigantes tecnológicos, como google, Microsoft, Amazon AWS, Apple y Nvidia.

Keras, ofrece dos formas de crear modelos de *Deep Learning*, uno por medio del modelo Sequential y otro por medio de lo que se conoce como API funcional de

Tema 5. Redes neuronales y deep learning

Keras.

En todos los ejemplos que hemos utilizado en este trabajo, usamos el modo Sequential, en el cual cada capa se va declarando en el orden en que en definitiva tendrá dentro del modelo. Se inicia con la definición de la capa de entrada y sus parámetros y luego las capas ocultas o intermedias y finalmente se declara la capa de salida.

En esta forma de definir un modelo, luego de que este ha sido completamente definido, se compila y luego se entrena. Cuando concluye el entrenamiento el modelo puede guardarse para luego desplegarlo o simplemente se usa para predecir.



Pytorch {XE “PyTorch” if “Definido por el usuario” }

Este es un framework desarrollado por Facebook AI Research Lab. (FAIR) Su liberación inicial fue en septiembre del 2016. Se basa en Torch y puede considerarse como un front end de este, basado en Python.

Este framework ofrece soporte para aceleración por medio de GPU, integración con Numpy y diferenciación automática. Este ha sido una pieza clave en el desarrollo de aplicaciones de Deep Learning, como el famoso y controvertido AutoPilot de Tesla.

Este framework ofrece dos modos de trabajo, uno llamado «eager» que básicamente se utiliza en el proceso de desarrollo y un modo llamado «grafo» que en algunos artículos se traduce como gráfico y que se usa mayormente para producción por su velocidad.

Tiene soporte nativo para ONNX, el estándar de intercambio de redes neuronales abierto, lo que hace que los modelos desarrollados con Pytorch se pueden desplegar

Tema 5. Redes neuronales y deep learning

en sistemas compatibles con este estándar. ONNX fue creado por Facebook, actualmente llamada Meta, y Microsoft, con el fin de poder aumentar la interoperabilidad de los modelos, en 2017.

Aunque la interfase en Python es la más pulida, Pytorch también tiene una interfase C++.

Al igual que Keras, tiene varios modelos preentrenados que se pueden usar de forma inmediata en sus proyectos de Deep Learning.



MXNET

Mxnet, es la plataforma de código abierto de Apache para Deep Learning. Este framework está, explícitamente, desarrollado con escalabilidad y eficiencia en mente. Está construido sobre CUDA y desarrollado en C++. Sin embargo, tiene soporte para Python, C++, Julia, Javascript, Go, R, scala, Perl.

Este framework, ha sido elegido por Amazon, para sus servicios de Deep Learning y puede usarse en Sagemaker, la suit de desarrollo de Aprendizaje automático del gigante.

Aunque se ha trabajado en el uso de keras como front end de mxnet, ha sido Apache Gluon, la interfase que más atención ha recibido. Gluon, les ofrece a los desarrolladores de modelos, los bloques constructivos necesarios para su trabajo, como capas predefinidas, funciones de pérdidas y optimizadores. Con esta interfase, se puede combinar el estilo de programación imperativo y el simbólico, obteniendo lo mejor ambos mundos.

La lista de frameworks para redes neuronales y en especial para *Deep Learning* ha

Tema 5. Redes neuronales y deep learning

crecido un montón y nos deja con muchas opciones. Aunque desde el punto de vista práctico es recomendable dominar uno, es básico para los científicos de datos o Ingenieros de *Machine Learning*, seguir los desarrollos de equipos de investigadores y entender su código podría suponer aprender el framework que los soporta. Por eso no es descabellado, coquetear con algunos de los más prometedores.

No tocaremos el resto de los frameworks de la lista anterior, pues la documentación de ellos es abundante y disponible libremente en internet. Nos concentramos en este trabajo en Keras/Tensorflow.

Dicho esto, pasamos a la última parte de esta conferencia, en la cual trataremos, aunque de forma somera lo que respecta al aprendizaje con reforzamiento profundo.

Aprendizaje con refuerzo

En años recientes hemos sido testigos de acontecimientos en el mundo de la Inteligencia Artificial y en Aprendizaje automático que han dejado estupefactos a más de uno. Noticias como la victoria de AlphaGo sobre alguno de los mejores maestros de Go del mundo, como el 9 dan Coreano Lee Sedol. El 19 de noviembre de 2019 Sedol, anunció su retirada del juego activo profesional por el simple hecho de que «ya no podría ser el mejor jugador del mundo debido a la dominación de Agentes con IA» y se refirió a ellos como “una entidad a la que no puede derrotarse». Sin embargo, muy pronto el algoritmo extendido de AlphaGo, llamado AlphaZero, venció a este, 100 a 0.

El 13 de abril de 2019, OpenAI Five, un agente de AI embebido en Dota2, venció al equipo campeón mundial de Juegos Electrónicos, el Team OG. Este agente ganó el 99.4% de aproximadamente 7000 juegos. Estos dos agentes se basan en Aprendizaje con Reforzamiento.

En este tipo de aprendizaje, a un Learner o aprendiz, que en lo adelante llamaremos agente, se le permite interactuar libremente con el ambiente. En cada instante de

Tema 5. Redes neuronales y deep learning

tiempo t , el agente muestrea el estado del ambiente y en función de este, toma decisiones, a partir de un conjunto de ellas disponibles para ese estado. Como resultado de las acciones, el ambiente retorna una recompensa y cambia a otro estado, repitiéndose el ciclo: estado, acción, recompensa y nuevo estado.

Esta recompensa puede ser positiva y en ese caso hablamos de aprendizaje con refuerzo positivo, o negativa y entonces hablamos de aprendizaje con refuerzo negativo y que se asocia con una penalización o castigo.

Esta recompensa, puede verse como un valor escalar, es decir, un valor numérico y podría tomar valores positivos para aumentar la recompensa o negativo para disminuirla.

El objetivo del accionar del agente dentro del ambiente es maximizar la recompensa acumulada a largo plazo.

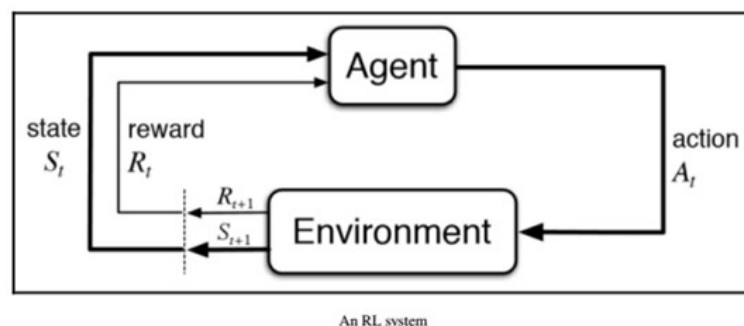


Fig. 26. Aprendizaje con reforzamiento.

De manera general, en aprendizaje con reforzamiento estaremos tratando con conceptos tales como agente, recompensa, ambiente o entorno, estado, función de valor y política, por tanto, sería conveniente que definir cada uno de estos conceptos con el fin de entender de qué hablamos.

Agente

Tema 5. Redes neuronales y deep learning

El elemento central de todo el aprendizaje con reforzamiento es el agente. Un agente, no es más que un programa de ordenador o robot, al que debemos dotar de los mecanismos necesarios para accionar en su ambiente y a partir del cual, florezca una conducta inteligente. Estos deben, al interactuar con el ambiente, y tomar decisiones, alcanzar una meta. En aprendizaje con reforzamiento, las decisiones de que acción tomar se basan en la recompensa recibida por el agente.

Un agente es una entidad que actúa basado en las observaciones
(percepción) de su ambiente.

Los humanos y los robots son ejemplos de agentes. Sin embargo, los agentes con los que trataremos en esta introducción son programas de ordenador, que toman decisiones a través de la interacción con su ambiente.

Todo el Aprendizaje con reforzamiento está basado en la «hipótesis de la recompensa» que plantea que cualquier objetivo/meta puede ser alcanzado por medio de la maximización de la recompensa esperada.

Recompensa

La recompensa, es dada al agente como respuesta a una acción tomada en un instante de tiempo t . El valor de esta recompensa habla de cuan bien o mal el agente lo hizo al elegir la acción.

Esta recompensa tiene forma de valor numérico y la meta del agente es maximizar su suma a largo plazo. Por ejemplo, una recompensa puede verse como la cantidad de puntos alcanzados en un juego de Atari. En Trading, la recompensa puede ser +1 para cada dólar ganado y -1 por cada uno perdido.

Las recompensas, pueden ser inmediatas o puede retrasarse en el tiempo. Por ejemplo, si el agente aprende un juego de mesa como el Go o el ajedrez, la

Tema 5. Redes neuronales y deep learning

recompensa puede ser +1 por cada juego ganado. Esto significa, que el agente no tiene realimentación por cada movimiento hecho y tiene que esperar al final del juego para obtenerla.

Ambiente o entorno

Este, representa el problema o la tarea que tratamos de resolver. Si nos enfrentamos a la tarea de conducción automática de un coche, este representa el coche en sí mismo y el entorno en que se mueve, con todas las variables provenientes de los sensores y las variables que usamos para controlarlo.

En caso de un juego de Atari, por ejemplo, la consola es el ambiente, y recibimos de él, en el caso de un juego de tenis, la posición de la bola y la de la raqueta. Las acciones con las que podemos controlar la posición de la raqueta se las pasamos de alguna forma y recibimos de él, el resultado de nuestra acción.

Estado

El estado, es una especie de instantánea de las propiedades del ambiente en un instante de tiempo t . El estado de un ambiente es lo que sabemos de él en un instante de tiempo determinado y es usado por el agente para tomar decisiones. En dependencia de su nivel de visibilidad, los ambientes pueden clasificarse en: completamente observables, parcialmente observables.

En un ambiente completamente observable, el estado de este está completamente disponible para los agentes, en cambio, en uno parcialmente observable, solo se tiene acceso a parte de este.

Tanto el ambiente como el agente tienen su propia representación del estado.

Para desarrollar la intuición de lo que representa un ambiente parcialmente observable, imagine que el agente trata de aprender a jugar cartas, en cada instante de tiempo, él solo tiene acceso a sus propias cartas y no a las de sus oponentes.

Tema 5. Redes neuronales y deep learning

Esto significa que tiene información limitada y por tanto el ambiente es parcialmente observable.

Modelo

El modelo, es la representación interna que va construyendo un agente, de su ambiente. Entre más detallado y preciso sea el modelo, mejores acciones se tomarán durante cada momento y un estado determinado.

Un modelo, permitiría a un agente predecir con certeza el estado en que estaría el ambiente si se tomará una determinada acción.

Un modelo está compuesto por dos piezas claves:

- ▶ Un modelo de transición entre estados.
- ▶ Un modelo de recompensa.

El modelo de transición entre estados podría ser o una distribución de probabilidades o una función que predice la probabilidad de pasar al estado S_1 en el próximo momento $t+1$ dado el estado S y la acción A en el tiempo t .

Por otro lado, el modelo de recompensa R , predice la próxima recompensa inmediata que se obtendría si tomáramos la acción A mientras estamos en el estado S en el momento t .

Los métodos que usan modelos del ambiente se denominan Métodos Basados en Modelos, mientras los que no se denominan Model-free o libres de modelo.

Función de valor

Una función de valor es aquella que permite determinar de forma cuantitativa el valor de un estado o de una acción. Hay dos tipos de funciones de valor:

- ▶ Funciones de estado-valor. (V_s)

Tema 5. Redes neuronales y deep learning

► Funciones de acción-valor. ($Q(s,a)$)

En la primera se representa el estimado del agente de cuan bueno sería estar en el estado S en el tiempo t . Es decir, es la predicción del agente de la recompensa obtenida si se terminara en el estado S . En cambio, la segunda, es el estimado del agente de cuan bueno sería tomar la acción a , en el estado S .

Es importante que distingamos de manera clara lo que se entiende por recompensa y lo que se entiende por valor. Cuando tomamos decisiones y elegimos acciones, no lo hacemos basados en la recompensa en sí misma, sino en el valor del estado o de la acción, pues ese nos traerá mayor recompensa.

A diferencia de la recompensa, que se da de forma directa por el ambiente, las estimaciones de valor son mucho más complicadas de obtener, pues hay que estimarlas a partir de la secuencia de observaciones que toma el agente a lo largo de su ciclo de vida.

De hecho, uno de los principales componentes de un modelo de Aprendizaje con reforzamiento es el método por medio del cual estimamos de manera eficientes esos valores.

Políticas

Una política, describe que acción es tomada para cada estado S . Esta puede verse como una función que mapea, estados a acciones. Hay dos tipos de Políticas: Las deterministas y las estocásticas.

En una política determinista, hay una y solo una acción a para cada estado S . en cambio, en una política estocástica para cada estado S hay múltiples acciones con un valor de probabilidad para cada una.

Esto es lo que en psicología se llama reglas de estímulos-respuestas o asociaciones.

Tema 5. Redes neuronales y deep learning

La conducta de un agente, en un determinado ambiente, depende de la política escogida.

Procesos de decisión de Markov. (MDP: Markov decision Process)

Está claro de la idea de interacción en cada instante de tiempo con un ambiente y la toma de decisiones del agente basado en el estado de este, que este tipo de problema puede verse como uno de toma de decisiones secuenciales y por tanto puede modelarse como un Proceso de Decisión de Markov.

Un Proceso de Decisión de Markov, puede entenderse como una n-tupla (S, A, R, P) formada por los siguientes elementos:

- ▶ Un conjunto no vacío S, llamado espacio de estados.
- ▶ Un conjunto no vacío A, llamado espacio de acciones.
- ▶ Una función $R: S \times A \rightarrow R$ conocida como función de recompensa esperada.
- ▶ Una función $P: S \times A \rightarrow P(S)$ donde $P(S)$ es el espacio de distribuciones de probabilidad sobre S. Se llama función de probabilidad de transición. Para cada par (s, a) nos retorna una distribución de probabilidad sobre los estados.

Un proceso de Markov, adquiere su nombre a partir de que cumpla con la propiedad de Markov. En un proceso que cumple con esta, el próximo estado o estado futuro, se basa, exclusivamente en el estado presente. Por tanto, la propiedad de Markov reza: «El estado futuro, en un proceso de Markov, solo depende del estado actual, pues este ya contiene la información suficiente del pasado».

Esa propiedad se expresa matemáticamente de la siguiente forma:

$$P(s_{t+1} | s) = P(s_{t+1} | s_1, \dots, s_t)$$

Por tanto, la probabilidad de transitar de un estado a otro es:

Tema 5. Redes neuronales y deep learning

$$P_{ss'} = P(S_{t+1} = s' \vee S = s)$$

Donde $P_{ss'}$ es la probabilidad de transición entre el estado s a el estado s' .

Finalmente, R , es la recompensa que recibimos al transitar de un estado s a un estado s' . Y que matemáticamente podemos expresar como sigue:

$$R_s = E[R_{s+1} \vee s]$$

Que significa que R_s , la recompensa del estado s es el valor esperado de estar en ese estado. R es la función de recompensa. Y la llamada recompensa acumulada con descuento sería:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{k-1} R_{t+k}$$

Donde G_t es la recompensa total esperada con descuento o descontada y γ se llama factor de descuento y toma valores entre $[0,1]$.

El horizonte de tiempo desde $T=0$ hasta cuando el ambiente termina se llama episodio. Mientras que a la tupla (s_t, a_t, r_t) se le llama experiencia. Esto quiere decir que una experiencia está compuesta por un estado, la acción que se tomó y la recompensa que se obtuvo de su ejecución.

Una trayectoria es una secuencia de experiencias, sobre un episodio:

$$t =$$

Un agente, normalmente, necesita muchos episodios para aprender una política óptima.

El retorno, es la suma total de la recompensa acumulada durante un episodio y la meta u objetivo de un agente RL es maximizar el retorno. Matemáticamente, entonces, el retorno puede expresarse de la siguiente manera:

Tema 5. Redes neuronales y deep learning

$$R_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots + \gamma^T r_T = \sum_{t=0}^T \gamma^t r_t$$

Donde γ es el factor de descuento y toma valores en el intervalo $[0,1]$. Mientras este es más pequeño menos peso se le asigna a las recompensas futuras, y viceversa.

Conociendo el retorno, entonces podemos definir cómo sería el objetivo a seguir por un agente y toma la forma siguiente:

$$j(T) = E[R_t] = \sum_{t=0}^T \gamma^t r_t$$

Esto lo que quiere decir es, que nuestro objeto a maximizar es el valor esperado del retorno. Lo que también quiere decir que el valor esperado es igual al promedio del retorno tomado de muchos episodios.

Compromiso exploración-explotación

Usualmente explotación significa usar el conocimiento existente para maximizar la efectividad del agente. Esta efectividad es evaluada en función de la recompensa esperada. Esto significa que el agente siempre usa la acción que más recompensa le genere con vistas a acumular la mayor cantidad posible en una secuencia. Por otra parte, la exploración, significa la búsqueda de otras acciones que acometan mayores recompensas y aumenten el conocimiento existente.

Un agente se enfrenta a un compromiso entre explotar el conocimiento que tiene sobre las acciones que más recompensa le generan y explorar otra que aún no conoce. Por tanto, se han desarrollado políticas que aunque la mayor parte del tiempo están ejecutando acciones de las cuales se conoce de antemano, por la experiencia acumulada, o por una estimación y de forma ocasional exploran otras.

En Aprendizaje con Reforzamiento se pueden aprender tres tipos de elementos:

- La Política.

Tema 5. Redes neuronales y deep learning

- ▶ La función de Valor.
- ▶ Un modelo.

Todos los algoritmos que se han desarrollado dentro de este campo, se pueden clasificar por el tipo de elemento que aprenden. De esta forma podemos esbozar una taxonomía que de alguna forma nos da la idea de los métodos que se usan para resolver el problema de RL. Teniendo en cuenta lo anterior, podemos decir que la división básica de los algoritmos de Aprendizaje con reforzamiento es entre los que aprenden un modelo o se les da y se denominan métodos o algoritmos basados en modelos y aquellos que no aprenden un modelo o no lo usan y que se denominan métodos *model-free*.

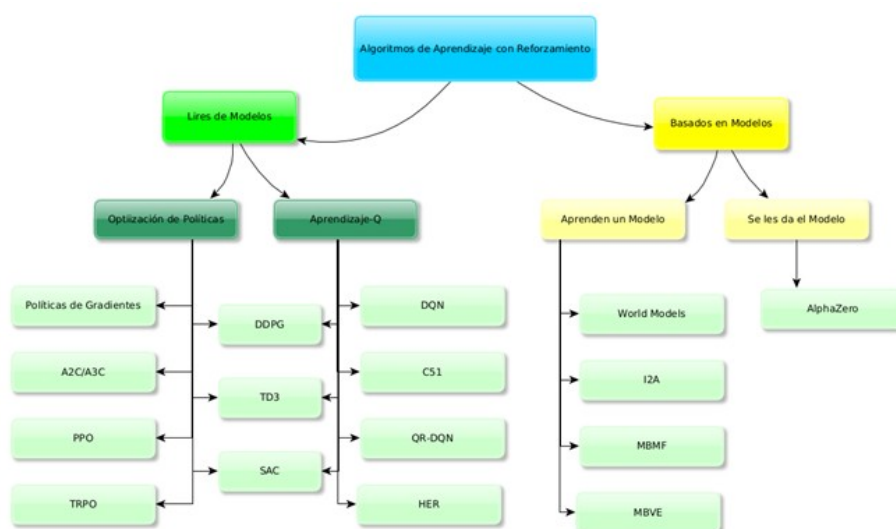


Fig. 27. Una primera aproximación a la clasificación de los métodos de Reinforcement Learning.

En la fig.27, se muestra una primera aproximación de una de las posibles clasificaciones de los algoritmos de Reinforcement Learning. Específicamente la que separa a los algoritmos en dos clases basado en el aprendizaje o no de un modelo. Pero ¿qué se entiende por modelo en RL? Para explicar esto primero hay que recordar que un MDP o Proceso de Decisión de Markov, está definido por una n -

Tema 5. Redes neuronales y deep learning

tupla (S,A,P,R,γ) . Donde S es el espacio de todos los estados por los que puede transitar el proceso, A es el espacio de todas las acciones disponibles, P es la función de transición entre estados y especifica la probabilidad de que el proceso haga una transición del estado s al s' basado en la acción a tomada en el estado s . R , es la función de recompensa y γ es llamado factor de descuento.

Para un agente que aprende por reforzamiento, los espacios de estado S y de acciones A y el factor de descuento γ son conocidos de antemano, sin embargo, la función de transición entre estados, que encierra la dinámica del proceso y la función de recompensa son desconocidos. Los métodos basados en modelo interactúan con el ambiente hasta que se consigan suficientes muestras con el fin de poder derivar las funciones de transferencia $P(s' \mid a, s)$ y la de recompensa $R(a, s)$. A estas funciones derivadas es a lo que se le llama modelo y métodos basados en modelo a aquellos algoritmos que los aprenden.

Los métodos basados en modelos se dividen, a su vez, en los que aprenden propiamente el modelo y aquellos a los que el modelo se les da, por medio o algún mecanismo.

Por ejemplo, AlphaGo, es un método que pertenece a la clase de los basados en modelo; pero a los cuales el modelo se les da ya completamente especificado. En este método, las reglas del juego GO, se le dan codificadas en el mismo lenguaje de programación.

En el grupo de los que aprenden el modelo, están algoritmos como I2A (Modelo llamado Imaginación Aumentada), World Models y otros. Estos métodos, inicialmente derivan el modelo interactuando con el ambiente y luego lo usan con el fin de optimizar su política.

Una de las ventajas de usar un modelo, es que permite anticipar la recompensa y los estados futuros, lo que hace que se pueda planificar mejor la secuencia de acciones

Tema 5. Redes neuronales y deep learning

que optimicen la recompensa esperada. Sin embargo, uno de los problemas de este tipo de modelo es que usualmente el problema de derivar un modelo a partir de la interacción con el ambiente es difícil, debido a la complejidad de su dinámica. Esto hace que los modelos derivados sean inseguros y sesgados, provocando que las políticas derivadas también lo sean y que muchas veces colapsen en la práctica.

Por otro lado, están los métodos que no aprenden un modelo del ambiente, sino que usan los valores de S , A y r de forma directa. A este tipo de modelos es a los que se les denomina model-free o libres de modelos. Ejemplos de modelos que se clasifican dentro de este grupo están: Deep Q Network o DQN, Los métodos de Política Gradiente PG, Deep Deterministic Policy Gradient o DDPG, entre otros. Sin embargo, este tipo de modelo también sufre de algunas restricciones, pues a la hora de entrenar a un agente, digamos, un conductor automático en un ambiente real, puede que sufra de accidentes al tener que tomar acciones al azar para explorar el ambiente.

Los métodos model-free, pueden dividirse además en dos grandes grupos basados en el tipo de función que aprenden: basados en valor y los basados en política. Sin embargo, hay método que pueden considerarse híbridos, al aprender ambas funciones, por ejemplo, el método Actor-Critic.

Los métodos basados en valor, usualmente aprenden a optimizar una función $Q(s,a)$, que mapea acciones a estados. Las ventajas de este tipo de modelo es que son más eficientes a la hora de aprender el modelo, en cuanto al número de muestras que necesitan para aprenderlo, no se quedan atascados en mínimos locales y la varianza de la función de estimación es baja.

Los métodos más conocidos de este apartado son los métodos como Q-Learning, SARSA, DQN, Dueling DQN, los métodos de Aprendizaje por diferencias temporales, etc.

Sin embargo, estos modelos no pueden manejar problemas donde el espacio estado-

Tema 5. Redes neuronales y deep learning

acción sea continuo, entre otros problemas.

Los métodos basados en política tratan de optimizar la política, valga la redundancia, de forma directa, por ejemplo, actualizándola de forma iterativa, hasta que se maximiza el retorno acumulado. Algunos de los algoritmos más conocidos que se clasifican bajo esta denominación están: PPO, TRPO y PG.

Sin embargo, métodos híbridos, se ha hecho populares, entre ellos el más popular es el marco Actor-Critica. Esta combina los méritos de los métodos basados en Valor y los métodos basados en política. Aunque estos métodos heredan las ventajas de sus correspondientes partes, también heredan sus desventajas. En este caso han aparecido variantes de AC usando Deep Learning, como A3C, DDPG, TD3 y SAC.

Aunque no la hemos reflejado en la taxonomía de la fig.27, existe una clasificación extra y que se trata en la mayoría de la literatura y por tanto la vamos a explicar de manera muy escueta en esta introducción y es la clasificación de los métodos en Monte Carlos y métodos de aprendizaje por Diferencias Temporales. (MC y TD).

Los métodos de aprendizaje por Diferencias Temporales, es un intermedio entre la Programación Dinámica y los Métodos de Monte Carlos. Tanto Programación Dinámica (DP) como TD (Diferencias Temporales) usan Bootstrapping para la estimación. Ni TD, ni MC necesitan un conocimiento completo del ambiente, como DP. Lo que hace diferente a MC de TD es la manera en que se actualiza el conocimiento. Mientras que MC tiene que esperar a que termine el episodio para actualizar, TD actualiza en cada paso de tiempo. Esto hace que TD tenga sesgos más grandes, mientras los métodos MC tengan mayor varianza.

Tema 5. Redes neuronales y deep learning

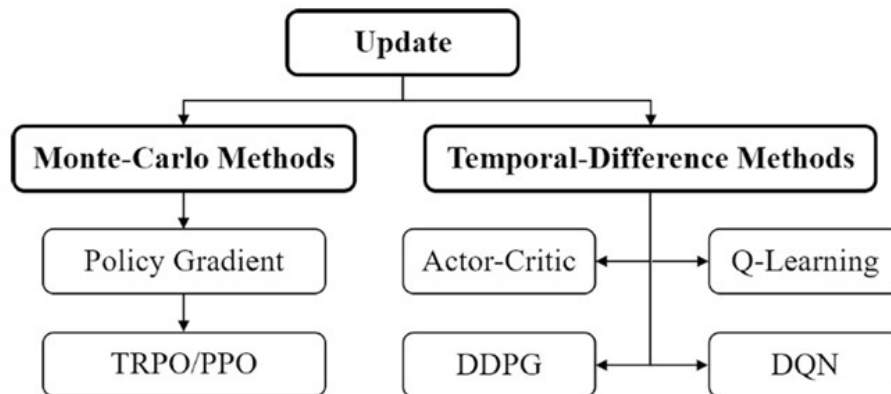


Fig. 28. Clasificación según el tipo de Update.

Otra de las formas de clasificar los algoritmos y métodos del Aprendizaje con Reforzamiento es basado en la actualización de la política y se dividen en dos grupos, los métodos llamados On-Policy y los llamados Off-Policy.

Los métodos que pertenecen al grupo On-policy, evalúan y mejoran la misma política que es usada para tomar las decisiones. Mientras que los métodos Off-Policy, evalúan o mejoran una política diferente de la que es usada para generar los datos. Los primeros, los On-Policy, requieren que el agente en sí mismo interactúe con el ambiente, es decir, que la política que interactúa con el ambiente y la que se pretende mejorar, tiene que ser la misma. Los métodos, Off-Política, por el contrario, no tienen que ceñirse a esa restricción e incluso la experiencia de otro agente interactuando con el ambiente, puede utilizarse para actualizar la política de otro.

Un método, muy conocido de los On-Policy es el llamado SARSA, que su nombre indica, Estado, Acción, Recompensa, Estado, Acción. Este método, selecciona una acción, basado en la política actual, la ejecuta y entonces actualiza la política. Esto quiere decir que la política que se usa para elegir la acción y la que se actualiza es la misma.

Este método actualiza la función Q de la siguiente forma:

Tema 5. Redes neuronales y deep learning

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R_t + \gamma Q(S_{t+1}, A_{t+1}) - Q(S,A)]$$

Por otra parte, un método muy conocido de los que se clasifican como Off-Policy es Q-Learning. En este tipo de algoritmo la política que se actualiza y la que se aprende no es la misma, ya que este adopta una operación Max y una política $\epsilon - greedy$ cuando se están eligiendo las acciones.

Q-Learning actualiza la función Q de la siguiente forma:

$$Q(S,A) \leftarrow Q(S,A) + \alpha [R_t + \gamma \max Q(S_{t+1}, A_{t+1}) - Q(S,A)]$$

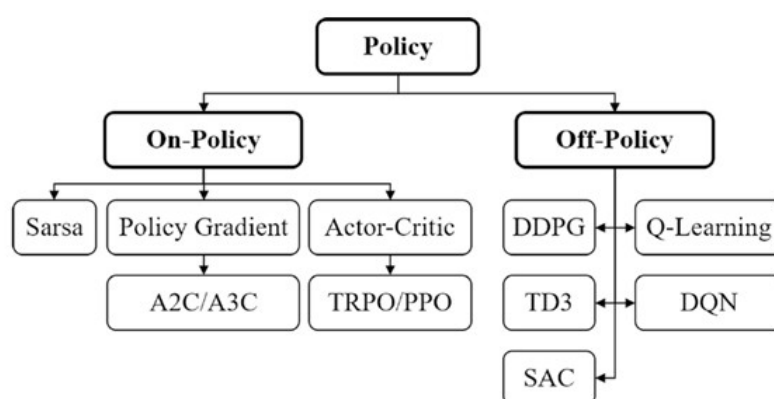
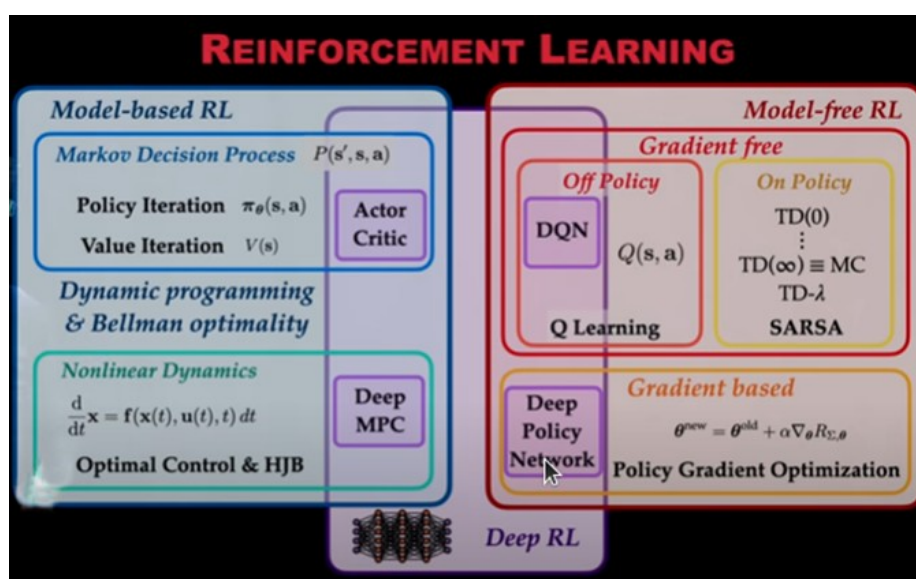


Fig. 29. Clasificación según la política.



Tema 5. Redes neuronales y deep learning

Fig. 29. Otra visión de la clasificación de los métodos de Reinforcement Learning.

Antes de estudiar algunos métodos específicos, intentaremos explorar métodos como programación dinámica, monte carlos y diferencias temporales, pues son la base de las ideas principales que forman el campo.

Programación Dinámica y la ecuación de optimalidad de Bellman

La idea y surgimiento de la programación dinámica proviene de los trabajos de Richard Bellman alrededor de los años 5. Es un método usado para resolver problemas de optimización en la toma de decisiones secuenciales. La idea, es que si la solución global óptima se puede dividir en soluciones más pequeñas, entonces la solución óptima de estos, será la conjunción de esas soluciones óptimas más pequeñas.

Por tanto, se pueden resolver por medio de esta técnica aquellos problemas que tienen una subestructura óptima en la cual se puede encontrar, como ya habíamos dicho, cuya solución global se pueden encontrar encontrando la solución de los problemas que componen sus subestructuras.

El ejemplo más simple que podemos usar para ejemplificar dicho método es la solución de la sucesión de Fibonacci. En este tipo de sucesión el elemento F_n es calculado como la suma de $F_{n-1} + F_{n-2}$, de la siguiente forma.

$$F_n = F_{n-1} + F_{n-2}$$

Esto quiere decir, que la solución al problema global, lo hemos descompuesto en la solución a subproblemas.

Por tanto, la solución a problemas por medio de programación dinámica incluye los siguientes pasos:

- Dividir el problema global, en subproblemas más pequeños.

Tema 5. Redes neuronales y deep learning

- ▶ Resolver estos subproblemas usando este mismo esquema de manera recursiva.
- ▶ Usar el resultado de la solución a estos subproblemas para armar la solución al problema global.

Bajo estos conceptos, Bellman desarrolla su principio de optimalidad y llega a su famosa ecuación. Este principio raza que «dada una secuencia óptima de decisiones, toda subsecuencia de ella es también óptima».

En el contexto que nos ocupa, el de aprendizaje con reforzamiento, que se modela por medio de procesos de decisión de Markov, la ecuación de Bellman toma la forma de una ecuación de valor.

Para explicar esta ecuación es necesario recordar que habíamos definido ya la idea de recompensa total descontada, que se expresaba matemáticamente de la siguiente forma:

$$R_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots + \gamma^T r_T = \sum_{t=0}^T \gamma^t r_t$$

Donde r es la recompensa correspondiente al incremento de tiempo t y que γ es el factor de descuento y que tomaba valores en el intervalo $[0,1]$ y su función era asignar peso o importancia a las recompensas futuras. De hecho, cuando $\gamma = 1$, es similar a no usar descuento y tratamos todas las recompensas de la misma manera. Sin embargo, cuando usamos $\gamma < 1$, entonces las recompensas futuras tendrán menos peso que las más recientes.

Esta expresión puede escribirse usando una forma recursiva, de la siguiente forma:

$$R_t = r_0 + \gamma (R_{t+1}) = \sum_{t=0}^T \gamma^t r_t$$

Si tenemos en cuenta que la función $V(s)$ representa el valor del estado s y que es

Tema 5. Redes neuronales y deep learning

igual a recompensa descontada a partir del estado s hasta el final del episodio, entonces:

$$V(s) = E(R_t \vee s = s)$$

Entonces

$$V(s) = r_0 + \gamma(V(s_{t+1}))$$

Y es a esta la que se le llama ecuación de Bellman para la función de valor.

Esta función que se ha dado en esta sección corresponde a un sistema determinista, sin embargo, no sería de utilidad para un sistema probabilístico.

De manera general, entonces la ecuación de Bellman se da, de la forma siguiente:

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} P(s', r|s, a) [r + \gamma V_{\pi}(s')]$$

Sin embargo, uno de los problemas de este método es que necesita el conocimiento de las probabilidades de transición entre estados, cosa que no siempre es posible en los tipos de problemas que tratan de solucionarse en aprendizaje con reforzamiento.

En aras de mantener a un mínimo la complejidad matemática de este tema, nos hemos apartado un tanto del rigor matemático y nos concentramos en que se comprendan las ideas esenciales. No es nuestra intención hacer un tratamiento completo ni riguroso del campo del Reinforcement Learning en esta introducción.

Método de Montecarlo

Este método no está directamente asociado al aprendizaje con reforzamiento, este, en vez, es un método que permite aproximar funciones cuando, o son muy complejas o no se tiene una descripción analítica de ella. La idea detrás de este método es muestrear de forma aleatoria los valores de la función y luego usar la media empírica para obtener el resultado.

Tema 5. Redes neuronales y deep learning

Este método fue popularizado por Stanislaw Ulam, un matemático que trabajó en el Laboratorio de Investigaciones de Los Alamos, en Estados Unidos, durante el desarrollo del proyecto Manhattan alrededor de 1940 (El proyecto Manhattan está asociado al desarrollo de la Bomba Atómica Norteamericana).

Con el fin que atrapen intuitivamente cómo funciona el método, expondremos un ejemplo extendido en muchos artículos que se basa en la estimación del valor de π , usando el método de Montecarlo.

Como sabemos de matemática, π es el símbolo que representa la relación entre la longitud de una circunferencia y su radio y que es constante. Su valor, aproximado es 3.1416.

El enfoque para estimar este valor de π usando el método de Montecarlo, es tomar un círculo de radio 1, centrado en el origen y circunscribirlo en un cuadrado. Sus respectivas áreas serían πr^2 y $(2r)^2$. Por tanto:

$$\frac{\text{área del círculo}}{\text{área del cuadrado}} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}$$

Bien, hasta aquí, sabemos que el área del cuadrado es 4, pero como no sabemos cuál es el valor de π no podemos saber el área del círculo.

Entonces para estimar el valor de π , podemos considerar usar un cuadrante de la forma que mostramos a continuación:

Tema 5. Redes neuronales y deep learning

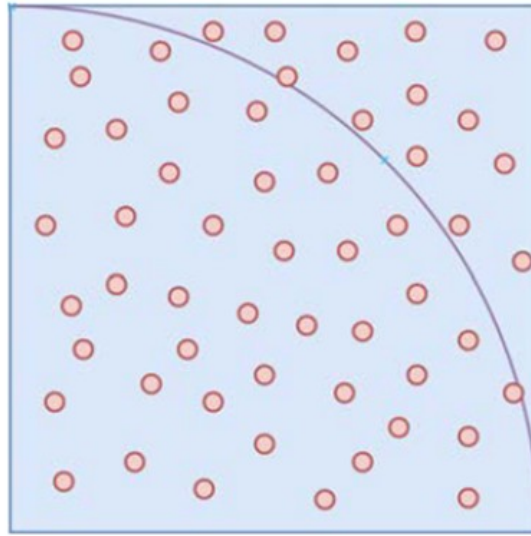


Fig. 30. Muestro por el método de Montecarlo para estimar π .

Si, tomamos muestras en cada uno de estos puntos, ubicados dentro del cuadrado usando una distribución aleatoria uniforme, cada uno de los puntos que caen dentro del círculo estarían a una distancia menor que 1 de su centro, dado que:

$$\sqrt{(x-0)^2 + (y-0)^2} < 1$$

Si, tomamos todos los puntos que caen dentro del círculo y lo dividimos entre la cantidad total, y el resultado lo multiplicamos por 4, el valor comenzará a aproximarse el valor de π . Entre más puntos pongamos, distribuidos de forma aleatoria y uniforme, más preciso será el valor y cercano al valor real de π .

Usando una idea similar se aplica este método en aprendizaje con reforzamiento. La idea sería muestrear tantas trayectorias como sean posibles y promediar sus resultados. De esta forma se estiman los valores de la política o la función de valor.

Una de las ventajas de usar este método es que no necesitan conocer las probabilidades de transición entre estados o las recompensas.

Tema 5. Redes neuronales y deep learning

Este procedimiento pueden probarlo usando el código que les mostramos a continuación.

```
from random import *
from math import sqrt
ins = 0
n = 100000
for j in range(0, n):
    x = (random()-.5) * 2
    y = (random()-.5) * 2
    if sqrt(x*x+y*y)<=1:
        ins+=1
pi = 4 * ins / n
print(pi)
```

En la medida que se agregan puntos, el valor resultante se acerca mucho más al valor de π .

Método de las Diferencias Temporales

Este es un método importante dentro del aprendizaje con reforzamiento. Este método, a diferencia de los métodos de Montecarlo, que usan la secuencia completa de un episodio y por tanto tienen que esperar a que ellos terminen, los métodos basados en diferencias temporales solo esperan porque termine el paso actual. Otra de las ventajas de este tipo de método es que no necesitan un modelo del ambiente. Este es mucho más fácil de explicar y de implementar.

De manera general en aprendizaje con reforzamiento este método se aplica de la siguiente forma:

Tema 5. Redes neuronales y deep learning

```
Inicializa  $V(s)$  arbitrariamente y  $\pi$  a la política a evaluar
Repite (para cada episodio):
  Inicializa  $s$ 
  Repite (para cada paso del episodio):
     $a \leftarrow$  acción dada por  $\pi$  para  $s$ 
    Realiza acción  $a$ ; observa la recompensa,  $r$ , y el siguiente estado,  $s'$ 
     $V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$ 
     $s \leftarrow s'$ 
  hasta que  $s$  sea terminal
```

Fig. 31. Algoritmo genérico de RL usando TD(0).

La idea básica detrás de este método es actualizar el valor de $V(s)$ usando la diferencia entre el valor del estado futuro con el valor del estado actual. La forma es que se actualiza el valor de $V(s)$ es:

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

Donde $V(s)$ es el valor del estado actual y que se va a actualizar, α es la tasa de aprendizaje o learning rate, r la recompensa inmediata asociada a la acción tomada en ese estado s . γ como ya lo definimos anteriormente es el factor de descuento y $V(s')$ es el valor del estado futuro, al que pasará el ambiente luego de tomar la acción a en el estado s .

A este tipo de método se le conoce como TD(0), pues solo usamos un time step en el futuro.

Los algoritmos Q-Learning y SARSA que explicaremos a continuación son métodos basados en diferencias temporales.

OpenAI Gym

Gym, es quizás, una de las librerías más conocidas en el campo de aprendizaje con reforzamiento. Esta, es una plataforma que permite construir, evaluar o comparar algoritmos de aprendizaje con reforzamiento, compatible con la mayoría de los frameworks de python, como TensorFlow, Pytorch o Keras. OpenAI también brinda

Tema 5. Redes neuronales y deep learning

una extensión de Gym llamada Universe.

Algunas otras de las plataformas conocidas son las siguientes:

- ▶ DeepMain LAB.
- ▶ RL-Glue.
- ▶ Malmö.
- ▶ VizDoom.

Una de las ventajas de usar Gym, es que tiene una interface, sencilla y fácil de entender por medio de la cual interactúan el agente de aprendizaje con el ambiente. Esta librería tiene cientos de ambientes preconstruido que facilitan la tarea de desarrollar los agentes.

De manera general el mecanismo que se utiliza para desarrollar un agente usando Gym es el siguiente:

```
import gym
episodios=1000
env = gym.make("CartPole-v0")
estado = env.reset()
for episodio in range(episodios):
    env.render()
    action = env.action_space.sample()
    estado, reward, done, info = env.step(action)
    if done:
        break
env.close()
```

En la primera línea en el código del recuadro anterior, se importa la librería gym y en la tercera, se construye el ambiente, en este caso usaremos uno llamado CartPole, en su versión 0. Este ambiente consiste en un elemento vertical delgado llamado

Tema 5. Redes neuronales y deep learning

Pole que está suspendido sobre un pequeño carrito que puede moverse a la izquierda y la derecha. El objetivo del ambiente es desarrollar un agente capaz de mantener el pole en posición vertical la mayor cantidad de tiempo posible. El ambiente retorna una recompensa de +1 por cada instante de tiempo en que el pole está en posición vertical, pero termina el ambiente cuando este sobrepasa los 12 grados de inclinación, el carrito se separa del centro al menos en 2,5 unidades o sale de la zona de visión y finalmente cuando se alcanzan 200 time steps.

Luego que se crea el ambiente con `env = gym.make("CartPole-v0")` , se inicializa el ambiente y se obtiene el primer estado que se almacena en la variable `estado`.

En este ambiente, el estado es un arreglo de cuatro elementos que representan la posición del carrito, su velocidad, el ángulo del pole y su velocidad angular

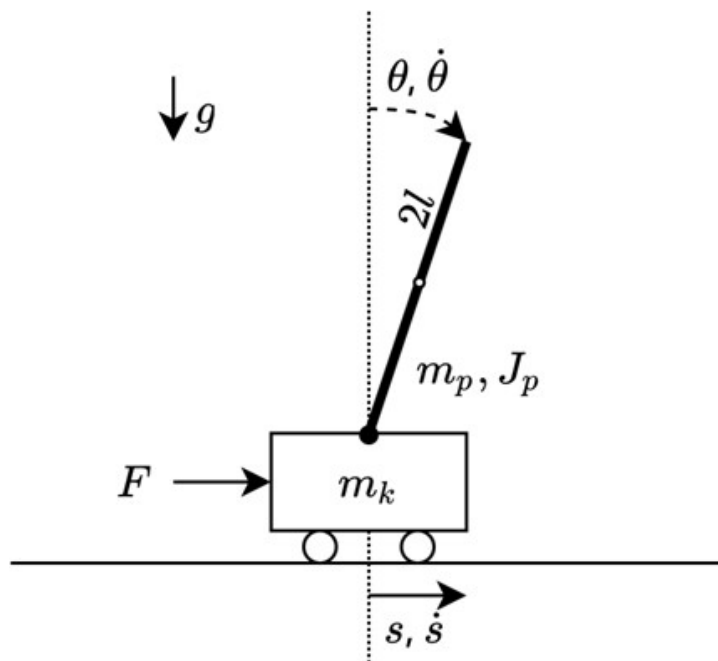


Fig. 32. Esquema representativo del Ambiente CartPole de Gym.

Este estado, tendría la forma siguiente:

`[-0.03718334 -0.03640839 0.00338674 -0.03613728]`

Tema 5. Redes neuronales y deep learning

Es importante que subrayemos que los componentes de este estado son variables continuas y por tanto a la hora de trabajarlo sería necesario discretizar-las.

Continuamos entonces con la explicación del ciclo de control, por medio del cual un agente interactúa con su ambiente. Luego que se tiene el estado inicial, se entra en un lazo, que en este caso está representado por medio de uno de tipo for y que se ejecutará una cantidad igual a la cantidad de episodios que necesitamos para que nuestro agente aprenda a controlar el ambiente.

Luego, visualizamos en nuestra pantalla el estado del sistema, que lo hacemos con la ejecución de `env.render()`.

Ahora ejecutaremos lo que viene a ser uno de los tres pilares fundamentales en aprendizaje con reforzamiento y es la elección de una acción en función del estado actual y normalmente a eso se le llama la política.

En este caso particular, la acción la escogemos de manera aleatoria desde el espacio de acciones de este ambiente, usando la siguiente expresión:

```
action = env.action_space.sample()
```

En `CartPole-v0`, el espacio de acciones está compuesto por dos acciones, 0 mueve el carrito a la izquierda y 1 que mueve el carrito a la derecha.

Luego que se ha escogido la acción por medio de una política determinada, es hora de ejecutar dicha acción, y lo hacemos de la siguiente forma:

```
estado, reward, done, info = env.step(action)
```

Luego que ejecutamos la acción, el ambiente retorna un nuevo estado y la recompensa de la transición a este. Y finalmente se cierra el ambiente.

Como ya veremos en la próxima sección, donde estudiaremos uno de los métodos más usados en aprendizaje con reforzamiento llamado Q-Learning, a este trozo de

Tema 5. Redes neuronales y deep learning

código le falta una de los tres elementos esenciales de RL y el mecanismo por medio del cual el agente aprende de la interacción con el ambiente.

Q-Learning

Q-Learning es uno de los métodos más usados en aprendizaje con reforzamiento, no solo por la calidad de las soluciones, sino, por la simplicidad a la hora de implementar y de entenderlo.

Como ya explicamos anteriormente, en aprendizaje con reforzamiento los agentes aprenden usando uno o varios de los tres elementos disponibles, aprenden o una política, aprenden un modelo del ambiente o una función de valor. Claro que hay algunos que combinan estos elementos y pueden clasificarse como híbridos.

También explicamos que las taxonomías se basan, para categorizar los métodos, en los elementos que usan para aprender.

Entonces, Q-Learning, es uno de los métodos que aprende una de las funciones de valor, específicamente aprende la función $Q(s,a)$ que cuantifica el valor de las acciones tomadas en el estado s . Esto lo hace uno de los métodos de aprendizaje con reforzamiento basado en valor. Tampoco necesita conocimiento del ambiente y no construye ningún modelo de este, por tanto, es también un método model-free.

De manera general, el algoritmo que se presenta a continuación muestra cómo funciona Q-Learning.

Tema 5. Redes neuronales y deep learning

Algorithm 1: Q-Learning

Result: Off-policy control for estimating $\pi \simeq \pi_*$

Parameters: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

Initialize $Q(s, a) \forall s \in \mathcal{S}^+, a \in \mathcal{A}(s)$ arbitrarily.

Set $Q(\text{terminal}, \cdot) = 0$

for each episode **do**

 Initialize S ;

for $step = 0$ **to** T **do**

 Choose A from S using policy derived from Q (e.g., ϵ -greedy);

 Take action A , observe R, S' ;

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$;

$S \leftarrow S'$;

end

end

El paso inicial del aprendizaje vía Q-Learning es construir una tabla que contenga todos los posibles estados correlacionados con las acciones posibles en cada uno de ellos y los inicialicemos de manera aleatoria o le asignamos valores de cero.

1	Q-Table	0	1
2	Estado 1	0	0
3	Estado 2	0	0
4	.	0	0
5	.	0	0
6	.	0	0
7	.	0	0
8	.	0	0
9	Estado n	0	0
10			

Fig. 33. Q-Table.

En la Figura 33 mostramos como debe ser una tabla Q con los valores inicializado en cero.

El siguiente paso es la elección de una acción usando una política determinada. Veamos en detalles que es una política y cual usa Q-Learning específicamente.

Ya hemos explicado que una política en el contexto de aprendizaje con reforzamiento es la estrategia que se usa para escoger una acción a en un estado s . Una política

Tema 5. Redes neuronales y deep learning

sería, por ejemplo, elegir siempre la acción que mayor recompensa nos genere y a este tipo de política es a la que se llama avara.

Sin embargo, como también explicamos ya, los agentes que aprenden por medio de reforzamiento enfrentan lo que se conoce como compromiso exploración explotación. Por tanto, para resolver ambos temas, es decir, escoger las acciones que más recompensa nos generen y a la vez explorar lo suficiente, Q-Learning usa una política llamada ϵ -greedy.

En este tipo de política ϵ -greedy, se establece un valor de ϵ , de manera que la mayoría de las veces se escoja una acción basada en los $Q(s,a)$ aprendidos y el resto de las veces permita explorar acciones nunca antes exploradas.

La variante de esta política ϵ -greedy que usamos en la implementación que les mostramos varía el valor de ϵ en cada episodio, de forma que al inicio del entrenamiento se explore más y luego al final las decisiones de la acción a escoger se basen en los valores de $Q(s,a)$.

Bueno, luego de explicar que es y cómo es la política usada por Q-Learning, el próximo paso a ejecutar, con la acción ya seleccionada es el de ejecutar dicha acción. Al ejecutar esta acción el ambiente, en este caso, CartPole, responde con un nuevo estado y una recompensa.

Con este nuevo estado y la recompensa adquirida es hora de aprender, si, este paso se basa en una ecuación como la de Bellman para la función Q que permite actualizar los valores de Q para los pares de estados-acciones. Estos valores actualizados permiten mejorar en cada iteración el proceso de toma de decisiones.

La manera en que se actualizan los valores de Q es la siguiente:

$$Q(s,a) \leftarrow (1 - \alpha) Q(s,a) + \alpha \left[r + \gamma * \max_a \left(Q(s',a) \right) \right]$$

Donde s es el estado actual y s' es el estado al que ha transitado el ambiente

Tema 5. Redes neuronales y deep learning

producto a la ejecución de la acción a . Es decir, el estado futuro.

Luego, si el episodio no ha concluido por algunas de las razones se repite el proceso.

A continuación, iremos mostrando la implementación de Q-Learning por partes.

```
#####
#                               Implementación de Q-Learning tabular                               **
#####
import gym
import numpy as np
from collections import defaultdict
import random
import matplotlib.pyplot as plt
#####
# Construye el ambiente con con el que interactúa el agente
env = gym.make("CartPole-v0").env
# Asegura la reproductibilidad de los resultados
np.random.seed(0)
env.seed(0)
# Número de Bins para discretizar los estados
numBins = 20
bins = [
    np.linspace(-4.8, 4.8, numBins),
    np.linspace(-4, 4, numBins),
    np.linspace(-.418, .418, numBins),
    np.linspace(-4, 4, numBins)
]
obsSpaceSize = len(env.observation_space.high)

# vamos a crear la tabla Q(s,a) con forma [20,20,20,20,2]
Q = np.random.uniform(low=-2, high=0, size=(numBins) * obsSpaceSize + [env.action_space.n])
#####
```

Fig. 34. Primera parte de la implementación de Q-Learning.

Es importante subrayar que, aunque ya lo hemos comentado, los valores que componen el estado retornado por CartPole, corresponden a variables continuas y por tanto antes de poder manejarlo es necesario discretizarlos.

Este proceso de discretización se lleva a cabo con la función: `get_discrete_state(state, bins, obsSpaceSize)`.

En la siguiente imagen se muestran las funciones que implementan las tres componentes necesarias, la política, la ejecución de las acciones y la función que aprende los valores Q.

Tema 5. Redes neuronales y deep learning

```
#####
alpha = 0.1 #learning rate
discount_factor = 0.9
epsilon = 1
max_epsilon = 1
min_epsilon = 0.01
decay = 0.001
episodes = 25000
max_steps = 200
#####
#Discretiza el estado, pues CartPole retorna un estado con componentes continuos
def get_discrete_state(state, bins, obsSpaceSize):
    stateIndex = []
    for i in range(obsSpaceSize):
        stateIndex.append(np.digitize(state[i], bins[i]) - 1) #
    return tuple(stateIndex)
#####
def policy(state): #Implementamos una e-greedy
    #Elijo un valor de forma aleatoria de una distribución normal con media 0 y varianza 1
    if random.uniform(0, 1) > epsilon:
        action = Q_guetAction(state)
    else:
        action = np.random.randint(0, env.action_space.n)
    return action
#####
def Q_guetAction(state):
    discreteState = get_discrete_state(state, bins, obsSpaceSize)
    action = np.argmax(Q[discreteState])
    return action
#####
def Q_update(state, new_state, action, reward):
    discreteState = get_discrete_state(state, bins, obsSpaceSize)
    New_discreteState = get_discrete_state(new_state, bins, obsSpaceSize)
    maxFutureQ = np.max(Q[New_discreteState]) # Value de Q-Value para el estado futuro
    currentQ = Q[discreteState + (action,)] # Valor de Q-Value para el estado actual
    #(1 - alpha) *
    newQ = (1 - alpha) * currentQ + alpha * (reward + discount_factor * maxFutureQ)
    Q[discreteState + (action,)] = newQ # Actualiza Q
#####
```

Fig. 35 Segunda parte de la Implementación de Q-Learning.

En la primera parte del código mostrado en esta imagen, se definen los hiperparámetros que guían el funcionamiento del agente y su interacción con el ambiente, veamos cuáles son:

- ▶ **alpha**: Es el learning Rate o tasa de aprendizaje. Debe tomar valores entre 0 y 1.
- ▶ **discount_factor**: Determina el peso que se les asigna a los valores de las observaciones en cada time step. Debe tomar valores entre 0 y 1.
- ▶ **épsilon**: Representa el valor que determina el compromiso entre exploración y explotación. En esta implementación se pone a 1, pues para cada episodio su valor decrece en un valor fijo con el fin de hacer que al final del entrenamiento se explore menos.

El resto de los hiperparámetros son bien simples de entender.

En esta sección se muestran las implementaciones de las funciones que componen los elementos esenciales de un agente con aprendizaje con reforzamiento. La

Tema 5. Redes neuronales y deep learning

primera de las cuatro, como ya dijimos en la sección anterior es la que se encarga de discretizar los componentes del estado. La segunda de las funciones es la política, la cual implementa la llamada e-greedy.

La última de las funciones implementa la manera en que se actualizan los valores de Q que ya habíamos explicado.

```
training_rewards = []
epsilons = []
done=False
env.render()
for episode in range(epsisodes):
    total_training_rewards = 0
    state = env.reset()
    for _ in range(max_steps):
        #Elige una acción según la política y basado en el estado actual
        action = policy(state)
        #Ejecuta la acción en el estado actual y obtiene la recompensa y el nuevo estado
        new_state, reward, done, info = env.step(action)
        #Actualiza los valores de Q(s,a)
        Q.update(state,new_state, action,reward)
        total_training_rewards += reward
        state=new_state
        #Si desea monitorear de forma visual el proceso de aprendizaje descomente
        #la siguiente línea aunque suele demorar el proceso
        #env.render()
        if done==True:
            break
    print(episode,total_training_rewards)
    #Ajusta el valor epsilon de forma que en la medida que pasan los episodios se explora menos
    epsilon = min_epsilon + (max_epsilon - min_epsilon)*np.exp(-decay*episode)
    #Compilando los valores totales de la recompensa
    training_rewards.append(total_training_rewards)
    epsilons.append(epsilon)
env.close()
#print(Q)
#Visualizando la recompensa total contra episodios
x = range(epsisodes)
plt.plot(x, training_rewards)
plt.xlabel('Episodio')
plt.ylabel('Recompensa Total')
plt.title('Recompensa Total sobre todos los episodios')
plt.show()
#Visualizando epsilon por episodios
plt.plot(epsilons)
plt.xlabel('Episodios')
plt.ylabel('Epsilon')
plt.title("Epsilon por episodios")
plt.show()
```

Fig. 36 Tercera parte y ultima de la Implementación de Q-Learning.

En esta sección de la implementación se muestra la parte que corresponde al ciclo de control, donde el agente interactúa con el ambiente. Esta parte está bastante comentada y coincide con la explicación que ya dimos del lazo de control general que usamos para los agentes que interactúan con la librería Gym.

Tema 5. Redes neuronales y deep learning

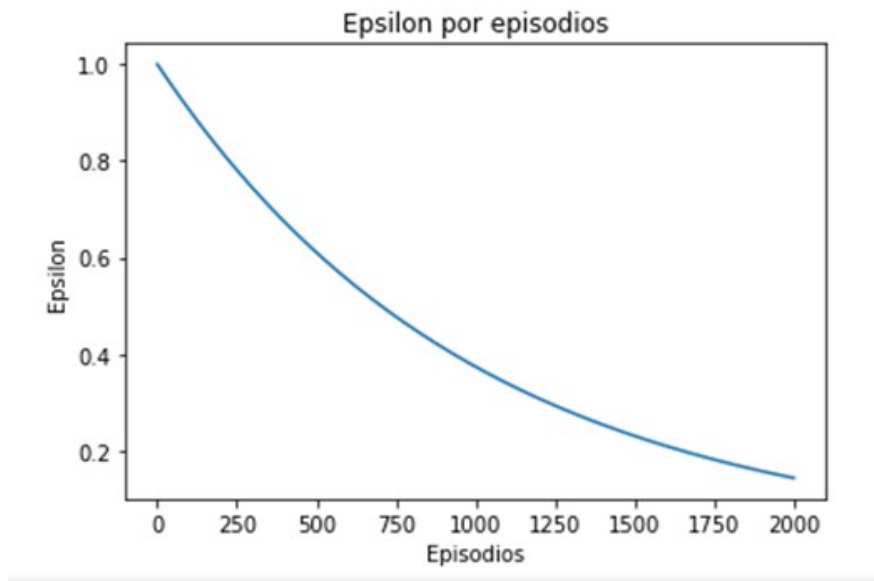


Fig. 37 Muestra como disminuye épsilon por cada episodio.

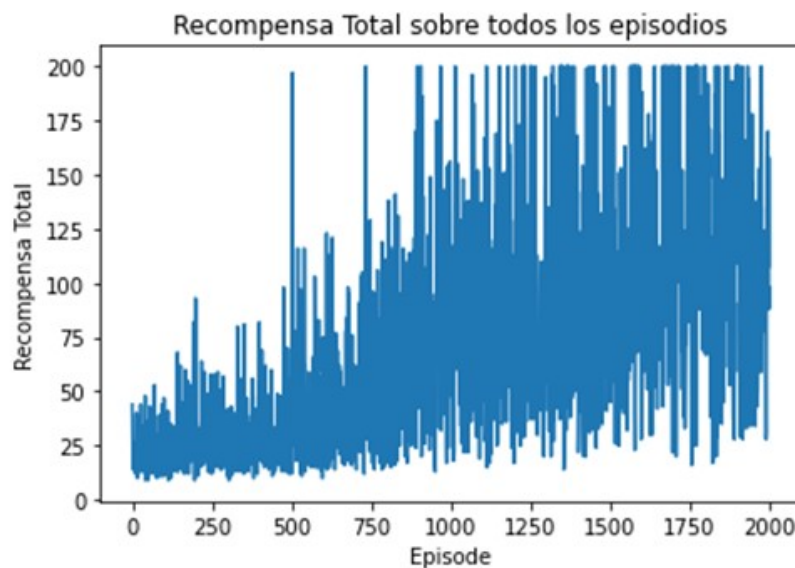


Fig. 38. Muestra como aumenta la recompensa total en cada episodio.

Conclusiones

Hasta aquí llegamos con este recorrido por las técnicas de aprendizaje profundo y aprendizaje con reforzamiento. Ambos campos, bastos y complejos, llenos de

Tema 5. Redes neuronales y deep learning

formalismos matemáticos y sus sutilezas. Para sacar provecho de estas páginas es importante que no descansen y lo que no entienda de aquí lo busque en internet. Hay infinidad de fuentes de donde puede aprender, unas buenas y otras no tanto pero que irían cimentando lo que ha aprendido hasta aquí.

Por no abarrotar en demasía estas páginas no hemos abarcado más técnicas y nos hemos concentrado en las que consideramos básicas, por tanto con el fin de ampliar su conocimiento es sumamente importante que consulten la mayor cantidad de literatura posible.

Tema 6. Despliegue de modelos de aprendizaje automático

6.1. Introducción y objetivos

En esta unidad estudiaremos una de las etapas fundamentales en el ciclo de vida de los modelos de aprendizaje automático y es la etapa de despliegue de los modelos a ambientes de producción.

Es importante que destaquemos que, aunque esta etapa es tan importante como todas las anteriores se le ha prestado mucho menos atención en la literatura y por eso hacemos énfasis a la hora de entender todo el proceso de aprendizaje.

Esta unidad está compuesta por una sola lección y no profundizamos en el tema y solo hacemos una introducción pues es un tema que está ligado a lo que se conoce como MLOps y que requeriría de un libro solamente para eso.

De esta forma hemos querido solamente llamar la atención sobre el tema y dar una introducción rápida sobre este asunto.

En esta lección cubrimos los aspectos que creemos que son esenciales en el ciclo de vida de un modelo, revisamos las características más generales de algunas de las plataformas que creemos son las más importantes en lo que a manejo del ciclo de vida de un modelo se refiere y luego ponemos un ejemplo práctico de despliegue de un modelo que usa una red neuronal en la plataforma Tensorflow Serving, que forma parte de la plataforma TensorFlow extended, o TFX, desarrollada por Google.

Tema 6. Despliegue de modelos de aprendizaje automático

Los **objetivos** que se persiguen en este tema son:

- ▶ Conocer las etapas del ciclo de vida de un modelo de aprendizaje y sus cuestiones esenciales.
- ▶ Conocer las plataformas más importantes que permiten la gestión completa del ciclo de vida de un modelo.
- ▶ Aprender a desplegar un modelo de aprendizaje en TensorFlow Serving.

Tema 6. Despliegue de modelos de aprendizaje automático

6.2. Despliegue de modelos de aprendizaje automático

En todas las lecciones anteriores hemos tratado muchas de las aristas que conforman lo que se conoce como aprendizaje automático, hemos visto como desarrollamos modelos de aprendizaje automático supervisado, como hacemos análisis exploratorio de los datos, aprendizaje profundo, entre otras cosas. Sin embargo, es preciso que nos preguntemos, luego que tenemos los modelos de aprendizaje automático desarrollado, ¿Qué hacemos con ellos?

El objetivo de esta lección es precisamente responder esa pregunta, por tanto nuestros objetivos son:

- ▶ Describir las principales formas de desplegar un modelo para su uso.
- ▶ Describir las principales plataformas para el despliegue en producción de los modelos de aprendizaje automático.
- ▶ Desarrollar un ejemplo de despliegue de un modelo usando Tensorflow Serving.

Al final de esta lección, por tanto, deben ser capaces de desplegar modelos a un ambiente de producción como TensorFlow Serving y entender plataformas de producción más complejas que manejen el ciclo de vida completo de un modelo de aprendizaje automático.

Introducción

En muchos de los cursos que se ofertan hoy en día, incluso en la multitud de propaganda que se dedica a las ciencias de datos, aprendizaje automático y profundo, se trata, casi exclusivamente el proceso de desarrollo de los modelos de aprendizaje automático y se excluye de forma deliberada cómo usar estos modelos en ambientes de producción. Aunque eso ha comenzado a cambiar recientemente

Tema 6. Despliegue de modelos de aprendizaje automático

con la aparición de conceptos como MLOps.

Con el fin de dar una visión completa del uso del aprendizaje automático en la solución de problemas prácticos en las organizaciones, no puede faltar, por tanto, lo referente al despliegue y consumo de dichos modelos entre otras actividades que se engloban en lo que se conoce como el ciclo de vida de un modelo de aprendizaje automático.

En la medida que las organizaciones se mueven desde la experimentación con aprendizaje automático al uso de este, en ambientes de producción, las complejidades propias del campo del aprendizaje automático se suman a las complejidades de las organizaciones modernas. Y comienzan a surgir interrogantes: ¿Cómo pasamos de manejar un modelo de aprendizaje automático a manejar cientos de ellos o incluso miles? ¿Cómo desplegamos los modelos a producción y que herramientas usamos? ¿Cómo integramos los requerimientos de los modelos de aprendizaje con los requerimientos de las aplicaciones que soportan la operación de la empresa? ¿Cómo mantenemos una evaluación constante de la calidad de los modelos desplegados en producción? ¿Cuáles son los riesgos que corremos al usar modelos de aprendizaje automático en ambientes de producción?

Estas son algunas de las dificultades a las que se enfrentan las organizaciones a la hora de pasar de experimentar con aprendizaje automático a usarlos en producción. La tasa de fallo de los proyectos de aprendizaje automático en las organizaciones es bastante alta y ronda el 85%. Esto habla bien claro de lo complejo que puede tornarse el asunto. Y es que no basta con tomar un puñado de datos y desarrollar un modelo, sino que tiene que ser visto como proceso que tiene que ser gestionado de forma adecuada y holística e integrado dentro del resto de los procesos de la empresa.

El proceso de desarrollo de un modelo de aprendizaje automático no es simplemente lineal, con un inicio y un fin bien delimitados, sino que tiene que verse como un ciclo.

Tema 6. Despliegue de modelos de aprendizaje automático

A este ciclo es a lo que se da en llamar Ciclo de vida de un modelo de Aprendizaje y está compuesto por las siguientes etapas:

- ▶ Desarrollo.
- ▶ Despliegue.
- ▶ Monitoreo.
- ▶ Iteración.
- ▶ Gobernanza.

Explicemos brevemente en que consiste cada una de estas fases o etapas.

Desarrollo: Durante esta etapa es usual iniciar el proceso con el establecimiento de los objetivos de negocio que puede ser tan sencillo como identificar correos como spam o no o tan complejos como la detección de fraude en un banco. Normalmente estos objetivos están asociados a presupuestos y requerimientos de recursos de computación y eficiencia que pueden ser traducidos a KPI.

Luego que quedan bien definidos estos objetivos es hora de que los científicos de datos de conjunto con expertos del negocio se sienten y cooperen y encuentren los conjuntos de datos necesarios para el trabajo y que estará regido en parte por las reglas de gobernanza de los datos de la organización.

Como los datos son el ingrediente principal de un proyecto de aprendizaje automático es esencial que se tenga algún conocimiento de ellos y aquí entra el proceso que ya estudiamos en una de nuestras lecciones llamado Análisis Exploratorio de Datos.

Durante el EDA, podremos hacernos algunas preguntas sobre los datos, hacer hipótesis y limpiar los datos, entre muchas otras actividades que ya vimos. Esta etapa nos lleva de forma directa a lo que ya también estudiamos y que llamamos

Tema 6. Despliegue de modelos de aprendizaje automático

ingeniería de rasgos o como se le conoce en su término en inglés «feature selection».

Luego que ya hemos construido el conjunto de datos y creemos que es el correcto podemos escoger un algoritmo o incluso varios y entrenarlos con los datos, con el fin de escoger el que mejor resultados muestre y luego someterlos a un proceso de evaluación.

Es importante destacar que este es un proceso iterativo en el que los modelos se van refinando y que puede llevar a la modificación del conjunto de datos también.

Uno de los problemas a los que se enfrentan los científicos de datos a la hora de desarrollar sus modelos a lo que se conoce como reproducibilidad. Y esto significa que cualquier otro científico de datos pueda obtener los mismos resultados sobre los mismos datos y bajo las mismas características del ambiente donde se desarrolló el modelo.

La reproducibilidad es la clave para que el modelo pueda ser desplegado en entornos de producción.

Despliegue: Esta etapa es completamente diferente a la anterior. Es en esta etapa en que los modelos desarrollados en la etapa anterior se ponen a disposición de los ingenieros de IT para que los puedan usar como parte de las soluciones de la empresa. Esta etapa tiene un conjunto de retos técnicos completamente diferentes a los de la etapa de desarrollo.

Esta etapa requiere de una colaboración especial entre los equipos de desarrollo y los de ingeniería, en muchas empresas llamado equipo de DEVOps.

Básicamente hoy dos tipos de despliegue de los modelos:

- ▶ Despliegue de un modelo como servicio.
- ▶ Despliegue de los modelos Embebidos.

Tema 6. Despliegue de modelos de aprendizaje automático

En el primer tipo el modelo se despliega dentro de un framework que brinda un endpoint REST API u otro, por medio del cual se accede a los modelos. En la segunda forma, el modelo se empaqueta dentro de una aplicación que es la que es publicada.

Uno de los problemas que se enfrentan en esta etapa es que los modelos que se despliegan necesitan tener las mismas condiciones que tenían en el ambiente de desarrollo. Esto quiere decir que las versiones de las librerías y paquetes tienen que coincidir con los de desarrollo o la calidad de los modelos puede verse afectada de manera importante.

Un modo de reducir las dependencias del modelo sobre el ambiente de producción sería exportar estos a formatos portables como PMML, PFA, ONNX.

Otra de las soluciones que actualmente es muy usada es el uso de contenedores, tales como Docker. Esta es una de las soluciones a los dolores de cabeza de las dependencias de los modelos en los ambientes de producción cuando se despliegan.

Docker, es una alternativa mucho más liviana que las máquinas virtuales permitiendo que las aplicaciones sean desplegadas en ambientes auto contenidos que machean exactamente los requerimientos de los modelos.

Otra de las ventajas de este tipo de solución es que los recursos necesarios para el modelo pueden ser escalados de manera sencilla usando múltiples contenedores manejados por orquestadores como Kubernetes que pueden desplegarse tanto on-premisa o en la nube.

Monitoreo: El despliegue del modelo a producción no es el paso final en el uso de un modelo o punto final en el proyecto de ciencia de datos o aprendizaje automático, luego que se despliega comienza una nueva etapa para el modelo ya que es necesario para la organización mantener los ojos puestos sobre el comportamiento

Tema 6. Despliegue de modelos de aprendizaje automático

del modelo.

Una de las cosas que complica el tema del despliegue y uso de modelos en producción es que la calidad del modelo es vista de manera diferente en dependencia de la perspectiva de cada uno de los actores que tienen relación con él. Por ejemplo el equipo de ciencia de datos debe estar pendiente del comportamiento del modelo respecto a los indicadores de calidad como el accuracy y el error medio cuadrático por ejemplo, mientras que los ingenieros de DEVOps, se preocupan por los recursos computacionales que usan y los tiempos de respuestas, al equipo de negocios, por otra parte, le interesa si el modelo está cumpliendo con los objetivos de negocios que se propusieron y si aporta valor a la organización. Esto habla, a grandes rasgos, de la complejidad y riesgos que implica el uso de modelos de aprendizaje automático en las organizaciones.

Por tanto, para tener control sobre todos estos aspectos, el modelo se somete a un monitoreo constante.

Iteración: En las etapas anteriores el modelo se desarrolla y valida y luego se despliega. Para estar observando su comportamiento se monitorea, sin embargo que pasa si el modelo se desvía de los objetivos de calidad?, para eso se añade la etapa iterativa. Es en esta etapa donde el modelo es mejorado y reelaborado si cambian los objetivos de la organización. En esta etapa el modelo se reentrena para adaptarse a los cambiantes datos del negocio.

En esta etapa pueden generarse nuevas versiones del modelo que se prueban en producción y sus métricas se comparan con los modelos desplegados anteriormente.

Gobernanza: La Gobernanza se refiere a los procesos y estructuras establecidos en una organización para la toma de decisiones. Esta puede incluir tanto los mecanismos formales, como lo son las estructuras y los procesos, como los informales: derivados de la cultura organizacional y sus valores.

Tema 6. Despliegue de modelos de aprendizaje automático

Por tanto, la gobernanza de una organización es el conjunto de controles impuestos en una empresa para garantizar que cumpla con sus responsabilidades con todas las partes interesadas, desde accionistas y empleados hasta el público y gobiernos nacionales. Estas responsabilidades incluyen obligaciones financieras, legales y éticas. La base de estos tres es el principio fundamental de la equidad.

Dentro de las regulaciones que caen bajo la Gobernanza de las organizaciones están las regulaciones el uso de los datos personales de los clientes, por ejemplo, la GDPR.

De todo esto se desprende la Gobernanza de los datos, que tienen que ver con las regulaciones correspondientes a la recolección y uso de los datos dentro de las organizaciones.

Las organizaciones, con el fin de manejar todos los requerimientos que impone el ciclo de vida de los modelos de aprendizaje automático han desarrollado plataformas que automatizan dichas actividades. De una forma u otra, estas plataformas, dan respuestas a las necesidades de las organizaciones en la gestión de todas estas etapas o ciclo de vida de los modelos de aprendizaje automático.

En la próxima sección revisaremos algunas de las plataformas más importantes que han ido surgiendo como resultado del intento de poner en producción los modelos de aprendizaje.

Plataformas de gestión del ciclo de vida de los modelos de aprendizaje automático

La mayoría de las empresas tecnológicas grandes, han desarrollado sus propias plataformas que gestionan de extremo a extremo todos los requerimientos y necesidades que impone el manejo y gestión del ciclo de vida de los modelos de aprendizaje automático. Empresas como Google, Amazon, IBM entre otras han desarrollado las suyas.

Tema 6. Despliegue de modelos de aprendizaje automático

Todas estas plataformas tratan de cumplir con determinados retos que les son comunes y difíciles de resolver, tales como:

- ▶ **Despliegue, automatización y escalabilidad:** Llevar los modelos de aprendizaje automático desde la etapa de desarrollo a la etapa de producción de manera oportuna no está dentro de las competencias de los equipos de ciencias de datos.
- ▶ **Reproducibilidad de los modelos y predicción:** Es difícil reproducir los resultados de los modelos en ambientes cambiantes sin hacer un seguimiento correcto de las versiones de las librerías y de los conjuntos de datos.
- ▶ **Diagnóstico, monitoreo de la calidad de los modelos y su deriva:** Normalmente existen muchos modelos corriendo en ambientes diferentes usando variados datasets y parámetros haciendo virtualmente imposible hacer un seguimiento del linaje de cada modelo.
- ▶ **Complimiento con las normas de Gobernanza y explicabilidad:** Es de suma importancia seguir los protocolos en lo referente al seguimiento y documentación de los modelos, los datos y los parámetros para mostrar a los auditores.
- ▶ **Colaboración:** Los equipos de ciencia de datos enfrentan innumerables retos a la hora de llevar los modelos desde desarrollo a producción.

Tema 6. Despliegue de modelos de aprendizaje automático

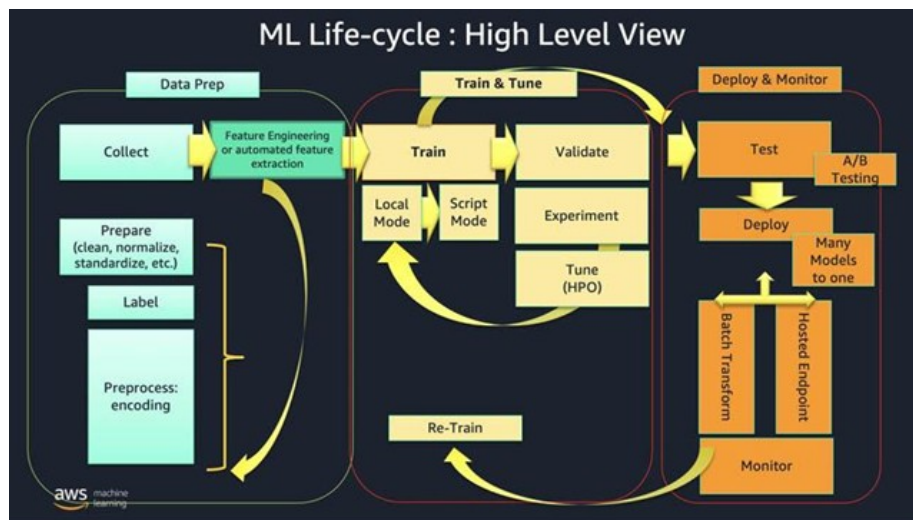


Fig.1. Ciclo de vida de un Modelo de Aprendizaje Automático (Amazon).

A grandes rasgos el ciclo de vida, se descompone en varias etapas y que vimos en la introducción se compacta en tres grandes etapas: La etapa de preparación de los datos, la etapa de entrenamiento y sintonía de los parámetros de modelo y la etapa de despliegue y monitoreo. En la Figura 1 pueden verse los elementos que conforman cada una de estas etapas y en la Figura 2 vemos los detalles.

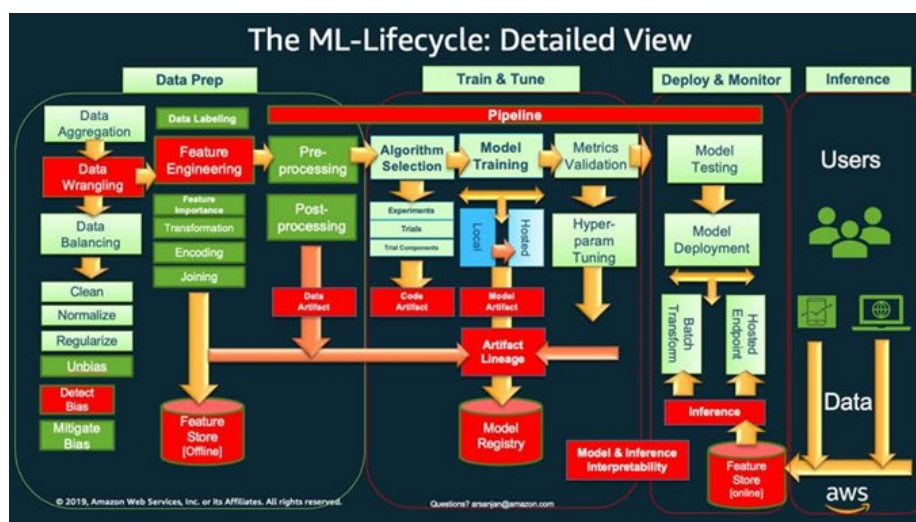


Fig. 2. Vista detallada de los elementos que conforman las etapas del ciclo de vida de un modelo de aprendizaje automático (Amazon).

Tema 6. Despliegue de modelos de aprendizaje automático

No vamos a tocar los detalles de cada uno de los componentes en cada una de las etapas del ciclo de pues ya lo vimos anteriormente, sin embargo, conviene verlos en la manera que se muestra en la figura 3, pues deja una idea clara de las actividades más importantes.

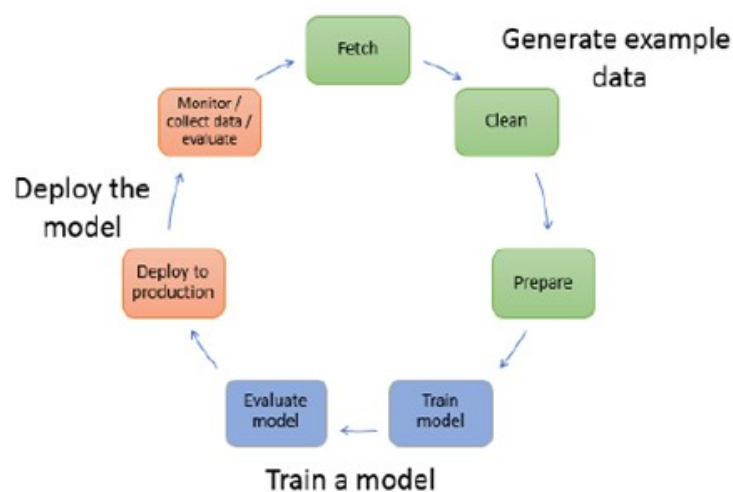


Fig. 3. Ciclo de vida de un modelo de aprendizaje. (Amazon)

El ciclo, como se muestra en la figura 3, comienza con la búsqueda de los datos, la limpieza de estos, luego se preparan para entrenar el modelo, evaluarlo y desplegarlo para luego monitorear su calidad de forma permanente hasta que se repite nuevamente el ciclo. Este ciclo se repite constantemente en función de los ajustes del modelo a probables derivas del modelo.

Tema 6. Despliegue de modelos de aprendizaje automático

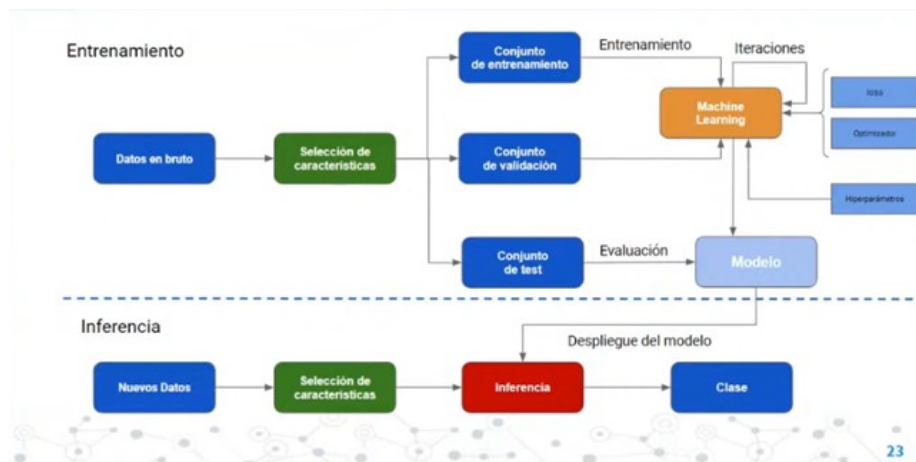


Fig.4. Actividades requeridas en cada uno de los estados de un modelo de aprendizaje automático.

Como se muestra en la imagen de la figura 4, un modelo de aprendizaje puede encontrarse en dos estados mutuamente excluyentes. El modelo puede estar en desarrollo o entrenamiento o en modo de inferencia. El modelo está en el modo de desarrollo cuando se está en el proceso de entrenamiento. Cuando el modelo se despliega a producción se pone en modo de inferencia y esto quiere decir que al modelo se le va a estar constantemente haciendo solicitudes para, clasificar imágenes, si fue entrenado con ese fin.

Las plataformas de gestión del ciclo de vida de modelos de aprendizaje tratan de abarcar la mayor cantidad de componentes dentro del ciclo de vida, automatizándolas.

Las soluciones en el mercado son numerosas sin embargo hay algunas que destacan. Entre ellas las que veremos a continuación.



Amazon SageMaker

Amazon SageMaker es una plataforma de aprendizaje automático que ayuda a crear,

Tema 6. Despliegue de modelos de aprendizaje automático

entrenar, administrar e implementar modelos de aprendizaje automático en un entorno listo para producción. SageMaker acelera la experimentación con herramientas especialmente diseñadas para el etiquetado, preparación de datos, ajuste de hiperparámetros, monitoreo y mucho más.

Esta plataforma no es gratis y funciona en la nube, así que es importante tener bien claro los elementos de precios a la hora de usarlo.

Los principales rasgos de esta plataforma son las siguientes:

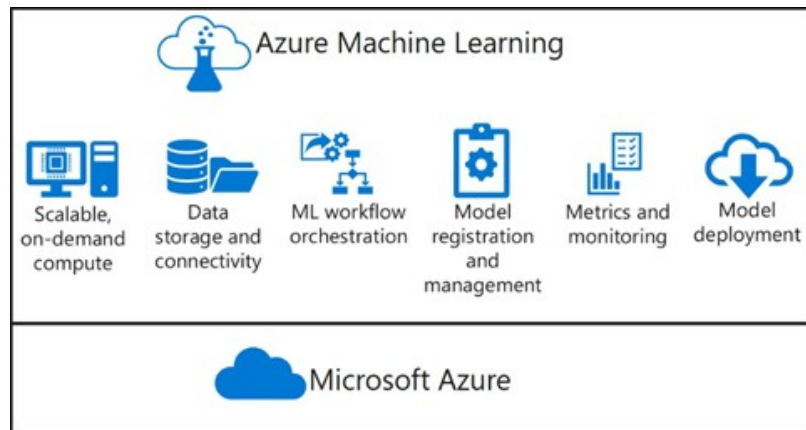
- ▶ En la plataforma vienen incluidos muchos algoritmo de aprendizaje automático, que incluye tanto supervisados como no supervisados.
- ▶ Esta es una plataforma de extremo a extremo que acelera el proceso de modelado y despliegue.
- ▶ Incluye AutoML, que automáticamente construye, entrena y ajusta un modelo de aprendizaje automático basado sobre los datos que se le pasan.
- ▶ Este permite integrar API y SDK lo que facilita la configuración y el uso de modelo en cualquier parte.
- ▶ Incluye más de 150 soluciones que se pueden desplegar en cualquier momento.



Azure Machine Learning

Azure Machine Learning, es una plataforma basada en la nube que permite entrenar, desplegar y monitorear los modelos de aprendizaje automático.

Tema 6. Despliegue de modelos de aprendizaje automático



Sus principales características son las siguientes:

- ▶ Soporta lenguajes de programación como python y R, sobre la cual pueden correr Jupyter Lab y R Studio.
- ▶ Tiene soporte para la mayoría de los frameworks de fuente abierta más usados en aprendizaje automático como TensorFlow, Scikit-learn, ONNX y pytorch.



Esta plataforma no es gratis, aunque en su esquema de precio concibe 12 meses gratis, luego del cual se cobra por uso.

Tema 6. Despliegue de modelos de aprendizaje automático



Google AI Platform

Esta es una plataforma extrema a extremo para aprendizaje automático y ciencia de datos. Soporta la mayoría de las necesidades de la administración del ciclo de vida de los modelos de aprendizaje automático.

Sus principales características son las siguientes:

- ▶ Bigquery y el almacenamiento en la nube ayuda a la hora del manejo de los conjuntos de datos, especialmente cuando son grandes.
- ▶ Pueden desarrollarse los modelos y todas las tareas asociadas sin necesidad de escribir ni una línea de código usando AutoML, puede usarse si lo prefiere Google Colab.
- ▶ Soporta muchos de los frameworks de fuente abierta más populares como KubeFlow, TensorFlow, Google Colab Notebooks.
- ▶ El despliegue puede hacerse usando AutoMLy puede ejecutar acciones en tiempo real sobre los modelos.
- ▶ Puede manejar y monitorear sus modelos haciendo uso de las tuberías.
- ▶ Cuenta con herramientas para la explicabilidad de los modelos, con el fin de entender la conducta de los modelos con el fin de mejorarlos.

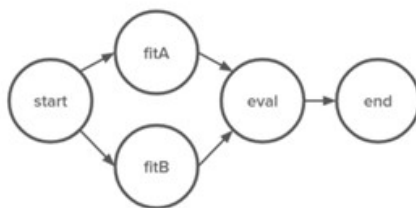
Esta es una plataforma en la nube y tampoco es gratis, por tanto es importante tenerlo en cuenta.

Tema 6. Despliegue de modelos de aprendizaje automático



MetaFlow

Esta es una plataforma para el manejo del ciclo de vida de los modelos de aprendizaje automático y basada en Python. Fue desarrollada por Netflix con el fin de incrementar la productividad de sus científicos.



```
class MyFlow(FlowSpec):  
  
    @step  
    def start(self):  
        self.data = load_data()  
        self.next(self.fitA, self.fitB)  
  
    @step  
    def fitA(self):  
        self.model = fit(self.data, model='A')  
        self.next(self.eval)  
  
    @step  
    def fitB(self):  
        self.model = fit(self.data, model='B')  
        self.next(self.eval)  
  
    @step  
    def eval(self, inputs):  
        self.best = max((i.model.score, i.model) for i in inputs)[1]  
        self.next(self.end)  
  
    @step  
    def end(self):  
        print('done!')
```

Sus principales rasgos son los siguientes:

- ▶ Es un proyecto de código abierto que puede integrarse con sagemaker y con otras librerías de aprendizaje automático.
- ▶ Tiene una API unificada que le permite manejar un proyecto desde el prototipo hasta el despliegue a producción.
- ▶ Cuenta con una interfaz gráfica que permite que el flujo de trabajo se diseñe en forma de un grafo acíclico dirigido.
- ▶ Luego que el modelo es desplegado a producción se puede hacer un seguimiento al

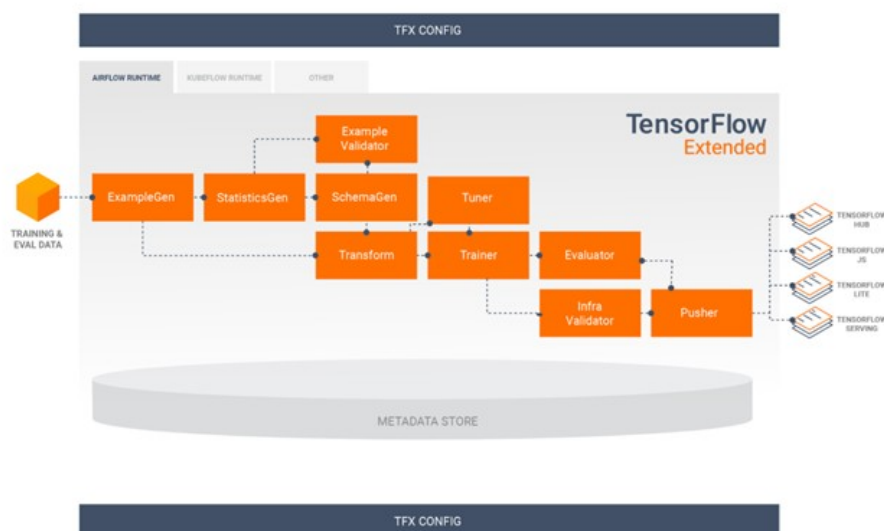
Tema 6. Despliegue de modelos de aprendizaje automático

versionado y los datos.



TensorFlow

TensorFlow Extended o TFX es una plataforma de escala de producción desarrollada por Google que puede desplegarse tanto on-premise o en la nube y basada en TensorFlow.



Sus principales características son las siguientes:

- ▶ Está concebida para manejar el flujo de trabajo necesario para la puesta en funcionamiento de un proyecto de aprendizaje automático de extremo a extremo.
- ▶ Esta plataforma permite orquestar el flujo de trabajo sobre muchas plataformas, tales como Apache Bean, KubeFlow, etc.

Tema 6. Despliegue de modelos de aprendizaje automático

- ▶ Contiene un kit de herramientas que permite compilar lo que se llaman tuberías o canalizaciones de aprendizaje automáticos.
- ▶ Maneja todas las tareas, desde la ingestión de los datos hasta el monitoreo de los modelos.

Hasta este punto hemos listado algunas de las principales plataformas de gestión del ciclo de vida de los modelos y no son solo estas, sin embargo, para mantener suficientemente simple esta lección no vamos a seguir mencionando ninguna otra.

Con el fin de ejemplificar la manera en que se puede poner un modelo en producción explicaremos una solución muy simple, haciendo uso de uno de los componentes de TensorFlow Extended, llamado TensorFlow Serving. Aunque desplegando de esta manera no podemos poner en práctica muchas de las tareas importantes del ciclo de vida, como el monitoreo de los modelos.

En la siguiente sección, por tanto, desarrollaremos un ejemplo que permite desarrollar, entrenar y desplegar un modelo de aprendizaje automático que luego puede ser usado por cualquier cliente.

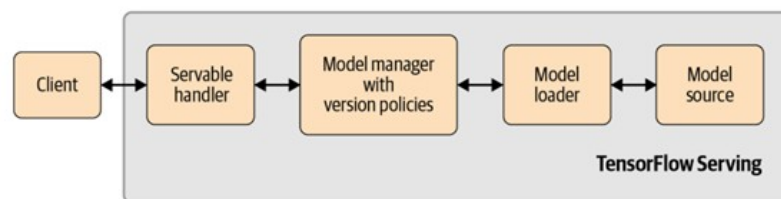


Fig.5. Arquitectura de Tensorflow Serving.

Para este ejemplo, decidimos usar como entorno de producción a TensorFlow Serving. Este es un servidor de modelos de aprendizaje automático a escala de producción desarrollado por Google para su propio uso y que forma parte de TFX.

En la Figura 5, mostramos la arquitectura general del servidor de modelos

Tema 6. Despliegue de modelos de aprendizaje automático

Tensorflow Serving y explicaremos grosso modo sus componentes antes de utilizarlo.

Comencemos de derecha a izquierda.

Model Source (Fuente de modelos): Este bloque es una estructura de módulos acoplables (Plugins) que permite cargar modelos o como se llaman en terminología de Tensorflow Serving, los servibles, desde muchos lugares diferentes. Tales como Google Cloud o AWS S3, el sistema de ficheros actual, etc. Una vez que se carga el modelo, se les notifica a los cargadores de modelos que son el próximo componente. En esencia este componente explora el sistema de ficheros en busca de modelos y sus versiones.

- ▶ **Model Loaders (Cargadores de Modelos):** La función de estos cargadores, es precisamente cargar los modelos desde las diversas fuentes sin importar el tipo de modelo que sea, el tipo de datos o el caso de uso. En esencia lo que hacen estos componentes es cargar y descargar los modelos de manera eficiente desde cualquier fuente.
- ▶ **Model Managers (Administradores de modelos):** Estos manejan el ciclo de vida de los modelos. Es decir, cuando el modelo se actualiza, cuales versiones del modelo se usa para inferencia, las reglas y políticas de inferencia entre otras cuestiones.
- ▶ **Servable Handler:** Este módulo brinda la API necesaria e interfaces para comunicarse con el Tensorflow Serving. Este servidor de modelos brinda dos tipos de interfaces o handler, una es REST API y la otra gRPC.

REST API es un protocolo usado por aplicaciones Web. Los clientes REST de las aplicaciones Web se comunican con esta por medio de métodos estándar HTTP, como GET, POST y DELETE. El intercambio de datos entre la aplicación y el cliente se hace en formato JSON.

Tema 6. Despliegue de modelos de aprendizaje automático

gRPC, por otro lado, es un protocolo de comunicaciones desarrollado inicialmente por Google. El formato de datos estándar usado es llamado Buffer de protocolo que permite la serialización de estructuras de datos. Este protocolo permite comunicaciones con baja latencia payloads más pequeñas que las de REST y es preferible cuando se trabajan con grandes conjuntos de datos para la inferencia.

Una de las principales ventajas de este servidor de modelos es que permite el despliegue de los modelos de manera fácil y sencilla.

Para instalar este servidor de modelos en Ubuntu, sería como sigue:

```
apt-get update && apt-get install tensorflow-model-server
```

Con el servidor instalado es hora que describamos cual es la idea detrás de nuestro ejemplo. Inicialmente vamos a desarrollar un modelo de aprendizaje automático que usa una red neuronal para aprender a clasificar flores de Iris, usando Keras. Ya en una de las lecciones anteriores lo tocamos. Luego que el modelo se entrena, se exporta a una carpeta, desde donde el servidor lo toma y lo pone en producción.

¿Qué significa esto de poner en producción el modelo? Pues bien, lo que significa es que ahora el modelo puede ser encuestado por medio de la red y no de forma directa empaquetándolo dentro de nuestra aplicación. Eso significa que cualquier aplicación que conozca como preguntar al modelo sobre una flor puede hacerlo.

El paso siguiente antes de todo es entrenar nuestro modelo y eso lo hacemos, como lo habíamos hecho en la lección que correspondía a redes neuronales, usando python y keras que es una especie de sabor de TensorFlow.

El código que usamos para entrenar el modelo lo mostramos en la imagen 5 y como puede verse el código comienza cargando las librerías que usaremos en el entrenamiento.

El próximo paso, es cargar el conjunto de datos que contiene 150 casos de flores, de

Tema 6. Despliegue de modelos de aprendizaje automático

tres categorías de Iris, Setosa, Virginica y Versicolor. Y los casos se distribuyen a razón de 50 para cada categoría.

```
import tensorflow.keras as keras
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from keras.layers.core import Dense, Activation, Dropout
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn.datasets as datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import os

# Cargamos el dataset
IDS = datasets.load_iris()
x_train, x_test, y_train, y_test = train_test_split(IDS.data, IDS.target, test_size = 0.15, random_state = 45)

# Convertimos a float, por defecto el tipo de datos uint8
x_train = x_train.astype('float32')
y_train = y_train.astype('float32')

# Normalizamos el conjunto de datos
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.fit_transform(x_test)

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

# Definimos el modelo que vamos a usar
model = tf.keras.Sequential([
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

# Con la red definida en la sección anterior ahora podemos entrenarla
model.compile(loss="categorical_crossentropy", optimizer="sgd", metrics = ['accuracy'])
model.fit(x_train, y_train, batch_size=100, epochs=30)

version = 1489706933289
MODEL_DIR = "/home/yero/Escritorio/Models/IrisModel"
export_path = os.path.join(MODEL_DIR, str(version))

# Ahora hacemos un test del modelo
tf.saved_model.save(model, export_path)
test_loss, test_acc = model.evaluate(x_test, y_test)
test_acc
```

Fig.5. Código para entrenar una Red Neuronal para aprender a clasificar Flores de Iris.

El conjunto de datos se almacena en IDS, que luego se divide en los conjuntos de entrenamiento y test.

El siguiente paso es normalizar los datos del conjunto y luego definimos el modelo que en este caso es una red neuronal con 4 capas, una de entrada, una de salida y dos ocultas. La capa de entrada espera un tensor de dimensión 4 y lo lleva a 512 neuronas y luego la segunda capa tendrá 256 neuronas y la tercera 128 y la capa de salida tendrá 3 neuronas que corresponde con las tres categorías en las que podemos clasificar las flores. Las funciones de activación de las tres primeras capas son de tipo relu y la capa de salida softmax.

Tema 6. Despliegue de modelos de aprendizaje automático

El tensor de entrada a la red neuronal tiene el aspecto siguiente:

```
[4.9, 3. , 1.4, 0.2]
```

Y el primero de los numeritos corresponde al atributo longitud del sépal, en este caso igual a 4.9, le sigue ancho del sépal y es igual a 3.0, el próximo atributo es la longitud del pétalo y es igual 1.4, mientras que el último es el ancho del pétalo que es igual a 0.2.

Este es el formato que le mandaremos al modelo que estará en producción para que nos retorne la categoría iris a la que pertenecen los datos que le enviamos.

Después que definimos la estructura que tiene nuestra red neuronal, debemos compilar el modelo y para eso decidimos que la red tendrá una función de pérdida basada en la entropía cruzada categórica y que usaremos como optimizador “sgd” que significa en español, gradiente descendiente estocástico.

Luego de eso, entrenamos el modelo y repetimos el proceso 30 veces. Luego de esto el modelo se guarda en una carpeta desde donde el servidor de modelos lo leerá y lo pondré en producción.

Para activar el servidor de modelos lo hacemos de la siguiente manera:

```
tensorflow_model_server --rest_api_port=8501 --model_name=IrisModel --  
model_base_path="/home/yero/Escritorio/Models/IrisModel"
```

Esto le indica el servidor de modelo que escuche por el puerto 8501 que por donde le mandaremos las solicitudes y que el modelo se llama IrisModel y que debe encontrarlo en la carpeta «/home/yero/Escritorio/Models/IrisModel».

Una vez que el modelo está en producción entonces ya podemos desarrollar un cliente que nos permita de forma remota clasificar flores de Iris mandando al clasificador las mediciones que hicimos a los pétalos u sépalos de la flor.

Tema 6. Despliegue de modelos de aprendizaje automático

Para este ejemplo desarrollamos un pequeño cliente que encuesta el modelo e imprime los resultados. Veamos.

```
#*****
#                               Cliente REST API para TensorFlow Serving                               **
#*****
import requests
import json

data = json.dumps({"inputs": [[5.4, 3.9, 1.3, 0.4]]})
headers = {"content-type": "application/json"}
json_response = requests.post('http://localhost:8501/v1/models/IrisModel/predict', data=data, headers=headers)
predictions = json.loads(json_response.text)
print(predictions)

{'outputs': [[0.196431, 0.0772045553, 0.726364374]]}
```

Al ejecutar este cliente, él se conecta con el servidor de modelos y le pasa al modelo en producción los datos correspondientes a las mediciones de la flor y este retorna tres numeritos, pueden verlos al final del código del cliente. Estos corresponden a la probabilidad de pertenencia de la flor a cada una de las categorías y escogemos la mayor de las probabilidades y es la 0.77 y eso nos indica que la flor pertenece a la categoría 2 es decir a versicolor.

Bueno, hasta aquí esta lección que, aunque corta ha hecho una introducción a un tema basto y complejo como lo es la puesta en producción de los modelos de aprendizaje automático.

Conclusiones

Hasta aquí hemos llegado en lo que corresponde al tema del manejo del ciclo de vida de los modelos de aprendizaje automático y su puesta en producción. Aunque es un mundo basto y complejo la idea inicial es solo hacer una pequeña introducción para despertar el interés sobre el tema y descubrir el velo que lo cubre.

Sin duda este tema esté estrechamente relacionado con lo que se conoce como MLOps u Operación de Aprendizaje automático y que requiere de una atención mucho más profunda y amplia con el fin de hacerle justicia a su vital importancia para las organizaciones.